

JAVA PAS À PAS

Introduction à la programmation et au langage Java

Robert Godin, Daniel Lemire

Cinquième édition, octobre 2026

PREFACE

1. CONCEPTS DE BASE

- 1.1 COMPOSANTES MATERIELLES D'UN ORDINATEUR (*HARDWARE*)
 - 1.1.1 *Processeur et mémoire*
 - 1.1.2 *Unités périphériques*
- 1.2 LE LOGICIEL
 - 1.2.1 *Le binaire, le langage machine et la compilation*
 - 1.2.2 *Étapes de création et d'exécution d'un programme Java avec le JDK sous Windows*

2. INTRODUCTION A LA PROGRAMMATION JAVA

- 2.1 COMMENTAIRE JAVA
- 2.2 IMPORTATION DE CLASSES
- 2.3 PACKAGES
- 2.4 NOTION DE CLASSE ET DE METHODE
- 2.5 LE NOM D'UNE CLASSE
- 2.6 LA METHODE *MAIN()*
- 2.7 CORPS D'UNE METHODE
 - 2.7.1 *Déclaration de variables*
 - 2.7.2 *Types prédéfinis de Java*
 - 2.7.3 *Appel de méthode de classe, paramètres et énoncé d'affectation*
 - 2.7.4 *Expression*
 - 2.7.5 *Expression de type String*
- 2.8 DIAGRAMME DE SEQUENCE UML
- 2.9 EXCEPTIONS
- 2.10 SYNTAXE DES IDENTIFICATEURS JAVA
- 2.11 DISPOSITION DU TEXTE
- 2.12 INITIALISATION DE VARIABLE A LA DECLARATION
- 2.13 METHODE *SYSTEM.OUT.PRINTLN()*
- 2.14 CLASSE *SCANNER*

3. STRUCTURES DE CONTROLE

- 3.1 LA SEQUENCE
- 3.2 LA BOUCLE AVEC L'ENONCE *WHILE*
- 3.3 QUALITÉ DU LOGICIEL, TESTS ET DÉBOGAGE
- 3.4 LA BOUCLE AVEC L'ENONCE *FOR*
- 3.5 LA DECISION AVEC *IF*

4. TYPES ET EXPRESSIONS JAVA

- 4.1 TYPE PRIMITIF ET LITTERAL
- 4.2 TYPES ET EXPRESSIONS NUMÉRIQUES
- 4.3 EXPRESSIONS BOOLÉENNES
- 4.4 TRAITEMENT DE CARACTÈRES
 - 4.4.1 *Type String, objets et classes*

- 4.5 FONCTIONS MATHÉMATIQUES : JAVA.LANG.MATH
- 4.6 SOMMAIRE DES OPÉRATIONS ET PRIORITÉS
- 5. GRAPHISME 2D ET CONCEPTS DE PROGRAMMATION OBJET**
 - 5.1 DESSIN AVEC LES CLASSES GRAPHICS ET UNE SOUS-CLASSE DE JFrame
 - 5.2 SIMPLIFICATION DU PROGRAMME PAR UNE METHODE AVEC PARAMETRES
 - 5.3 TRAITEMENT DES EVENEMENTS DE SOURIS (INTERFACE *MouseListener*)
 - 5.4 CONSTANTES (FINAL)
 - 5.5 SOMMAIRE D'UNE DECLARATION DE CLASSE
- 6. INTRODUCTION A L'ANIMATION 2D**
 - 6.1 UNE PREMIERE TENTATIVE D'ANIMATION
 - 6.2 ANIMATION PAR DOUBLE TAMPON
- 7. DEVELOPPEMENT DE CLASSES : CONCEPTION OBJET**
 - 7.1 DECOUPAGE D'UN PROGRAMME EN CLASSES
 - 7.2 COMPILATION ET EXECUTION D'UN PROGRAMME COMPOSE DE PLUSIEURS CLASSES ET DE PACKAGES
 - 7.3 LIMITER LA REPETITION DE CODE PAR LA CREATION D'UNE SUPER-CLASSE
- 8. ANIMATION 2D ET DEVELOPPEMENT D'UN JEU SIMPLE**
 - 8.1 ANIMATION AVEC UN *TIMER* DANS UNE SOUS-CLASSE DE *JPanel*
 - 8.2 ISOLER LE MONDE A ANIMER DU MECANISME D'ANIMATION
 - 8.3 DEVELOPPEMENT DU JEU
 - 8.4 GENERIQUES
 - 8.5 AUTRES COLLECTIONS
- 9. TRAITEMENT DE FICHIERS**
 - 9.1 FICHIER BINAIRE (FILEOUTPUTSTREAM, FILEINPUTSTREAM)
 - 9.2 DATAINPUTSTREAM ET DATAOUTPUTSTREAM
 - 9.3 FICHIER TEXTE
 - 9.3.1 *Représentation interne des caractères et traitement des fins de ligne*
 - 9.3.2 *Analyse lexicale avec la classe StreamTokenizer*
 - 9.3.3 *Traitement d'un document XML avec SAX et DOM*
 - 9.4 GESTION DE FICHIERS ET REPERTOIRES AVEC JAVA.IO.FILE
 - 9.4.1 *Dialogue de sélection de fichier avec la classe JFileChooser*
 - 9.5 FICHIER D'OBJETS EN JAVA
 - 9.5.1 *Fichier sérial d'objets en Java*
 - 9.5.2 *Fichier à adressage relatif en Java avec RandomAccessFile*

À PROPOS DES AUTEURS

Préface

Cet ouvrage présente les concepts de base de la programmation et du langage Java. Le livre s'adresse à un auditoire très large, aussi bien un débutant qui désire apprendre la programmation pour le plaisir qu'à un étudiant qui entreprend une carrière d'informaticien. L'approche proposée introduit graduellement les concepts de base de la programmation et leur incarnation dans le langage Java à l'aide d'une série d'exemples et d'exercices. Les exercices apparaissent avec leurs solutions mais il est important de prendre le temps de les faire pour en tirer les pleins bénéfices.

Un aspect qui motive l'emploi de Java comme premier langage est sa popularité. Sa conception par James Gosling, alors qu'il travaillait pour la compagnie Sun Microsystems, a été motivée par la possibilité d'écrire un code qui soit le moins dépendant possible de la plate-forme d'exploitation visée (*Write Once Run Anywhere* - WORA). Java est devenu un langage de développement très répandu pour une grande variété d'applications dans toutes sortes de contextes d'exploitation. La plupart des applications roulant sur les téléphones intelligents Android sont écrites en Java ou dans une variante du Java (comme Kotlin). Les systèmes de mégadonnées comme Apache Spark ou Elasticsearch sont souvent écrits en Java.

Afin de promouvoir l'aspect ludique de la programmation, le développement d'applications graphiques en deux dimensions et d'un jeu interactif simple est proposé pour illustrer des concepts de programmation importants, tels que la programmation objet et la modularisation du code, dans un contexte non trivial.

La programmation est une activité très gratifiante parce que les efforts fournis sont récompensés par des créations concrètes et utiles. Un grand sentiment d'accomplissement peut être éprouvé au fur et à mesure du développement d'un logiciel. Le plaisir est d'autant plus grand que le projet est d'envergure. Cependant, l'initiation à la programmation comporte aussi un grand potentiel de frustrations, surtout dans les premiers temps. En particulier, un certain nombre de détails au sujet des systèmes de développements de programmes doivent être maîtrisés dans les premières phases d'apprentissage afin d'atteindre les activités plus intéressantes qui sont au cœur de la programmation. Nous vous encourageons à faire preuve de

persévérance et de patience pour surmonter cette première difficulté car les efforts seront grandement récompensés par la suite.

Il faut toujours garder à l'esprit que la seule lecture d'un livre sur la programmation ne suffit jamais pour apprendre à programmer. Il vous faut programmer pour apprendre. Vous devrez faire les exercices, tester les exemples, les modifier, etc. Il faut être curieux pour apprendre.

1. Concepts de base

« *L'art de douter est le meilleur secret pour apprendre* » — Marcel Prévost

Un ordinateur doit être programmé pour accomplir une tâche.

Programme, programmation, langage de programmation, logiciel (software), matériel (hardware)

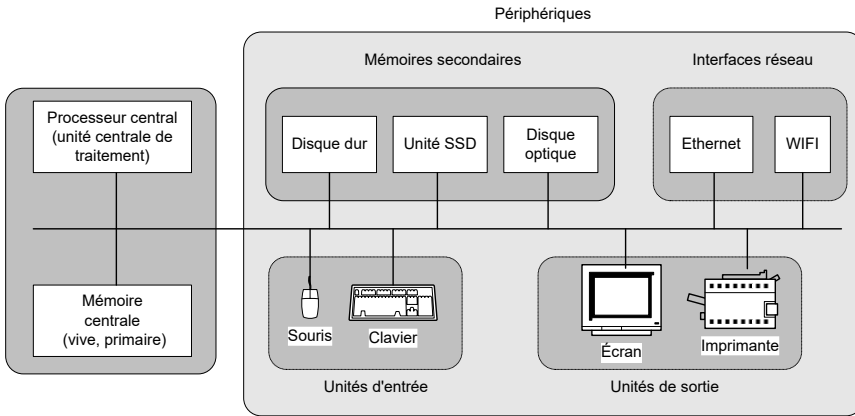
Un *programme* est un ensemble organisé d'instructions que l'ordinateur exécute afin d'accomplir une tâche. La *programmation* est le processus de production d'un programme. Un programme est exprimé à l'aide d'un *langage de programmation*. Il existe une grande variété de langages de programmation qui ont été développés tels que Java, Python, C, C++, C#, JavaScript, Swift, PHP, Scala, Go, R, etc.

Le terme logiciel (*software*) désigne l'ensemble des programmes, par opposition au matériel (*hardware*) de l'ordinateur qui correspond à l'ensemble des composantes physiques.

Avant d'aborder la programmation et Java, il est utile d'introduire certaines notions de base et situer la programmation dans son contexte.

1.1 Composantes matérielles d'un ordinateur (*hardware*)

La description d'un ordinateur faite dans cette section est une introduction à un sujet complexe. Parfois, certains aspects sont simplifiés ou omis afin de s'en tenir à l'essentiel pour la compréhension des concepts de base de la programmation. Un ordinateur est constitué de plusieurs *composantes matérielles* qui collaborent entre elles afin de produire un traitement. L'*architecture matérielle* d'un ordinateur est la description de ses composantes matérielles et de leurs interrelations. Les composantes matérielles d'un ordinateur typique sont illustrées à la Figure 1 : processeur central, mémoire centrale, bus et unités périphériques.

Figure 1. Composantes matérielles (*hardware*) d'un ordinateur typique.

Bus

Le bus permet la communication entre les composantes de l'ordinateur. Une composante branchée au bus peut envoyer des informations à une autre composante ou recevoir des informations d'une autre composante branchée au même bus.

Pour permettre les échanges ordonnés entre les différentes composantes, il doit y avoir un mécanisme qui empêche les conflits entre plusieurs composantes qui veulent utiliser le bus en même temps. Les composantes doivent respecter des règles précises pour établir la communication. L'ensemble des règles est appelé le *protocole de communication du bus*.

1.1.1 Processeur et mémoire

Le cœur d'un ordinateur est composé d'un *processeur central* et d'une *mémoire centrale*. Avant qu'un programme ne puisse être exécuté, celui-ci doit être placé dans la mémoire centrale de l'ordinateur.

Mémoire centrale (main memory, primary storage), principale, vive, primaire ou volatile

La mémoire centrale contient temporairement les *instructions* et *données* d'un programme en cours de traitement. L'ordinateur exécute des instructions placées en mémoire centrale. Ces instructions manipulent des données qui

doivent aussi résider en mémoire centrale. Un programme est habituellement chargé en mémoire centrale à partir d'une unité périphérique (souvent une mémoire secondaire tel que le disque dur) avant d'être exécuté.

Adresse-mémoire

La mémoire centrale est constituée d'une séquence de *cases* (*cellules*, *mots*) de taille fixe. Une case de la mémoire centrale est identifiée par une *adresse* (*adresse-mémoire*). Dans le cas le plus simple, l'adresse est un entier dans un intervalle de 0 à $n-1$ ¹, où n est la taille de la mémoire centrale. La taille d'une case peut varier selon le processeur utilisé.

Les cases de la mémoire centrale peuvent contenir des instructions ou des données. Le contenu des cases peut être modifié par les instructions des programmes.

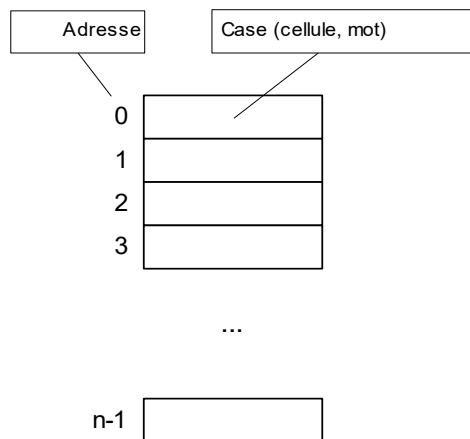


Figure 2. Mémoire centrale.

La mémoire centrale est caractérisée par sa rapidité d'accès d'une part et sa volatilité (non-permanence) d'autre part. La vitesse est importante pour que le processeur puisse accéder rapidement aux instructions et aux données en mémoire centrale lors de l'exécution d'un programme. On désigne aussi la mémoire centrale par le terme *Random Access Memory* (RAM) à cause de cette capacité à pouvoir accéder rapidement à n'importe quelle case à tout

¹ Par exemple, si la taille de la mémoire est $n=16$, les cases seront numérotées de 0 à 15. En réalité, le schéma d'adressage peut être plus compliqué...

moment. Sauf pour une petite partie appelée le ROM (*Read Only Memory*), le contenu de la mémoire centrale n'est pas permanent. Il est perdu lorsque le courant électrique qui alimente l'ordinateur est interrompu. Le ROM et les mémoires secondaires permanentes (tel que le disque) peuvent être utilisées pour conserver l'information en permanence au-delà des interruptions de courant. Il faut comprendre que les interruptions de courant ne sont pas toujours volontaires et peuvent provenir, par exemple, d'une panne d'électricité. Il est donc important de placer, en mémoire secondaire ou en ROM, les éléments qui doivent être conservés de manière *persistante*, i.e. survivre aux programmes ou aux anomalies de fonctionnement.

En pratique, le programmeur n'accède pas directement aux adresses en mémoire correspondant à du stockage physique. Le système d'exploitation offre au logiciel un accès à une mémoire virtuelle. L'adresse virtuelle est convertie par le système d'exploitation, au besoin, en adresse physique. Le programme reçoit sa mémoire en bloc appelée pages. Les pages peuvent avoir différentes tailles selon le système d'exploitation et l'architecture matérielle, mais occupent généralement au moins 4 kilo-octets. Par ailleurs, au sein d'un langage de programmation comme Java, mêmes les adresses virtuelles ne sont pas accessibles au programmeur.

Processeur central (Central Processing Unit - CPU), ou unité centrale de traitement (UCT)

Le *processeur central* est la composante qui coordonne l'exécution d'un programme. Il effectue inlassablement le traitement suivant :

Chercher la prochaine instruction en mémoire centrale²
Exécuter l'instruction
Chercher la prochaine instruction en mémoire centrale
Exécuter l'instruction
Etc.

Chacune des instructions d'un programme produit un traitement relativement simple. Par exemple, une instruction peut additionner le contenu de deux cases de la mémoire centrale dont les adresses sont x et y , et placer le résultat dans une troisième case dont l'adresse est z .

² La réalité est un peu plus complexe. L'accès à la mémoire centrale peut être accéléré par l'intermédiaire d'une [antémémoire](#) (*cache memory*).

Mémoire centrale de l'ordinateur

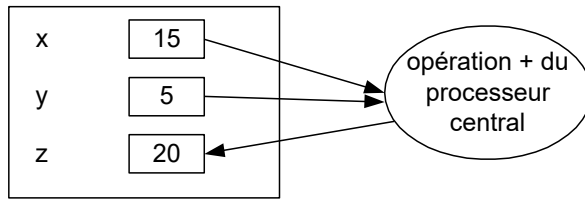


Figure 3. Instruction effectuée par le processeur central.

Le processeur central est lui-même typiquement constitué de deux composantes : l'Unité de Contrôle (UC) et l'Unité Arithmétique et Logique (UAL). L'UC est responsable de déterminer la séquence d'exécution des instructions. L'UAL exécute les instructions telles que l'addition illustrée à la Figure 3.

Les processeurs effectuent généralement leurs opérations sur des valeurs qui sont stockées au sein de registres : les registres sont des unités de mémoire particulières faisant généralement 64 bits et permettant de représenter différentes valeurs (nombres, adresses, etc.). Les processeurs n'ont que quelques dizaines de registres au plus : les registres doivent constamment être déchargés en mémoire ou rechargés à partir de la mémoire.

La puissance d'un ordinateur vient de sa capacité à exécuter un très grand nombre d'opérations simples à une vitesse extrême. Un processeur typique peut exécuter des milliards d'instructions par seconde.

Les processeurs exécutent leurs instructions à une cadence fixe, souvent précisée en gigahertz (GHz). À chaque seconde, le processeur bénéficie d'un certain nombre de cycles. Les instructions démarrent et se terminent toujours lors d'un cycle. Toute chose étant égale, un processeur ayant une cadence plus rapide (plus de cycles par seconde) sera plus rapide ; cependant il utilisera aussi davantage d'énergie. Les processeurs ajustent donc parfois leur vitesse en fonction de la charge de calcul.

La plupart de nos processeurs sont superscalaires : ils peuvent exécuter plusieurs instructions par cycle. Ainsi un processeur pourrait exécuter plusieurs additions en simultanée. Certains processeurs peuvent faire 4 ou 6 additions par cycle. Par ailleurs, la plupart des processeurs bénéficient

d'instructions spécialisées appelées SIMD (single instruction multiple data) qui fonctionnent sur des registres plus volumineux (faisant 128 bits, 256 bits ou plus) capables de représenter plusieurs valeurs à la fois (par exemple, 4 entiers). La plupart du temps, le programmeur n'a pas à se soucier de la manière dont sont exécutées les instructions : si on fait exception de la performance, il importe peu de savoir combien d'opérations sont traitées par cycle.

Un aspect important du fonctionnement de l'ordinateur est la manière de déterminer la *prochaine instruction* à exécuter. Il y a trois mécanismes fondamentaux à cet effet.

1. *Séquence*. L'adresse en mémoire centrale de la *prochaine instruction* est normalement celle qui suit l'adresse de l'instruction précédente. Donc, par défaut, les instructions sont exécutées en séquence. Cependant, si c'était toujours le cas, l'unité centrale de traitement exécuterait les instructions jusqu'à ce qu'elle aboutisse à un cul de sac, soit la dernière adresse de la mémoire centrale et ne pourrait continuer !
2. *La boucle*. Des instructions peuvent spécifier l'adresse de la prochaine instruction à exécuter. Par exemple, une instruction peut provoquer le saut à une adresse précédente ce qui permet de répéter un ensemble d'instructions en revenant constamment au début de la séquence.
3. *Décision (sélection, embranchement ou choix)*. Certaines instructions peuvent choisir l'adresse de la prochaine instruction en fonction d'une condition. Par exemple, si le contenu de la case d'adresse x est 0, sauter à l'adresse y sinon continuer normalement en séquence. C'est ce genre d'instruction qui permet à l'ordinateur de « prendre des décisions » et de modifier son comportement au besoin.

Ces trois manières d'organiser l'exécution des instructions, la séquence, la boucle et la décision, sont des mécanismes de base de la plupart des langages de programmation.

En pratique, la plupart des CPU adoptent une architecture multi-cœur où chaque cœur est un processeur capable d'exécuter ses propres instructions. Il peut y avoir 2, 4, 6 ou même 64 cœurs (ou processeurs) dans un même CPU. Ces processeurs coexistent dans la même unité et ils ont accès aux

mêmes ressources matérielles. L'unité centrale doit coordonner les processeurs.

Les processeurs sont souvent beaucoup plus rapides que la mémoire et l'accès à la mémoire souffre d'un temps d'accès (latence). En général, plusieurs dizaines de nanosecondes sont nécessaires pour qu'un processeur puisse avoir accès à une valeur en mémoire. Un tel délai peut représenter des dizaines de cycles pendant lesquels le processeur ne pourrait rien faire. Pour améliorer la performance, les processeurs ont donc leur propre mémoire appelée mémoire tampon. Cette mémoire sert à la fois à stocker des données et des instructions. Il y a plusieurs couches de mémoire tampon allant de la plus petite et la plus rapide (L1) à la plus lente et la plus volumineuse (L3 sur les processeurs x64). Le temps d'accès à la mémoire stockée en L1 n'est que de quelques cycles, mais ne permet que de stocker quelques kilo-octets. Il serait complexe pour le processeur de gérer la mémoire avec une granularité très fine (par exemple, octet par octet). Ainsi la mémoire tampon gère la mémoire en blocs de mémoire appelée lignes. Les lignes font souvent entre 64 octets et 128 octets. Le processeur ne peut donc pas demander la valeur d'un octet spécifique : il faut accéder à une ligne entière. Les lignes sont alignées avec les adresses de la mémoire virtuelle : les premiers 64 octets forment une ligne et ainsi de suite. Dans une architecture multi-cœur, les différents processeurs ont des mémoires tampons distinctes, mais aussi des mémoires tampons partagées. Cette mémoire tampon garde une copie des données récemment traitées. Il faut néanmoins souvent tout de même avoir accès à la mémoire centrale, mais le processeur passe généralement par la mémoire tampon. La plupart des processeurs peuvent faire plusieurs requêtes d'accès à la mémoire en même temps.

1.1.2 Unités périphériques

Les unités périphériques permettent d'échanger de l'information entre la mémoire centrale de l'ordinateur et le monde extérieur.

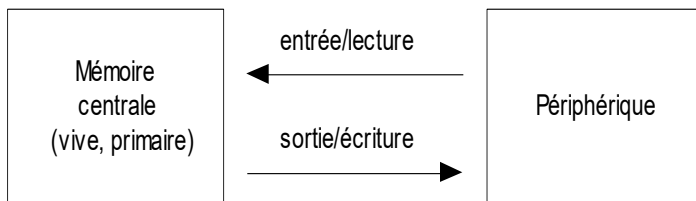


Figure 4. Opération d'entrée/lecture et de sortie/écriture

Entrée, lecture (input)

Du point de vue de la mémoire centrale, une opération de transfert d'une information d'une unité périphérique vers la mémoire centrale est appelée une *opération d'entrée* ou de *lecture*.

Sortie, écriture (output)

Un transfert inverse de la mémoire centrale vers une unité périphérique est une *opération de sortie* ou d'*écriture*. Des instructions d'entrées/sorties sont prévues pour déclencher les opérations d'entrée/sortie.

Périphérique d'entrée

Les *périphériques d'entrée* permettent de transmettre de l'information du monde extérieur à la mémoire centrale. Par analogie avec l'humain, les périphériques d'entrée sont les sens de l'ordinateur. En particulier, certains périphériques permettent aux humains de communiquer avec l'ordinateur. La souris, l'écran tactile, la caméra, le microphone et le clavier sont des unités d'entrée bien connus. La *manette de jeu* et le *bâton joyeux (joystick)* sont employés pour les jeux. Au-delà de ces périphériques d'entrée usuels, toutes sortes de capteurs existent pour saisir des données de diverses natures.

Périphérique de sortie

Les *périphériques de sortie* permettent à l'ordinateur de transmettre de l'information au monde extérieur et en particulier à l'humain. L'*écran* et l'*imprimante* sont des périphériques de sortie typiques. Les *haut-parleurs* ou les *écouteurs* sont aussi fréquemment utilisés.

Divers périphériques d'entrée/sorties plus adaptés à des applications particulières sont de plus en plus répandus. Les *interfaces réseau (modem, carte réseau, ...)* et les *mémoires secondaires (disque dur, mémoire SSD, disque optique, ...)*

sont aussi des périphériques d'entrée/sortie. Plusieurs de ces périphériques permettent à la fois les entrées et les sorties. Par exemple, il est possible de lire et d'écrire des informations sur le disque dur. Il est possible d'envoyer et de recevoir des informations par un réseau. Cependant, il est impossible d'écrire sur certains disques optiques.

Mémoire secondaire (secondary storage), de masse, auxiliaire, permanente, externe, stable, non volatile ou persistante

Une *mémoire secondaire* est une mémoire habituellement plus lente que la mémoire centrale mais qui a la caractéristique d'être permanente. Son contenu ne disparaît pas lorsque le courant électrique est interrompu. Les informations (données et programmes) qui doivent être conservées en permanence sont donc placées en mémoire secondaire. Ce type de mémoire est aussi appelé mémoire *de masse, auxiliaire, permanente, externe, stable, non volatile* ou *persistante*.

Il est possible de transférer des informations (données ou programmes) entre une mémoire secondaire et la mémoire centrale par des instructions d'entrées/sorties prévues à cet effet. Historiquement, la mémoire secondaire la plus répandue était l'*unité de disque magnétique (disque dur, disque rigide* ou tout simplement le *disque*). La *mémoire SSD (Solid State Drive)*, ou *disque SSD* est maintenant plus populaire à cause de sa rapidité en lecture par rapport au disque. Les autres types de mémoires secondaires moins rapides ont d'autres caractéristiques qui les rendent utiles (coût, capacité de stockage, etc.).

Dans certains cas, il est possible d'utiliser une mémoire secondaire distante qui n'est pas à proximité physiquement du processeur. On accède à cette mémoire par l'entremise d'un réseau avec ou sans fil.

Interface réseau

Une interface réseau permet à l'ordinateur de communiquer avec d'autres ordinateurs par l'intermédiaire d'un réseau d'ordinateurs. Il y a différents types de réseaux d'ordinateurs et la manière de brancher un ordinateur à un réseau varie d'un type de réseau à un autre.

Réseau local (*Local Area Network* - LAN), domestique, d'entreprise, Internet

Un réseau dit *local* permet la communication entre un ensemble limité d'ordinateurs situés à proximité³ les uns des autres. Un *réseau local domestique* permet la communication entre ordinateurs d'une résidence. Un *réseau local d'entreprise* permet la communication entre les ordinateurs à l'intérieur d'une entreprise. Le réseau *Internet* est un réseau planétaire qui permet la communication avec des centaines de millions d'ordinateurs répartis autour de la planète. Pour être plus précis, le réseau Internet permet la communication entre (*inter*) les réseaux locaux (*net*). C'est donc un réseau de réseaux ! Ainsi, un ordinateur branché à un réseau local peut accéder à Internet lorsque le réseau local est lui-même branché à Internet.

Deux interfaces réseau populaires sont l'interface avec fil *Ethernet* ou l'interface sans fils (WIFI) à un réseau local qui est lui-même relié au réseau Internet. Les appareils mobiles peuvent passer par le réseau cellulaire pour l'accès à Internet. Le branchement entre les périphériques et le bus suit des conventions basées des normes bien établies tel que ISA, PCI, PCMCIA, SCSI, etc. La pièce maîtresse de l'ordinateur sur laquelle sont installées les différentes composantes de l'ordinateur est la *carte mère* (*mother board*).

1.2 Le logiciel

Un ordinateur fonctionne en exécutant des programmes. On utilise souvent le terme *logiciel* (*software*) pour désigner les programmes. Lorsqu'on démarre un ordinateur⁴, il y a un premier programme qui est automatiquement exécuté, appelé le *programme de démarrage* (*boot program*). Ce premier programme est toujours dans la mémoire centrale à une adresse fixe, connue à l'avance. À cet effet, il y a une petite partie de la mémoire centrale qui est permanente, le ROM (*Read Only Memory*), et qui contient le programme de démarrage. Le programme de démarrage a pour rôle essentiel de placer en

³ La distance possible entre les ordinateurs d'un réseau local varie en fonction du matériel utilisé.

⁴ On allume l'ordinateur en enfonçant le bouton ON qui est parfois bien caché pour que les non-initiés éprouvent un sentiment d'humiliation la première fois qu'ils essaient de le faire fonctionner. Curieusement, il faut parfois enfoncer le bouton ON pour éteindre certains ordinateurs...

mémoire centrale un plus gros programme appelé le *système d'exploitation*⁵, à partir d'une mémoire secondaire, habituellement le disque dur⁶.

Système d'exploitation (operating system - OS)

Le système d'exploitation est un programme dont le rôle est de faciliter l'utilisation de l'ordinateur et en particulier des périphériques. Windows, Unix et macOS sont des exemples de systèmes d'exploitation. Sans système d'exploitation, un ordinateur est aussi utile qu'une aiguille dans une botte de foin.

Le système d'exploitation permet de faciliter l'utilisation de l'ordinateur et d'organiser l'exécution d'autres programmes appelés *programmes d'application* ou plus simplement *applications*. Après que le système d'exploitation est chargé en mémoire centrale, celui-ci se met en état d'attente d'une commande de l'utilisateur. À ce moment, l'utilisateur va souvent commander l'exécution d'un programme d'application.

Interface système (shell), interface en ligne de commande (Command Line Interface- CLI), interface à l'utilisateur graphique (Graphical User Interface - GUI)

Les systèmes d'exploitation offrent habituellement deux types d'interface permettant d'invoquer leurs fonctions. Les interface en ligne de commande (*Command Line Interface – CLI*) ou les *interfaces à l'utilisateur graphiques* conviviales (*Graphical User Interface - GUI*) basées sur l'utilisation des fenêtres, de menus, de boutons, etc. Dans le cas de Windows, l'interface en ligne de commande est aussi appelée *fenêtre de commande Windows (Command Prompt)* et Windows est l'interface graphique. Dans les systèmes d'exploitation UNIX, par analogie avec une noix, il est d'usage d'employer le terme *shell* (coquille) pour désigner une interface système qui permet de faire exécuter des commandes du *noyau (kernel)* du système d'exploitation.

⁵ Pour être plus précis, le programme de démarrage inclut déjà certaines parties du système d'exploitation nécessaires pour accéder aux unités périphériques. Dans les ordinateurs PC compatibles, cette portion du système d'exploitation est appelée le BIOS (*Basic Input Output System*).

⁶ Il est aussi possible de charger le système d'exploitation à partir d'une autre mémoire secondaire (clé USB, cédérom, etc.) mais ceci est habituellement effectué dans des circonstances spéciales, par exemple, lorsqu'un problème survient avec le disque dur.

Programme d'application (ou simplement application)

Les programmes d'applications accomplissent des tâches utiles. Par exemple, parmi les applications d'usage courant, il y a les *programmes de traitement de textes* (e.g. Bloc-notes, Wordpad de Windows, Word de Microsoft Office, Emacs de UNIX), les *chiffriers électroniques* (e.g. Excel de Microsoft Office), les *surfeurs WEB* (e.g. Google Chrome, Safari, Microsoft Edge), les *systèmes de courriel* (e.g. Outlook de Microsoft, Google GMail), les *réseaux sociaux* (e.g. Facebook, Twitter, Instagram).

Cet ouvrage traite du développement de programmes d'application en Java. Concrètement, avant qu'il ne soit chargé en mémoire centrale pour être exécuté, un programme d'application se trouve habituellement en mémoire secondaire, par exemple, sur disque dur. Un des rôles importants du système d'exploitation est de permettre d'organiser les programmes et les données placées en mémoire secondaire afin de pouvoir les retrouver.

Fichier (file), système de gestion de fichiers (file system)

La partie du système d'exploitation qui s'occupe de l'organisation des données et des programmes en mémoire secondaire est le *système de gestion de fichiers (file system)*. Un *fichier* peut contenir un ou une partie d'un programme ou encore des données pour un programme.

On utilise souvent le terme *document* pour désigner un fichier de données. En effet, un fichier peut par exemple contenir un texte produit avec un programme de traitement de texte. Le fichier est alors un document contenant des données du point de vue de l'application de traitement de texte. Le système de gestion de fichier permet de regrouper des fichiers dans un *dossier* (aussi appelé *répertoire*). Un dossier peut lui-même regrouper d'autres dossiers, ainsi de suite, résultant en une *hiérarchie de dossiers*.

Pour identifier un dossier ou un fichier, on peut employer son *chemin (path)*. Le chemin indique la séquence des dossiers qui mènent à l'élément visé.

Sous Windows, le chemin commence tout d'abord par le nom de *volume* dans lequel se situe le fichier ou dossier que l'on recherche, puis vient ensuite l'ordre des dossiers à parcourir (les noms des dossiers sont séparés par des «\»). Le nom de volume identifie une unité périphérique d'entrée/sortie telle

qu'un disque dur, un disque SSD, etc. Ainsi, dans l'exemple de chemin suivant, il y a un disque dur nommé C. Pour arriver au fichier *HelloWorld.java*, il faut donc partir du disque C, aller dans le dossier *Users*, puis ensuite, dans *Robert*, et ensuite dans *Documents* pour enfin arriver au fichier *HelloWorld.java*. Ce fichier contient un programme en Java comme nous le verrons par la suite.

C:\Users\Robert\Documents\HelloWorld.java

Il est à noter qu'après le nom de volume, il faut mettre un « : » avant le « \ ».

D'autres systèmes d'exploitation comme Linux, FreeBSD ou macOS utilisent des chemins qui débutant toujours par la racine (« / ») et omettant le nom du volume.

/users/lemire/Documents/HelloWorld.java

Les systèmes de gestion de fichier modernes permettent le stockage dans des serveurs distants en donnant l'impression d'une unité de stockage locale. De plus en plus, le stockage est effectué dans le *nuage informatique* (*infonuagique*, *cloud computing*). Les grandes entreprises offrent des services de stockage dans le nuage qui incorporent des mécanismes de sécurité, de fiabilité et de partage plus élaborés que le stockage sur des unités locales.

Parallélisme simulé : pseudo-parallélisme

Les ordinateurs permettent aussi de faire exécuter plusieurs programmes en même temps. Mais comment ceci est-il possible même quand il n'y a qu'un seul processeur ! En réalité, dans un système monoprocesseur (à un seul processeur), il n'y a effectivement qu'un seul programme à la fois qui est exécuté. Le système d'exploitation fournit l'illusion d'une exécution simultanée de plusieurs programmes en allouant à chacun des programmes à tour de rôle une petite partie du temps du processeur central. Mais ceci se fait tellement rapidement que l'utilisateur ne peut le percevoir même s'il est un « petit vite ». Le terme *pseudo-parallélisme* désigne cette illusion. Au-delà du pseudo-parallélisme, les ordinateurs modernes sont souvent pourvus de plusieurs processeurs qui permettent donc l'exécution en parallélisme réel de plusieurs traitements.

Les unités périphériques sont aussi souvent pourvues de processeurs simples qui peuvent exécuter des opérations d'entrée/sortie en parallèle avec le traitement du processeur central. Pour coordonner les différents traitements entre tous ces processeurs, des mécanismes d'interruption sont prévus afin qu'un processeur puisse signaler à un autre processeur qu'une tâche est terminée, et que ce dernier puisse réagir à cet événement. Ceci entraîne une interruption du cours normal d'exécution de la boucle de traitement de base. La synchronisation et coordination des différentes tâches parallèles d'un ordinateur peut devenir très complexe et c'est le rôle du système d'exploitation de fournir des services à cet effet.

1.2.1 Le binaire, le langage machine et la compilation

Les instructions qui composent un programme en mémoire centrale sont exprimées sous forme d'un code appelé le *langage machine*. Ce langage varie en fonction du type de processeur visé. Une instruction en langage machine est composée d'une suite de symboles binaires. Un symbole binaire est soit un « 0 » ou un « 1 ». On désigne par *langage binaire*, un langage qui se limite à l'utilisation de deux symboles. Chacun des symboles d'une séquence de symboles binaires (0 ou 1) est appelé un *bit*. Une séquence de 8 bits est appelée un *octet* (*byte*). Chacune des cases de la mémoire peut contenir un nombre fixe de bits, habituellement, 8 bits (1 octet). La taille d'une instruction est un multiple de la taille d'une case mémoire. Il peut même y avoir des tailles d'instruction variables pour un même processeur.⁸

Les données manipulées par un programme sont aussi codées en mémoire sous forme binaire. Par exemple, un nombre entier est souvent représenté par une séquence de 32 bits (4 octets). Le nombre entier est codé selon un système de numération binaire (base 2). Par exemple, l'entier 25 en décimal est représenté sur huit bits (un octet) par :

$$\begin{aligned}
 00011001_2 &= 0 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 \\
 &\quad + 1 \times 2^4 + 1 \times 2^3 \\
 &\quad + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0
 \end{aligned}$$

⁷ En anglais, le terme octet existe aussi, mais on lui préfère le terme *byte*. Généralement, un *byte* est constitué de 8 bits.

⁸ C'est le cas pour les processeurs x86/x64 commercialisés par Intel et AMD.

$$= 25_{10}$$

Chacun des bits correspond à un exposant en base 2.

Les mémoires modernes ont des tailles impressionnantes. Les unités de mesure suivantes sont employées pour les tailles des mémoires et données. Elles sont basées sur le système international qui repose sur la base 10.

Nom	Symbole	Nombre d'octets
Kilo-octet	Ko	10^3
mégaoctet	Mo	10^6
gigaoctet	Go	10^9
téraoctet	To	10^{12}
pétaoctet	Po	10^{15}
exaoctet	Eo	10^{18}
zettaoctet	Zo	10^{21}
yottaoctet	Yo	10^{24}

En anglais, on emploie les acronymes KB, MB, GB, TB, PB où B correspond à *Byte*.⁹ À cause de l'emploi des nombres binaires en informatique, on a traditionnellement employé le préfixe *kilo* pour désigner 2^{10} , *méga* pour 2^{20} et *giga* pour 2^{30} . Depuis 1998, la norme CEI 60027-2 propose plutôt les préfixes du tableau suivant pour les nombres en base 2. Cependant, ces conventions sont plus ou moins respectées en pratique.

⁹ En anglais, le B majuscule désigne un *byte*. Tant en français qu'en anglais, le b minuscule désigne le bit. Il ne faut donc pas confondre KB et Kb.

Nom	Symbole	Nombre d'octets
kibioctet	Kio	$2^{10} = 1\,024$
mébioctet	Mio	$2^{20} = 1\,048\,576$
gibioctet	Gio	$2^{30} = 1\,073\,741\,824$
tébioctet	Tio	$2^{40} = 1\,099\,511\,627\,776$
pébioctet	Pio	$2^{50} = 1\,125\,899\,906\,842\,624$
exbioctet	Ei	$2^{60} = 1\,152\,921\,504\,606\,846\,976$
zébioctet	Zi	$2^{70} = 1\,180\,591\,620\,717\,411\,303\,424$
yobioctet	Yi	$2^{80} = 1\,208\,925\,819\,614\,629\,174\,706\,176$

Une chaîne de caractère est typiquement représentée en utilisant une séquence d'octets. Chacun des caractères est codé en suivant une norme de codage. Le code ASCII est une des premières normes employées pour représenter les caractères par des codes de la taille d'un octet. L'exemple suivant montre le codage pour une suite d'octets. La première ligne montre le code binaire. La deuxième ligne indique les caractères correspondants.

01100001	01100010	01100011	00001101	00001010	00110001	00110010	00001101	00001010
a	b	c	\r	\n	1	2	\r	\n

Sous Windows, la fin de ligne est représentée par la séquence des caractères spéciaux ASCII, *retour de chariot* (`"\r"`) et *saut de ligne* (`"\n"`)¹⁰. Le code ASCII ne permet pas de traiter les caractères de toutes les langues. La norme *Unicode* (www.unicode.org) est une norme plus générale qui permet d'encoder les caractères d'un grand nombre de langues. Sur Internet, on représente souvent Unicode en utilisant le code UTF-8. Le code ASCII est un cas particulier du code UTF-8. Le code UTF-8 permet de représenter une gamme plus riche de caractères (appelés parfois graphèmes) en utilisant des séquences de 2, 3 ou 4 octets. On distingue les octets ASCII des autres au sein d'une séquence UTF-8 par le fait que leur bit de poids fort est 0. Unicode permet aussi de combiner plusieurs graphèmes pour faire un même caractère. Le Java utilise le code UTF-16 qui représente chaque graphème en utilisant 2 ou 4 octets ; le code UTF-16 est incompatible avec ASCII. Le code UTF-8 est plus concis que le code UTF-16, sauf lors que le texte

¹⁰ La manière de représenter les fins de ligne peut différer en fonction de la plate-forme. Sous macOS et Linux, par exemple, on emploie uniquement le *saut de ligne* (`"\n"`).

comporte beaucoup de caractères asiatiques. En pratique, il est souvent nécessaire de faire une conversion entre UTF-16 et UTF-8 lorsqu'on programme en Java, en particulier quand on doit communiquer avec le web.

Une image est représentée par un quadrillage de pixels (*picture element*). Pour une image en noir et blanc, chacun des pixels est représenté par un bit qui correspond à la couleur noir ou blanc (0 ou 1). Pour une image en couleur, le système RVB (RGB) encode chacun des pixels par trois entiers entre 0 et 255 qui correspondent aux trois couleurs, rouge (*Red*), vert (*Green*) et bleu (*Blue*).

Dans le cas des premiers ordinateurs, il fallait les programmer directement en langage machine, ce qui était très fastidieux et peu productif. Aujourd'hui, les humains programment à l'aide de langages de programmation dits évolués, tel que Java. Les langages évolués cachent le codage binaire employé pour les données. Les données sont représentées dans une forme interprétable par l'humain. Par exemple, les nombres entiers sont représentés en décimal dans les programmes Java.

Cependant, comme l'ordinateur ne comprend pas directement le Java, il faut un moyen intermédiaire qui permet d'exécuter le programme Java sur une machine. Deux approches sont possibles, la compilation et l'interprétation. Le processus de compilation et d'exécution d'un programme Java est illustré à la Figure 5.

Compilateur

Un *compilateur* est un programme qui traduit un programme, écrit en un langage évolué, en un programme en langage machine. Le programme en langage évolué est appelé le *programme source* ou encore *code source*. Et le programme en *langage machine* résultant de la traduction est appelé le *programme objet* ou encore le *code objet*. Les langages C, C++, Rust, Swift et Fortran s'exécutent normalement par l'entremise d'un compilateur.¹¹

¹¹ En pratique, plusieurs compilateurs vont utiliser des langages intermédiaires au lieu de compiler directement en langage machine.

Code-octet Java, machine virtuelle Java¹²

Le cas de Java est un peu plus compliqué parce que le compilateur Java traduit le programme source en un programme objet intermédiaire, correspondant au langage machine de la *machine virtuelle Java* (*Java Virtual Machine* -JVM). Le langage machine de la machine virtuelle Java est appelé *code-octet* (*byte-code*) Java (ou simplement *code-octet*). La machine virtuelle Java est une machine théorique qui n'existe pas vraiment, du moins pour l'instant. Il manque donc quelque chose pour faire exécuter le programme objet en *code-octet* sur une machine réelle ! Cependant, comme le langage code-octet est proche du langage machine des machines réelles, le processus de traduction restant est assez facile à réaliser. Il y a trois manières d'exécuter les programmes en *code-octet Java* sur la machine réelle visée :

- ◆ *Interprète de code-octet.* Un petit programme spécial, appelé *interprète de code-octet*, simule une machine Java. Comme la machine n'existe pas réellement mais qu'elle est simulée, elle est dite *virtuelle*. En d'autres mots ce petit programme donne l'illusion d'une machine dont le langage est le *code-octet*. L'interprète de code-octet est un programme en langage machine de la machine réelle.
- ◆ *Compilateur de code-octet.* Un second niveau de compilation traduit les instructions code-octet en instructions du langage machine de la machine réelle. Un compilateur JIT (*Just-In-Time*) effectue la traduction au moment d'exécuter le code-octet.
- ◆ *Processeur Java.* La troisième approche consisterait à construire un processeur dont le langage est le *code-octet* (*code-octet*) Java.

L'avantage d'utiliser un langage intermédiaire, le code-octet, dans le processus de traduction est l'indépendance du code objet vis-à-vis la machine réelle utilisée. Le même programme objet en code-octet Java peut être exécuté sur n'importe quelle machine pourvu qu'une machine virtuelle Java soit disponible sur celle-ci. L'indépendance de la plate-forme d'exécution est une considération importante à l'origine de Java qui vise à

¹² Le modèle en code-octet de Java n'est pas unique. La plateforme .NET de Microsoft et son langage C# sont similaires.

être utilisé dans des contextes variés. D'autres langages de programmation comme Kotlin peuvent utiliser le même code-octet que Java.

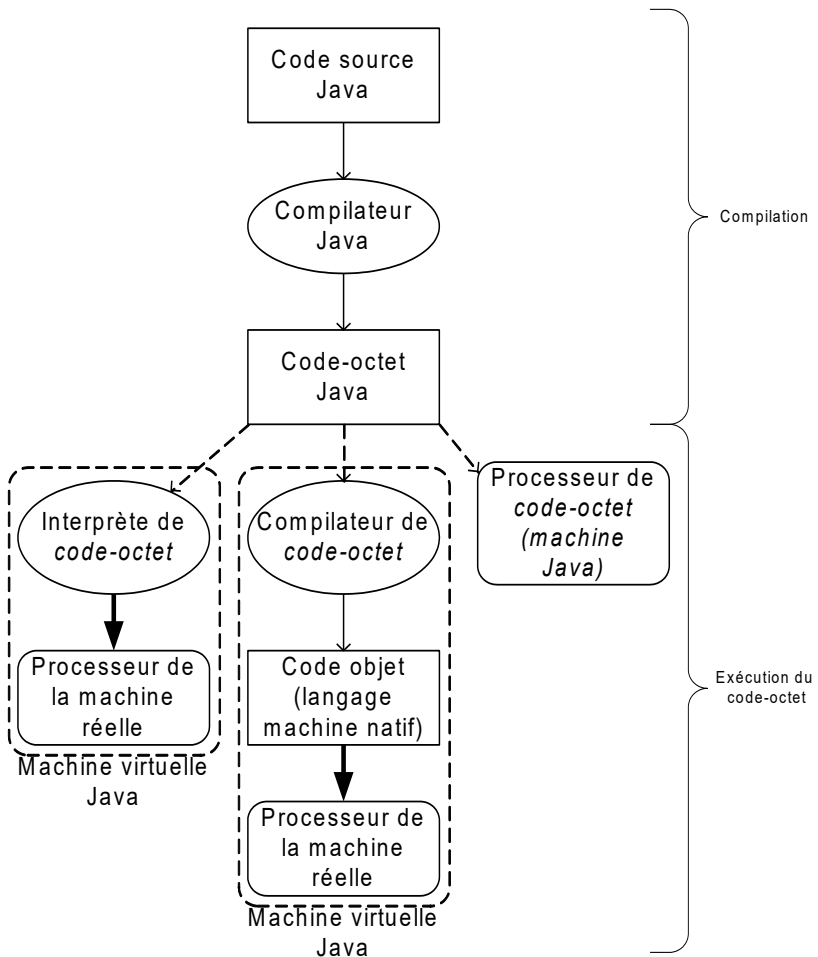


Figure 5. Compilation et exécution d'un programme Java.

Interprète

Un interprète est un programme qui exécute les instructions du langage une à la fois au fur et à mesure qu'elles sont lues. Cette approche permet de simplifier le processus d'exécution d'un programme en évitant l'étape intermédiaire de compilation. L'inconvénient est une exécution moins rapide

que dans l'approche avec compilation qui permet d'optimiser l'exécution du code d'une manière plus globale. Certains langages de programmation sont plus appropriés à l'interprétation que la compilation, tel que Python, Javascript, etc. Certains langages normalement interprétés (Python, JavaScript) bénéficient aussi d'un compilateur qui permet d'accélérer une partie du code. Dans la pratique, il y a donc souvent des modèles hybrides, utilisant à la fois des interprètes et des compilateurs.

1.2.2 Étapes de création et d'exécution d'un programme Java avec le JDK sous Windows

Pour illustrer les concepts précédents de manière concrète, cette section montre comment écrire, compiler et faire exécuter un petit programme Java très simple sous le système d'exploitation Windows. À cet effet, les outils de base offerts de Java (le Java Development Kit, ou JDK, de la plateforme Java SE) offerts gratuitement sur le site Adoptium sont installés.¹³ Le lien actuel est à l'adresse suivante :

<https://adoptium.net/>

Pour une démonstration d'installation, vous pouvez vous rendre sur le site YouTube suivant :

https://www.youtube.com/watch?v=Tk6u3Wm_s

Quand vous consultez une vidéo, prenez en compte que l'apparence du site, du système et d'autres éléments peuvent varier dans le temps. Une vidéo n'est pas un guide mais une illustration.

Il est aussi possible de passer par le site d'Oracle :

<https://www.oracle.com/java/technologies/downloads/>

Bien que le logiciel d'Oracle soit un excellent choix, plusieurs trouveront préférable de passer par Adoptium.

Voici un exemple de scénario d'installation de la version du JDK pour le système d'exploitation Windows 10 et macOS, en utilisant le logiciel

¹³ Attention à ne pas installer le *Java Runtime Environment* (JRE). Le JRE permet d'exécuter des programmes Java, mais il ne permet pas de compiler de nouveaux programmes.

d'Oracle. Si vous employez un autre système d'exploitation ou une autre version du JDK, les détails des manipulations peuvent varier. Par exemple, sous Linux on peut installer le JDK avec une commande telle que `apt-get install default-jdk` (ubuntu). Les instructions d'installation sont parfois un peu mystérieuses pour un débutant. Si c'est votre cas, il peut être nécessaire de recourir aux services d'un informaticien expérimenté.

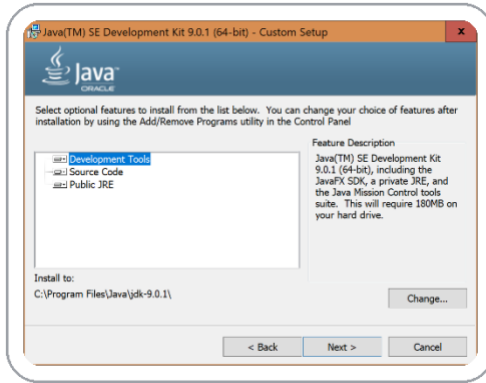
- **Exemple d'installation du JDK avec Windows**

1. Télécharger le programme d'installation Oracle
2. Exécuter le programme d'installation. Une fenêtre indique que le programme d'installation s'exécute.



3. Répondre aux questions du programme d'installation. Vous pouvez accepter les options définies par défaut en cliquant sur le bouton *Next* à chacun des dialogues proposés.

Le dialogue suivant permet de spécifier le dossier de base de l'installation du JDK. Dans cet exemple, la valeur de défaut a été acceptée. Il est habituellement préférable de ne pas la changer parce que les programmes qui utilisent le JDK peuvent la retrouver plus facilement.

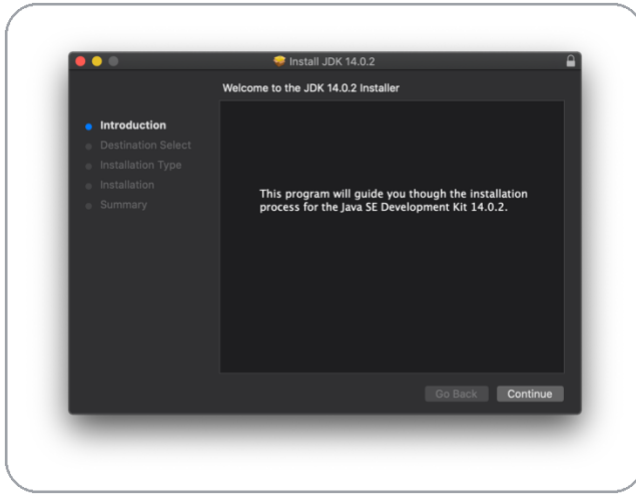


La fenêtre de dialogue suivante indique que le processus s'est déroulé correctement.



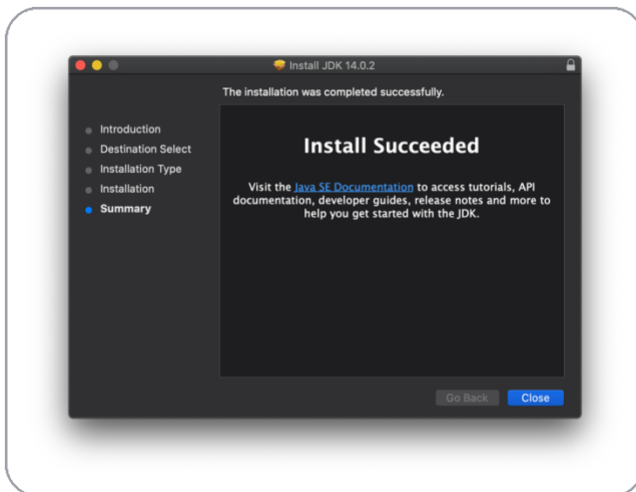
- **Exemple d'installation du JDK avec macOS**

1. Télécharger le programme d'installation
2. Exécuter le programme d'installation. Une fenêtre indique que le programme d'installation s'exécute.



3. Répondre aux questions du programme d'installation. Vous pouvez accepter les options définies par défaut en cliquant sur le bouton *Continue* à chacun des dialogues proposés.

La fenêtre de dialogue suivante indique que le processus s'est déroulé correctement.



Parmi les outils inclus dans le JDK, il y a un *compilateur Java* et un environnement d'exécution Java incluant une *machine virtuelle Java* (*Java Virtual Machine* – JVM) qui peut exécuter le code-octet produit par le compilateur Java. Cet environnement contient aussi d'autres éléments nécessaires à l'exécution des programmes Java dont des *bibliothèques de classes réutilisables* (cette notion sera expliquée par la suite). Cet ensemble d'outil est rudimentaire d'un point de vue convivialité et il existe des environnements de développement Java intégrés (*Integrated Development Environment* - IDE) plus élaborés tel que IntelliJ IDEA (www.jetbrains.com/idea), Eclipse (www.eclipse.org), NetBeans (netbeans.apache.org) ou Visual Studio Code (code.visualstudio.com). Ces environnements facilitent la tâche du programmeur en simplifiant l'utilisation des outils du JDK. Ces IDE facilitent le processus d'édition, de compilation, de déploiement et d'exécution des programmes Java.

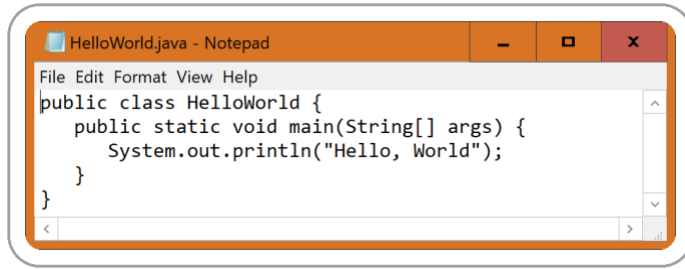
Le scénario qui suit est un exemple du processus de création et d'exécution d'un petit programme Java à l'aide des outils de base du JDK (le compilateur Java et la JVM Java) sous Windows. Il peut être utile d'invoquer ces outils directement au moins une fois afin de se familiariser avec le mécanisme. Une approche similaire est possible sous macOS et Linux.

- **Étape 1 : édition du texte du programme source Java avec Bloc-notes (Windows)**

Le texte d'un programme Java est produit avec un éditeur de texte. Voici le texte du programme de notre exemple :

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World");  
    }  
}
```

Ce genre de programme est une tradition typique dans l'apprentissage d'un nouveau langage de programmation. Il ne fait qu'afficher la phrase « Hello, World » sur l'unité périphérique de sortie standard qui est habituellement une fenêtre à l'écran de l'ordinateur. Dans la figure suivante, l'éditeur de texte *Bloc-notes* de Windows est utilisé pour éditer le texte.



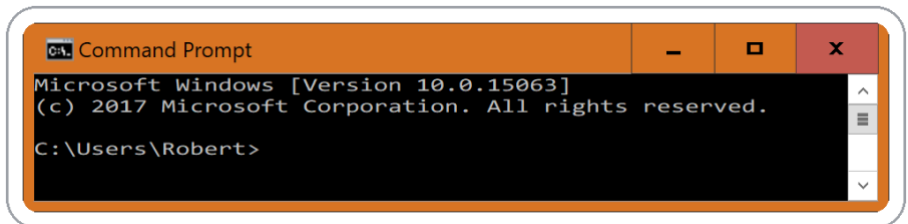
Le fichier est sauvegardé sous le nom *HelloWorld.java* dans le dossier dont le chemin est :

C:\Users\Robert\Documents

Par convention, un programme source Java a l'extension « *.java* ».

- **Invocation d'une fenêtre de commande Windows**

Pour compiler le programme source Java, il faut d'abord invoquer une fenêtre de commande Windows (*Command Prompt*).



Il y a différentes manières d'invoquer cette fenêtre, dont le menu des programmes Windows dans le coin inférieur gauche de l'écran.

- **Compilation et exécution du programme source Java en fenêtre de commande**

La commande « *CD Documents* » est donnée pour naviguer dans le répertoire du programme source Java :

Ensuite, le programme source est compilé en tapant le chemin qui mène au compilateur Java (programme *javac.exe*) suivi du nom du fichier¹⁴ :

```
"C:\Program Files\Java\jdk-9.0.1\bin\javac" HelloWorld.java
```

Dans cet exemple, le chemin est encadré par des guillemets parce qu'il contient un espace. L'extension « .java » est optionnelle.

La commande *javac* invoque le compilateur Java (programme nommé *javac.exe*) qui produit le code-octet dans un fichier nommé *HelloWorld.class*. Par défaut, le programme objet compilé en code-octet possède le même préfixe que le programme source auquel le compilateur ajoute l'extension « .class ».

Erreur de compilation, erreur de syntaxe

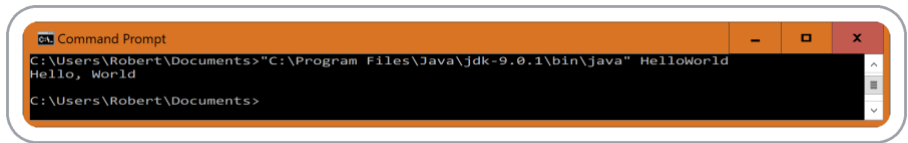
Si le programme source est incorrect selon les règles de syntaxe du langage Java, des messages d'erreur sont affichés afin de faciliter le repérage des erreurs. Ce sont des erreurs parfois appelées erreurs de compilation ou erreurs de syntaxe.

Attention !

¹⁴ Il est important ici de connaître le chemin exact de la commande *javac.exe*. En cas de doute, vous pouvez naviguer sur votre disque dans les dossiers pour vérifier l'emplacement.

Les messages d'erreur des compilateurs ne sont pas toujours faciles à déchiffrer ... S'il y a des erreurs, il faut les corriger dans le programme source à l'aide de l'éditeur de texte et tenter de compiler le programme à nouveau. En pratique, il y a habituellement plusieurs cycles d'édition/compilation avant qu'un programme ne soit correct d'un point de vue de la compilation.

Le programme est exécuté avec la machine virtuelle Java en tapant le chemin de la machine virtuelle (programme *java.exe*) suivi du nom du fichier du code octet (*HelloWorld.class*) mais en omettant l'extension « .class ».



Le texte « Hello, World » est affiché ! L'exécution du programme Java comme telle est précédée de deux étapes :

Chargement (load). Avant d'être exécuté le code-octet du programme doit d'abord être chargé en mémoire principale à partir du fichier d'extension *.class* en mémoire secondaire.

Vérification. Ensuite, le code-octet est vérifié afin de déterminer s'il respecte certaines contraintes, en particulier, concernant la sécurité. Java inclut des mécanismes sophistiqués de sécurité. Ces mécanismes permettent d'empêcher qu'un programme Java ne produise des effets indésirables.

C'est seulement après la vérification que le programme est effectivement exécuté.

Erreur d'exécution

Dans un scénario typique de développement d'un programme, il y souvent des erreurs d'exécution qui peuvent conduire à l'interruption du programme ou à la production d'un résultat incorrect. Il faut alors corriger les erreurs dans le code source et recommencer. On distingue deux types d'erreurs. Il y a des erreurs à la compilation (par ex., lors de la production du code-objet), et il y a des erreurs à l'exécution. Les erreurs ayant lieu lors de l'exécution du

programme sont parfois plus difficiles à régler et elles peuvent occasionner plus de désagréments.

- **Mettre à jour les variables d'environnement *Path* et *Classpath* de Windows**

Notez que si vous avez installé Java à partir du site d'Adoptium, alors vos variables d'environnement ont été modifiées lors de l'installation par défaut. Vous pouvez donc sauter cette étape.

Cette étape n'est pas nécessaire mais elle facilite l'utilisation des outils du JDK en évitant de devoir spécifier les chemins complets pour retrouver les outils et les programmes. La variable d'environnement *Path* identifie des chemins (de dossiers) que le système d'exploitation Windows parcourt afin de retrouver les programmes à exécuter. Dans la variable *Path*, le « ; » sépare les chemins les uns des autres. En ajoutant à la variable *Path* le chemin « C:\Program Files\Java\jdk-9.0.1\bin » qui contient les outils du JDK (le compilateur *javac.exe*, la machine virtuelle *java.exe*, etc.), il n'est pas nécessaire de spécifier ce chemin à chaque fois que l'on veut invoquer ces outils.

La variable d'environnement *Classpath* contient par défaut le chemin « . » qui représente le *dossier courant*. La notion de dossier courant sera développée par la suite. Le *Classpath* indique à la JVM (*Java Virtual Machine*) où trouver les programmes Java lorsque le chemin n'est pas spécifié. Si le dossier courant apparaît en premier, la JVM va toujours chercher dans le dossier courant en premier afin de retrouver les programmes Java. On peut ajouter d'autres chemins pour simplifier l'invocation des outils.

Une autre possibilité consiste à spécifier les options `-sourcepath` ou `-classpath` en invoquant les outils Java. Référez-vous à la documentation du JDK pour plus de détails. Ceci illustre la complexité de l'utilisation de ces outils. Comme nous le verrons par la suite, la situation devient encore plus compliquée lorsque les programmes sont composés de plusieurs fichiers et de packages. Un avantage important de l'utilisation d'un IDE est d'éviter d'avoir à spécifier tous ces paramètres.

- **Mettre à jour les variables d'environnement *Path* et *Classpath* sous macOS et Linux**

Sous macOS et Linux, on peut aussi modifier les variables d'environnement au besoin. Pour modifier la variable `PATH`, il suffit généralement d'éditer le fichier texte « `.profile` » (en le créant au besoin) dans le répertoire de l'utilisateur comme `/home/monnom` ou `/Users/monnom`. Il suffit d'y ajouter la ligne « `export PATH=/chemin/vers/java:$PATH` ». La modification de la variable `CLASSPATH` est similaire. L'installation de Java par l'entremise d'Adoptium vous dispense, par défaut, d'ajuster les variables d'environnement.

Attention au conflit entre plusieurs installations du JDK

S'il y a plusieurs installations du JDK sur votre système, les chemins des différentes installations peuvent entrer en conflit. En effet, les chemins sont parcourus en séquence par Windows lors de la recherche d'un programme. C'est le premier rencontré qui sera choisi. Il faut donc placer le chemin voulu en premier pour qu'il soit sélectionné. Ceci peut entraîner des conflits ennuyeux lorsque le chemin prioritaire doit varier selon le contexte. Si vous êtes débutant avec Windows, n'installez pas plusieurs JDK !

2. Introduction à la programmation Java

« L'enseignement de l'informatique ne peut faire de personne un programmeur expert plus que l'étude des pinceaux et du pigment peut faire de quelqu'un un peintre expert. » — Eric S. Raymond

Ce chapitre introduit les principes de base de la programmation avec le langage Java. Le programme suivant est utilisé pour introduire quelques concepts fondamentaux. Dans un premier temps, chacune des lignes du programme sera examinée l'une après l'autre.

Ce programme permet de saisir deux nombres entiers par le clavier de l'ordinateur dans des fenêtres de dialogue et d'en afficher la somme dans une autre fenêtre.

Exemple. [JavaPasAPas/chapitre_2/Exemple1.java](#)¹⁵

```

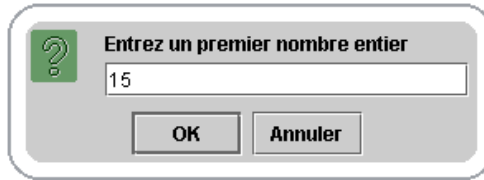
/**
 * Exemple1.java
 * Ce programme saisit deux entiers et en affiche la somme
 */
import javax.swing.JOptionPane; // Importe la
// classe javax.swing.JOptionPane
public class Exemple1{
    public static void main (String[] args) {
        // Déclaration de variables
        String chaine1, chaine2; // Les entiers lus
        // sous forme de String
        int entier1, entier2, somme; // Les entiers à
        // additionner et la somme
        // Saisir les deux chaînes de caractères qui
        // représentent des nombres entiers
        chaine1 = JOptionPane.showInputDialog(
            "Entrez un premier nombre entier");
        chaine2 = JOptionPane.showInputDialog(
            "Entrez un second nombre entier");
        // Convertir les deux chaînes de caractères en entiers
        entier1 = Integer.parseInt(chaine1);
        entier2 = Integer.parseInt(chaine2);
        // Calculer la somme des deux entiers
        somme = entier1 + entier2;
        // Afficher la somme avec JOptionPane.showMessageDialog
        JOptionPane.showMessageDialog(null,"La somme des deux entiers est "
            + somme);
        // Appel de System.exit(0) nécessaire à cause des appels à
        // JOptionPane.showInputDialog
        // et JOptionPane.showMessageDialog
        System.exit(0);
    }
}

```

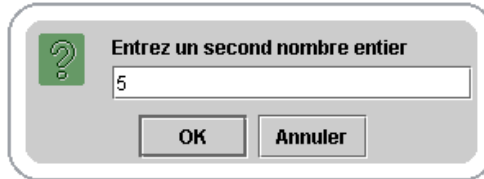
Exercice. Éditer, compiler et faire exécuter ce programme.

Le scénario suivant illustre le résultat de cette exécution. D'abord la fenêtre de dialogue suivante est affichée. Elle vous permet d'entrer un nombre entier dans la zone d'édition. Lorsque le nombre est entré, il faut cliquer sur le bouton OK pour poursuivre.

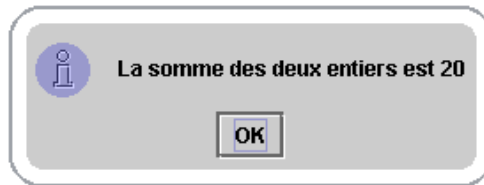
¹⁵ Ce titre est un lien vers le répertoire Github qui contient le code des exemples et exercices : <https://github.com/RobertGodin/JavaPasAPas>



Ensuite, la fenêtre de dialogue suivante est affichée et permet de saisir un second nombre entier.



Enfin, la fenêtre suivante affiche la somme des deux entiers lus. Il faut cliquer sur le bouton OK pour terminer le programme.



Examinons maintenant les détails du code du programme.

2.1 Commentaire Java

Le programme *Exemple1* débute par un commentaire :

```
/**  
 * Exemple1.java  
 * Ce programme saisit deux entiers et en affiche la somme  
 */
```

Toute portion du texte source qui débute par `/*` et se termine par `*/` est considérée comme un commentaire en Java et n'a aucun effet du point de vue de l'exécution du programme. En d'autres termes, on peut enlever tous les commentaires sans changer le fonctionnement du programme. L'objectif d'un commentaire est de faciliter la compréhension du programme par les humains (programmeurs). Un commentaire de ce type peut s'étendre sur une ou plusieurs lignes.

Souvent, par souci d'élégance, on débute un tel commentaire par deux astérisque (`/**`), suivi de lignes qui débutent par un espace suivi d'un astérisque, celui-ci étant aligné avec le second astérisque de la première ligne. On termine le commentaire avec une ligne distincte contenant un seul astérisque aligné sur les autres.

Une autre façon de spécifier un commentaire consiste à le débiter par `//` comme dans :

```
import javax.swing.JOptionPane; // Importe la classe javax.swing.JOptionPane
```

Le texte qui suit le `//` est considéré comme un commentaire. Un tel commentaire se termine automatiquement à la fin de la ligne courante et ne peut donc pas chevaucher plusieurs lignes.

Notation <fin de ligne>

La notation <fin de ligne> est employée pour représenter la fin de ligne dans le texte de cet ouvrage. La représentation exacte de la fin de ligne peut différer selon le codage employé pour les caractères.

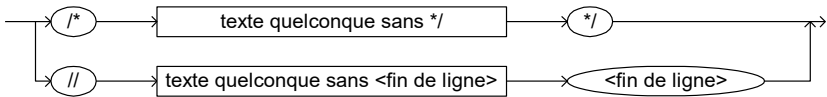
Le *diagramme syntaxique* suivant montre la forme générale d'un commentaire Java.

Diagramme syntaxique

Un diagramme syntaxique est un graphe qui permet de représenter les règles de syntaxe d'un langage de programmation. Le sens des flèches indique comment enchaîner les différents éléments. Un ovale contient des caractères spécifiques. Un rectangle fait référence à une autre règle de syntaxe.

Dans le diagramme suivant, le rectangle contenant le titre « texte quelconque sans `*/` » représente une séquence de caractères quelconque qui ne peut contenir la suite des deux caractères `*/`. La règle représentée par le rectangle devrait être détaillée dans un autre diagramme syntaxique. Comme le sens de la règle est facile à comprendre, la règle détaillée est omise ici.

commentaire :



2.2 Importation de classes

Il ne serait pas très pratique devoir écrire entièrement un programme logiciel en utilisant un seul fichier. Les langages de programmation permettent donc de répartir le code en plusieurs fichiers. Et, en particulier, le langage Java vient avec une librairie de code (appelée parfois la librairie standard) très riche que nous pouvons utiliser dans tous les programmes Java. Cette librairie est organisée en « classes », un concept sur lequel nous reviendrons sous peu. La ligne

```
import javax.swing.JOptionPane; // Importe la classe javax.swing.JOptionPane
```

indique que notre programme utilise par la suite un « bout de programme » qui est défini ailleurs. Le bout de programme est une *classe* au sens de Java. Le nom complet de la classe est *javax.swing.JOptionPane*. Le *import* a pour effet de simplifier l'écriture du programme car il est suffisant d'utiliser le nom de classe *JOptionPane* par la suite pour identifier la classe plutôt que d'écrire son nom complet. Techniquement, donc, cette ligne débutant par « import » n'est pas nécessaire : elle équivaut à dire que partout où nous rencontrons *JOptionPane*, on fait référence à *javax.swing.JOptionPane*.

2.3 Packages

Le préfixe « *javax.swing* » du nom complet de classe *javax.swing.JOptionPane* représente un nom de *package*. Un *package* au sens de Java est tout simplement un regroupement de classes.¹⁶ Ce regroupement de classes en *packages* permet d'organiser les classes afin de les retrouver plus facilement. Le rôle des *packages* est analogue aux dossiers (*répertoires*) des systèmes de gestion de fichiers. Un *package* peut contenir des classes ainsi que des *packages*. Dans notre exemple, le *package swing* fait partie du *package javax*. Le nom complet d'une classe doit ainsi contenir la liste des *packages* séparés par des points (.). L'ordre des *packages* doit respecter l'ordre de la hiérarchie des contenus.

¹⁶ En français, nous pourrions utiliser le terme *paquetage*.

La figure suivante montre les trois *packages* principaux de Java avec la notation UML. UML ([*Unified Modeling Language*](#)) est une notation graphique normalisée qui permet de représenter divers aspects des logiciels.¹⁷



Figure 6. Principaux packages de Java.

Le *package javax* contient le *package swing* ainsi que d'autres *packages* illustrés à la figure suivante.

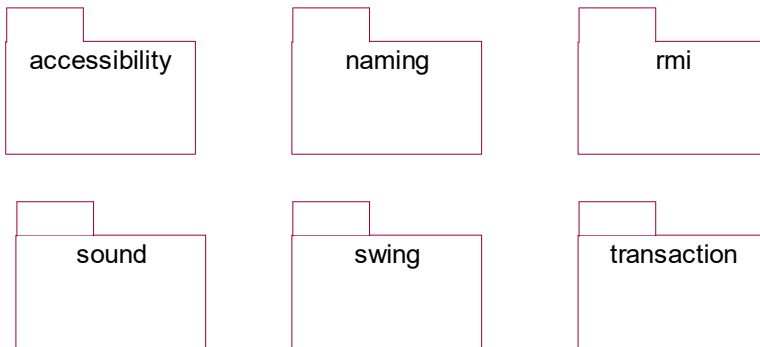


Figure 7. Sous-packages de *javax*.

Enfin, le *package swing* contient la classe *JOptionPane* ainsi que d'autres packages et classes visant le développement d'interfaces à l'utilisateur graphiques. Une classe est représentée par un rectangle en UML. Le rectangle contient le nom de la classe.

¹⁷ Bien que nous utilisons la notation UML dans ce manuel, il n'est pas nécessaire de la maîtriser. Elle peut être comprise de manière intuitive.

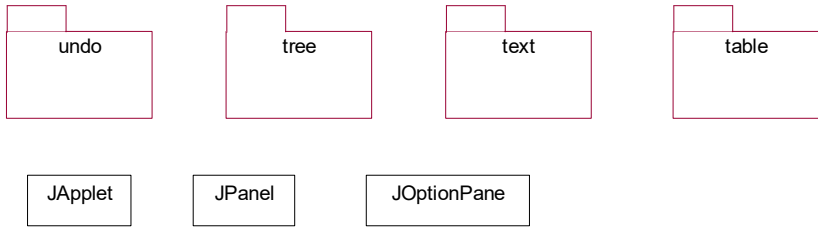


Figure 8. Partie du contenu du package *swing*.

Ainsi, le nom complet de la classe *JOptionPane* est *javax.swing.JOptionPane* parce que la classe *JOptionPane* fait partie du package *swing* qui lui-même fait partie du package *javax*.

2.4 Notion de classe et de méthode

Qu'est-ce qu'une classe ? Pour l'instant, il serait périlleux de tenter de décrire tous les détails de ce concept. En première approximation, un programme Java est composé d'un ensemble de classes.¹⁸ Une classe est donc en quelque sorte un morceau d'un programme Java. Une classe regroupe des *méthodes* ; une *méthode* représente une *action (instruction, opération)* qui porte un nom et que l'on peut appeler (invoquer) pour effectuer une tâche particulière. Une méthode peut elle-même appeler d'autres méthodes.

Notre programme appelle la méthode *showInputDialog()* de la classe *javax.swing.JOptionPane* qui est une classe prédéfinie en Java. Le JDK contient un grand nombre de classes prédéfinies que tout programme Java peut utiliser. Ces classes prédéfinies fournissent toutes sortes de méthodes que l'on peut appeler pour effectuer diverses tâches sans avoir à les programmer à chacune des utilisations.

Il ne faut pas oublier d'indiquer par des directives *import* les classes prédéfinies nécessaires à notre programme à moins de préciser le nom complet à chacune des utilisations. Sinon, une erreur sera soulevée à la compilation du programme.

¹⁸ Tous les langages de programmation n'ont pas cette caractéristique. Certains langages n'ont pas de classes du tout (comme le C) alors que d'autres permettent d'utiliser des classes, mais sans obligation que tout le code soit au sein de classes (comme le C++).

2.5 Le nom d'une classe

Notre petit exemple de programme est formé d'une seule classe. Le nom de classe (ici *Exemple1*) est défini à la ligne suivante :

```
public class Exemple1{
```

Le contenu de la classe, incluant ses méthodes, vient après son nom et il est placé entre une accolade ouvrante et une accolade fermante. Chacune des classes possède un nom permettant aux autres classes d'y faire référence. Le mot réservé *public* signifie que la classe est publique au sens où elle peut être utilisée par les autres classes sans restriction. Le sens de cette notion sera précisé ultérieurement.

2.6 La méthode *main()*

Au sein de la classe *Exemple1*, la ligne

```
public static void main (String[] args) {
```

début la définition de la méthode nommée *main*. Après le nom *main*(), une liste de paramètres formels est spécifiée entre parenthèses :

```
(String[] args)
```

La méthode a donc seul paramètre, de type *String[]*, soit un tableau de chaînes de caractères. Nous reviendrons sur les paramètres par la suite. Dans un premier temps, ces paramètres ne seront pas utilisés et vous pouvez les ignorer.

Une classe comporte en général un ensemble de méthodes. Chacune des méthodes porte un nom permettant de l'identifier. Java doit être à même de toujours distinguer les méthodes. Les méthodes sont donc uniques en ce qui a trait à leur « signature » : si une classe a deux méthodes du même nom, ces méthodes se distinguent en ce qui a trait aux paramètres. La méthode *main(String[])* est une méthode spéciale qui est exécutée au démarrage du programme. Lors de l'exécution de notre exemple *Exemple1* par la commande

```
java Exemple1
```

que l'on entre à l'invite de commande, le traitement démarre toujours par l'exécution de la méthode *main(String[])* de la classe *Exemple1*.

Les mots réservés *public static void* qui précèdent le nom de la méthode décrivent certaines caractéristiques de la méthode. Pour le moment, vous pouvez ignorer le sens de ces caractéristiques. Pour les curieux, voici une description sommaire :

- *public* signifie que la méthode peut être appelée de partout et en particulier en dehors de la classe elle-même ;
- *static* signifie que c'est une méthode de classe (par opposition à une méthode d'objet) ;
- *void* signifie que la méthode ne retourne rien.

2.7 Corps d'une méthode

Après les paramètres, vient le corps de la méthode entre accolades. Le corps d'une méthode spécifie ce que la méthode fait. Dans le corps, on retrouve entre autres des énoncés de déclarations de variables et des énoncés d'actions (aussi appelés *instructions* ou *opérations*) qui précisent le traitement à effectuer.

2.7.1 Déclaration de variables

La ligne suivante dans le corps de la méthode *main()*

```
String chaine1, chaine2; // Les entiers sous  
// forme de chaînes
```

est une déclaration de deux variables nommées *chaine1* et *chaine2*. Ces deux variables servent à mémoriser les deux séquences de chiffres qui représentent les entiers à additionner.

Variable

Une variable est un contenant pour une *valeur*. Une variable est donc en quelque sorte une petite mémoire qui permet de stocker une donnée pour utilisation ultérieure. À un moment donné de l'exécution d'un programme, une variable contient une et une seule valeur. Cependant, une variable peut changer de valeur à la suite de l'exécution d'instructions du programme. Votre ordinateur et Java se chargent de stocker la valeur en question : elle

peut être stockée au sein d'un ou plusieurs registres, au sein de la mémoire tampon et au sein de mémoire centrale de l'ordinateur.¹⁹

Une variable est identifiée par un nom. Supposons dans un premier temps que le nom est unique dans le corps d'une méthode. Le nom lui-même est sans signification d'un point de vue du langage de programmation mais devrait être choisi avec soin afin d'indiquer aux programmeurs le rôle de la variable dans le contexte du programme.

Une variable ne peut contenir n'importe quoi. Une déclaration de variables débute par un *type* qui détermine l'ensemble des valeurs possibles de la variable. Dans notre exemple, **String** est le type de la variable *chaine1* ainsi que de *chaine2*. Ceci signifie que *chaine1* ne peut contenir qu'une chaîne de caractères. Il en est de même pour *chaine2*. Une chaîne de caractère est une séquence de caractères où chacun des caractères provient d'un code préétabli (tel que *Unicode*). Dans le cas du Java, le code utilisé par l'interface (*String*) est UTF-16 : dans cet encodage, chaque caractère occupe deux octets au minimum, alors que certains caractères comme les émojis nécessitent quatre octets.

La ligne suivante déclare deux variables qui serviront à mémoriser les entiers eux-mêmes, car *int* représente le type entier en Java.

```
int entier1, entier2, somme; // Les entiers à additionner
```

Après ces déclarations, on peut imaginer que cinq zones de mémoire ont été réservés pour les cinq variables tel qu'illustré à la figure suivante :

¹⁹ Dans la pratique, la valeur d'une variable peut être représentée de différentes manières et il arrive fréquemment qu'il y ait plusieurs copies de la valeur dans votre ordinateur. Par exemple, une variable contenant un entier pourra être chargée à partir de la mémoire centrale dans un registre d'un cœur de votre processeur, et en ce faisant être propagé dans la mémoire tampon. Un autre cœur de votre processeur pourra charger la valeur de cette variable au même moment dans un autre registre. La plupart du temps, vous n'avez pas à vous préoccuper de ces détails.

Mémoire centrale de l'ordinateur

chaîne1	
chaîne2	
entier1	
entier2	
somme	

Au départ, le contenu des variables est vide. En Java, cela signifie que si on tente d'accéder à la valeur de la variable, nous obtiendrons une erreur lors de la compilation du programme.

2.7.2 Types prédéfinis de Java

Java inclut un ensemble de types prédéfinis. Le tableau suivant énumère les types dits primitifs avec les valeurs possibles.

Type primitif	Valeurs
boolean	true/false
char	Caractère selon l'encodage UTF-16 (mot de 16 bits)
byte	Octet en binaire (8 bits) entre -128 (-2^7) et 127 (2^7-1)
short	Entier (précision de 16 bits) entre -32 768 (-2^{15}) et 32 767 ($2^{15}-1$)
int	Entier (précision de 32 bits) entre -2 147 483 648 (-2^{31}) et 2 147 483 647 ($2^{31}-1$)
long	Entier (précision de 64 bits) entre -9 223 372 036 854 775 808 (-2^{63}) et 9 223 372 036 854 775 807 ($2^{63}-1$)
float	Nombre à virgule flottante (précision de 32 bits selon le code IEEE 754-1985) entre -3.4×10^{38} et 3.4×10^{38} (7 chiffres significatifs)
double	Nombre à virgule flottante (précision de 64 bits IEEE 754-1985) entre -1.7×10^{308} et 1.7×10^{308} (15 chiffres significatifs)

Figure 9. Types primitifs de Java.

En informatique, on définit l'ensemble des nombres positifs comme étant les nombres plus grands que zéro. Les nombres négatifs sont les nombres plus petits que zéro. Les nombres entiers (par ex., short, int) comportent une seule valeur nulle (0) alors que les nombres à virgule flottante (float, double) comportent deux valeurs nulles : le zéro négatif (-0) et le zéro positif (+0). Quand on écrit un nombre avec un point décimal en Java (par ex., 3.1416), celui-ci est interprété comme un nombre à virgule flottante ayant une précision de 64 bits. Les nombres à virgule flottante comportent aussi les valeurs infinies (Double.POSITIVE_INFINITY et Double.NEGATIVE_INFINITY). La division d'un nombre non nul par une valeur nulle en Java donne une valeur infinie. La division d'une valeur nulle par une autre valeur nulle donne une valeur spéciale (NaN pour Not a Number) qui a la propriété unique de ne pas être égale à elle-même : l'expression $0.0/0.0 == 0.0/0.0$ est fausse en Java. En général, il est possible de représenter l'ensemble des nombres réels entre -1.7×10^{308} et 1.7×10^{308} avec 15 chiffres de précision, mais pas 16 chiffres de précision. Par exemple, les

nombres comportant 16 chiffres significatifs 0.8825149536132812 et 0.8825149536132813 sont représentés en nombre à virgule flottante comme étant 115673 fois 2 à la puissance -17 ce qui est le nombre 0.88251495361328125. Nous avons donc que l'expression 0.8825149536132812 == 0.8825149536132813 est vraie en Java.

Le type **String** est aussi un type prédéfini mais n'est pas un type primitif. En fait, **String**, dont le nom complet est *java.lang.String*, est une classe Java qui fait partie du package *java.lang*. Ainsi un type peut être soit un type primitif ou une classe. La différence sera expliquée ultérieurement.

2.7.3 Appel de méthode de classe, paramètres et énoncé d'affectation

La ligne

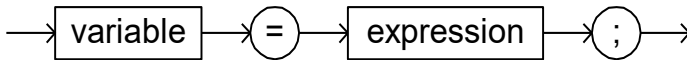
```
chaine1 = JOptionPane.showInputDialog("Entrez un premier nombre entier");
```

représente un *appel* (ou invocation) de la *méthode de classe* *javax.swing.JOptionPane.showInputDialog()* et l'affectation du résultat de l'appel de la méthode à la variable *chaine1*.

Le terme *affectation* signifie : prendre la valeur de ce qui est produit dans l'expression à droite, et donner cette valeur à la variable qui est à gauche. Le « *javax.swing.* » est omis étant donné la clause *import*.

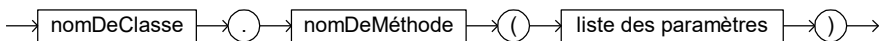
Cette ligne est un exemple d'un *énoncé d'affectation* simple dont la forme générale est :

énoncé d'affectation (cas simple):

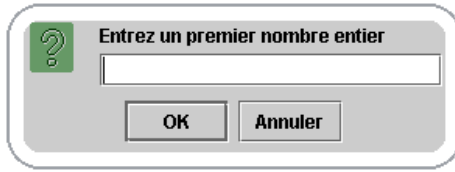


L'expression dans notre exemple est un appel d'une méthode de classe dont la forme générale est :

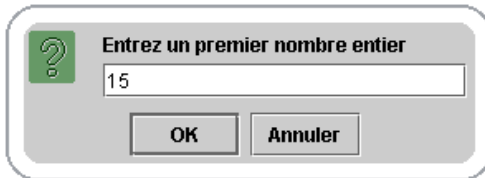
appel de méthode de classe :



La méthode provoque d'abord l'affichage de la fenêtre de dialogue suivante :



Cette fenêtre de dialogue permet à l'utilisateur du programme d'entrer une séquence de caractères qui vise à représenter le premier nombre entier à additionner. Par exemple, dans la figure suivante l'utilisateur a entré la séquence des deux chiffres "15".



Lorsque l'utilisateur clique OK, la séquence de caractères saisie est retournée par la méthode et stockée dans la variable *chaine1* par l'énoncé d'affectation. Un appel de méthode doit retourner quelque chose pour qu'il puisse apparaître dans la partie droite d'une affectation. Ce ne sont pas toutes les méthodes qui retournent quelque chose.

Mémoire centrale de l'ordinateur

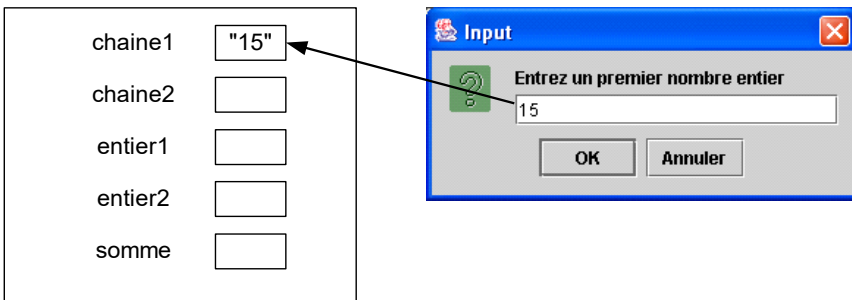


Figure 10. Effet de `chaine1 = JOptionPane.showInputDialog("Entrez un premier nombre entier");`

Un aspect important à saisir dans cette opération est le fait que la séquence de caractères lue n'est pas interprétée comme un nombre entier à ce point-ci mais comme une chaîne de caractères (type **String**).

Dans l'appel d'une méthode, il faut préciser les valeurs des *paramètres* entre parenthèses après le nom de la méthode. Un paramètre représente une valeur qui est utilisée par la méthode. Un paramètre de méthode est analogue à un paramètre de fonction en mathématiques. Par opposition aux paramètres en mathématiques, un paramètre de méthode n'est pas limité à des valeurs numériques. Le texte

```
"Entrez un premier nombre entier"
```

est la valeur du paramètre de la méthode `JOptionPane.showInputDialog()` dans notre exemple. Il représente un titre qui est affiché dans la fenêtre de dialogue. En Java, une séquence de caractères entre guillemets (") est interprétée comme une chaîne de caractères, c'est-à-dire une valeur de type **String**. Le type de la valeur passée en paramètre doit toujours être conforme au type attendu par la méthode. Dans le cas de la méthode `showInputDialog()`, la méthode attend un paramètre de type **String**.

Notation pour les méthodes

Pour désigner une méthode dans le texte, on utilise souvent la notation *nomMéthode(listeParamètres)* ou *nomMéthode()*. Les paramètres ne sont pas toujours mentionnés. Pour `showInputDialog`, la notation `showInputDialog()` a déjà été employée. Pour préciser la nature des paramètres, on utiliserait la notation

showInputDialog(String titre)

On peut ainsi observer que la méthode a besoin d'un paramètre appelé *titre* de type **String**. Le nom du paramètre (ici *titre*) ne fait pas partie de la signature de la méthode en Java.

La méthode produit en résultat un autre **String** ("15" dans notre exemple) qui est la séquence de caractères saisie. Pour récupérer ce résultat de l'appel de la méthode, la valeur retournée est affectée à la variable *chaîne1*. Le symbole = représente une *opération d'affectation* qui signifie de prendre le résultat de ce qui est à droite du = et de le placer dans la variable qui est à gauche du =. Pour que cela soit acceptable, il faut que le type de ce qui vient de la partie droite soit compatible avec le type de la variable de la partie gauche. Dans notre cas, il faut que la méthode retourne un **String**.

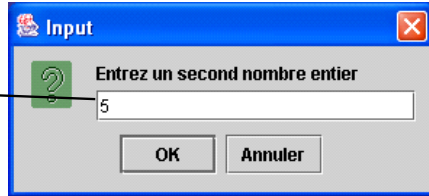
La ligne suivante permet de saisir la séquence des chiffres du deuxième entier dans la variable *chaine2* :

```
chaine2 = JOptionPane.showInputDialog("Entrez un second nombre entier");
```

Dans la figure suivante, l'utilisateur a entré "5" :

Mémoire centrale de l'ordinateur

chaine1	"15"
chaine2	"5"
entier1	
entier2	
somme	



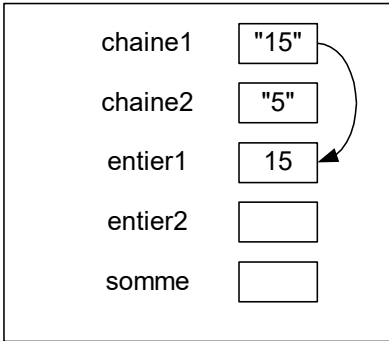
Avant de pouvoir les additionner, il faut convertir les chaînes de caractères en nombres entiers. La ligne suivante convertit la chaîne de caractères *chaine1* en un nombre entier qui est affecté à la variable *entier1* de type *int*.

```
entier1 = Integer.parseInt(chaine1);
```

En effet, la chaîne de caractères contient une série de valeurs à 16 bits, et dans ce cas deux valeurs correspondant à 1 et 5. Il faut convertir ces valeurs en un entier (type *int*) occupant 32 bits. En Java, on représente la valeur correspondante (soit 15) avec le mot à 32 bits : 00000000000000000000000000001111.

Voici l'effet dans notre exemple.

Mémoire centrale de l'ordinateur



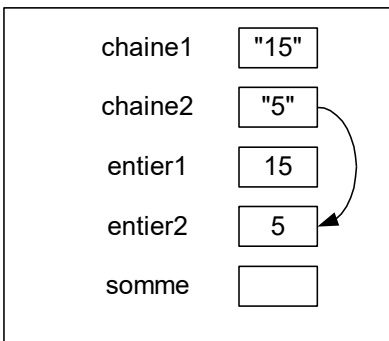
La méthode `Integer.parseInt()` accepte un paramètre **String** et retourne la conversion du **String** en un entier de type `int`²⁰. Vous vous demandez peut-être pour quelle raison il n'y a pas de clause *import* pour la classe `Integer` contrairement à `JOptionPane` ? La raison est que `Integer` fait partie du package `java.lang` (nom complet `java.lang.Integer`) dont toutes les classes sont importées automatiquement dans tous les programmes Java.

La ligne suivante convertit le deuxième entier :

```
entier2 = Integer.parseInt(chaine2);
```

Dans notre exemple, cela produit l'effet suivant en mémoire :

Mémoire centrale de l'ordinateur



²⁰ Si la chaîne contient autre chose que des chiffres, une erreur sera provoquée.

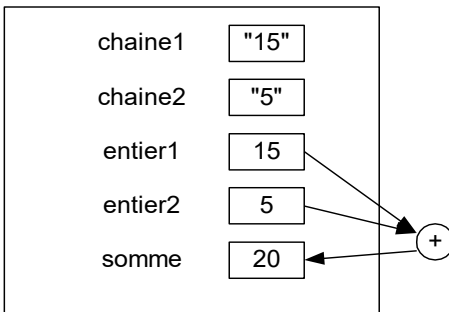
2.7.4 Expression

La ligne suivante calcule la somme des deux entiers et affecte le résultat à la variable *somme* :

```
somme = entier1 + entier2;
```

Cet exemple permet d'illustrer que la partie droite d'une instruction d'affectation peut être une expression analogue à une expression mathématique. Dans le cas de types numériques, l'expression peut inclure des opérations arithmétiques typiques telles que l'addition (+) et la soustraction (-), la multiplication (*), la division réelle (/), des parenthèses, etc.²¹ L'expression peut aussi inclure des appels de méthodes. Java inclut un grand nombre de méthodes pour les fonctions numériques (dans la classe *java.lang.Math*).

Mémoire centrale de l'ordinateur

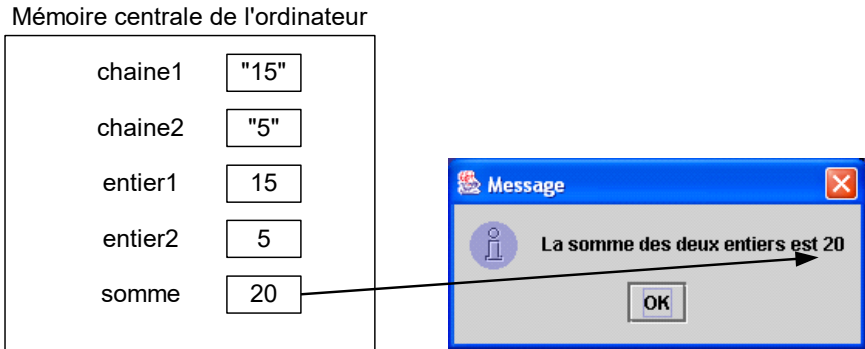


La ligne suivante affiche la somme dans une fenêtre de dialogue :

```
JOptionPane.showMessageDialog(null,"La somme des deux entiers est "
+ somme);
```

Dans notre exemple, on obtient :

²¹ Il faut que le type du résultat de l'évaluation de l'expression soit compatible avec le type de la variable de la partie gauche de l'opération d'affectation.



La méthode nécessite deux paramètres qui sont séparés par une virgule. Selon la convention Java, tous les paramètres passés à une méthode doivent être séparés par des virgules. Le premier paramètre (valeur spéciale *null*) n'est pas utile pour notre exemple et nous n'expliquerons pas son rôle pour le moment. Le deuxième paramètre est un **String** qui est affiché dans la fenêtre de dialogue.

2.7.5 Expression de type **String**

Dans notre exemple, la valeur du paramètre est en réalité produite par une expression :

```
"La somme des deux entiers est " + somme
```

Cette expression devrait paraître curieuse pour un non-initié. En effet, l'expression additionne un **String** ("La somme des deux entiers est ") à une valeur d'une variable de type *int* (20 dans notre exemple) ? En réalité, le « + » dans cette expression représente une opération de concaténation de deux chaînes de caractères. Le compilateur Java fait cette interprétation car le premier opérande est un **String**. Ainsi, le symbole + possède un sens différent en fonction du contexte. On dit que le symbole + est *surchargé*, car il a plus d'un sens.

La concaténation a pour effet de placer deux chaînes bout à bout. Mais, la variable *somme* n'est pas un **String** mais un *int* ? Le compilateur Java effectue automatiquement une conversion de l'entier en un **String**. Dans notre exemple, le *int* 20 dans *somme* est converti en **String** "20". La concaténation des deux **String**

```
"La somme des deux entiers est " + "20"
```

produit le **String**

"La somme des deux entiers est 20".

C'est ce **String** qui est ensuite passé en paramètre à la méthode `JOptionPane.showMessageDialog()` pour être affiché dans la fenêtre de dialogue.

Cette conversion automatique, de type entier au type **String**, n'est pas unique au Java. Il permet d'écrire rapidement du code sans trop se soucier du type des variables. Par contre, on peut toujours écrire du code plus explicite :

```
"La somme des deux entiers est " + Integer.toString(somme)
```

Enfin, la ligne suivante doit être appelée pour terminer le programme correctement. Cette ligne n'est pas toujours nécessaire. Elle l'est lorsque le programme utilise des éléments graphiques tels que les fenêtres de dialogue. C'est le cas de notre programme qui fait appel aux méthodes `JOptionPane.showInputDialog()` et `JOptionPane.showMessageDialog()`.

```
System.exit(0);
```

2.8 Diagramme de séquence UML

La Figure 11 montre un *diagramme de séquence* UML qui illustre la séquence d'exécution des méthodes dans le contexte de notre petit scénario. Chacun des rectangles de la partie supérieure correspond à une classe. Sous chacune des classes une ligne verticale, représente l'évolution dans le temps de la classe. D'abord, la méthode `main()` de la classe *Exemple1* est appelée automatiquement par l'intermédiaire de la commande `java Exemple1`. Ceci est illustré par une flèche étiquetée par le nom de la méthode appelée. La flèche part de la ligne sous `java.exe` et aboutit à la classe *Exemple1* de notre programme. Un rectangle mince vertical sous une classe signifie que la méthode est en exécution. La méthode `main()` appelle en premier la méthode `showInputDialog("Entrez un premier nombre entier")` de *JOptionPane* qui s'exécute. Après son exécution, elle retourne le **String** "15" qui correspond au premier entier. La fin de l'exécution d'une méthode est illustrée par une flèche pointillée étiquetée de la valeur retournée par la méthode. Ensuite, la méthode `main()` poursuit son travail en appelant de nouveau la méthode

`showInputDialog("Entrez un second nombre entier")` qui retourne le **String** "5" au second appel. La méthode `main()` appelle alors `parseInt("15")` de la classe `Integer` qui retourne le *int* 15. Ensuite, `main()` appelle `parseInt("5")` de la classe `Integer` qui retourne le *int* 5. La méthode `main()` calcule alors la *somme* et appelle `showMessageDialog("La somme des deux entiers est 20")` pour l'afficher. Enfin, `main()` appelle `exit(0)` de la classe `System`.

Ainsi un programme Java effectue un traitement par le résultat de l'interaction entre plusieurs classes par l'intermédiaire d'appels de méthodes d'une classe à l'autre. Dans notre premier exemple, une seule classe a été définie et cette classe a été utilisée trois fois. Sauf dans les cas les plus simples, le codage d'un programme Java nécessite habituellement la définition de plusieurs classes.

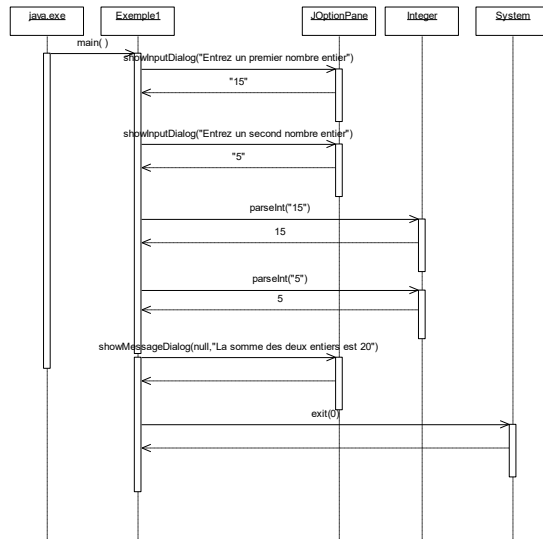


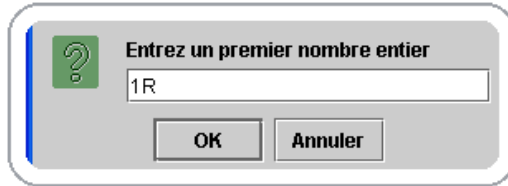
Figure 11. Diagramme de séquence de l'exécution de *Exemple1*.

2.9 Exceptions

Lors de l'exécution d'un programme, divers problèmes peuvent se produire. Par exemple, si l'utilisateur entre autre chose qu'une chaîne de caractère qui représente un nombre entier dans le dialogue de saisie, la méthode `Integer.parseInt()` ne pourra pas convertir la chaîne en un entier ! Ceci provoquera une interruption anormale du programme. Ce genre

d'interruption est appelé une *exception* en Java. Lorsqu'une exception est levée, le programme est interrompu et un message d'erreur est affiché. Ce message permet de retracer l'origine de l'exception.

Par exemple, supposons que l'utilisateur entre la chaîne "1R" plutôt que "15" :



Lorsque l'appel de la méthode `Integer.parseInt("1R")` est effectué, une exception est levée et le message d'erreur suivant est retourné :

```
java.lang.NumberFormatException: For input string: "1R"
    at java.lang.Integer.parseInt(Integer.java:580)
    at java.lang.Integer.parseInt(Integer.java:615)
    at Exemple1.main(Exemple1.java:13)
```

Ce message indique la classe de l'exception (`java.lang.NumberFormatException`) et la valeur de paramètre qui a causé le problème (1R). Ensuite, la séquence des appels de méthode qui a conduit au problème est donnée en ordre inverse des appels. Ici, on peut voir que la méthode `main` de la classe `Exemple1` a appelé la méthode `parseInt` de la classe `java.lang.Integer`. La méthode `java.lang.Integer.parseInt(java.lang.String)` appelle une autre méthode `java.lang.Integer.parseInt(java.lang.String, int)` du même nom mais avec deux paramètres ? D'une part, ceci montre que les méthodes que notre programme appelle peuvent elles-mêmes appeler d'autres méthodes. D'autre part, nous verrons plus loin qu'il est possible de définir plusieurs méthodes différentes du même nom sans qu'il n'y ait d'ambiguïté lorsque les paramètres sont différents. Le compilateur peut lever l'ambiguïté par la forme des paramètres passés à l'appel.

2.10 Syntaxe des identificateurs Java

Les noms de variables, méthodes et classes doivent respecter des règles précises qui s'appliquent à tous les identificateurs Java. Un identificateur Java doit débuter par une lettre suivie d'une suite d'un nombre quelconque de caractères limités aux lettres, chiffres, \$ et _. Les lettres peuvent être

majuscules ou minuscules et la casse est importante. Par exemple, « somme » et « Somme » sont deux identificateurs différents en Java. Certains identificateurs sont dits réservés et ne peuvent servir de nom de variable, méthode ou classe. Ces identificateurs réservés ont un sens prédéfini en Java. Par exemple, les identificateurs *class*, *import* et *int* sont réservés.

2.11 Disposition du texte

Les règles de Java concernant la disposition du texte sont assez flexibles au sens où les éléments du langage peuvent être séparés par une suite quelconque d'espaces blancs. Le terme *espace blanc* désigne un *espace*, une marque de *tabulation* ou une *fin de ligne*. Un compilateur Java ne prend donc pas en compte les espaces blancs lorsqu'il traite votre code informatique. Cependant, il est utile d'utiliser une manière systématique de disposer les différentes parties d'un programme source afin d'en faciliter la lecture. N'oubliez pas qu'un code qui se lit bien sera plus facile à corriger et à modifier. Certains éditeurs mettent automatiquement en forme votre code informatique pour en assurer la lisibilité.

Certains langages de programmation, comme les langages Python et le Swift, prennent en compte les espaces et les tabulations. D'autres langages, comme le langage Go, disposent d'une norme fixe pour l'utilisation des espaces et des tabulations.

Lorsque vous travaillez en équipe, ou lorsque vous entendez contribuez au code de quelqu'un d'autre, il est mal vu de reformatter leur code. Vous devez alors modifier le moins possible leur code.

2.12 Initialisation de variable à la déclaration

Dans *Exemple1*, les variables sont déclarées au début du corps de la méthode *main()*. Ceci n'est pas obligatoire. En fait, il est possible de retarder la déclaration à sa première utilisation comme illustré dans *Exemple2*.

Exemple. `JavaPasAPas/chapitre_2/Exemple2.java`

```

/**
 * Exemple2.java
 * Ce programme saisit deux entiers et en affiche la somme
 */
import javax.swing.JOptionPane; // Importe la classe
javax.swing.JOptionPane
public class Exemple2{

    public static void main (String[] args) {

        // Saisir les deux chaînes de caractères qui représentent des
        // nombres entiers
        String chaine1 = JOptionPane.showInputDialog(
            "Entrez un premier nombre entier");
        String chaine2 = JOptionPane.showInputDialog(
            "Entrez un second nombre entier");

        // Convertir les chaînes en entiers
        int entier1 = Integer.parseInt(chaine1);
        int entier2 = Integer.parseInt(chaine2);

        // Calculer la somme des deux entiers
        int somme = entier1 + entier2;

        // Afficher la somme avec JOptionPane.showMessageDialog
        JOptionPane.showMessageDialog(null,"La somme des deux entiers est "
            + somme);

        // Appel de System.exit(0) nécessaire à cause des appels à
        // JOptionPane.showInputDialog et JOptionPane.showMessageDialog
        System.exit(0);
    }
}

```

La ligne suivante déclare le type de *chaine1* et l'initialise en même temps par l'appel à *showInputDialog()*.

```

String chaine1 = JOptionPane.showInputDialog(
    "Entrez un premier nombre entier");

```

Il est permis de déclarer une variable n'importe où dans le corps d'une méthode et il est permis de l'initialiser au moment de sa déclaration.

Erreur de programmation

L'utilisation d'une variable non déclarée provoque une erreur de compilation.

Déclarer une variable déjà déclarée provoque une erreur de compilation.

L'utilisation de la valeur d'une variable non initialisée provoque une erreur à l'exécution.

Exercice. Modifiez le programme *Exemple1* afin qu'il lise trois entiers (plutôt que deux) et en affiche la somme.

Solution. La solution suivante est une adaptation directe de *Exemple1.java*. La nouvelle variable *chaine3* de type **String** est introduite pour y lire la troisième chaîne de caractère ainsi qu'une variable *entier3* pour l'entier correspondant.

JavaPasAPas/chapitre_2/Exercice1.java

```

/**
 * Exercice1.java
 * Lire 3 entiers et en afficher la somme
 */
import javax.swing.JOptionPane;
public class Exercice1{

    public static void main (String[] args) {

        String chaine1 = JOptionPane.showInputDialog(
            "Entrez un premier nombre entier");
        String chaine2 = JOptionPane.showInputDialog(
            "Entrez un second nombre entier");
        String chaine3 = JOptionPane.showInputDialog(
            "Entrez un troisième nombre entier");

        int entier1 = Integer.parseInt(chaine1);
        int entier2 = Integer.parseInt(chaine2);
        int entier3 = Integer.parseInt(chaine3);

        int somme = entier1 + entier2 + entier3;

        JOptionPane.showMessageDialog(null,"La somme des trois entiers est
"
            + somme);
        System.exit(0);
    }
}

```

Exercice. Écrivez un programme qui effectue le même traitement que le précédent mais en utilisant une seule variable de type **String** (plutôt que trois) et une variable entière plutôt que quatre !

Solution. La solution suivante réutilise la même variable *chaine* pour lire chacune des chaînes de caractère qui représente un entier et la même variable *somme* pour cumuler les valeurs intermédiaires.

JavaPasAPas/chapitre_2/Exercice2.java

```
/**
 * Exercice2.java
 * Lire 3 entiers et en faire la somme. Utiliser seulement une variable
 * String
 * et deux variables int.
 */
import javax.swing.JOptionPane;
public class Exercice2{

    public static void main (String[] args) {

        String chaine = JOptionPane.showInputDialog (
            "Entrez un entier dans cette case");
        int somme = Integer.parseInt(chaine);

        chaine = JOptionPane.showInputDialog (
            "Entrez un entier dans cette case");
        somme = somme + Integer.parseInt(chaine);

        chaine = JOptionPane.showInputDialog (
            "Entrez un entier dans cette case");
        somme = somme + Integer.parseInt(chaine);

        JOptionPane.showMessageDialog(null,"La somme des trois entiers est
"
            + somme);
        System.exit(0);
    }
}
```

Minimiser la quantité de mémoire consommée

Un aspect à considérer dans le développement d'un programme est la quantité de mémoire consommée. La solution *Exercice2* peut sembler préférable à *Exercice1* de ce point de vue. Cependant, ceci n'est pas le seul aspect à considérer dans l'élaboration d'un programme. Parfois, le fait de minimiser la mémoire consommée de manière extrême peut introduire d'autres défauts tel qu'un temps de traitement plus élevé ou un manque de clarté d'un point de vue de la lisibilité du programme. Par ailleurs, dans le cas qui nous concerne, même si nous utilisons une seule variable, il y aura tout de même trois instances de la classe String.

2.13 Méthode `System.out.println()`

Dans les exemples précédents, nous avons employé la méthode `showMessageDialog()` pour afficher les résultats du programme à l'écran. Une autre manière fréquemment employée consiste à utiliser la méthode `System.out.println()` pour afficher un message dans la fenêtre de commande. De tels messages sont notamment utiles pour mieux comprendre le déroulement du code.

Exemple. Le programme suivant reprend *Exemple1* mais en affichant la somme avec `System.out.println()`.

```
JavaPasAPas/chapitre_2/ExemplePrintln.java
/**
 * ExemplePrintln.java
 * Ce programme saisit deux entiers et en affiche la somme avec
 * System.out.println()
 */
import javax.swing.JOptionPane; // Importe la classe
javax.swing.JOptionPane
public class ExemplePrintln{

    public static void main (String[] args) {

        // Déclaration de variables
        String chaine1, chaine2;    // Les entiers sous forme de chaînes
        int entier1, entier2, somme; // Les entiers à additionner

        // Saisir les deux chaînes de caractères qui représentent des
        // nombres entiers
        chaine1 = JOptionPane.showInputDialog(
            "Entrez un premier nombre entier");
        chaine2 = JOptionPane.showInputDialog(
            "Entrez un second nombre entier");

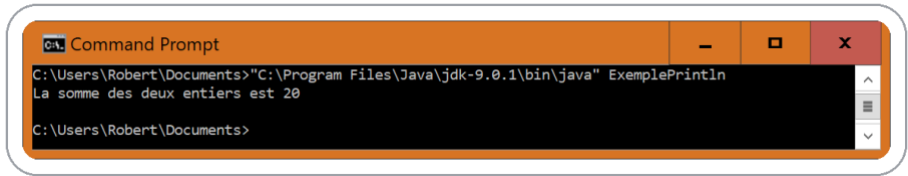
        // Convertir les chaînes en entiers
        entier1 = Integer.parseInt(chaine1);
        entier2 = Integer.parseInt(chaine2);

        // Calculer la somme des deux entiers
        somme = entier1 + entier2;

        // Afficher la somme avec System.out.println()
        System.out.println("La somme des deux entiers est " + somme);

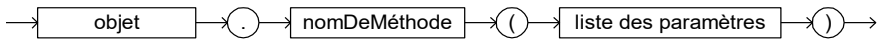
        System.exit(0);
    }
}
```


Sous Windows, le résultat est affiché dans la fenêtre de commande plutôt que dans une fenêtre de dialogue tel qu'illustré par la figure suivante :



La méthode *println()* est une méthode d'objet et non pas une méthode de classe par opposition à *JOptionPane.showInputDialog()* qui est une méthode de classe. On peut voir la différence par la forme de l'appel. L'appel d'une méthode d'objet est de la forme suivante :

appel de méthode d'objet :



L'appel débute par un objet plutôt qu'une classe. Dans notre exemple, le préfixe « *System.out* » représente un objet et non pas une classe. Cet objet correspond à l'unité périphérique de sortie standard de Java qui est la fenêtre de commande lorsque l'on utilise le JDK.

La méthode *println()* représente une méthode d'objet. Qu'est-ce qu'un objet ? Nous ne tenterons pas une explication de cette notion à ce point-ci. Mais, notons qu'une classe peut contenir des méthodes de classe et des méthodes d'objet. Dans un premier temps, dans nos premiers exemples de programmes Java, vous pouvez ignorer la différence entre un appel de méthode de classe ou d'objet.

A noter que l'appel de la méthode *println()* ne retourne rien et l'appel n'est donc pas placé dans une affectation. Il y a des méthodes qui retournent des valeurs et d'autres qui ne retournent rien. Évidemment, même si la méthode ne retourne rien, elle peut avoir un effet ! Dans le cas de *println()*, la méthode affiche quelque chose sur la fenêtre de commande.

Print() vs *println()*

La méthode `println()` provoque un saut de ligne après avoir affiché la chaîne alors que la méthode `print()` fait la même chose que `println()` mais sans saut de ligne.

Exercice. Écrivez un programme Java qui lit deux entiers et affiche la différence des deux nombres.

Exercice. Écrivez un programme Java qui lit trois entiers a , b et c . Le programme affiche le résultat de :

$ab-c$.

Exercice. Écrivez un programme Java qui lit trois entiers a , b et c . Le programme affiche le résultat de :

$(a-b)c$.

Exercice. Écrivez un programme Java qui lit deux nombres réels a et b (type `float`) et affiche a/b .

Exercice. Écrivez un programme Java qui lit deux nombres réels (type `double`) qui représentent la largeur et la hauteur d'un rectangle et affiche la surface du rectangle.

2.14 Classe Scanner

La classe **Scanner** introduite à la version 1.5 de Java permet de saisir des données correspondant à des types primitifs Java directement à la fenêtre de commande (console). Les exercices et exemples des prochains chapitres peuvent être effectué avec **Scanner** plutôt qu'avec les fenêtres de dialogue si la version de Java employée le permet.

Exemple. Le programme suivant saisit deux entiers à la console avec la classe **Scanner** et en affiche la somme.

```
/**
 * ExempleScanner.java
 * Ce programme saisit deux entiers a la console avec Scanner et en
 * affiche
 * la somme avec System.out.println()
 */

import java.util.Scanner;

public class ExempleScanner {
    public static void main (String[] args) {

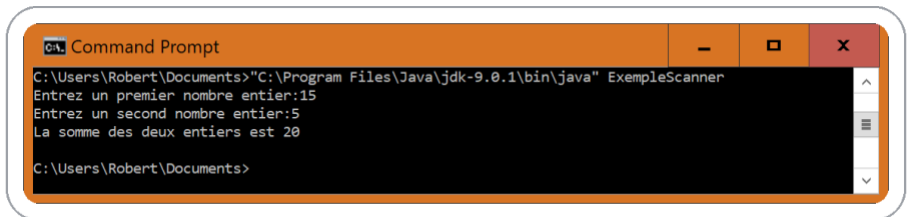
        int entier1, entier2, somme;
        Scanner unScanner = new Scanner(System.in);

        // Saisir les deux entiers avec Scanner
        System.out.print("Entrez un premier nombre entier:");
        entier1 = unScanner.nextInt();
        System.out.print("Entrez un second nombre entier:");
        entier2 = unScanner.nextInt();

        // Calculer la somme des deux entiers
        somme = entier1 + entier2;

        // Afficher la somme avec System.out.println()
        System.out.println("La somme des deux entiers est " + somme);
    }
}
```

Exemple d'exécution :



```
Command Prompt
C:\Users\Robert\Documents>"C:\Program Files\Java\jdk-9.0.1\bin\java" ExempleScanner
Entrez un premier nombre entier:15
Entrez un second nombre entier:5
La somme des deux entiers est 20
C:\Users\Robert\Documents>
```

Exercice. Reprenez les exercices précédents en effectuant la saisie des données avec **Scanner**.

3. Structures de contrôle

« *Les tests montrent la présence, pas l'absence de bogues.* » — Edsger W. Dijkstra

Dans le corps d'une méthode, il y a trois manières fondamentales d'enchaîner les actions : la *séquence*, la *boucle* et la *décision*. Ce chapitre introduit les énoncés de base Java qui permettent d'exprimer ces trois types d'enchaînement.

3.1 La séquence

Les exemples vus jusqu'à présent ont utilisé une séquence. Une séquence d'énoncés est de la forme générale suivante :

$$\text{énoncé}_1 \text{ énoncé}_2 \dots \text{énoncé}_n$$

Les énoncés sont placés en séquence les uns après les autres et sont exécutés dans cet ordre. La Figure 12 montre une représentation graphique d'une séquence avec un *diagramme d'activité* UML. Un rectangle aux coins arrondis représente une activité. Dans notre cas, une activité correspond à un énoncé Java. Les flèches indiquent l'ordre d'exécution des activités. Le point noir représente le début et le point noir encerclé la fin de l'exécution.

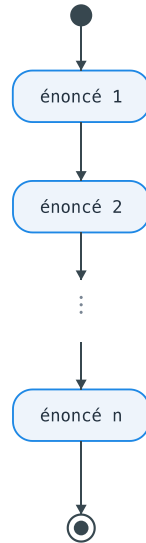
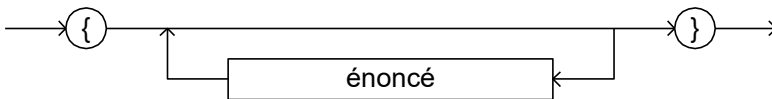


Figure 12. Diagramme d'activité UML pour une séquence

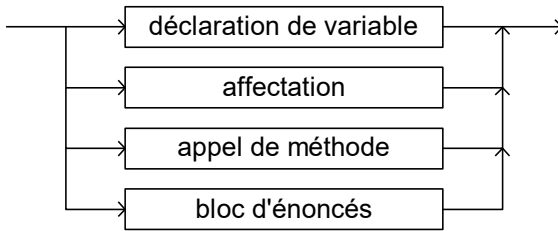
Dans *Exemple1* du chapitre 2, il n'y a pas seulement des énoncés qui correspondent à des actions mais aussi des déclarations. On peut ainsi mélanger les énoncés de déclaration et d'actions dans la séquence. Une séquence d'énoncés (action ou déclaration) placés entre accolades est appelé un *bloc d'énoncés* Java (ou simplement *bloc*). La syntaxe d'un bloc est illustrée à la figure suivante :

bloc d'énoncés :



Le corps d'une méthode est essentiellement un bloc Java. Il est à noter qu'il est permis d'avoir un seul énoncé dans le bloc. Les différents types d'énoncés seront étudiés en détails. Jusqu'à présent, nous avons rencontrés trois sortes d'énoncés : énoncé de déclaration de variable, d'affectation et d'appel de méthode. Un bloc Java est lui-même considéré comme un énoncé. On obtient ainsi le diagramme syntaxique suivant :

énoncé :



On peut donc imbriquer un bloc Java dans un autre bloc Java.

Dans l'exemple suivant, les deux énoncés de saisie de chaîne de *Exemple1* ont été regroupés en un bloc qui est imbriqué dans le bloc de la méthode *main()*.

Exemple. [JavaPasAPas/chapitre_3/ExempleBloc.java](#)

```

/**
 * ExempleBloc.java
 * Modification de Exemple1 avec un bloc imbriqué
 */
import javax.swing.JOptionPane; // Importe la classe
javax.swing.JOptionPane
public class ExempleBloc{

    public static void main (String[] args) {

        // Déclaration de variables
        String chaine1, chaine2;    // Les entiers sous forme de chaînes
        int entier1, entier2, somme; // Les entiers à additionner

        // Saisir les deux chaînes de caractères qui représentent des
        // nombres entiers
        {
            chaine1 = JOptionPane.showInputDialog(
                "Entrez un premier nombre entier");
            chaine2 = JOptionPane.showInputDialog(
                "Entrez un second nombre entier");
        }

        // Convertir les chaînes en entiers
        entier1 = Integer.parseInt(chaine1);
        entier2 = Integer.parseInt(chaine2);

        // Calculer la somme des deux entiers
        somme = entier1 + entier2;
    }
}
  
```

```
// Afficher la somme avec JOptionPane.showMessageDialog
JOptionPane.showMessageDialog(null,"La somme des deux entiers est "
    + somme);

// Appel de System.exit(0) nécessaire à cause des appels à
// JOptionPane.showInputDialog et JOptionPane.showMessageDialog
System.exit(0);
}
}
```

Ce programme produit le même effet que *Exemple1*. Dans cet exemple, l'utilisation du bloc imbriqué n'a aucune utilité. Nous verrons par la suite l'importance de cette notion.

3.2 La boucle avec l'énoncé while

Imaginons que l'on veuille afficher les entiers de 1 à 5. On pourrait produire ce résultat avec le programme suivant :

Affichage des entiers de 1 à 5.

Exemple. [JavaPasAPas/chapitre_3/Afficher12345.java](#)

```
/**
 * Afficher12345.java
 * Afficher les entiers de 1 à 5
 */
import javax.swing.JOptionPane;
public class Afficher12345{

    public static void main (String[] args) {

        JOptionPane.showMessageDialog(null,"Valeur du compteur: "+1);
        JOptionPane.showMessageDialog(null,"Valeur du compteur: "+2);
        JOptionPane.showMessageDialog(null,"Valeur du compteur: "+3);
        JOptionPane.showMessageDialog(null,"Valeur du compteur: "+4);
        JOptionPane.showMessageDialog(null,"Valeur du compteur: "+5);
        System.exit(0);
    }
}
```

S'il fallait afficher les entiers de 1 à 1 000 000, le programme serait long à écrire... Et même si vous aviez la patience de le faire, ce serait sans aucun doute inefficace parce que le processeur devrait charger et interpréter un grand nombre d'instructions redondantes. Par ailleurs, si le nombre de

répétition n'est pas connu au moment d'écrire le code, par exemple si c'est l'utilisateur ou la taille du fichier qui détermine le nombre de répétition, il peut être tout simplement impossible de procéder avec une répétition explicite dans le code. Pour éviter de répéter les énoncés dans le programme, on peut employer une répétition (aussi appelée *boucle* ou *itération*). L'énoncé *while* Java est un des énoncés Java qui permet d'effectuer une répétition.

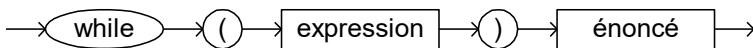
ExempleWhile illustre la notion de répétition avec compteur en employant un énoncé *while*. Ce programme a le même effet que le précédent.

Exemple. [JavaPasAPas/chapitre_3/ExempleWhile.java](#)

```
/**
 * ExempleWhile.java
 * Exemple d'utilisation d'une boucle while avec un compteur
 */
import javax.swing.JOptionPane;
public class ExempleWhile{
    public static void main (String[] args) {
        int compteur = 1;
        while(compteur <= 5){
            JOptionPane.showMessageDialog(null,
                "Valeur du compteur: "+compteur);
            compteur = compteur + 1;
        }
        System.exit(0);
    }
}
```

Voici la syntaxe d'un énoncé *while* :

énoncé while :



L'expression entre parenthèses doit être une *expression booléenne*, aussi appelée *condition*, dont la valeur est vraie (*true*) ou fausse (*false*). Si cette condition est respectée (i.e. la valeur retournée par l'*expression* est *true*), l'énoncé après le *while* est répété en boucle jusqu'à ce que la condition ne soit plus respectée (i.e. la valeur retournée par l'*expression* est *false*).

Il faut prendre soin, lorsque l'on conçoit une boucle, de s'assurer que celle-ci puisse se terminer : une boucle peut être infinie et ne jamais se terminer. Il s'agit généralement d'une erreur.

La figure suivante illustre l'enchaînement des énoncés du programme par un diagramme d'activité UML. Un losange représente une condition. Les flèches qui partent de la condition montrent les deux enchaînements possibles selon le résultat de la condition. Le diagramme montre bien le concept de répétition dans l'enchaînement des énoncés.

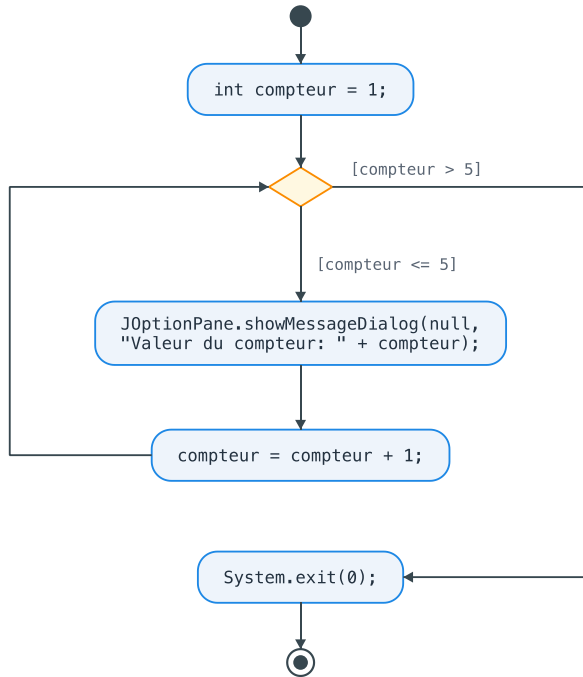


Figure 13. Diagramme d'activité pour le programme.

S'il y a plus d'un énoncé à répéter, comme c'est le cas de notre exemple, il faut les regrouper en un bloc, donc mettre ces énoncés entre accolades. Il est à noter que s'il y a un seul énoncé, les accolades sont facultatives. Dans notre exemple, la condition est

```
(compteur <= 5){
```

Donc, tant que cette condition s'avère vraie (tant que le compteur sera plus petit ou égal à 5), les énoncés

```
JOptionPane.showMessageDialog(null, "Valeur du compteur: "+compteur);
    compteur = compteur + 1;
```

s'exécuteront en boucle. Dès que le compteur dépasse la valeur de cinq, ces énoncés arrêtent de s'exécuter, et le programme passe à l'énoncé suivant, qui est *System.exit(0)*. Ce dernier énoncé met fin au programme.

Expression booléenne

Le nombre de répétition est contrôlé par une expression booléenne. Une expression booléenne peut être formée en comparant des valeurs à l'aide des opérateurs de comparaison du tableau suivant. Il y a quelques différences avec la notation mathématique usuelle.

Opérateur de comparaison	Signification
<	Plus petit que
<=	Plus petit ou égal à
>	Plus grand que
>=	Plus grand ou égal à
==	Égal à
!=	N'est pas égal à

Figure 14. Opérateurs de comparaison.

Exercice. Modifiez l'exemple précédent afin qu'il affiche les entiers 0, 2, 4, 6, 8, 10.

Solution. [JavaPasAPas/chapitre_3/ExerciceWhile1.java](#)

```
/**
 * ExerciceWhile1.java
 * Afficher les valeurs 0,2,4,6,8,10
 */
import javax.swing.JOptionPane;
public class ExerciceWhile1{
    public static void main (String[] args) {
        int compteur = 0;
        while(compteur <= 10){
            JOptionPane.showMessageDialog(null,
                "Valeur du compteur: "+compteur);
            compteur = compteur + 2;
        }
        System.exit(0);
    }
}
```

Exercice. Modifiez l'exemple afin d'afficher 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5.

Solution. `JavaPasAPas/chapitre_3/ExerciceWhile2.java`

```
/**
 * ExerciceWhile2.java
 * Afficher les valeurs de compteur 5,4,3,2,1,0,-1,-2,-3,-4,-5
 */
import javax.swing.JOptionPane;
public class ExerciceWhile2{
    public static void main (String[] args) {
        int compteur;
        compteur = 5;
        while(compteur >= -5){
            JOptionPane.showMessageDialog(null,
                "Valeur du compteur:"+compteur);
            compteur = compteur - 1;
        }
        System.exit(0);
    }
}
```

Exercice. Reprenons l'exemple de lecture d'entiers afin d'en afficher la somme. Nous avons vu le cas de deux et de trois entiers. Maintenant cherchons à extrapoler jusqu'à 10 entiers ! Il serait très long de répéter dix fois la lecture, la conversion et l'accumulation dans somme. Une solution naturelle est d'utiliser un énoncé *while*. Dans cet exercice, utilisez une boucle *while* pour lire dix entiers et en afficher la somme.

Solution. `JavaPasAPas/chapitre_3/ExerciceWhile3.java`

```
/**
 * ExerciceWhile3.java
 * Lire dix entiers et en afficher la somme avec un while
 */
import javax.swing.JOptionPane;
public class ExerciceWhile3 extends Object {
    public static void main (String[] args) {
        String serie;
        int entier;
        int compteur = 1;
        int somme = 0;

        while (compteur <=10){
            serie = javax.swing.JOptionPane.showInputDialog(
                "Entrez un entier");
            entier = Integer.parseInt (serie);
            somme = somme + entier;
            compteur = compteur + 1;
        }
    }
}
```

```

        JOptionPane.showMessageDialog(null,"La somme des dix entiers est "
            + somme);
        System.exit(0);
    }
}

```

Exercice. Supposons que le nombre d'entiers à lire est inconnu à l'avance. Une technique souvent employée pour arrêter la répétition est l'utilisation d'une valeur spéciale appelée sentinelle qui provoque l'arrêt de la répétition. Par exemple, supposons que le nombre 0 représente la sentinelle. Vous devez donc écrire un programme qui lit une série d'entier jusqu'à ce que l'entier 0 soit entré et produit la somme de ces entiers.

Solution. [JavaPasAPas/chapitre_3/ExerciceWhileSentinelle.java](#)

```

/**
 * ExerciceWhileSentinelle.java
 * Lire une suite d'entiers jusqu'à ce que l'entier 0 soit entré et
 * afficher la somme
 * des entiers lus.
 */
import javax.swing.JOptionPane;
public class ExerciceWhileSentinelle {
    public static void main (String[] args) {
        String serie;
        int somme = 0;
        int entier = 1; // N'importe quelle valeur différente de 0 ferait
        // l'affaire
        while (entier != 0) {
            serie = JOptionPane.showInputDialog("Entrez un nombre");
            entier = Integer.parseInt (serie);
            somme = somme + entier;
        }
        JOptionPane.showMessageDialog(null,
            "La somme de tous les nombres est de " +somme+ ".");
        System.exit(0);
    }
}

```

3.3 Qualité du logiciel, tests et débogage

Un aspect fondamental de la qualité d'un programme est la validité des résultats produits par rapport à sa spécification. Une pratique courante en développement de logiciel est de vérifier que les résultats corrects sont produits pour un ensemble de cas de tests.

Exemple. Pour tester le programme précédent, on pourrait employer plusieurs combinaisons de données en entrée et vérifier que pour chacun des cas de tests, le bon résultat est produit.

Numéro de test	Input	Output
1	15 120 30 0	165
2	10 -5 20 0	30
3	0	0
4	2a	Exception

Ce genre de test est dit *fonctionnel* étant donné qu'il vérifie que le fonctionnement du programme est valide par rapport à ce qu'il doit faire (*spécification fonctionnelle*). Dans des programmes plus complexes, d'autres aspects peuvent aussi être mesurés tel que le temps de calcul, la mémoire consommée ou d'autres aspects dits *non fonctionnels*.

Il existe des méthodes et logiciels qui visent à automatiser le processus de vérification par des tests. Généralement, même si tous les tests produisent le résultat correct, ceci ne garantit pas que le programme fonctionne correctement dans tous les cas possibles. Il est habituellement trop complexe en pratique de tester tous les cas. Cependant en choisissant les cas de tests d'une manière judicieuse, on peut obtenir un grand niveau de confiance au sujet du fonctionnement du programme. Différentes stratégies peuvent être employées à cet effet.

Dans l'approche de test par boîte noire ou opaque (*black box testing*), les tests sont choisis sans examiner le code lui-même. On cherche à choisir les cas de tests de manière à produire différentes combinaisons d'input qui couvrent les différentes possibilités prévues dans la spécification du programme. Dans l'approche de test par boîte blanche ou transparente (*white box testing, glass box testing*), les tests sont conçus en tenant compte du code. En particulier, il faut tenter de parcourir toutes les parties du code par l'ensemble des tests.

Lorsqu'un test ne produit pas le résultat voulu, on dit qu'il y a un ou plusieurs bogues (*bug*) dans le programme. Le débogage est le processus d'élimination des bogues. Le débogage peut ressembler à un travail de fin limier qui consiste à trouver le code coupable en examinant les indices produits par l'exécution du programme. Il existe des outils débogueurs qui facilitent le dépistage des bogues par exemple en permettant d'exécuter les énoncés pas à pas et en inspectant les valeurs des variables. En l'absence d'un tel outil, on peut ajouter des énoncés qui affichent l'état de variables à différents endroits du programme pour en suivre le déroulement d'une manière plus détaillée.

Exercice. Vérifiez si les bons résultats sont produits avec les tests précédents pour la solution du dernier exercice.

3.4 La boucle avec l'énoncé *for*

L'utilisation d'une répétition avec compteur est très fréquente. La boucle *for* simplifie l'écriture de telles boucles.

Le programme suivant produit le même effet que *ExempleWhile* en affichant les entiers de 1 à 5. Dans un énoncé *for*, l'initialisation du compteur, l'expression de fin de répétition et la mise-à-jour du compteur sont regroupés entre parenthèses après l'identificateur réservé *for*.

Exemple. [JavaPasAPas/chapitre_3/ExempleForSimple.java](#)

```
/**
 * ExempleForSimple.java
 * Exemple d'utilisation d'un énoncé for qui affiche les entiers de 1 à 5
 */
import javax.swing.JOptionPane;
public class ExempleForSimple {
    public static void main (String[] args) {
        for (int compteur = 1; compteur <=5; compteur = compteur + 1)
            JOptionPane.showMessageDialog(null,"Valeur du compteur:
"+compteur);
        System.exit(0);
    }
}
```

Une abréviation syntaxique souvent employée est l'emploi de l'opérateur de post-incrémentation (++) qui a l'effet d'incrémenter de 1.

Exemple. Le *for* suivant est équivalent au précédent.

```
for (int compteur = 1; compteur <=5; compteur++)
```

La syntaxe du *for* est :

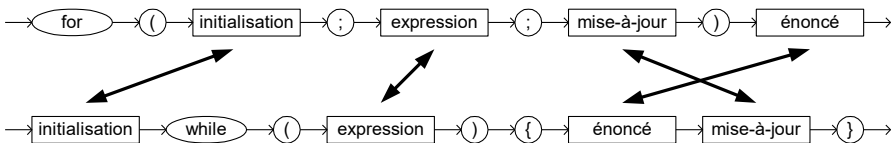
énoncé *for* :



A noter qu'il n'y a qu'un énoncé à répéter dans notre exemple, et qu'il n'est pas nécessaire de mettre un bloc pour l'énoncé à répéter. En effet, dans l'exemple, il n'y a pas d'accolades. Cependant, s'il y avait plusieurs énoncés, il aurait été nécessaire de mettre des accolades avant et après les énoncés à répéter.

L'énoncé *for* fonctionne de façon semblable au *while*. La figure suivante montre comment transformer un énoncé *for* en un *while* (il suffit tout simplement de changer les emplacements de certaines composantes). On peut donc se passer du *for* et toujours utiliser le *while*. Le *for* vise tout simplement à simplifier l'écriture des programmes. D'autre part, le *for* n'est pas limité au cas d'un compteur. N'importe quelle expression peut être employée pour décider de la poursuite de la répétition.

énoncé *for* :



Exemple. Les deux bouts de code suivants font la même chose :

```
for (int compteur = 1; compteur <=5; compteur = compteur + 1)
    JOptionPane.showMessageDialog(null,"Valeur du compteur: "+compteur);
```

```
int compteur = 1;
while(compteur <= 5){
    JOptionPane.showMessageDialog(null,
        "Valeur du compteur: "+compteur);
    compteur = compteur + 1;
}
```


Il est possible d'omettre l'initialisation, l'expression ou la mise-à-jour du *for* mais en laissant les « ; ».

L'exemple suivant reprend l'exercice de lecture d'une série d'entiers avec sentinelle vu précédemment mais en employant un *for* plutôt qu'un *while*. La partie mise-à-jour du *for* est vide mais le dernier « ; » à l'intérieur des parenthèses du *for* doit être présent.

Exemple. `JavaPasAPas/chapitre_3/ExempleForSentinelle.java`

```
/**
 * ExempleForSentinelle.java
 * Lire une suite d'entiers jusqu'à ce que l'entier 0 soit entré et
 * afficher la somme
 * des entiers lus. Exemple illustrant un for sans la partie mise-à-jour.
 */
import javax.swing.JOptionPane;
public class ExempleForSentinelle {
    public static void main (String[] args) {
        String serie;
        int somme = 0;
        for (int entier = 1; entier != 0;){
            serie = JOptionPane.showInputDialog("Entrez un nombre");
            entier = Integer.parseInt (serie);
            somme = somme + entier;
        }
        JOptionPane.showMessageDialog(null,
            "La somme de tous les nombres est de " +somme+ ".");
        System.exit(0);
    }
}
```

Exercice. Trouver une justification au besoin de conserver le dernier ; du *for* précédent malgré l'omission de la partie mise-à-jour (indice : essayez de penser comme un ordinateur...).

Exercice. Affichez les entiers 0, 2, 4, 6, 8, 10 avec un *for*.

Exercice. Affichez 5, 4, 3, 2, 1, 0, -1, -2, -3, -4, -5 avec un *for*.

Exercice. Utilisez un *for* pour lire dix entiers et en afficher la somme.

Exercice. Utilisez un *for* pour lire une série d'entier jusqu'à ce que l'entier 0 soit entré (cas de sentinelle) et afficher la somme de ces entiers.

Exercice. Affichez le résultat suivant sur la sortie standard (avec `System.out.print()` et `System.out.println()`)

```
1
12
123
1234
12345
123456
1234567
12345678
123456789
```

Avec *for* imbriqués :

Solution. [JavaPasAPas/chapitre_3/ExerciceForFor.java](#)

```
/**
 * ExerciceForFor.java
 */
import javax.swing.JOptionPane;
public class ExerciceForFor {
    public static void main (String[] args) {
        for (int compteur1 = 1; compteur1 <=9; compteur1 = compteur1 + 1){
            for (int compteur2 = 1; compteur2 <=compteur1; compteur2 = compteur2
                + 1)
                System.out.print(compteur2);
            System.out.println();
        }
    }
}
```

3.5 La décision avec *if*

L'énoncé *if* permet au programme de prendre une décision au sujet des actions à exécuter en fonction d'une condition à évaluer.

Le programme suivant lit un entier (*unInt*) et indique s'il est plus grand que 10 ou non. Pour déterminer le message à afficher, une décision est prise en comparant l'entier lu à 10 dans une condition (*unInt* > 10). Selon le résultat de la condition, le *if* permet de choisir entre les deux énoncés alternatifs à exécuter. La première alternative suit la condition et elle est exécutée si la condition est évaluée à vrai (*true*). Sinon (i.e. la valeur de l'expression est *false*), l'énoncé qui suit l'identificateur *else* est exécuté.

Exemple. JavaPasAPas/chapitre_3/ExempleIf.java

```

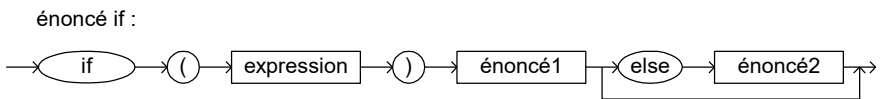
/**
 * ExempleIf.java
 * Petit exemple illustrant l'énoncé if.
 */
import javax.swing.JOptionPane;
public class ExempleIf{
    public static void main (String[] args) {
        String unString = JOptionPane.showInputDialog(
            "Entrez un premier nombre entier");
        int unInt = Integer.parseInt(unString);

        // Exemple d'énoncé if
        if (unInt > 10)
            JOptionPane.showMessageDialog(null,unInt
                + " est plus grand que 10");
        else
            JOptionPane.showMessageDialog(null,unInt
                + " n'est pas plus grand que 10");

        System.exit(0);
    }
}

```

La syntaxe du *if* est :



Lors de l'utilisation d'un *if*, l'*énoncé1* suivant l'expression ne sera exécutée que si l'expression (la condition) se révèle vraie. L'énoncé ne s'exécute qu'une seule fois. Si l'expression se révèle fausse, l'*énoncé2* suivant le mot *else* sera exécuté (une seule fois). Dans notre exemple, si la condition

```
(unInt > 10)
```

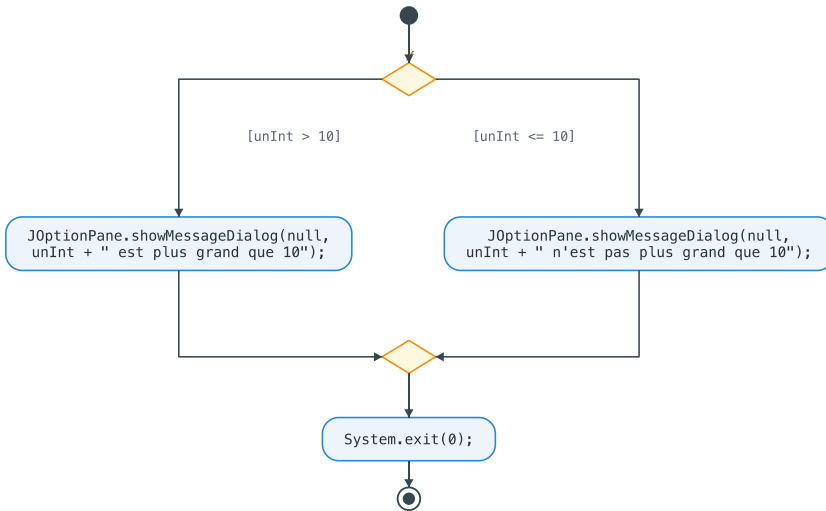
se révèle vraie, l'énoncé

```
JOptionPane.showMessageDialog(null,unInt
    + " est plus grand que 10");
```

sera exécuté. Si l'expression se révèle fausse, l'énoncé qui sera exécuté est

```
JOptionPane.showMessageDialog(null,unInt
    + " n'est pas plus grand que 10");
```

Le diagramme d'activité suivant illustre cet enchaînement.



Il est à noter que s'il y a plusieurs énoncés à exécuter dans la partie *énoncé1* (cas vrai) ou *énoncé2* (cas faux), il faut les regrouper en un bloc, se débutant par l'accolade ouvrante et se terminant par l'accolade fermante.²² Aussi, la partie *else* est optionnelle. En son absence, lorsque l'expression de condition est fausse, rien n'est exécuté. Ceci peut causer une ambiguïté potentielle illustrée par l'exemple suivant.

Illustration du *else* ambigu.

Exemple. `JavaPasAPas/chapitre_3/ExempleElseAmbigu.java`

```

/**
 * ExempleElseAmbigu.java
 * Petit exemple illustrant l'ambiguïté du else.
 */
import javax.swing.JOptionPane;
public class ExempleElseAmbigu{
    public static void main (String[] args) {
        String chaine1 = JOptionPane.showInputDialog(
            "Entrez un premier nombre entier");
        String chaine2 = JOptionPane.showInputDialog(
            "Entrez un second nombre entier");
        int entier1 = Integer.parseInt(chaine1);
        int entier2 = Integer.parseInt(chaine2);
    }
}

```

²² Il est parfois recommandé de toujours mettre des accolades autour des énoncés découlant d'une clause if afin d'éviter les ambiguïtés.

```

// If ambigu
if (entier1 > 10)
    if (entier2 > 10)
        JOptionPane.showMessageDialog(null, entier1 + " et " + entier2
            + " sont plus grands que 10");
    else
        JOptionPane.showMessageDialog(null, entier1
            + " est inférieur ou égal à 10");

    System.exit(0);
}
}

```

Voici un scénario possible avec ce programme ! Voyez-vous le problème ?

Le problème vient du fait que la disposition du texte donne l'impression que le *else* est associé au premier *if*. Ce n'est pas le cas ! Java associe toujours le *else* au *if* le plus rapproché ! La disposition suivante suggère la bonne interprétation :

```

if (entier1 > 10)
    if (entier2 > 10)
        JOptionPane.showMessageDialog(null, entier1 + " et " + entier2
            + " sont plus grands que 10");
    else
        JOptionPane.showMessageDialog(null, entier1
            + " est inférieur ou égal à 10");

```

Pour forcer le *else* à être associé au premier *if*, il faut utiliser un bloc Java afin de forcer la terminaison du second *if*.

Forcer l'association du *else* avec un *if* éloigné par l'introduction d'un bloc.

Exemple. JavaPasAPas/chapitre_3/ExempleIfIfElse.java

```

/**
 * ExempleIfIfElse.java
 * Introduction d'un bloc pour terminer un if sans else
 */
import javax.swing.JOptionPane;
public class ExempleIfIfElse{
    public static void main (String[] args) {
        String chaine1 = JOptionPane.showInputDialog(
            "Entrez un premier nombre entier");
        String chaine2 = JOptionPane.showInputDialog(
            "Entrez un second nombre entier");
        int entier1 = Integer.parseInt(chaine1);
    }
}

```

```

int entier2 = Integer.parseInt(chaine2);

// If ambigu
if (entier1 > 10) {
    if (entier2 > 10)
        JOptionPane.showMessageDialog(null, entier1 + " et " + entier2
            + " sont plus grands que 10");
    }
else
    JOptionPane.showMessageDialog(null, entier1
        + " est inférieur ou égal à 10");

System.exit(0);
}
}

```

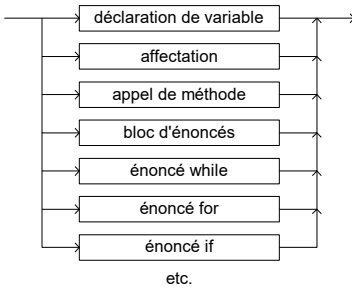
Voici un scénario avec cette nouvelle version :

The image shows three sequential Java Swing dialog boxes:

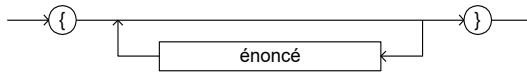
- Dialog 1:** Titled "Entrez un premier nombre entier" (Enter a first integer). It features a green question mark icon, an input field containing the number "7", and two buttons: "OK" and "Annuler" (Cancel).
- Dialog 2:** Titled "Entrez un second nombre entier" (Enter a second integer). It features a green question mark icon, an input field containing the number "5", and two buttons: "OK" and "Annuler" (Cancel).
- Dialog 3:** An information dialog titled "7 est inférieur ou égal à 10" (7 is less than or equal to 10). It features a purple information icon (a lowercase 'i' inside a circle) and a single "OK" button.

Nous avons maintenant vu les trois manières d'enchaîner les énoncés : séquence, boucle et choix. Il est possible de combiner ces trois types d'énoncés de manière quelconque. Les diagrammes syntaxiques suivants résument les différents cas d'énoncés vus jusqu'à présent :

énoncé :



bloc d'énoncés :



Dans un bloc d'énoncé, il peut y avoir un while, dans le while, un if et dans le if, un bloc, etc.

Exercice. Lire deux entiers et afficher la division du premier par le deuxième. Si le diviseur est 0, afficher un message à cet effet.

Solution. [JavaPasAPas/chapitre_3/ExerciceIf1.java](#)

```

/**
 * ExerciceIf1.java
 * Lire deux entiers et afficher la division entière.
 * Si le diviseur est 0 afficher un message à cet effet.
 */
import javax.swing.JOptionPane;
public class ExerciceIf1 {

    public static void main (String[] args) {

        String chaine1 = JOptionPane.showInputDialog(
            "Entrez un premier nombre entier");
        String chaine2 = JOptionPane.showInputDialog(
            "Entrez un second nombre entier");
        int entier1 = Integer.parseInt(chaine1);
        int entier2 = Integer.parseInt(chaine2);

        if (entier2 == 0)
            JOptionPane.showMessageDialog(null,
                "La division est impossible");
        else
            JOptionPane.showMessageDialog(null, entier1 + "/" + entier2 +
                "=" +
                    + entier1 / entier2);
        System.exit(0);
    }
}

```

Exercice. Lire deux entiers et afficher le maximum des deux. S'ils sont égaux, afficher n'importe lequel des deux.

Solution. `JavaPasAPas/chapitre_3/ExerciceIfMax2.java`

```
/**
 * ExerciceIfMax2.java
 * Lire trois entiers et afficher le maximum des trois
 */
import javax.swing.JOptionPane;
public class ExerciceIfMax2{

    public static void main (String[] args) {

        String chaine1 = JOptionPane.showInputDialog(
            "Entrez un premier nombre entier");
        String chaine2 = JOptionPane.showInputDialog(
            "Entrez un second nombre entier");

        int entier1 = Integer.parseInt(chaine1);
        int entier2 = Integer.parseInt(chaine2);

        if (entier1 > entier2)
            JOptionPane.showMessageDialog(null,"Le maximum des deux entiers est "
                + entier1);
        else
            JOptionPane.showMessageDialog(null,"Le maximum des deux entiers est "
                + entier2);

        System.exit(0);
    }
}
```

Exercice. Lire deux entiers et afficher le plus grand des deux s'il y en a un qui est le plus grand, sinon afficher qu'ils sont égaux.

Exercice. Lire trois entiers et afficher le maximum des trois.

Solution. `JavaPasAPas/chapitre_3/ExerciceIfMax3.java`

```
/**
 * ExerciceIfMax3.java
 * Lire trois entiers et afficher le maximum des trois
 */
import javax.swing.JOptionPane;
public class ExerciceIfMax3{

    public static void main (String[] args) {

        String chaine1 = JOptionPane.showInputDialog(
            "Entrez un premier nombre entier");
```



```

String chaine2 = JOptionPane.showInputDialog(
    "Entrez un second nombre entier");
String chaine3 = JOptionPane.showInputDialog(
    "Entrez un troisième nombre entier");

int entier1 = Integer.parseInt(chaine1);
int entier2 = Integer.parseInt(chaine2);
int entier3 = Integer.parseInt(chaine3);

if (entier1 > entier2)
    if (entier1 > entier3)
        JOptionPane.showMessageDialog(null,
            "Le maximum des trois entiers est " + entier1);
    else
        JOptionPane.showMessageDialog(null,
            "Le maximum des trois entiers est " + entier3);
else
    if (entier2 > entier3)
        JOptionPane.showMessageDialog(null,
            "Le maximum des trois entiers est " + entier2);
    else
        JOptionPane.showMessageDialog(null,
            "Le maximum des trois entiers est " + entier3);
System.exit(0);
}
}

```

On peut imaginer la complexité de cette méthode si on accroît le nombre d'entiers à lire. Dans l'exercice suivant, cherchez à utiliser une boucle pour simplifier le code.

Exercice. Lire 5 entiers et afficher l'entier maximal (le plus grand des cinq entiers).

```

Solution. JavaPasAPas/chapitre_3/ExerciceWhileIf.java
/**
 * ExerciceWhileIf.java
 * Lire 5 entiers et afficher l'entier maximal
 */
import javax.swing.JOptionPane;
public class ExerciceWhileIf{
    public static void main (String[] args) {
        String chaine = JOptionPane.showInputDialog("Entrez un nombre");
        int plusGrand = Integer.parseInt (chaine);
        for (int compteur = 1; compteur < 5; compteur=compteur+1){
            chaine = JOptionPane.showInputDialog("Entrez un nombre entier");
            int entier = Integer.parseInt (chaine);
            if (entier > plusGrand) {
                plusGrand = entier;
            }
        }
    }
}

```

```
}  
    JOptionPane.showMessageDialog(null, "L'entier le plus grand est "  
        + plusGrand);  
    System.exit(0);  
}  
}
```

Cette solution montre un exemple de *if* imbriqué dans une boucle.

Exercice *. Afficher les nombres premiers plus petits que 100.

Exercice. Lire une note entre 0 et 100 inclusivement et afficher la lettre correspondante selon le barème suivant :

0<=note<60	E
60<=note<70	D
70<=note<80	C
80<=note<90	B
90<=note<=100	A

4. Types et expressions Java

«N'importe quel idiot peut écrire du code qu'un ordinateur peut comprendre. Les bons programmeurs écrivent du code que les humains peuvent comprendre.» — Martin Fowler

Dans les premiers chapitres, nous avons rencontré les types *int* et **String** et quelques expressions simples. Ce chapitre approfondit les notions de type et d'expression Java.

4.1 Type primitif et littéral

Chacun des types primitifs correspond à un ensemble de valeurs décrit dans le tableau de la Figure 15. Une variable d'un type primitif contient une valeur parmi l'ensemble des valeurs du type.

Type primitif	Valeurs	Exemples de littéraux	Signification
boolean	true ou false	true false	Vrai faux
char	Caractère selon le code standard UNICODE (16 bits)	'a' 'A' '5' ' ' '\" '\u0061' '\u000a'	La lettre a (minuscule) Lettre A (majuscule) Le chiffre 5 Un espace Le caractère ' Le caractère a dont le code Unicode est 0061 (en hexadécimal) Le caractère non imprimable qui correspond à un saut de ligne
byte	Octet en binaire (8 bits) entre -	(byte)15	Conversion explicite d'un littéral int

Type primitif	Valeurs	Exemples de littéraux	Signification
	128 (-2^7) et 127 (2^7-1)		
short	Entier (précision de 16 bits) entre -32 768 (-2^{15}) et 32 767 ($2^{15}-1$)	(short)325	Conversion explicite d'un littéral int
int	Entier (précision de 32 bits) entre -2 147 483 648 (-2^{31}) et 2 147 483 647 ($2^{31}-1$)	3015 -3015 0X2A 017	Par défaut, un littéral entier est de type <i>int</i> Le moins unaire est employé pour un nombre négatif Représentation hexadécimale du int 42 ($2*16+10$) Représentation octale du int 15 ($1*8 + 7$)
long	Entier (précision de 64 bits) entre -9 223 372 036 854 775 808 (-2^{63}) et 9 223 372 036 854 775 807 ($2^{63}-1$)	3015L 3015l	Le L ou l à la fin du littéral entier indique que le nombre entier est de type <i>long</i>
float	Nombre réel (précision de 32 bits selon le code standard IEEE 754-1985) entre - $3.4*10^{38}$ et $3.4*10^{38}$ (7 chiffres significatifs)	45.032F 45.032f 32F 32f 12.453E5F 12.053E-5F	Le F ou f à la fin du littéral numérique indique que le nombre doit être de type <i>float</i> 1245300 (La notation E_n représente 10^n) 0.0001205
double	Nombre réel (précision de 64 bits selon le code standard IEEE 754-1985) entre - $1.7*10^{308}$ et $1.7*10^{308}$ (15 chiffres significatifs)	45.032 45.032D 45.032d 32D 32d 12.453E5 12.053E-5 124.	Par défaut, un littéral numérique avec partie décimale est de type <i>double</i> Le D ou d à la fin indique que le nombre doit être de type <i>double</i> 1245300 (La notation E_n représente 10^n) 0.0001205

Figure 15. Types primitifs de Java.

Littéral (*literal*)

Une valeur particulière d'un type primitif exprimée par une chaîne de caractères dans le code source d'un programme est appelée un *littéral*. Le tableau précédent montre des exemples de littéraux pour chacun des types primitifs.

4.2 Types et expressions numériques

Les types primitifs numériques *byte*, *short*, *int*, *long* correspondent à des entiers de différentes tailles : ils utilisent une quantité variable de mémoire, et ils peuvent représenter des nombres plus ou moins volumineux. À l'exception du type « char » qui peut être considéré comme un type représentant des entiers (de 0 à 65536), les types entiers en Java sont toujours signés dans le sens où ils permettent de représenter à la fois les nombres positifs et négatifs. En contraste, dans des langages comme Go, Swift, C#, C++, etc., il existe des types d'*entiers signés* et *non-signés* (*unsigned*). Notez qu'il y a toujours une valeur négative de plus grande amplitude que n'importe quelle valeur positive (par exemple, -128, -32768, -2147483648, etc.) ce qui signifie qu'on ne peut toujours prendre la valeur absolue : par exemple, la valeur 2147483648 ne peut pas être représentée par un « int » alors que -2147483648 est parfaitement légitime.

Les types *float* et *double* sont des nombres réels avec partie fractionnaire. En Java, on ne peut pas représenter les valeurs réelles (comme π). On utilise plutôt des nombres à virgule flottante. Ainsi donc, on peut utiliser des « double » pour consacrer 64 bits afin de représenter des nombres. On utilise alors la norme « binary64 » qui accorde 53 bits à la mantisse d'une représentation binaire. En d'autres mots, on peut pratiquement représenter n'importe quel nombre de la forme $m \cdot 2^p$ tant que m n'excède pas 2^{53} . En particulier, le type « double » en Java peut représenter tous les entiers (positifs et négatifs) qui n'excèdent pas une magnitude de 2^{53} . Quand un nombre ne peut pas être représenté, Java va trouver le nombre à virgule flottante le plus proche. Si nous arrivons exactement entre deux nombres à virgule flottante, comme c'est le cas avec le nombre 90000000000000002.5, Java va choisir le nombre le plus proche qui a une mantisse paire (ici 90000000000000002). Tous les nombres décimaux à 6 chiffres significatifs peuvent être représentés avec le type *float*. Par contre, certains nombres comprenant 6 chiffres significatifs correspondent à plus d'un nombre de

type *float*. On peut distinguer deux nombres de type *float* en utilisant 9 chiffres significatifs. Tous les nombres décimaux à 15 chiffres significatifs peuvent être représentés avec le type *double*; on peut distinguer deux nombres de type *double* en utilisant 17 chiffres significatifs. Dans la pratique, nous utilisons donc souvent soit 9 chiffres significatifs, soit 17 chiffres significatifs afin de stocker en format décimal les nombres. Par exemple, si on sait que le résultat sera stocké sous la forme d'un double, il est inutile de saisir le nombre π avec plus de précision que 3.1415926535897932. Les types *float* et *double* peuvent aussi avoir pour valeur *-infini*, *+infini* ainsi qu'une valeur spéciale NaN (*not-a-number*) que nous pouvons générer en divisant zéro par zéro (par exemple).

L'exemple suivant illustre le comportement du zéro positif, du zéro négatif et du NaN en Java. Il affiche ceci à l'écran :

Exemple. [JavaPasAPas/chapitre_3/ExempleZero.java](#)

```
true
-Infinity
Infinity
false
NaN
false
```

```
public class ExempleZero {
    public static void main(String[] s) {
        double minus_zero = -0.0;
        double plus_zero = +0.0;
        System.out.println(minus_zero == plus_zero);
        System.out.println(1/minus_zero);
        System.out.println(1/plus_zero);
        System.out.println(1/minus_zero == 1/plus_zero);
        double n = 0.0 / 0.0;
        System.out.println(n);
        System.out.println(n == n);
    }
}
```

La Figure 15 montre les conventions Java pour exprimer les littéraux des types numériques. Java permet de former des expressions numériques complexes de manière analogue aux expressions mathématiques. Le tableau suivant montre les opérations numériques binaires de Java.

Opération binaire	Signification
+	Addition Exemples : $4 + 5$ donne 9, $4.2 + 16.3$ donne 20.5
-	Soustraction Exemples : $5 - 2$ donne 3, $20.5 - 16.3$ donne 4.2
*	Multiplication Exemple : $3 * 4$ donne 12
/	Division Exemples : $16 / 5$ donne 3, $16.0 / 5.0$ donne 3.2
%	Reste après division entière Exemples : $16 \% 4$ donne 0, $16 \% 5$ donne 1

La division des nombres entiers positifs se fait comme à la petite école : le résultat de la division est un entier qui représente le nombre de fois que le diviseur peut être soustrait au numérateur sans obtenir un résultat négatif. Par exemple, $4/4$, $5/4$ et $7/4$ donnent toutes comme réponse le quotient 1. Le reste de la division est obtenu avec le symbole `%`. On peut vérifier que l'entier x divise l'entier y en comparant $y \% x$ avec la valeur zéro.

La division par zéro (avec `/` ou `%`) dans le cas des entiers génère une erreur et peut terminer un programme. La division par zéro dans le cas des nombres à virgule flottante est permise, mais elle génère une valeur infinie ou la valeur NaN.

Dans le cas d'une opération arithmétique dont le résultat ne peut pas être représenté par le type choisi, un résultat incorrect peut être obtenu. Il est de votre responsabilité de vérifier que le résultat du calcul peut être représenté.

Exemple. Le `int` 2000000000 ajouté au `int` 2000000000 donne la valeur -294967296 :

```
int x = 2000000000;
int y = 2000000000;
int z = x + y; // = -294967296
```

Il est permis de former des expressions arithmétiques complexes en combinant les opérations au besoin. Comme pour les conventions mathématiques usuelles, lorsqu'il y a plusieurs opérations, les opérations à plus grande priorité sont effectuées en premier. À priorité égale, les opérations sont effectuées de gauche à droite. Le tableau suivant montre la priorité des opérateurs en ordre décroissant de priorité. Le + et - *unaires* servent à préciser le signe d'une valeur numérique comme dans +3 ou -15, le + étant toujours facultatif.

Opération	Priorité
(,)	0
+, - unaires	1
*, /, %	2
+, - binaires	3

Exemple illustrant les priorités. L'expression suivante

$$3 + 2 * 6 - 3 - 2 * 4$$

est équivalente à

$$(((3 + (2 * 6)) - 3) - (2 * 4))$$

dont le résultat est 4. L'évaluation procède donc selon les étapes suivantes :

$$\begin{aligned}
 &(((3 + (2 * 6)) - 3) - (2 * 4)) \\
 &(((3 + 12) - 3) - (2 * 4)) \\
 &(((3 + 12) - 3) - 8) \\
 &((15 - 3) - 8) \\
 &(12 - 8) \\
 &4
 \end{aligned}$$

Les parenthèses permettent de modifier cet ordre d'évaluation au besoin.

Exercice. Quel est le résultat de l'expression suivante :

$$2+4*2*5+10/2$$

Réécrire l'expression avec des parenthèses qui reflètent la priorité d'évaluation des opérations.

Conseil

Ne vous fiez pas à la priorité, et mettez des parenthèses en cas de doute !

Lorsque des opérandes de types différents sont combinés, Java effectue des conversions de type automatiques en convertissant à un type unique tous les opérandes de l'expression.

Exemple. L'expression suivante

```
3.4 + 7
```

fait intervenir le *double* 3.4 et le *int* 7. Le *int* sera converti automatiquement en un *double* avant d'effectuer l'opération.

La conversion cherche à éviter la perte d'information en faisant une *promotion* à un type plus général. Le tableau suivant montre les promotions valides en Java pour les types numériques. Les conversions sont appliquées non seulement à l'évaluation d'expressions mais aussi lors de l'affectation du résultat de l'expression à une variable et lors du passage d'un paramètre.

Type	Promotions automatiques valides
double	aucune
float	double
long	float ou double
int	long, float ou double
short	int, long, float ou double
byte	short, int, long, float ou double

Lorsqu'une conversion non valide est voulue par le programmeur, il est possible de forcer une conversion par une opération de conversion (*cast*) dont la syntaxe est :

```
(nomDuType)valeur
```

La valeur sera alors convertie dans le type entre parenthèses. La conversion peut entraîner une perte de précision comme l'illustre l'exemple suivant.

Exemple. Le *double* 15.2 est converti en *int* 15 et affecté à la variable *unInt* :

```
int unInt = (int)15.2
```

Sans conversion explicite, une erreur serait levée car cette promotion automatique n'est pas valide en Java. La conversion explicite est potentiellement dangereuse. En tant que programmeur, vous avez la responsabilité de vérifier que le nouveau type peut représenter la valeur. Considérons l'exemple suivant.

Exemple. Le *int* 131072 est converti en *short* 0 et affecté à la variable *unShort* :

```
short unShort = (short)131072
```

Conseil de génie logiciel

Il est préférable de spécifier explicitement les conversions dans les expressions pour clarifier le comportement désiré.

Exemple. L'expression suivante

```
3.4 + 7
```

est équivalente à

```
3.4 + (double)7
```

La deuxième formulation est préférable car elle clarifie la conversion désirée.

4.3 Expressions booléennes

Le type *boolean* comprend deux littéraux, *true* et *false*. Ce type est souvent utilisé dans les conditions des énoncés *if* et des énoncés de boucle (*while*, *for*, *do*). Nous avons vu comment formuler des expressions booléennes simples avec les opérateurs de comparaison (<, ==, >, etc.) dans le chapitre 3. Il est aussi possible de formuler des expressions booléennes complexes avec les opérateurs logiques (et, ou et négation) du tableau suivant :

Opérateur logique	Signification
&	et
	ou
!	négation

Exemple. JavaPasAPas/chapitre_3/ExempleLogique.java

Le programme suivant illustre les opérateurs logiques &, | et !.

```
/**
 * ExempleLogique.java
 * Petit exemple illustrant l'énoncé if.
 */
import javax.swing.JOptionPane;
public class ExempleLogique{
    public static void main (String[] args) {
        String unString = JOptionPane.showInputDialog("Entrez un premier
nombre entier");
        int unInt = Integer.parseInt(unString);
        if (unInt > 10 & unInt < 20) {
JOptionPane.showMessageDialog(null,unInt + " est entre 10 et 20"); }
        if (unInt == 100 | unInt == 200) {
JOptionPane.showMessageDialog(null,unInt + " est 100 ou 200"); }
        if (!(unInt > 30)) { JOptionPane.showMessageDialog(null,unInt + "
n'est pas plus grand que 30"); }
        System.exit(0);
    }
}
```

La condition

```
(unInt > 10 & unInt < 20)
```

est évaluée à vrai si *unInt* est à la fois plus grand que 10 et plus petit que 20. Lorsque l'on utilise un &, il faut que les deux parties de la condition soient vraies pour que le résultat soit vrai.

La condition

```
if (unInt == 100 | unInt == 200)
```

est évaluée à vrai si *unInt* est égal à 10 ou *unInt* est égal à 20. Une seule des deux parties doit être vraie pour que le résultat soit vrai. Le ou n'est pas exclusif, c'est-à-dire que si les deux parties sont vraies, le résultat est vrai.

La condition

```
!(unInt > 30)
```

est évaluée à vrai si *inInt* > 30 est faux, donc si *inInt* <= 30.

Java inclut aussi le && et le ||. Le && est une variante du & qui court-circuite l'évaluation lorsque possible. Le & évalue toujours les deux parties de la condition alors que le && n'évalue pas la deuxième partie si la première est fausse. On dit que l'évaluation est court-circuitée. En effet, si la première partie est fausse, le résultat de la condition sera nécessairement faux. Il n'est donc pas nécessaire d'évaluer la deuxième partie.

Opérateur logique	Signification
&&	et (évaluation court-circuité)
	ou (évaluation court-circuité)

Exemple. L'expression

$$2 > 3 \ \&\& \ 2 < 10$$

donne *false*. Comme $2 > 3$ donne *false*, il n'est pas nécessaire d'évaluer la deuxième partie $2 < 10$. Le && évite d'évaluer la seconde partie, $2 < 10$. Dans la plupart des circonstances, le résultat de & et && est identique. Cependant, si l'évaluation de la partie droite peut avoir un effet autre que l'évaluation de la condition elle-même, le résultat ne sera pas toujours le même.

Le programme suivant vérifie si l'entier saisi est un diviseur de 10. Il vérifie si l'entier est non nul et si le reste après division entière est nul. Dans le cas où l'entier est nul, il ne faudrait pas effectuer la division qui provoquerait une exception en Java et l'arrêt du programme. En utilisant le &&, la division ne sera pas effectuée.

Exemple. [JavaPasAPas/chapitre_3/ExempleEtCourtccircuite.java](#)

```
/**
 * ExempleEtCourtccircuite.java
 * Petit exemple illustrant l'énoncé if.
 */
import javax.swing.JOptionPane;
public class ExempleEtCourtccircuite{
    public static void main (String[] args) {
        String unString = JOptionPane.showInputDialog(
            "Entrez un premier nombre entier");
        int unInt = Integer.parseInt(unString);

        // Exemple de l'opérateur &
        if (unInt != 0 && 10 % unInt == 0)
            JOptionPane.showMessageDialog(null,unInt
                + " est un diviseur de 10");
        else
            JOptionPane.showMessageDialog(null,unInt
                + " n'est pas un diviseur de 10");
        System.exit(0);
    }
}
```

Dans le cas du `||`, si la première partie est vraie, la deuxième partie de la condition n'est pas évaluée, car le résultat doit être vrai. L'évaluation se fait toujours de la gauche vers la droite. La première condition à gauche est toujours évaluée en premier.

4.4 Traitement de caractères

Le type *char* correspond souvent en pratique à l'ensemble des caractères. Cet ensemble est défini par le standard Unicode (www.unicode.org). Les codes sont énumérés dans

<http://www.unicode.org/Public/UNIDATA/UnicodeData.txt>

Plus précisément, un *char* en Java permet de représenter n'importe quel caractère Unicode pouvant être représenté avec un mot de 16 bits dans la norme UTF-16. Certains caractères spécialisés, comme les émojis, requièrent 32 bits et donc deux mots de 16 bits (deux *char*). La norme Unicode permet aussi de combiner plusieurs caractères pour former un seul caractère visible (par exemple, les émojis « famille »).

On peut aussi voir le type *char* comme une valeur entière entre 0 et 65536. Le type *char* occupe autant d'espace en mémoire que le type *short*, mais le type *short* est utilisé pour représenter les valeurs entières entre -32768 et 32767.

La Figure 15 montre des exemples de littéraux du type *char*. Un caractère imprimable est représenté entre apostrophes. On peut aussi employer le code Unicode du caractère selon le format :

```
'\uyyyy'
```

où *yyyy* est une suite de quatre chiffres hexadécimaux qui correspond au code Unicode hexadécimal du caractère. Le `\u` est une *séquence* dite *d'échappement* qui altère l'interprétation normale par le compilateur Java. La séquence indique au compilateur que ce qui suit est le code Unicode du caractère et non pas le caractère lui-même. Le tableau suivant montre d'autres séquences d'échappement prévues en Java.

Séquence d'échappement	Code Unicode	Description
<code>\b</code>	<code>\u0008</code>	backspace BS
<code>\t</code>	<code>\u0009</code>	tabulation HT
<code>\n</code>	<code>\u000a</code>	saut de ligne LF
<code>\f</code>	<code>\u000c</code>	saut de page FF
<code>\r</code>	<code>\u000d</code>	retour de chariot CR
<code>\"</code>	<code>\u0022</code>	guillemets "
<code>\'</code>	<code>\u0027</code>	apostrophe '
<code>\\</code>	<code>\u005c</code>	backslash \

4.4.1 Type *String*, objets et classes

Le type *char* ne permet que de manipuler un caractère à la fois. Il est souvent nécessaire de manipuler une chaîne de caractères, par exemple pour afficher un message ou pour saisir une donnée. Java inclut le type **String** à cet effet. Le type **String** est aussi un type prédéfini mais n'est pas un type primitif. En fait, **String**, dont le nom complet est *java.lang.String*, est une classe Java qui fait partie du package *java.lang*. Le type d'une variable peut aussi être une classe. Dans ce cas, une *valeur* de la variable est une référence à un *objet* de la classe. Par analogie à un type primitif qui correspond à un ensemble de valeurs, une classe correspond à un ensemble d'objets. La notion d'objet est

fondamentale en Java ainsi que dans les autres langages objet. Par analogie avec les types primitifs, on peut dire qu'un objet est analogue à une valeur possible pour une classe. La différence entre une valeur d'un type primitif et une référence à un objet d'une classe n'est pas apparente dans les exemples que nous avons rencontrés jusqu'à présent. Nous allons maintenant faire ressortir certains aspects fondamentaux qui distinguent les objets (de classes) et les valeurs (de types primitifs).

- **Constructeur d'objet**

Pour créer un nouvel objet, il faut normalement employer un *constructeur d'objet*.

L'exemple de programme suivant permet d'illustrer concrètement la notion d'objet et de constructeur d'objet pour la classe **String**.

Exemple. [JavaPasAPas/chapitre_3/ExempleCreationObjetString.java](#)

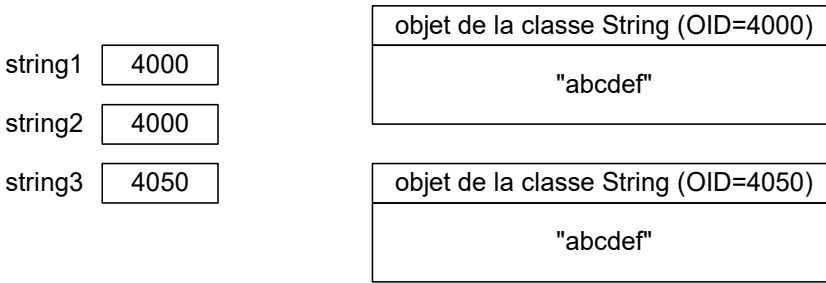
```
public class ExempleCreationObjetString{
    public static void main (String[] args) {
        String string1 = new String("abcdef");
        String string2 = string1;
        String string3 = new String("abcdef");

        System.out.println(string1 == string2); //true
        // string1 et string3 sont deux objets différents
        System.out.println(string1 == string3); //false
        // par contre, string1 et string3 ont le même contenu
        System.out.println(string1.equals(string3)); //true
    }
}
```

Après les trois affectations

```
String string1 = new String("abcdef");
String string2 = string1;
String string3 = new String("abcdef");
```

le résultat suivant est produit :



L'appel `new String("abcdef")` dans la ligne suivante

```
String string1 = new String("abcdef");
```

crée un objet de la classe **String** dont le contenu est "abcdef". L'objet contient la chaîne de caractères "abcdef". Il *contient* la chaîne mais n'est pas la chaîne ! Lorsqu'un objet est créé, un identifiant d'objet (OID) lui est assigné automatiquement. Dans notre exemple, l'objet crée a l'OID = 4000. Cette valeur n'est donnée qu'à titre d'exemple et n'a pas d'importance en soi. On ne doit pas se préoccuper de la manière dont les OID sont générés. En fait, l'OID n'est pas visible dans le programme Java et n'est pas manipulable directement. Un OID est en quelque sorte une adresse pour retrouver un objet. Il est analogue à un numéro d'assurance sociale pour un citoyen. Le numéro en soi n'a pas d'importance. Ce qui compte, c'est qu'il permet d'identifier un citoyen sans ambiguïté. L'adresse en mémoire centrale est une manière de réaliser un OID, mais il y a aussi d'autres implémentations possibles.

Dans l'énoncé

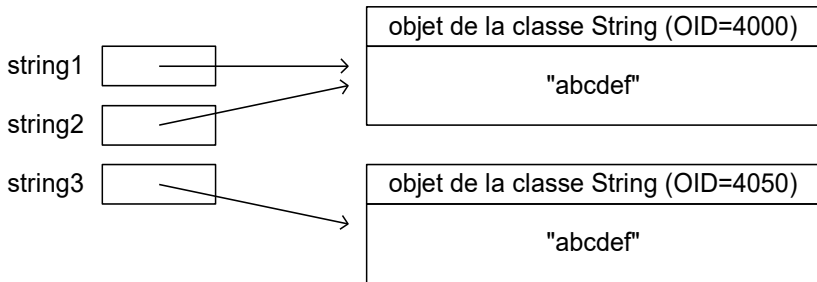
```
String string1 = new String("abcdef");
```

l'objet de la classe **String** créé dans la partie droite est affecté à la variable *string1* de la partie gauche. Le type de *string1* doit être le même que celui de l'objet créé²³. C'est pourquoi, le type de *string1* est **String**. Il est fréquent de rencontrer en Java ce genre d'énoncé où une variable de type *ClasseX* est déclarée et on lui affecte un objet de type *ClasseX* créé par `new ClasseX()`.

Lorsqu'un objet est affecté à une variable, c'est l'OID de l'objet qui est placé dans la variable. On dit alors que la variable contient une *référence* à l'objet.

²³ Nous verrons plus loin que ce n'est pas nécessairement le même type dans certaines circonstances

Souvent les références sont représentées graphiquement par des flèches tel qu'illustré dans la figure suivante car les valeurs exactes des OID sont sans importance. Ce qui compte, c'est que la variable fasse référence au bon objet.



Un objet est créé avec un constructeur d'objet. Un constructeur d'objet est une méthode spéciale dont le rôle est de créer un objet d'une classe. Il est appelé en utilisant la syntaxe suivante :

appel de constructeur d'objet :



Un constructeur porte le même nom que la classe. De ce point de vue, une classe est comme un moule à objet. La classe **String** est donc un moule pour construire des objets de type **String**.

Une valeur de type **String** ne peut pas être modifiée. Une fois que la valeur « abcdef » a été assignée à la variable, on ne peut plus changer la chaîne. C'est un choix spécifique à cette classe puisqu'il aurait été possible pour les créateurs du Java de faire en sorte que la classe **String** puisse modifier son contenu.

La ligne

```
String string2 = string1;
```

affecte le contenu de *string1* à *string2*. Ceci ne copie pas l'objet mais plutôt l'OID de l'objet. En conséquence, les deux variables font maintenant référence au même objet !

La ligne

```
String string3 = new String("abcdef");
```

crée un **autre** objet de la classe **String**, dont l'OID = 4050. Cet autre objet contient aussi "abcdef" mais c'est un objet différent du premier !

Par opposition aux objets, il n'y a pas de distinction entre une valeur et son contenant pour les types primitifs. Comment fait-on la différence entre la référence à l'objet et le contenu de l'objet dans un programme ? La réponse à cette question est illustrée par le reste du code du programme.

Dans le cas d'objets, le « == » Java compare les références aux objets et non pas le contenu des objets. Ainsi le test `string1 == string2` dans

```
System.out.println(string1 == string2); //true
```

produit la valeur *true* car les deux variables font référence au même objet mais `string1 == string3` dans

```
System.out.println(string1 == string3); //false
```

est *false* car les deux variables `string1` et `string3` font référence à des objets différents ! Pour comparer le contenu des objets **String**, on peut utiliser la méthode `equals()` de la classe **String**. Ainsi le test `string1.equals(string3)` dans

```
System.out.println(string1.equals(string3)); //true
```

produit la valeur *true* parce que le contenu des deux objets est le même.

Attention !

Une erreur fréquente en Java est de confondre `==` et `equals()`.

La méthode `equals()` de la classe **String** est une méthode d'objet. La possibilité d'appeler des méthodes sur les objets est un autre aspect qui les distingue des valeurs des types primitifs. Nous avons déjà dit qu'une classe regroupe un ensemble de méthodes. Parmi ces méthodes, il y a des méthodes de classe et des méthodes d'objets. On ne peut appeler une méthode de classe sur un objet ou une méthode d'objet sur une classe.

- **Documentation des classes et méthodes**

Un aspect important de la programmation Java est le fait qu'un grand nombre de classes et de méthodes sont déjà définies et mises à la disposition

du programmeur. Le programmeur doit pouvoir facilement retrouver les méthodes et les classes. À cet effet, le programmeur peut consulter la documentation des classes et méthodes prédéfinies. Cette documentation est accessible sur le site de Oracle. Pour la version 8, vous pouvez y accéder par :

<https://docs.oracle.com/en/java/javase/21/docs/api/>

Cette documentation est sous forme HTML, et elle peut être consultée à partir d'un fureteur Web. Notez que même si les versions de Java se succèdent rapidement, il est souvent possible de tout faire avec les classes de la version 8. Il y a d'ailleurs des bénéfices à ne pas trop rapidement adopter des fonctions et des classes qui ne sont disponibles qu'avec des versions récentes du Java.

Exercice. À ce point-ci, vous devriez vous familiariser un peu avec cette documentation en cherchant la classe **String**. Vous pouvez la retrouver dans la liste *All Classes* du panneau inférieur gauche. Cliquez sur **METHOD** de la rubrique **SUMMARY NESTED** dans le panneau de droite en haut et vous obtenez la liste des méthodes de la classe **String**.

- La documentation montre pour chacune des méthodes, le type de ce qui est retourné sous la première colonne du tableau *Method summary*.
 - L'identificateur réservé *void* signifie que la méthode ne retourne rien.
- Les méthodes de classe sont distinguées des méthodes d'objet par l'identificateur réservé *static* qui apparaît avant le type de ce qui est retourné. Par exemple, la méthode **copyValueOf(char[] data)** est une méthode de classe alors que **charAt(int index)** est une méthode d'objet.
- Dans la deuxième colonne du tableau *Method Summary*, la liste des paramètres apparaît après le nom de la méthode. Les paramètres sont séparés par des virgules.
 - Pour chacun des paramètres, il y a son type suivi d'un nom de paramètre. Le nom n'a pas d'importance comme tel.

Lorsqu'on appelle la méthode, les paramètres doivent apparaître dans le même ordre et doivent être du bon type.

La classe **String** inclut plusieurs autres méthodes d'objet visant la manipulation de chaînes de caractères.

Exemple. Le programme suivant illustre quelques méthodes de la classe **String**.

```
import javax.swing.JOptionPane; // Importe la classe
javax.swing.JOptionPane
public class ExempleMethodesDeString{

    public static void main (String[] args) {

        String string1 = new String("abcdef");
        String string2 = new String("cd");
        System.out.println("String string1 = new String(\"abcdef\")");
        System.out.println("String string2 = new String(\"cd\")");
        System.out.println("La longueur de string1 est : "
            + string1.length());
        System.out.println("Le caractère en position 2 de string1 est : "
            + string1.charAt(2));
        System.out.println("La sous-chaîne en position 2 de string1 est : "
            + string1.substring(2));
        System.out.println(
            "La sous-chaîne qui débute en position 2 et finit en 4 est : "
            + string1.substring(2,5));
        System.out.println(
            "La première occurrence de string2 dans string1 est à la "
            + "position : "
            + string1.indexOf(string2));
        System.out.println("La concaténation de string2 à string1 donne : "
            + string1.concat(string2));
    }
}
```

Résultat affiché :

```
String string1 = new String("abcdef")
String string2 = new String("cd")
La longueur de string1 est :6
Le caractère en position 2 de string1 est :c
La sous-chaîne en position 2 de string1 est :cdef
La sous-chaîne qui débute en position 2 et finit en 4 est :cde
La première occurrence de string2 dans string1 est à la position :2
La concaténation de string2 à string1 donne :abcdefcd
```

Notez dans la ligne suivante l'utilisation de la séquence d'échappement `\`. Ceci est nécessaire pour inclure un guillemet dans une chaîne de caractères étant donné que le guillemet est aussi le délimiteur de fin de chaîne.

La méthode `length()` retourne la taille d'un **String**. Elle compte le nombre de mots de 16 bits contenu dans la chaîne de caractères. Dans plusieurs cas, le résultat de la méthode donne le nombre de caractères UTF-16 : tant qu'un caractère n'appartient à un plan supplémentaire. Les plans supplémentaires comprennent les emojis, les symboles mathématiques et musicaux et autres symboles spécialisés.

```
System.out.println("La longueur de string1 est :"  
+ string1.length());
```

L'appel `string1.charAt(2)` retourne le *caractère* (ou plutôt le mot de 16 bits) en position 2 de `string1`, ce qui correspond au **troisième** caractère de "abcdef", c'est-à-dire le caractère « c » car les positions des caractères sont numérotées à partir de la position 0 !

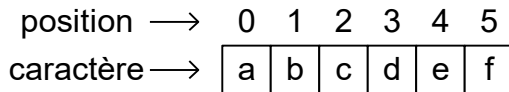


Figure 16. Numérotation des positions des caractères à partir de 0 !

Le fait d'inclure 0 comme indice peut sembler étrange à un non-initié ...

L'appel `string1.substring(2)` de la ligne suivante retourne un objet de la classe **String** qui contient la sous-chaîne qui débute en position 2 (troisième caractère).

```
System.out.println("La sous-chaîne en position 2 de string1 est :"  
+ string1.substring(2));
```

L'appel `string1.substring(2,5)` de la ligne suivante retourne un objet de la classe **String** qui contient la sous-chaîne qui débute en position 2 (troisième caractère) et se termine en position 5 (excluant la position 5, ce qui correspond au cinquième caractère).

```
System.out.println(
    "La sous-chaîne qui débute en position 2 et finit en 4 est : "
    + string1.substring(2,5));
```

A noter que les deux appels utilisent le même nom de méthode mais avec des paramètres différents ! En réalité, ces deux appels invoquent deux méthodes différentes. D'ailleurs, dans l'extrait suivant de la documentation, la méthode *substring()* apparaît deux fois.

String	substring(int beginIndex) Returns a string that is a substring of this string.
String	substring(int beginIndex, int endIndex) Returns a string that is a substring of this string.

Ces deux méthodes de même nom désignent en réalité deux méthodes différentes. Le compilateur peut distinguer les méthodes de même nom par le fait que la nature des paramètres est différente. Dans **substring(int beginIndex)**, il n'y a qu'un paramètre de type *int*. Cette méthode retourne un objet **String** dont la chaîne est la sous-chaîne qui débute à la position *beginIndex* et termine au dernier caractère du **String**. Dans **substring(int beginIndex, int endIndex)**, il y a deux paramètres et la méthode retourne un objet **String** qui débute à la position *beginIndex* et se termine à la position *endIndex*. Ce sont bien deux méthodes différentes mais apparentées.

Surcharge d'un nom de méthode

Le fait d'employer le même nom pour désigner plusieurs méthodes différentes distinguées par la nature des paramètres est appelé la *surcharge* des noms de méthodes.

Ensuite, l'appel *string1.indexOf(string2)* retourne la position de la première occurrence de la chaîne de *string1* dans *string2*.

```
System.out.println(
    "La première occurrence de string2 dans string1 est à la "
    + "position : "
    + string1.indexOf(string2));
```

Enfin, l'appel `string1.concat(string2)` retourne un objet **String** formé de la concaténation de la chaîne de `string2` à la fin de celle de `string1`.

```
System.out.println("La concaténation de string2 à string1 donne : "
    + string1.concat(string2));
```

Exercice. Lire un **String**, en afficher la taille, et compter le nombre d'occurrences de la lettre «a» dans le **String**.

- **Littéral *String***

À cause de l'importance de la manipulation de chaînes de caractères, Java prévoit un raccourci pour la création d'objets de la classe **String**. Plutôt que d'utiliser un constructeur pour créer un objet de la classe **String**, Java permet d'utiliser un *littéral String*. Un littéral **String** est une séquence de caractères entre guillemets.

Le littéral est en fait un raccourci pour désigner un objet de la classe **String**. Par exemple, il est permis d'écrire le code suivant :

```
String string1 = "abc";
```

Cette utilisation de littéraux donne l'illusion que **String** est un type primitif. Mais cette vue n'est qu'une partie du portrait car un littéral **String** désigne un objet. Cet énoncé assigne un objet de la classe **String** qui contient la suite de caractères "abc" à la variable `string1`. Ceci produit presque le même effet que :

```
String string1 = new String("abc");
```

L'effet de ces deux énoncés n'est cependant pas exactement le même mais dans la majorité des cas, la différence n'est pas importante. L'exemple suivant illustre la différence.

L'exemple de programme suivant illustre quelques subtilités concernant l'usage de littéraux **String**. Le principe important à saisir est le suivant : l'utilisation d'un littéral qui est connu à la compilation du programme est

toujours remplacé par une référence à un objet de la classe **String** qui contient la chaîne de caractère du littéral. S'il y a plusieurs occurrences du même littéral, disons "abc", c'est toujours le même objet qui est utilisé. Ceci permet une certaine économie de mémoire car il n'est pas nécessaire de créer un objet différent à chaque utilisation du même littéral. Cependant, si un objet est créé à l'exécution (par *new* par exemple), ce ne sera pas le même objet !

Exemple. `JavaPasAPas/chapitre_3/ExemplesString.java`

```
public class ExemplesString{

    public static void main (String[] args) {

        String string1 = "abc";
        String string2 = "def";
        String string3 = "abcdef";
        String string4 = new String("abcdef");

        // Tous les littéraux identiques (à la compilation) sont traduits
        // par une référence au même objet
        System.out.println(string3 == "abcdef"); // true
        System.out.println("abc"+"def" == "abcdef"); //true

        // Cependant, si le littéral est calculé à l'exécution, ce n'est pas
        // le cas
        System.out.println(string1 + string2 == "abcdef"); //false

        // Le constructeur String produit toujours un objet différent de
        // l'objet
        // correspondant au littéral
        System.out.println(string4 == "abcdef"); //false

        // La méthode intern() de la classe String permet de convertir
        // la référence à l'objet correspondant au littéral
        System.out.println((string1 + string2).intern() == "abcdef"); //true
        System.out.println(string4.intern() == "abcdef"); //true

        // La méthode equals() permet de comparer le contenu de l'objet
        // plutôt que la référence
        System.out.println((string1 + string2).equals("abcdef")); //true
        System.out.println(string4.equals("abcdef")); //true
    }
}
```

La condition dans

```
System.out.println(string3 == "abcdef"); // true
```

retourne true car *string3* fait référence au même objet que "abcdef".

La condition dans

```
System.out.println("abc"+"def" == "abcdef"); //true
```

retourne aussi true car la concaténation "abc"+"def" est calculée au moment de la compilation, ce qui produit le littéral "abcdef".

La condition dans

```
System.out.println(string1 + string2 == "abcdef"); //false
```

retourne false car la concaténation *string1* + *string2* ne peut être calculée à la compilation. Donc, l'objet créé ne sera pas le même que l'objet correspondant au littéral "abcdef".

La condition

```
System.out.println(string4 == "abcdef"); //false
```

retourne false car le constructeur *String()* produit toujours un objet distinct et donc différent de l'objet correspondant au littéral.

Les deux conditions dans

```
System.out.println((string1 + string2).intern() == "abcdef"); //true  
System.out.println(string4.intern() == "abcdef"); //true
```

retournent true car la méthode *intern()* convertit la référence à une référence à l'objet correspondant au littéral "abcdef".

Les deux conditions dans

```
System.out.println((string1 + string2).equals("abcdef")); //true  
System.out.println(string4.equals("abcdef")); //true
```

retournent *true* car ce n'est pas la référence à l'objet qui est comparée mais bien le contenu, c'est-à-dire la chaîne de caractère elle-même. La morale de cet exemple : il est habituellement préférable de comparer les **String** avec *equals()*.

Un autre aspect qui porte souvent à confusion pour un novice est la distinction entre

- le littéral qui représente la chaîne vide ""

- le littéral *null*

L'exemple suivant illustre la différence entre ces concepts.

Exemple. [JavaPasAPas/chapitre_3/ExemplesStringVide.java](#)

```
public class ExemplesStringVide{

    public static void main (String[] args) {

        String string1 = "";
        String string2 = new String("");
        String string3 = new String();
        String string4 = null;

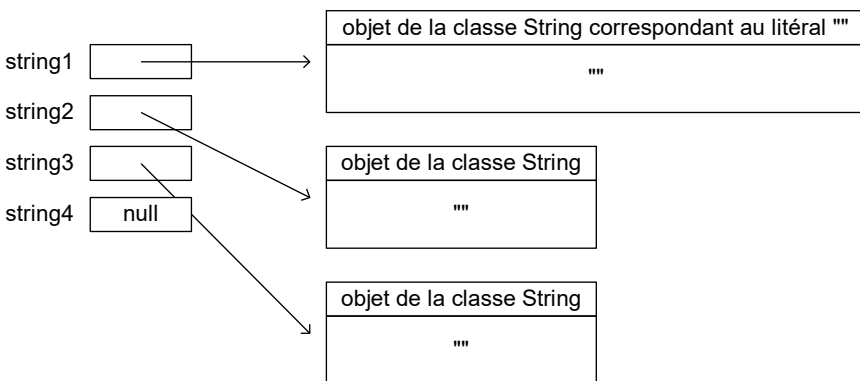
        System.out.println(string1 == ""); // true
        System.out.println(string1.equals("")); //true
        System.out.println(string1 == null); //false

        System.out.println(string2 == ""); // false
        System.out.println(string2.equals("")); //true
        System.out.println(string2 == null); //false

        System.out.println(string3 == ""); // false
        System.out.println(string3.equals("")); //true
        System.out.println(string3 == null); //false

        System.out.println(string4 == ""); //false
        // System.out.println(string4.equals("")); provoquerait une
        exception
        // à l'exécution
        System.out.println(string4 == null); //true
    }
}
```

La figure suivante montre l'effet du programme.



String1 fait référence à l'objet **String** qui correspond au littéral de la chaîne vide `""`. *String2* fait référence à un autre objet qui contient aussi la chaîne vide. Il en est de même pour *string3*. Ceci signifie que le constructeur de *String* sans paramètre *String()* initialise automatiquement son contenu à la chaîne vide. Enfin, *string4* contient la référence *null*. Le littéral spécial *null* signifie que la variable ne fait référence à aucun objet. L'expression *string4==null* permet de détecter cette situation. Lorsque l'objet fait référence à *null*, l'accès à son contenu provoque une exception Java à l'exécution. Ce serait le cas de l'énoncé suivant car la méthode *equals()* doit extraire le contenu de l'objet *string4* mais cet objet n'existe pas !

```
// System.out.println(string4.equals("")); provoquerait une exception
// à l'exécution
```

Enfin, notons que l'accès à une variable non initialisée provoque une erreur de compilation. Java vous protège contre l'accès aux variables non initialisées parce que de tels accès sont souvent la source de problèmes et de bogues.

En Java, nous distinguons la déclaration d'une variable (sans initialisation) et son initialisation (ou attribution de valeur). Avant de pouvoir utiliser une variable, celle-ci doit avoir été initialisée. Au sein d'une fonction, la simple déclaration d'une variable ne suffit par à l'initialiser et Java refusera l'accès à une variable non-initialisée.

```
Exemple. JavaPasAPas/chapitre_3/ExempleStringNonInitialise.java
public class ExempleStringNonInitialise{
    public static void main (String[] args) {
        String unString;
        System.out.println(unString == null); // erreur de compilation car
non
        // initialisé
    }
}
```

Java représente ses caractères sous le format UTF-16. La notion de caractère au sein d'une chaîne de caractères en Java présume que tous les caractères occupent 16 bits. Or les caractères des plans supplémentaires (comme les emojis) occupent 32 bits. Le code suivant va donc afficher 8 pour la longueur de la chaîne et va retourner un caractère invalide après l'appel à *charAt*. Le

traitement des chaînes de caractères dans de tels cas peut se faire avec les méthodes *codePointAt* et *offsetByCodePoints*. Il s'agit d'un sujet avancé.

Exemple. [JavaPasAPas/chapitre_3/ExempleEmoji.java](#)

```
public class ExempleEmoji {
    public static void main(String[] args) {
        String s = "😄😄👉👍";
        System.out.println(s.length());
        System.out.println(s.charAt(1));
    }
}
```

4.5 Fonctions mathématiques : *java.lang.Math*

Au-delà des opérations de base permises dans les expressions arithmétiques, le package *java.lang.Math* contient plusieurs méthodes pour le calcul de fonctions mathématiques courantes.

L'exemple suivant montre quelques exemples de fonctions mathématiques usuelles.

Exemple. [JavaPasAPas/chapitre_3/ExemplesMath.java](#)

```
public class ExemplesMath {

    public static void main (String[] args) {
        System.out.println("Math.log(1.0)="+Math.log(1.0));
        System.out.println("Math.exp(1.0)="+Math.exp(1.0));
        System.out.println("Math.cos(0)="+Math.cos(0));
        System.out.println("Math.sin(0)="+Math.sin(0));
        System.out.println("Math.sqrt(4)="+Math.sqrt(4));
    }
}
```

Résultat affiché :

```
Math.log(1.0)=0.0
Math.exp(1.0)=2.718281828459045
Math.cos(0)=1.0
Math.sin(0)=0.0
Math.sqrt(4)=2.0
```

Les classes *java.math.BigInteger* (pour les entiers) et *java.math.BigDecimal* (pour les nombres décimaux) permettent de traiter des nombres d'une précision plus grande que ce qui est permis avec les types primitifs.

4.6 Sommaire des opérations et priorités

Le tableau suivant montre la liste des opérations pour les expressions Java en ordre décroissant de priorité.

Opérateur	
++ --	Post-incrémentation Post-décrémentation
++ -- + - ! ~ (type)	Pré-incrémentation Pré-décrémentation + unaire - unaire négation logique complément (niveau bit) conversion de type
* / %	Multiplication binaire Division Reste après division entière
+ -	Addition binaire Soustraction binaire
<< >> >>>	Décalage à gauche (niveau bit) Décalage à droite (niveau bit) Décalage à droite sans signe (niveau bit)
< > <= >= instanceof	Plus petit que Plus grand que Plus petit ou égal à Plus grand ou égal à Est une instance de
== !=	Est égal à Est différent de
&	Et (niveau bit / logique)
^	Ou exclusif (niveau bit / logique)
	Ou inclusif (niveau bit / logique)
&&	Et logique
	Ou logique
?:	Expression conditionnelle

Opérateur	
=	Affectation
+=	Auto-addition
-=	Auto-soustraction
/=	Auto-division
%=	Auto-reste
^=	Auto-ou-exclusif (niveau bit)
=	Auto-ou (niveau bit)
<<=	Auto-décalage à gauche (niveau bit)
>>=	Auto-décalage à droite (niveau bit)
>>>=	Auto-décalage à droite sans signe (niveau bit)

5. Graphisme 2D et concepts de programmation objet

« *La meilleure façon de prédire l'avenir est de l'inventer.* » — *Alan Kay*

Ce chapitre examine les principaux mécanismes de dessins en deux dimensions (2D) de Java et approfondit les concepts de programmation objet suivants : sous-classe, interface, variables de classe et d'objet, méthode de classe et d'objet.

5.1 Dessin avec les classes *Graphics* et une sous-classe de *JFrame*

Étudions d'abord un exemple de programme qui dessine un crabe simple dans une fenêtre.

Le programme *ExempleDessin2DDansJFrame.java* dessine un crabe, l'un des envahisseurs du jeu que nous développerons au chapitre 8, dans une fenêtre produite par la classe *javax.swing.JFrame*.

Exemple. [JavaPasAPas/chapitre_4/ExempleDessin2DDansJFrame.java](#)

```
import java.awt.*;
import javax.swing.JFrame;

public class ExempleDessin2DDansJFrame extends JFrame {

    public ExempleDessin2DDansJFrame() {
        super("Exemples de dessin avec les méthodes de Graphics");
    }
}
```



```
this.setDefaultCloseOperation(EXIT_ON_CLOSE);
this.setSize(400, 400);
this.setVisible(true);
}

// La méthode paint() est appelée automatiquement lors de la création
// du JFrame
// La méthode paint() fait le dessin d'un crabe
public void paint(Graphics g) {

    // Il faut appeler la méthode paint() de la super-classe
    super.paint(g);

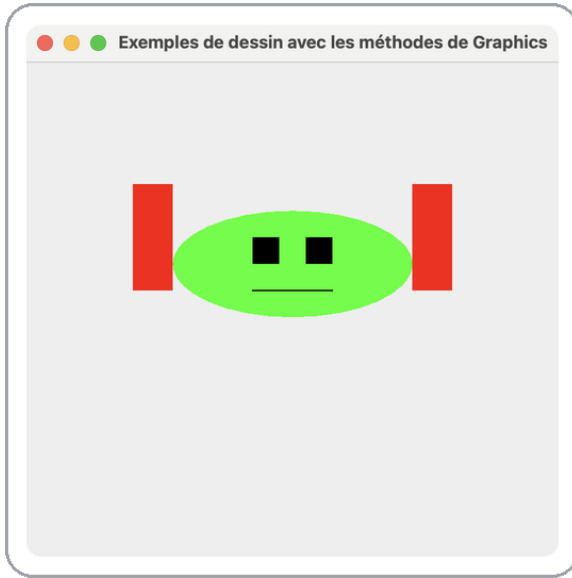
    g.setColor(Color.green);
    g.fillOval(110, 140, 180, 80); // Le corps

    g.setColor(Color.black);
    g.fillRect(170, 160, 20, 20); // L'oeil gauche
    g.fillRect(210, 160, 20, 20); // L'oeil droit
    g.drawLine(170, 200, 230, 200); // La bouche

    g.setColor(Color.red);
    g.fillRect(80, 120, 30, 80); // La pince gauche
    g.fillRect(290, 120, 30, 80); // La pince droite
}

public static void main(String[] args) {
    new ExempleDessin2DDansJFrame();
}
}
```

Voici le résultat de l'exécution du programme :



Le dessin est effectué dans une fenêtre graphique qui correspond à un espace à deux dimensions (2D) illustré à la figure suivante. L'axe des x est l'axe horizontal et l'axe des y , le vertical. Par opposition à la convention mathématique usuelle, l'axe des y est orienté vers le bas. Les figures graphiques font référence aux coordonnées de cet espace.

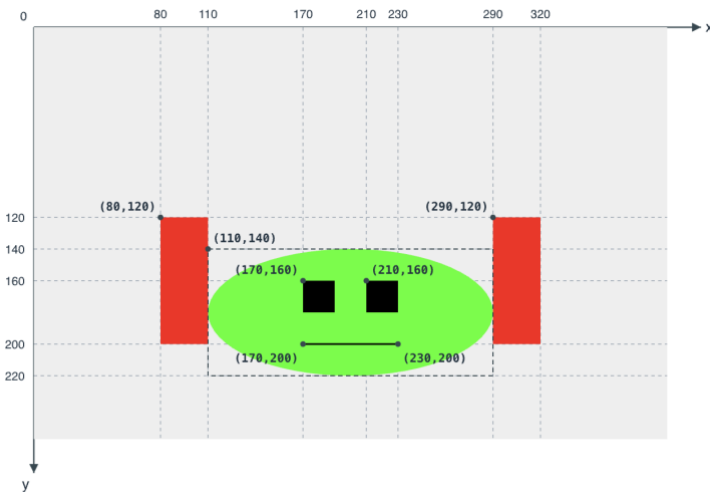


Figure 17. Coordonnées du crabe.

Certains aspects sembleront flous à ce point-ci et seront détaillés par la suite. La clause `import java.awt.*` apparaît en début de programme étant donné que les classes `java.awt.Graphics` et `java.awt.Color` sont utilisées. Le `*` permet d'importer toutes les classes du package `java.awt` sans devoir spécifier chacune des classes individuellement.

```
import java.awt.*;
```

La méthode `main()` ne fait que créer un objet de la classe `ExempleDessin2DDansJFrame` par :

```
new ExempleDessin2DDansJFrame();
```

Cet objet représente la fenêtre dans laquelle est effectué le dessin.

- **Notion de sous-classe**

La classe `ExempleDessin2DDansJFrame` est une sous-classe de la classe `JFrame`. Ceci est exprimé par la clause `extends JFrame` dans la ligne suivante.

```
public class ExempleDessin2DDansJFrame extends JFrame {
```

Inversement, on dit que `JFrame` est la super-classe de `ExempleDessin2DDansJFrame`. La figure suivante montre un diagramme de classe UML qui illustre ce concept. Un tel diagramme est utile pour comprendre l'organisation des classes d'un programme Java. Chacune des classes est représentée par un rectangle. Le nom de la classe apparaît dans la partie supérieure, et les méthodes de la classe dans la partie inférieure. La flèche représente une relation de sous-classe (aussi appelée relation d'héritage ou encore de *généralisation / spécialisation*). À noter que le diagramme est partiel. La classe `JFrame` contient beaucoup plus de méthodes et elle est elle-même sous-classe d'une autre classe.

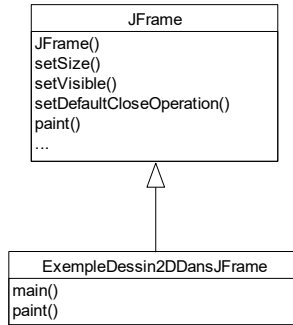


Figure 18. Représentation d'une sous-classe en UML.

En définissant une classe *X* comme une sous-classe d'une autre classe *Y*, la classe *X* hérite de toutes les méthodes définies dans la classe *Y*. Ainsi, en définissant la classe *ExempleDessin2DDansJFrame* comme une sous-classe de *JFrame* à l'aide de l'utilisation de la directive *extends*, la classe *ExempleDessin2DDansJFrame* hérite d'un ensemble de méthodes définies dans la classe *JFrame* pour la manipulation des fenêtres. Ceci entraîne qu'un objet de la classe *ExempleDessin2DDansJFrame* est aussi considéré comme un objet de la super-classe *JFrame*. Il s'agit d'un exemple de ce que nous appelons parfois du *polymorphisme* : une instance de la classe *ExempleDessin2DDansJFrame* est aussi une instance de la classe *JFrame* ce qui implique qu'une classe peut prendre plusieurs formes (littéralement : poly [plusieurs] morph [forme]). Une sous-classe est parfois appelée une « classe héritée » ou une « classe dérivée ».

Dans le diagramme, on ne répète pas les méthodes héritées dans la sous-classe. Ceci est implicite. Les méthodes d'objet héritées de la classe *JFrame* peuvent être appelées sur un objet de la classe *ExempleDessin2DDansJFrame* comme si elles y avaient été définies. Le mécanisme d'héritage permet ainsi de réutiliser les méthodes de la classe *JFrame* dans la classe *ExempleDessin2DDansJFrame* sans avoir à les répéter. En particulier, dans notre exemple, les méthodes *setDefaultCloseOperation()*, *setSize()*, *setVisible()*, *paint()* sont héritées de la classe *JFrame* mais elles sont utilisées sur un objet de la sous-classe *ExempleDessin2DDansJFrame* dans notre exemple. Cette manière de programmer en créant une sous-classe d'une classe existante en réutilisant le code d'une classe existante tout en ajoutant un comportement plus spécialisé est une approche typique et très puissante de la

programmation objet. Dans notre exemple, ceci nous permet de créer des fenêtres contenant des dessins d'une manière très simple sans avoir à en programmer tous les détails.

Rappelons la syntaxe de la création d'un objet :

appel de constructeur d'objet :



Ainsi, le `new ExempleDessin2DDansJFrame()` dans la méthode `main()` crée un objet de la classe `ExempleDessin2DDansJFrame`. Concrètement, le `new` appelle une méthode dite *constructeur* d'objet qui doit porter le même nom que celui de la classe.

Dans notre exemple, le `new ExempleDessin2DDansJFrame()` provoque l'appel de la méthode `ExempleDessin2DDansJFrame()` qui est une méthode *constructeur d'objet* de la classe `ExempleDessin2DDansJFrame`. Le corps de la méthode constructeur est utilisé pour initialiser certains aspects d'un objet au moment de sa création.

La ligne suivante déclare le constructeur d'objet :

```
public ExempleDessin2DDansJFrame() {
```

La ligne suivante spécifie un titre qui apparaît dans le haut de la fenêtre.

```
super("Exemples de dessin avec les méthodes de Graphics");
```

L'identificateur réservé *super* signifie d'appeler la méthode constructeur correspondante de la super-classe `JFrame`. La méthode correspondante est le constructeur de la super-classe qui a les mêmes paramètres. Il y a donc, dans la classe `JFrame`, une méthode constructeur qui prend un titre (**String**) en paramètre. Cette manière d'appeler le constructeur d'une super-classe permet, dans une sous-classe, de compléter le travail du constructeur de la super-classe en ajoutant des aspects particuliers à la sous-classe.

La ligne suivante spécifie qu'il faut terminer le programme (provoque un appel de `System.exit(0)`) lorsque l'utilisateur clique dans le X dans le coin supérieur droit de la fenêtre. Ceci est spécifié par l'appel de méthode d'objet `setDefaultCloseOperation(int operation)` de la classe `JFrame`.

```
this.setDefaultCloseOperation(EXIT_ON_CLOSE);
```

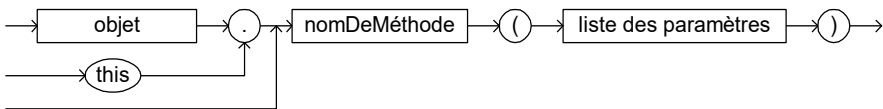
Identificateur réservé *this*

L'identificateur réservé *this* représente une référence à l'objet « *ceci* » qui est en processus de construction par le constructeur. Comme cet objet est en phase de construction, il ne peut y avoir une variable qui contient cet objet pour y faire référence. Le *this* est un identificateur réservé qui permet de désigner l'objet en phase de construction.

Le constructeur ne construit qu'un objet à la fois et il n'y a donc pas d'ambiguïté au sujet de l'objet représenté par *this*. Lorsqu'un appel de méthode est fait sans spécifier un objet en préfixe, c'est comme si on mettait le *this* en préfixe. Le *this* n'est donc pas obligatoire. Dans notre exemple, on pourrait remplacer `this.setDefaultCloseOperation(EXIT_ON_CLOSE)` par `setDefaultCloseOperation(EXIT_ON_CLOSE)`. Cependant, dans certains cas, il faut mettre le *this* pour éviter certaines ambiguïtés. Nous y reviendrons.

Voici le diagramme de syntaxe d'un appel de méthode d'objet tenant compte de ces deux nouvelles possibilités. Rappelons qu'une méthode d'objet est une méthode qu'on appelle sur un objet.

appel de méthode d'objet :



La méthode `setSize()` spécifie la largeur (400) et la hauteur (400) de la fenêtre :

```
this.setSize(400, 400);
```

La méthode `setVisible()` rend la fenêtre visible. Ceci provoque indirectement l'appel de la méthode `paint(Graphics g)` qui effectue le dessin.

```
this.setVisible(true);
```

Le dessin du crabe est alors effectué par la méthode *paint()*.

La méthode *paint()* et le multifenêtrage

L'appel à *paint()* n'est pas visible dans notre programme. Cet appel se passe au niveau de la super-classe *JFrame*. Le fait de ne pas voir les détails de cet appel est un aspect un peu surprenant de la programmation Java, surtout pour un novice. En fait, la méthode *paint()* est appelée automatiquement à chaque fois qu'il faut redessiner le contenu de la fenêtre. Ceci est nécessaire lorsqu'une partie cachée de la fenêtre est rendue visible au premier plan lors de la manipulation des fenêtres par l'utilisateur. Plusieurs opérations de manipulation de fenêtre conduisent à cette circonstance.

Donc, à chaque fois qu'on utilise la classe *JFrame*, la méthode *paint()* est employée pour afficher quelque chose à l'intérieur de la fenêtre. D'autres façons de procéder seront vues plus loin.

Regardons maintenant le détail de la méthode *paint()* qui effectue le dessin. La méthode est déclarée par :

```
public void paint (Graphics g) {
```

Dans notre code, cette méthode est une méthode d'objet de la classe *ExempleDessin2DDansJFrame*.

Rappelons le sens des identificateurs réservés *public void* :

- *public* signifie que la méthode peut être appelée de partout
- *void* signifie que la méthode ne retourne rien

L'absence de l'identificateur réservé *static* signifie que c'est une méthode d'objet.

Cette méthode existe déjà dans la super-classe ! Cependant, la méthode de la super-classe doit être redéfinie pour effectuer le dessin. Cette manière d'adapter une classe en définissant une sous-classe qui redéfinit quelques

méthodes de la super-classe afin de spécialiser le comportement de la sous-classe est typique de la programmation objet.

La méthode *paint()* a un paramètre appelé *g* dont le type est la classe *java.awt.Graphics*. Rappelons que le nom de paramètre est précisé par le programmeur et n'a pas d'importance d'un point de vue de l'exécution du programme. Ce nom de paramètre est utilisé dans le corps de la méthode pour désigner le paramètre. Le type du paramètre (**Graphics**) représente un espace 2D de dessin qui fait partie de la fenêtre. Un tel objet de la classe **Graphics** est appelé un *contexte graphique*. Les opérations de dessin dans la méthode *paint()* seront effectuées en appelant des méthodes d'objet de l'objet *g*. Lorsque la méthode *paint()* est appelée (on ne voit pas l'appel ici), l'objet **Graphics** de la fenêtre est passé en paramètre à la méthode *paint()*. Comme on ne voit pas l'appel à la méthode *paint()*, le programmeur curieux peut se sentir dans un état un peu flou ... Comment est créé l'objet **Graphics** ? Qui appelle *paint()* ?

La méthode *paint()* de la classe *ExempleDessin2DDansJFrame* appelle tout d'abord la méthode *paint()* de la super-classe **JFrame**. Notez la syntaxe spéciale *super.paint()*. Par convention Java, l'usage du préfixe *super* désigne la méthode de même nom et paramètres de la super-classe. L'appel à la méthode *paint()* de la super-classe est une convention préétablie des classes d'interface graphique de Java, car la méthode *paint()* de la super-classe effectue certaines opérations nécessaires au bon fonctionnement de fenêtres à structure complexe tel qu'une fenêtre **JFrame**.

```
// Il faut appeler la méthode paint() de la super-classe
super.paint(g);
```

Redéfinition d'une méthode par spécialisation et surcharge dynamique

Après avoir appelé le *paint()* de la super-classe, le *paint()* de la sous-classe ajoute les opérations de dessin. De ce point de vue la méthode *paint()* de la sous-classe est une spécialisation de la méthode *paint()* de la super-classe. On dit que la méthode *paint()* de la sous-classe *ExempleDessin2DDansJFrame* est une redéfinition de la méthode *paint()* de la super-classe **JFrame** par spécialisation de la méthode de la super-classe. Le résultat est qu'il y a

plusieurs méthodes *paint()* avec le même nom et les mêmes paramètres ! Comment savoir quelle méthode doit être appelée ? Java détermine la méthode appropriée en fonction du type de l'objet qui est désigné pour l'appel de la méthode. Si c'est un objet de la classe **JFrame**, c'est le *paint()* de **JFrame** qui est appelé. Si c'est un objet de la classe *ExempleDessin2DDansJFrame*, c'est le *paint()* de *ExempleDessin2DDansJFrame* qui est appelé. Ce principe est appelé *surcharge dynamique* (ou encore *polymorphisme dynamique*) d'un nom de méthode.

Identificateur réservé *super*

Lorsqu'une sous-classe redéfinit une méthode d'une super-classe, la syntaxe *super.nomMéthode()* désigne l'appel de la méthode *nomMéthode()* de la super-classe. Dans le cas d'un constructeur, il suffit d'utiliser *super* tout court pour désigner le constructeur correspondant de la super-classe, c'est-à-dire celui qui a les mêmes paramètres.

Voyons maintenant les méthodes de dessin. La ligne suivante spécifie que la couleur courante de dessin sera le vert.

```
g.setColor(Color.green);
```

La méthode **setColor(Color c)** est une méthode d'objet de la classe **Graphics** qui fixe la couleur utilisée pour les méthodes de dessin. Le paramètre *Color.green* est un objet de la classe **Color** qui correspond à une *constante de classe* définie dans la classe **Color** pour représenter la couleur verte. La notion de constante de classe sera détaillée par la suite. La classe **Color** inclut plusieurs autres constantes pour définir les principales couleurs.

La ligne suivante dessine l'ovale vert du corps du crabe.

```
g.fillOval(110, 140, 180, 80); // Le corps
```

La méthode **fillOval(int x, int y, int width, int height)** dessine un ovale plein inscrit dans un rectangle dont la coordonnée du coin supérieur gauche est (x, y) la largeur est *width* et la hauteur est *height*. Comme la largeur est plus grande que la hauteur, ceci produit une ellipse aplatie (si la

largeur était égale à la hauteur, on obtiendrait un cercle). La couleur employée est celle qui a été prédéterminée par l'appel précédent à *setColor()*.

La ligne suivante établit la couleur noire comme couleur courante.

```
g.setColor(Color.black);
```

La ligne suivante dessine un carré noir qui correspond l'œil gauche. Les paramètres de **fillRect**(int x, int y, int width, int height) ont la même signification que pour *fillOval()*. Comme la hauteur est égale à la largeur, ceci produit un carré.

```
g.fillRect(170, 160, 20, 20); // L'oeil gauche
```

La ligne suivante dessine l'œil droit.

```
g.fillRect(210, 160, 20, 20); // L'oeil droit
```

La ligne suivante dessine une ligne qui correspond à la bouche.

```
g.drawLine(170, 200, 230, 200); // La bouche
```

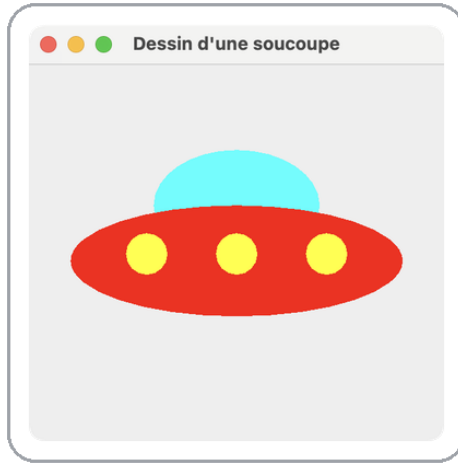
La méthode **drawLine**(int x1, int y1, int x2, int y2) dessine une ligne qui part de la coordonnées (x1,y1) et se termine à la coordonnée (x2,y2).

Enfin, les trois lignes suivantes dessinent les rectangles rouges des deux pinces.

```
g.setColor(Color.red);  
g.fillRect(80, 120, 30, 80); // La pince gauche  
g.fillRect(290, 120, 30, 80); // La pince droite
```

La classe **Graphics** contient plusieurs autres méthodes de dessin de formes de base.

Exercice. En vous inspirant de l'exemple précédent, écrivez un programme qui dessine un personnage de votre cru ou encore la soucoupe suivante. La taille de la fenêtre de la soucoupe est 300 par 300.



Solution. [JavaPasAPas/chapitre_4/ExerciceDessinSoucoupe.java](#)

```
import java.awt.*;
import javax.swing.JFrame;

public class ExerciceDessinSoucoupe extends JFrame {

    public ExerciceDessinSoucoupe() {
        super("Dessin d'une soucoupe");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(300, 300);
        this.setVisible(true);
    }

    public void paint(Graphics g) {
        super.paint(g);
        // Dessin de la soucoupe
        // Le dôme
        g.setColor(Color.cyan);
        g.fillOval(90, 90, 120, 80);
        // Le disque
        g.setColor(Color.red);
        g.fillOval(30, 130, 240, 80);
        // Les hublots
        g.setColor(Color.yellow);
        g.fillOval(70, 150, 30, 30);
        g.fillOval(135, 150, 30, 30);
        g.fillOval(200, 150, 30, 30);
    }

    public static void main(String[] args) {
        new ExerciceDessinSoucoupe();
    }
}
```

```
}
}
```

- **Création de plusieurs objets (fenêtres de dessin)**

L'exemple suivant crée trois fenêtres qui correspondent à trois objets de la classe *ExempleDessin2DDansJFrame*.

Exemple. Création de trois fenêtres de dessin.

```
import java.awt.*;
import javax.swing.JFrame;

public class ExempleDessin2DDansJFrame extends JFrame {

    public ExempleDessin2DDansJFrame() {
        super("Exemples de dessin avec les méthodes de Graphics");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 400);
        this.setVisible(true);
    }

    // La méthode paint() est appelée automatiquement lors de la création
    // du JFrame
    // La méthode paint() fait le dessin d'un crabe
    public void paint(Graphics g) {

        // Il faut appeler la méthode paint() de la super-classe
        super.paint(g);

        g.setColor(Color.green);
        g.fillOval(110, 140, 180, 80); // Le corps

        g.setColor(Color.black);
        g.fillRect(170, 160, 20, 20); // L'oeil gauche
        g.fillRect(210, 160, 20, 20); // L'oeil droit
        g.drawLine(170, 200, 230, 200); // La bouche

        g.setColor(Color.red);
        g.fillRect(80, 120, 30, 80); // La pince gauche
        g.fillRect(290, 120, 30, 80); // La pince droite
    }

    public static void main(String[] args) {
        new ExempleDessin2DDansJFrame();
        new ExempleDessin2DDansJFrame();
        new ExempleDessin2DDansJFrame();
    }
}
```

Chaque appel à `new ExempleDessin2DDansJFrame()` crée une nouvelle fenêtre.

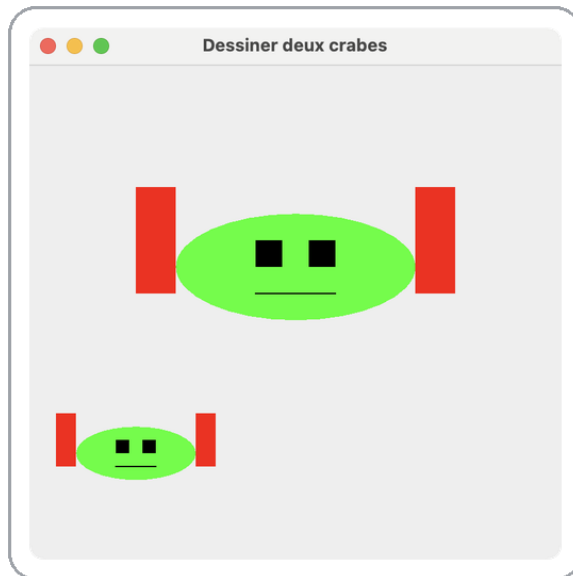
NB Si vous exécutez ce programme, les fenêtres sont superposées. Il faut les déplacer pour voir les trois.

5.2 Simplification du programme par une méthode avec paramètres

Cette section approfondit la création de méthode et l'utilisation des paramètres en montrant comment l'utilisation d'une méthode avec paramètre peut grandement simplifier un programme et ceci dans le contexte du dessin 2D. La notion de passage des paramètres sera étudiée en même temps.

L'exercice suivant permet de motiver l'utilisation d'une méthode avec paramètres.

Exercice. Dessiner deux crabs de taille et position différentes tel qu'illustré par la figure suivante :



Solution. [JavaPasAPas/chapitre_4/Exercice2Crabes.java](#)

```
import java.awt.*;
import javax.swing.*;

public class Exercice2Crabes extends JFrame {

    public Exercice2Crabes() {
        super("Dessiner deux crabes");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 400);
        this.setVisible(true);
    }

    public void paint(Graphics g) {
        super.paint(g);
        // Le premier crabe
        g.setColor(Color.green);
        g.fillOval(110, 140, 180, 80); // Le corps

        g.setColor(Color.black);
        g.fillRect(170, 160, 20, 20); // L'oeil gauche
        g.fillRect(210, 160, 20, 20); // L'oeil droit
        g.drawLine(170, 200, 230, 200); // La bouche

        g.setColor(Color.red);
        g.fillRect(80, 120, 30, 80); // La pince gauche
        g.fillRect(290, 120, 30, 80); // La pince droite

        // Le deuxième crabe
        g.setColor(Color.green);
        g.fillOval(35, 300, 90, 40); // Le corps

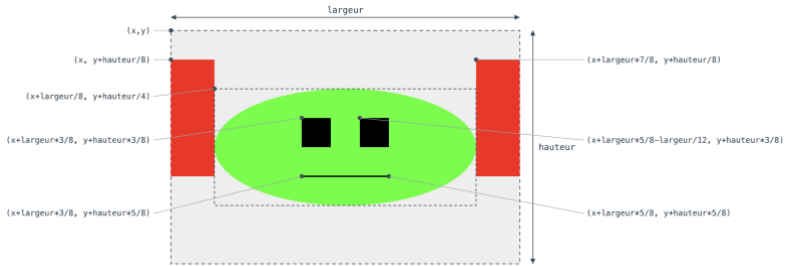
        g.setColor(Color.black);
        g.fillRect(65, 310, 10, 10); // L'oeil gauche
        g.fillRect(85, 310, 10, 10); // L'oeil droit
        g.drawLine(65, 330, 95, 330); // La bouche

        g.setColor(Color.red);
        g.fillRect(20, 290, 15, 40); // La pince gauche
        g.fillRect(125, 290, 15, 40); // La pince droite
    }

    public static void main(String[] args) {
        new Exercice2Crabes();
    }
}
```

La solution précédente rappelle les mêmes méthodes deux fois. Pour la position et la taille du crabe, il faut calculer de nouvelles valeurs des paramètres à chaque fois. Une manière plus élégante de traiter ce problème consiste à chercher une solution plus générale au dessin d'un crabe où sa position et sa taille sont variables. Une technique souvent employée en

graphisme 2D consiste à définir un rectangle englobant à l'intérieur duquel sera dessiné le crabe tel qu'illustré à la figure suivante. Le crabe est dessiné à l'échelle à l'intérieur du rectangle. Comme pour la méthode `fillRect()`, quatre variables sont définies pour représenter le rectangle englobant : les coordonnées x et y du coin supérieur gauche



, la *largeur* et la *hauteur* du rectangle englobant.

La version suivante dessine le même crabe que notre premier exemple mais en utilisant des variables qui représentent le rectangle englobant.

Exemple. [JavaPasAPas/chapitre_4/ExempleCrabeRectangleEnglobant.java](#)

```
import java.awt.*;
import javax.swing.JFrame;

public class ExempleCrabeRectangleEnglobant extends JFrame {

    public ExempleCrabeRectangleEnglobant() {
        super("Crabe avec rectangle englobant");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 400);
        this.setVisible(true);
    }

    public void paint(Graphics g) {
        super.paint(g);

        int x = 80;
        int y = 100;
        int largeur = 240;
        int hauteur = 160;

        // Crabe à l'échelle dans un rectangle englobant défini
        // par x,y,largeur,hauteur
        g.setColor(Color.green);
        // Le corps
        g.fillOval(
            x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

        g.setColor(Color.black);
        // L'oeil gauche
```

```

g.fillRect(
    x + largeur * 3 / 8,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// L'oeil droit
g.fillRect(
    x + largeur * 5 / 8 - largeur / 12,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// La bouche
g.drawLine(
    x + largeur * 3 / 8,
    y + hauteur * 5 / 8,
    x + largeur * 5 / 8,
    y + hauteur * 5 / 8);

g.setColor(Color.red);
// Les pinces
g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
g.fillRect(
    x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public static void main(String[] args) {
    new ExempleCrabeRectangleEnglobant();
}
}

```

On peut maintenant facilement redessiner le crabe deux fois en changeant la position et la taille par la modification des valeurs des variables x, y , *largeur* et *hauteur* mais en répétant exactement les mêmes instructions deux fois.

Exemple. [JavaPasAPas/chapitre_4/Exemple2CrabesRectangleEnglobant.java](#)

```

import java.awt.*;
import javax.swing.JFrame;

public class Exemple2CrabesRectangleEnglobant extends JFrame {

    public Exemple2CrabesRectangleEnglobant() {
        super("2 crabes avec rectangle englobant");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 400);
        this.setVisible(true);
    }

    public void paint(Graphics g) {
        super.paint(g);

        // Rectangle englobant du premier crabe
        int x = 80;

```



```
int y = 100;
int largeur = 240;
int hauteur = 160;

// Crabe à l'échelle dans un rectangle englobant
// défini par x,y,largeur,hauteur
g.setColor(Color.green);
// Le corps
g.fillOval(
    x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

g.setColor(Color.black);
// L'oeil gauche
g.fillRect(
    x + largeur * 3 / 8,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// L'oeil droit
g.fillRect(
    x + largeur * 5 / 8 - largeur / 12,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// La bouche
g.drawLine(
    x + largeur * 3 / 8,
    y + hauteur * 5 / 8,
    x + largeur * 5 / 8,
    y + hauteur * 5 / 8);

g.setColor(Color.red);
// Les pinces
g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
g.fillRect(
    x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);

// Rectangle englobant du deuxième crabe
x = 20;
y = 280;
largeur = 120;
hauteur = 80;

// Crabe à l'échelle dans un rectangle englobant
// défini par x,y,largeur,hauteur
g.setColor(Color.green);
// Le corps
g.fillOval(
    x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

g.setColor(Color.black);
// L'oeil gauche
g.fillRect(
    x + largeur * 3 / 8,
    y + hauteur * 3 / 8,
    largeur / 12,
```

```

        hauteur / 8);
// L'oeil droit
g.fillRect(
    x + largeur * 5 / 8 - largeur / 12,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// La bouche
g.drawLine(
    x + largeur * 3 / 8,
    y + hauteur * 5 / 8,
    x + largeur * 5 / 8,
    y + hauteur * 5 / 8);

g.setColor(Color.red);
// Les pinces
g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
g.fillRect(
    x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public static void main(String[] args) {
    new Exemple2CrabesRectangleEnglobant();
}
}

```

Cette solution oblige de répéter deux fois les mêmes énoncés. On peut éviter cette répétition en les regroupant dans une méthode et en appelant cette méthode à deux reprises.

Dans l'exemple suivant la méthode *paintCrabe*(Graphics g, int x, int y, int largeur, int hauteur) regroupe les énoncés de dessin du crabe. Plutôt que de répéter ces énoncés deux fois, il suffit d'appeler la méthode deux fois !

Exemple. [JavaPasAPas/chapitre_4/ExempleMethodePaintCrabe.java](#)

```

import java.awt.*;
import javax.swing.JFrame;

public class ExempleMethodePaintCrabe extends JFrame {

    public ExempleMethodePaintCrabe() {
        super("2 crabes avec méthode paintCrabe()");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 400);
        this.setVisible(true);
    }

    // Méthode qui dessine un crabe dans un objet Graphics g
    // à l'échelle dans un rectangle englobant de paramètres
    // x,y,largeur,hauteur
    public static void paintCrabe(

```

```
Graphics g, int x, int y, int largeur, int hauteur) {
    g.setColor(Color.green);
    // Le corps
    g.fillOval(
        x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

    g.setColor(Color.black);
    // L'oeil gauche
    g.fillRect(
        x + largeur * 3 / 8,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // L'oeil droit
    g.fillRect(
        x + largeur * 5 / 8 - largeur / 12,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // La bouche
    g.drawLine(
        x + largeur * 3 / 8,
        y + hauteur * 5 / 8,
        x + largeur * 5 / 8,
        y + hauteur * 5 / 8);

    g.setColor(Color.red);
    // Les pinces
    g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
    g.fillRect(
        x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public void paint(Graphics g) {
    super.paint(g);
    // Dessin du premier crabe
    paintCrabe(g, 80, 100, 240, 160);
    // Dessin du deuxième crabe
    paintCrabe(g, 20, 280, 120, 80);
}

public static void main(String[] args) {
    new ExempleMethodePaintCrabe();
}
}
```

La ligne suivante dans le code de l'exemple est une déclaration de la *signature* de la méthode `paintCrabe()`. Cette déclaration est similaire à celle de la méthode `main()` :

```
public static void paintCrabe(
    Graphics g, int x, int y, int largeur, int hauteur) {
```

Signature d'une méthode

La signature d'une méthode précise :

- le nom de la méthode
- la forme des paramètres
- la forme du résultat

La signature d'une méthode permet de déterminer comment appeler la méthode.

La méthode `paintCrabe()` fait partie de la classe `ExempleMethodePaintCrabe` car elle est déclarée dans le corps de la classe, c'est-à-dire entre les accolades qui suivent le nom de la classe. Dans cet exemple, on aurait aussi bien pu en faire une méthode d'objet en omettant le *static*. Nous reviendrons sur cet aspect.

La liste des paramètres entre virgules spécifie le type et le nom de chacun des paramètres. Le programmeur décide des noms, types et nombre des paramètres. Cette liste de paramètres qui apparaît dans la déclaration de la signature de la méthode représente les paramètres dits *formels*. D'autre part, lorsqu'on appelle la méthode, on doit spécifier les paramètres dits *réels*.

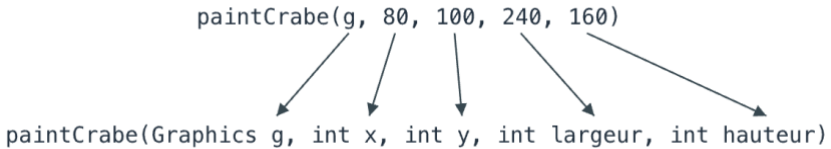
Dans le corps de la méthode, les énoncés font référence aux noms des paramètres formels comme s'ils étaient des variables. Par exemple, dans la ligne suivante, les variables `x`, `y`, `largeur` et `hauteur` sont les paramètres formels de la méthode `paintCrabe()` :

```
g.fillOval(  
    x + largeur / 8, y + hauteur / 4, largeur * 3 / 4,  
    hauteur / 2); // Le corps
```

Pour dessiner les deux crabes, la méthode `paint()` appelle deux fois la méthode `paintCrabe()` en précisant les paramètres réels. La ligne suivante est le premier appel.

```
paintCrabe(g, 80, 100, 240, 160);
```

Les paramètres `g`, `80`, `100`, `240`, `160` sont les paramètres réels. Il est à noter qu'un paramètre réel n'est pas nécessairement une constante. Ce peut aussi être un nom de variable, de paramètre, et même une expression. Lors de l'appel de la méthode, on peut imaginer que la valeur du paramètre réel est en quelque sorte affectée à la variable qui correspond au paramètre formel. Ensuite, la méthode est exécutée.



On obtient ainsi exactement le même effet que si l'on avait effectué la séquence suivante qui dessine le premier crabe dans *Exemple2CrabesRectangleEnglobant* :

```
int x = 80;
int y = 100;
int largeur = 240;
int hauteur = 160;

// Crabe à l'échelle dans un rectangle englobant
// défini par x,y,largeur,hauteur
g.setColor(Color.green);
g.fillOval(x + largeur / 8, y + hauteur / 4,
    largeur * 3 / 4, hauteur / 2); // Le corps
g.setColor(Color.black);
g.fillRect(x + largeur * 3 / 8, y + hauteur * 3 / 8,
    largeur / 12, hauteur / 8); // L'oeil gauche
g.fillRect(x + largeur * 5 / 8 - largeur / 12, y + hauteur * 3 / 8,
    largeur / 12, hauteur / 8); // L'oeil droit
g.drawLine(x + largeur * 3 / 8, y + hauteur * 5 / 8,
    x + largeur * 5 / 8, y + hauteur * 5 / 8); // La bouche
g.setColor(Color.red);
g.fillRect(x, y + hauteur / 8,
    largeur / 8, hauteur / 2); // La pince gauche
g.fillRect(x + largeur * 7 / 8, y + hauteur / 8,
    largeur / 8, hauteur / 2); // La pince droite
```

Passage de paramètre par valeur ou par référence

Le fait de copier la valeur du paramètre réel en l'affectant au paramètre formel correspond au concept de passage de paramètre par valeur. Ainsi, dans la méthode, si la valeur du paramètre formel est modifiée, ceci n'affecte

pas le paramètre réel. En Java, c'est la seule manière de passer un paramètre lorsqu'il s'agit d'un type primitif (int, double, etc.). D'autres langages offrent une autre option : le passage de paramètre par référence qui permet de modifier la valeur du paramètre réel. Dans le cas d'un paramètre objet en Java, la valeur passée est la référence à l'objet.

Pour le paramètre *g*, le contexte graphique (objet de la classe **Graphics**) de *paint()* est tout simplement passé à la méthode *paintCrabe()* qui va donc dessiner sur le même contexte graphique que si le dessin avait été effectué directement dans *paint()*. À noter que le nom du paramètre réel *g* est ici le même que celui du paramètre formel. Ceci n'est pas obligatoire. Par exemple, on aurait pu définir la méthode *paintCrabe()* de la manière suivante sans que cela n'ait d'effet sur le comportement du programme.

```
public static void paintCrabe(Graphics unGraphics,
    int x, int y, int largeur, int hauteur) {
    unGraphics.setColor(Color.green);
    unGraphics.fillOval(x + largeur / 8, y + hauteur / 4,
        largeur * 3 / 4, hauteur / 2); // Le corps
    unGraphics.setColor(Color.black);
    unGraphics.fillRect(x + largeur * 3 / 8, y + hauteur * 3 / 8,
        largeur / 12, hauteur / 8); // L'oeil gauche
    unGraphics.fillRect(x + largeur * 5 / 8 - largeur / 12,
        y + hauteur * 3 / 8,
        largeur / 12, hauteur / 8); // L'oeil droit
    unGraphics.drawLine(x + largeur * 3 / 8, y + hauteur * 5 / 8,
        x + largeur * 5 / 8, y + hauteur * 5 / 8); // La bouche
    unGraphics.setColor(Color.red);
    unGraphics.fillRect(x, y + hauteur / 8,
        largeur / 8, hauteur / 2); // La pince gauche
    unGraphics.fillRect(x + largeur * 7 / 8, y + hauteur / 8,
        largeur / 8, hauteur / 2); // La pince droite
}
```

Le nom en soi du paramètre formel n'a pas d'importance. L'important c'est de respecter l'ordre et le type des paramètres lors de l'appel.

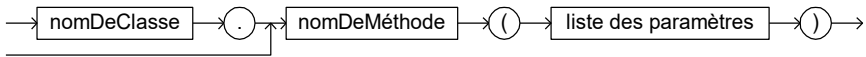
Même si le passage de paramètre est par valeur, c'est-à-dire qu'une copie de la référence à l'objet graphique *g* est passée à la méthode, la méthode peut modifier le contenu de l'objet en appelant les méthodes de modification de l'objet.

Raccourci pour l'appel d'une méthode de la même classe

La méthode `paintCrabe()` est une méthode de classe. Rappelons que l'appel d'une méthode de classe débute normalement par le nom de la classe. Dans le cas présent, il n'est pas nécessaire de mettre le nom de classe car la méthode appelée fait partie de la même classe que la méthode appelante. Ce raccourci est permis lorsqu'une méthode en appelle une autre de la même classe.

En considérant cette nouvelle possibilité, la syntaxe d'un appel de méthode de classe est donc :

appel de méthode de classe :



Exercice. Reprenez l'exemple de méthode avec votre personnage (ou la soucoupe). Définissez une méthode `paintXXX` (`Graphics g, int x, int y, int largeur, int hauteur`) qui dessine votre personnage et appelez-la à deux reprises avec des valeurs différentes pour les paramètres.

Solution avec la soucoupe :

```

JavaPasAPas/chapitre_4/ExerciceMethodePaintSoucoupe.java
import java.awt.*;
import javax.swing.JFrame;

public class ExerciceMethodePaintSoucoupe extends JFrame {

    public ExerciceMethodePaintSoucoupe() {
        super("2 soucoupes avec méthode paintSoucoupe()");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 400);
        this.setVisible(true);
    }

    // Méthode qui dessine une soucoupe dans un objet Graphics g
    // à l'échelle dans un rectangle englobant de paramètres
    // x,y,largeur,hauteur
    public static void paintSoucoupe(
        Graphics g, int x, int y, int largeur, int hauteur) {
        g.setColor(Color.cyan);
        // Le dôme
        g.fillOval(x + largeur / 4, y, largeur / 2, hauteur * 2 / 3);

        g.setColor(Color.red);
        // Le disque
    }
}

```

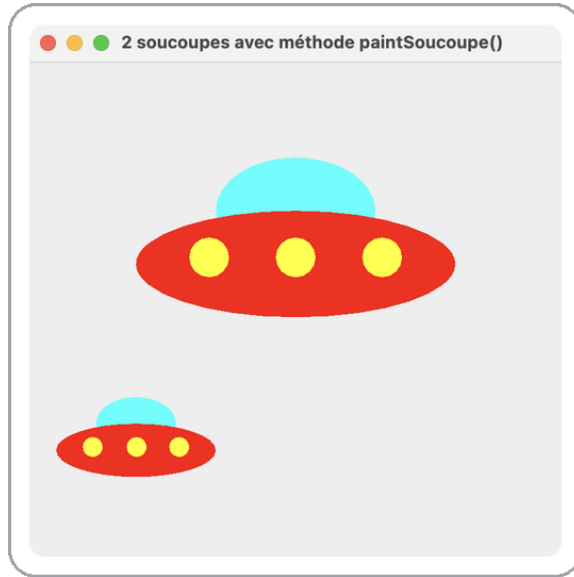
```
g.fillOval(x, y + hauteur / 3, largeur, hauteur * 2 / 3);

g.setColor(Color.yellow);
// Les hublots
g.fillOval(x + largeur / 6, y + hauteur / 2, largeur / 8, hauteur / 4);
g.fillOval(
    x + largeur / 2 - largeur / 16, y + hauteur / 2, largeur / 8,
    hauteur / 4);
g.fillOval(
    x + largeur * 5 / 6 - largeur / 8, y + hauteur / 2, largeur / 8,
    hauteur / 4);
}

public void paint(Graphics g) {
    super.paint(g);
    // Dessin de la première soucoupe
    paintSoucoupe(g, 80, 100, 240, 120);
    // Dessin de la deuxième soucoupe
    paintSoucoupe(g, 20, 280, 120, 60);
}

public static void main(String[] args) {
    new ExerciceMethodePaintSoucoupe();
}
}
```


Résultat :



Exercice. Dessiner plusieurs crabes et soucoupes dans la même fenêtre.

Solution. [JavaPasAPas/chapitre_4/ExercicePlusieursCrabesEtSoucoupes.java](#)

```
import java.awt.*;
import javax.swing.JFrame;

public class ExercicePlusieursCrabesEtSoucoupes extends JFrame {

    public ExercicePlusieursCrabesEtSoucoupes() {
        super("Rassemblement de crabes et de soucoupes");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 400);
        this.setVisible(true);
    }

    // Méthode qui dessine un crabe dans un objet Graphics g
    // à l'échelle dans un rectangle englobant de paramètres
    // x,y,largeur,hauteur
    public static void paintCrabe(
        Graphics g, int x, int y, int largeur, int hauteur) {
        g.setColor(Color.green);
        // Le corps
        g.fillOval(
            x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

        g.setColor(Color.black);
        // L'oeil gauche
        g.fillRect(
```

```
x + largeur * 3 / 8,
y + hauteur * 3 / 8,
largeur / 12,
hauteur / 8);
// L'oeil droit
g.fillRect(
    x + largeur * 5 / 8 - largeur / 12,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// La bouche
g.drawLine(
    x + largeur * 3 / 8,
    y + hauteur * 5 / 8,
    x + largeur * 5 / 8,
    y + hauteur * 5 / 8);

g.setColor(Color.red);
// Les pinces
g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
g.fillRect(
    x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

// Méthode qui dessine une soucoupe dans un objet Graphics g
// à l'échelle dans un rectangle englobant de paramètres
// x,y,largeur,hauteur
public static void paintSoucoupe(
    Graphics g, int x, int y, int largeur, int hauteur) {
    g.setColor(Color.cyan);
    // Le dôme
    g.fillOval(x + largeur / 4, y, largeur / 2, hauteur * 2 / 3);

    g.setColor(Color.red);
    // Le disque
    g.fillOval(x, y + hauteur / 3, largeur, hauteur * 2 / 3);

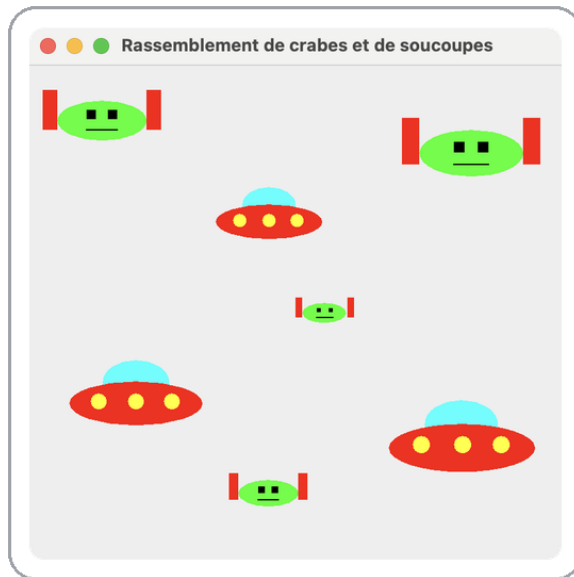
    g.setColor(Color.yellow);
    // Les hublots
    g.fillOval(x + largeur / 6, y + hauteur / 2, largeur / 8, hauteur / 4);
    g.fillOval(
        x + largeur / 2 - largeur / 16, y + hauteur / 2, largeur / 8,
        hauteur / 4);
    g.fillOval(
        x + largeur * 5 / 6 - largeur / 8, y + hauteur / 2, largeur / 8,
        hauteur / 4);
}

public void paint(Graphics g) {
    super.paint(g);
    paintCrabe(g, 10, 40, 90, 60);
    paintCrabe(g, 200, 200, 45, 30);
    paintCrabe(g, 150, 330, 60, 40);
    paintCrabe(g, 280, 60, 105, 70);
    paintSoucoupe(g, 140, 120, 80, 40);
    paintSoucoupe(g, 30, 250, 100, 50);
}
```

```
    paintSoucoupe(g, 270, 280, 110, 55);
}

public static void main(String[] args) {
    new ExercicePlusieursCrabesEtSoucoupes();
}
```

Résultat :



5.3 Traitement des événements de souris (interface *MouseListener*)

Dans les applications interactives, il faut pouvoir détecter les actions de l'utilisateur (déplacement de la souris, click de la souris, touche de clavier, etc.) et y réagir. Le package *java.awt* inclut les mécanismes à cet effet.

Le programme suivant illustre les mécanismes de base de détection des actions de la souris. Le programme répond à un *click de la souris* (bouton de gauche enfoncé avec Windows) en déplaçant le crabe à la position du click. La position du click est déterminée par la position du curseur de souris au moment où le bouton est enfoncé.

Exemple. [JavaPasAPas/chapitre_4/ExempleEvenementSouris.java](#)

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExempleEvenementSouris extends JFrame
    implements MouseListener {
    // Variables d'objet qui contiennent les coordonnées de la souris
    // Le premier sera dessiné à la coordonnée (0,0)
    private int x = 0; // Coordonnée x du crabe à dessiner
    private int y = 0; // Coordonnée y du crabe à dessiner

    public ExempleEvenementSouris() {
        super("Exemple de traitement d'événements de la souris");

        // Le paramètre this de addMouseListener() indique que l'objet qui
        // doit réagir aux événements de souris est l'objet
        // qui est créé par ce constructeur
        addMouseListener(this);

        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 600);
        this.setVisible(true);
    }

    // Méthode d'objet de la classe ExempleEvenementSouris
    // qui est appelée si le bouton de souris est enfoncé
    public void mousePressed(MouseEvent leMouseEvent) {
        // Place les coordonnées de la souris dans les variables x et y
        x = leMouseEvent.getX();
        y = leMouseEvent.getY();
        // repaint() provoque un nouvel appel à paint()
        repaint();
    }

    // Il faut absolument définir les autres méthodes pour les autres
    // événements de souris même s'ils ne font rien
    public void mouseClicked(MouseEvent leMouseEvent) {}

    public void mouseEntered(MouseEvent leMouseEvent) {}

    public void mouseExited(MouseEvent leMouseEvent) {}

    public void mouseReleased(MouseEvent leMouseEvent) {}

    public static void paintCrabe(
        Graphics g, int x, int y, int largeur, int hauteur) {
        g.setColor(Color.green);
```

```

// Le corps
g.fillOval(
    x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

g.setColor(Color.black);
// L'oeil gauche
g.fillRect(
    x + largeur * 3 / 8,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// L'oeil droit
g.fillRect(
    x + largeur * 5 / 8 - largeur / 12,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// La bouche
g.drawLine(
    x + largeur * 3 / 8,
    y + hauteur * 5 / 8,
    x + largeur * 5 / 8,
    y + hauteur * 5 / 8);

g.setColor(Color.red);
// Les pinces
g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
g.fillRect(
    x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public void paint(Graphics g) {
    super.paint(g);
    paintCrabe(g, x, y, 60, 40);
}

public static void main(String[] args) {
    new ExempleEvenementSouris();
}
}

```

Chaque fois que l'utilisateur emploie la souris, ceci produit un *événement (event) d'interface à l'utilisateur*. Un événement d'interface à l'utilisateur est le résultat d'une action de l'utilisateur sur un périphérique d'entrée tel que la souris ou le clavier. Pour réagir à l'événement, il faut programmer une méthode qui sera automatiquement activée lorsque l'événement est détecté. Cette méthode doit avoir une signature prédéfinie (par exemple, `mousePressed(MouseEvent e)`) pour un événement correspondant à enfoncer le bouton de la souris). Elle doit être incluse dans la classe d'un

objet qui est désigné par le programmeur comme écouteur (*listener*) de l'événement.

Dans notre exemple, l'objet écouteur est l'objet de la classe *ExempleEvenementSouris* qui représente une fenêtre `JFrame`. La ligne suivante du constructeur désigne *this* comme écouteur des événements de la souris. L'identificateur réservé *this* représente une référence à l'objet de la classe *ExempleEvenementSouris* qui est construit par le constructeur.

```
addMouseListener(this);
```

- **Notion d'interface Java**

Il faut que la classe de l'objet écouteur implémente des méthodes particulières pour répondre aux différents événements de la souris. Ceci est représenté par le fait que la classe de l'objet écouteur (*ExempleEvenementSouris*) doit implémenter l'interface `java.awt.event.MouseListener`. Ceci est spécifié dans la ligne suivante par la clause *implements MouseListener* dans la déclaration de la classe :

```
public class ExempleEvenementSouris extends JFrame implements MouseListener
```

Mettre en œuvre l'interface *java.awt.event.MouseListener* signifie que la classe doit contenir la définition d'un certain nombre de méthodes dont les signatures sont précisées dans l'interface Java désignée par le nom *java.awt.event.MouseListener*.

Qu'est-ce qu'une interface au sens de Java? Une interface Java est simplement un squelette de classe au sens où une interface contient la définition d'un ensemble de signatures de méthodes mais sans nécessairement spécifier le corps de la méthode. Par exemple, l'interface `java.awt.event.MouseListener` est définie de la manière suivante :

```
package java.awt.event;

public abstract interface MouseListener extends java.util.EventListener
{
    public abstract void mouseClicked(java.awt.event.MouseEvent e){}
    public abstract void mousePressed(java.awt.event.MouseEvent e){}
    public abstract void mouseReleased(java.awt.event.MouseEvent e){}
    public abstract void mouseEntered(java.awt.event.MouseEvent e){}
    public abstract void mouseExited(java.awt.event.MouseEvent e){}
}
```

Les méthodes de l'interface (*mouseClicked()*, *mousePressed()*, ...) ont un corps vide. Il est prévu que le corps de ces méthodes soit précisé dans la classe qui *implémente* l'interface.

Comme elle implémente l'interface `java.awt.event.MouseListener` (clause *implements java.awt.event.MouseListener*), la classe *ExempleEvenementSouris* doit contenir toutes ces méthodes. Ainsi en forçant une classe à implémenter une interface, la classe est contrainte à fournir les méthodes de cette interface. Cette manière de contraindre une classe à répondre à des méthodes garantit que la classe sera en mesure de fournir une réponse lors de l'exécution du programme. Le rôle des méthodes de l'interface `java.awt.event.MouseListener` est de spécifier comment réagir aux différents événements de la souris. En forçant un écouteur d'événement de souris à implémenter l'interface `java.awt.event.MouseListener`, il est garanti que l'objet écouteur puisse répondre aux différents événements de la souris. La souris peut générer plusieurs sortes d'événements tel que : bouton enfoncé, bouton cliqué, bouton relâché, curseur de souris déplacé hors des limites de la fenêtre, etc. Pour réagir à chacune des sortes d'événements, il faut définir la méthode correspondante de l'interface `MouseListener`. Par exemple, pour réagir à l'enfoncement du bouton de la souris, il faut définir la méthode `mousePressed(MouseEvent e)` dans la classe de l'écouteur *ExempleEvenementSouris*. Ces méthodes seront appelées automatiquement lorsque les événements se produisent.

Voici la méthode dans notre exemple :

```

public void mousePressed(MouseEvent leMouseEvent){
    x = leMouseEvent.getX(); // place la coordonnée x de la souris dans
la
    // variable x
    y = leMouseEvent.getY(); // place la coordonnée y de la souris dans
la
    // variable y
    // repaint() provoque un nouvel appel à paint()
    repaint();
}

```

La méthode a un paramètre qui est un objet de la classe `MouseEvent`. Cet objet contient des informations au sujet de l'événement de souris. La méthode d'objet `getX()` de `MouseEvent` retourne la coordonnée *x* du curseur au moment où le bouton de la souris est enfoncé et la méthode `getY()`, la coordonnée *y*. Ces coordonnées sont employées dans notre programme afin de pouvoir dessiner un crabe à cet endroit.

Ensuite, la méthode `repaint()` est appelée. Cette méthode est héritée de la classe `JFrame` qui est la super-classe de `ExempleEvenementSouris`. Elle provoque automatiquement un appel à `paint()` qui dessine un nouveau crabe à la position courante de la souris.

Méthodes `paint()`, `repaint()` et `update()`

Pourquoi ne devrait-on pas appeler `paint()` directement ? Ceci est dû au fait que la gestion de l'affichage est en réalité un processus complexe dans un contexte à multifenêtrage comme c'est le cas avec un système d'exploitation tel que Windows. Pour s'assurer que tout se passe correctement, il faut une coordination avec les autres événements des autres fenêtres. Il est prévu que la méthode `repaint()` effectue cette coordination avant que `paint()` ne soit appelée. En particulier, la méthode `repaint()` appelle la méthode `update()` qui efface le contenu du contexte graphique en rétablissant la couleur de fond et elle appelle `paint()` par la suite. D'autre part, c'est la méthode `repaint()` qui est appelée automatiquement lorsqu'il faut redessiner le contenu de la fenêtre lors des manipulations des fenêtres.

Les autres méthodes de l'interface `MouseListener` sont vides dans notre exemple car il n'y a rien à faire pour ces autres sortes d'événements :


```
public void mouseClicked(MouseEvent leMouseEvent){}  
public void mouseEntered(MouseEvent leMouseEvent){}  
public void mouseExited(MouseEvent leMouseEvent){}  
public void mouseReleased(MouseEvent leMouseEvent){}
```

Elles doivent tout de même être déclarées dans la classe *ExempleEvenementSouris* car elle implémente l'interface `MouseListener`. Nous verrons plus loin qu'il est possible d'éviter ces déclarations vides en employant une classe *Adaptor*.

Un peu comme pour la méthode *paint()*, cette manière de répondre aux événements peut sembler un peu mystérieuse car on ne voit nulle part l'appel aux méthodes de réponse à l'événement. Il faut comprendre que tout le code de gestion des événements se trouve en quelque part dans du code hérité de la super-classe `JFrame` et qu'il serait compliqué d'expliquer le détail de la gestion d'événement à ce point-ci.

- **Variables déclarées au niveau de la classe**

Vous avez peut-être noté que les variables *x* et *y* ne sont pas définies dans la méthode *mousePressed()* mais au début du corps de la classe dans les lignes suivantes :

```
private int x = 0; // Coordonnée x du crabe à dessiner  
private int y = 0; // Coordonnée y du crabe à dessiner
```

On appelle parfois ces variables des *attributs* de la classe. En Java, les attributs peuvent avoir différents degrés de visibilité indiqué par un terme tel que *public*, *private* ou *protected*. Si aucun de ces termes qualifiant la visibilité précède la déclaration de la variable, alors cette variable a une visibilité par défaut ce qui signifie que la variable est accessible par toutes les classes dans le même package, mais par seulement ces classes. Le terme *protected* signifie que l'attribut est accessible par toutes les classes dans le même package mais aussi par les classes dérivées. Le terme *public* signifie que l'attribut est librement accessible. Le *private* signifie que ces variables ne peuvent être accédées à partir des autres classes. Les méthodes d'une classe ont la même notion de visibilité : nous avons déjà vu que la méthode *main* d'un programme doit avoir la visibilité *public*.

Dans notre exemple, les variables sont des variables d'objet de la classe *ExempleEvenement.Souris*. L'emploi de variables d'objet permet à ces variables d'être accédées non seulement dans *mousePressed()* mais aussi dans la méthode *paint()* :

```
public void paint (Graphics g) {
    super.paint(g);
    paintCrabe(g,x,y,60,40);
}
```

L'utilisation de variables d'objet permet d'accéder aux mêmes variables dans les deux méthodes. Ainsi des données peuvent être partagées entre méthodes d'objet de la même classe. Si on avait déclaré les variables localement dans la méthode *mousePressed()*, la méthode *paint()* n'aurait pu y accéder. D'ailleurs, ceci aurait provoqué une erreur de compilation.

Exemple. Si vous compilez le programme suivant, vous obtiendrez une erreur de compilation indiquant que *x* et *y* n'existe pas dans *paint()* :

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExempleVariablesLocalesErreur extends JFrame
    implements MouseListener {

    public ExempleVariablesLocalesErreur() {
        super("Exemple de traitement d'événements de la souris");

        // Le paramètre this de addMouseListener() indique que l'objet qui
        // doit réagir aux événements de souris est l'objet
        // qui est créé par ce constructeur
        addMouseListener(this);

        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 600);
        this.setVisible(true);
    }

    // Méthode d'objet de la classe ExempleVariablesLocalesErreur
    // qui est appelée si le bouton de souris est enfoncé
    public void mousePressed(MouseEvent leMouseEvent) {
        // Place les coordonnées de la souris dans les variables x et y
        int x = leMouseEvent.getX();
        int y = leMouseEvent.getY();
        // repaint() provoque un nouvel appel à paint()
        repaint();
    }
}
```

```
// Il faut absolument définir les autres méthodes pour les autres
// événements de souris même s'ils ne font rien
public void mouseClicked(MouseEvent leMouseEvent) {}

public void mouseEntered(MouseEvent leMouseEvent) {}

public void mouseExited(MouseEvent leMouseEvent) {}

public void mouseReleased(MouseEvent leMouseEvent) {}

public static void paintCrabe(
    Graphics g, int x, int y, int largeur, int hauteur) {
    g.setColor(Color.green);
    // Le corps
    g.fillOval(
        x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

    g.setColor(Color.black);
    // L'oeil gauche
    g.fillRect(
        x + largeur * 3 / 8,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // L'oeil droit
    g.fillRect(
        x + largeur * 5 / 8 - largeur / 12,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // La bouche
    g.drawLine(
        x + largeur * 3 / 8,
        y + hauteur * 5 / 8,
        x + largeur * 5 / 8,
        y + hauteur * 5 / 8);

    g.setColor(Color.red);
    // Les pinces
    g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
    g.fillRect(
        x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public void paint(Graphics g) {
    super.paint(g);
    paintCrabe(g, x, y, 60, 40);
}

public static void main(String[] args) {
    new ExempleVariablesLocalesErreur();
}
}
```

L'erreur de compilation vient du fait qu'une variable déclarée localement dans une méthode ne peut être accédée dans une autre méthode.

Portée d'une variable, variable *locale*

La *portée* d'une variable déclarée dans une méthode est limitée au corps de la méthode. En d'autres mots, la variable déclarée dans une méthode est *locale* à la méthode.

Concrètement, ceci signifie que la variable disparaît en quelque sorte lorsque l'exécution de la méthode est terminée. Il serait possible de déclarer des variables *x* et *y* dans chacune des deux méthodes, *mousePressed()* et *paint()*, mais ceci n'aurait pas produit le résultat recherché car ces variables seraient en réalité des variables différentes même si elles portent le même nom !

Pour que le contenu d'une variable soit accessible à plusieurs méthodes de la même classe, il faut que la variable soit déclarée de manière globale au niveau de classe. D'autre part, une variable déclarée au niveau de la classe peut-être soit une variable de classe ou une variable d'objet.

Variable de classe (*static*)

L'identificateur réservé *static* indique que c'est une variable de classe. Quand la mention *static* est omise, on dit parfois qu'il s'agit d'une variable d'instance ou variable d'objet.

Dans l'exemple précédent, nous avons employé des variables d'objet. Mais, on aurait aussi bien pu les définir comme des variables de classe sans que cela n'affecte le comportement de l'exemple puisqu'il n'y a qu'un seul objet de la classe *ExempleEvenementSouris*.

Exercice. Ajoutez l'identificateur réservé *static* dans la déclaration des variables *x* et *y* de *ExempleEvenementSouris* et faites exécuter le programme.

```
private static int x = 0; // Coordonnée x du crabe à dessiner
private static int y = 0; // Coordonnée y du crabe à dessiner
```

Dans le contexte de cet exemple, le résultat est le même que si les variables étaient des variables d'objet. Voyons maintenant un cas où l'emploi d'une variable de classe ou d'objet ne produit pas le même effet parce qu'il y a plus d'un objet de la même classe.

Différence entre *variable d'objet* et *variable de classe*

Il y a une différence importante entre variable d'objet (ou variable d'instance) et variable de classe. Il n'y a qu'une valeur pour une variable de classe peu importe le nombre d'objets de la classe. Ceci signifie que tous les objets de la classe partagent la même variable. Dans le cas d'une variable d'objet, il y a en quelque sorte une variable différente pour chacun des objets.

Pour voir la différence entre variable d'objet et de classe, il faut créer au moins deux objets de la classe *ExempleEvenementSouris*, c'est-à-dire deux fenêtres.

L'exemple suivant illustre l'effet de l'utilisation de variables de classe dans le cas de plusieurs objets de la même classe.

Exemple. `JavaPasAPas/chapitre_4/ExempleEvenementSouris2Fenetres.java`

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExempleEvenementSouris2Fenetres extends JFrame
    implements MouseListener {
    // Variables de classe x et y
    private static int x = 0; // Coordonnée x du crabe à dessiner
    private static int y = 0; // Coordonnée y du crabe à dessiner

    public ExempleEvenementSouris2Fenetres() {
        super("Exemple de traitement d'événements de la souris");

        // Le paramètre this de addMouseListener() indique que l'objet qui
        // doit réagir aux événements de souris est l'objet
        // qui est créé par ce constructeur
        addMouseListener(this);

        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 600);
        this.setVisible(true);
    }

    // Méthode d'objet de la classe ExempleEvenementSouris2Fenetres
    // qui est appelée si le bouton de souris est enfoncé
    public void mousePressed(MouseEvent leMouseEvent) {
        // Place les coordonnées de la souris dans les variables x et y
        x = leMouseEvent.getX();
        y = leMouseEvent.getY();
        // repaint() provoque un nouvel appel à paint()
        repaint();
    }

    // Il faut absolument définir les autres méthodes pour les autres
```

```
// événements de souris même s'ils ne font rien
public void mouseClicked(MouseEvent leMouseEvent) {}

public void mouseEntered(MouseEvent leMouseEvent) {}

public void mouseExited(MouseEvent leMouseEvent) {}

public void mouseReleased(MouseEvent leMouseEvent) {}

public static void paintCrabe(
    Graphics g, int x, int y, int largeur, int hauteur) {
    g.setColor(Color.green);
    // Le corps
    g.fillOval(
        x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

    g.setColor(Color.black);
    // L'oeil gauche
    g.fillRect(
        x + largeur * 3 / 8,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // L'oeil droit
    g.fillRect(
        x + largeur * 5 / 8 - largeur / 12,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // La bouche
    g.drawLine(
        x + largeur * 3 / 8,
        y + hauteur * 5 / 8,
        x + largeur * 5 / 8,
        y + hauteur * 5 / 8);

    g.setColor(Color.red);
    // Les pinces
    g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
    g.fillRect(
        x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public void paint(Graphics g) {
    super.paint(g);
    paintCrabe(g, x, y, 60, 40);
    g.drawString("x=" + x + " y=" + y, 10, 550);
}

public static void main(String[] args) {
    new ExempleEvenementSouris2Fenetres();
    new ExempleEvenementSouris2Fenetres();
}
}
```

La figure suivante montre les deux fenêtres dont le crabe a été positionné à deux endroits différents en utilisant la souris. La ligne suivante a été ajoutée à la méthode *paint()* pour afficher la valeur des variables x et y . La méthode **drawString(String str, int x, int y)** a comme paramètres un **String** à afficher et les coordonnées de l'emplacement où l'on veut afficher ce **String**.

```
g.drawString("x="+x+" y="+y,10,550);
```

Dans la figure suivante produite par l'exécution du programme, le crabe de la fenêtre de gauche a été positionné avant celui de la fenêtre de droite. À ce point-ci de l'exécution du programme, les variables de classe x et y contiennent les coordonnées du crabe de la fenêtre de droite, soit $x = 288$ et $y = 442$, parce que le crabe de la fenêtre de droite a été positionné le dernier. Il n'y a pas de valeurs spécifiques à chacun des objets ! Les deux objets partagent la variable de classe.



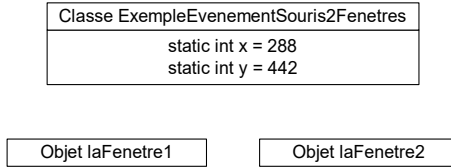
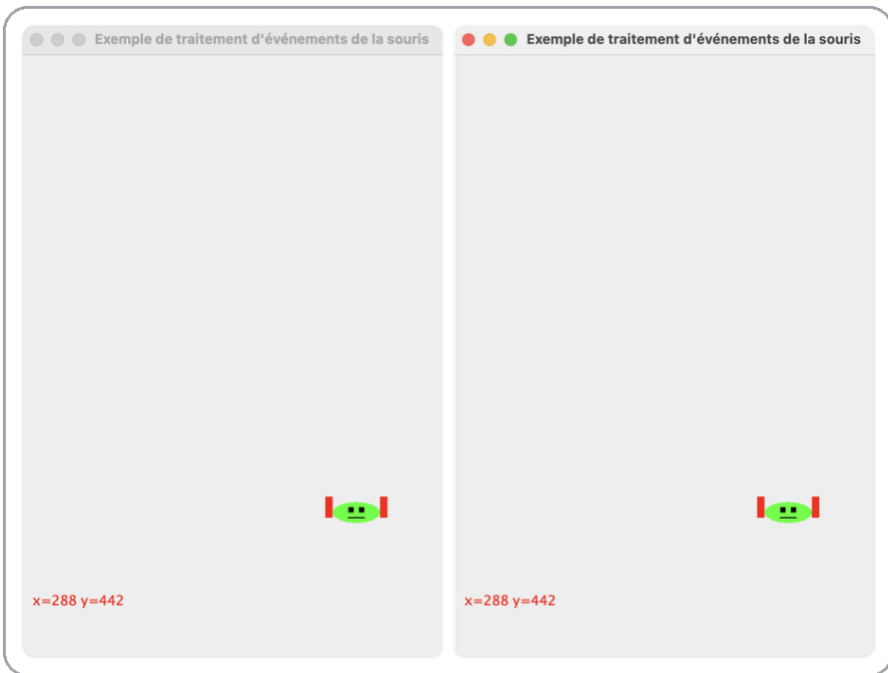


Figure 19. Variables de classe (static).

Si l'on réduit la fenêtre de gauche et on la réaffiche (le bouton de réduction est celui qui contient une petite barre horizontale dans le coin supérieur droit de la fenêtre), on obtient l'effet suivant :



Le crabe de la fenêtre de gauche est maintenant à la même position que celui de la fenêtre de droite ! Mais, nous ne l'avons pas repositionné ! Ceci est dû au fait qu'il n'y a qu'une seule valeur de x et y dans le cas de variables de classe. Lorsque la fenêtre de gauche est réaffichée, ceci provoque un appel automatique à la méthode `paint()` sur l'objet qui correspond à la fenêtre de gauche. La méthode `paint()` accède aux coordonnées x et y afin d'afficher le crabe. Mais comme, la valeur courante de x et y contient les coordonnées du crabe de la fenêtre de droite (288, 442) parce qu'il a été positionné le dernier,

la méthode *paint()* dessine le crabe au même endroit dans la fenêtre de gauche.

- **Variable d'objet**

Pour que chacune des deux fenêtres ait ses propres valeurs des coordonnées *x* et *y* du crabe, il faut en faire des variables d'objet. Dans ce cas, les deux objets ont leurs propres variables *x* et *y*. L'appel à *paint()* peut ainsi employer les valeurs de *x* et *y* particulières à l'objet.

Objet 1	Objet 2
int x = 62 int y = 84	int x = 288 int y = 442

Figure 20. Variables d'objet.

Dans *ExempleVariableDobjet*, les coordonnées du crabe sont des variables d'objet. Vous pouvez vérifier que le comportement des fenêtres est maintenant correct. Chacune des fenêtres a ses propres valeurs de *x* et *y* pour le crabe.

Exemple. **JavaPasAPas/chapitre_4/ExempleVariableDobjet.java**

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExempleVariableDobjet extends JFrame
    implements MouseListener {
    // Variables d'objet qui contiennent les coordonnées de la souris
    // Le premier sera dessiné à la coordonnée (0,0)
    private int x = 0; // Coordonnée x du crabe à dessiner
    private int y = 0; // Coordonnée y du crabe à dessiner

    public ExempleVariableDobjet() {
        super("Exemple de variable d'objet x et y");

        // Le paramètre this de addMouseListener() indique que l'objet qui
        // doit réagir aux événements de souris est l'objet
        // qui est créé par ce constructeur
        addMouseListener(this);

        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 600);
        this.setVisible(true);
    }
}
```

```
// Méthode d'objet de la classe ExempleVariableDobjet
// qui est appelée si le bouton de souris est enfoncé
public void mousePressed(MouseEvent leMouseEvent) {
    // Place les coordonnées de la souris dans les variables x et y
    x = leMouseEvent.getX();
    y = leMouseEvent.getY();
    // repaint() provoque un nouvel appel à paint()
    repaint();
}

// Il faut absolument définir les autres méthodes pour les autres
// événements de souris même s'ils ne font rien
public void mouseClicked(MouseEvent leMouseEvent) {}

public void mouseEntered(MouseEvent leMouseEvent) {}

public void mouseExited(MouseEvent leMouseEvent) {}

public void mouseReleased(MouseEvent leMouseEvent) {}

public static void paintCrabe(
    Graphics g, int x, int y, int largeur, int hauteur) {
    g.setColor(Color.green);
    // Le corps
    g.fillOval(
        x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

    g.setColor(Color.black);
    // L'oeil gauche
    g.fillRect(
        x + largeur * 3 / 8,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // L'oeil droit
    g.fillRect(
        x + largeur * 5 / 8 - largeur / 12,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // La bouche
    g.drawLine(
        x + largeur * 3 / 8,
        y + hauteur * 5 / 8,
        x + largeur * 5 / 8,
        y + hauteur * 5 / 8);

    g.setColor(Color.red);
    // Les pinces
    g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
    g.fillRect(
        x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public void paint(Graphics g) {
```

```
super.paint(g);
paintCrabe(g, x, y, 60, 40);
g.drawString("x=" + x + " y=" + y, 10, 550);
}

public static void main(String[] args) {
    new ExempleVariableDobjet();
    new ExempleVariableDobjet();
}
```

En conclusion, une variable de classe peut être employée si tous les objets de la classe partagent la même valeur pour cette variable. Si chacun des objets doit avoir sa propre valeur distincte pour la variable, il faut employer une variable d'objet.

- **Méthode d'objet ou de classe**

Une distinction analogue existe entre les méthodes d'objet et de classe. Une méthode de classe est appropriée lorsqu'elle n'utilise pas les variables d'objet. En revanche, si la méthode utilise des variables d'objet, il faut en faire une méthode d'objet.

Examinons le cas de la méthode *paintCrabe()* dans l'exemple précédent. C'est une méthode de classe et ceci est correct car elle n'utilise pas directement les variables d'objet *x* et *y*. En effet, elle utilise les paramètres *x* et *y* qui sont passés par l'appel *paintCrabe(g,x,y,60,40)* dans la méthode *paint()*.

```
public void paint (Graphics g) {
    super.paint(g);
    paintCrabe(g,x,y,60,40);
    g.drawString("x="+x+" y="+y,10,550);
}
```

Dans *paint()*, les variables *x* et *y* correspondent aux variables d'objet. Les valeurs des variables d'objet sont passées à la méthode de classe par l'appel *paintCrabe(g,x,y,60,40)*.

Regardons maintenant un autre exemple, où plutôt que de passer les valeurs de x et y à *paintCrabe()* par les paramètres, la méthode *paintCrabe()* utilise directement les variables d'objet x et y .

Exemple. [JavaPasAPas/chapitre_4/ExempleMethodeDobjetPaintCrabe.java](#)

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExempleMethodeDobjetPaintCrabe extends JFrame
    implements MouseListener {
    // Variables d'objet qui contiennent les coordonnées de la souris
    // Le premier sera dessiné à la coordonnée (0,0)
    private int x = 0; // Coordonnée x du crabe à dessiner
    private int y = 0; // Coordonnée y du crabe à dessiner

    public ExempleMethodeDobjetPaintCrabe() {
        super("Exemple de traitement d'événements de la souris");

        // Le paramètre this de addMouseListener() indique que l'objet qui
        // doit réagir aux événements de souris est l'objet
        // qui est créé par ce constructeur
        addMouseListener(this);

        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(400, 600);
        this.setVisible(true);
    }

    // Méthode d'objet de la classe ExempleMethodeDobjetPaintCrabe
    // qui est appelée si le bouton de souris est enfoncé
    public void mousePressed(MouseEvent leMouseEvent) {
        // Place les coordonnées de la souris dans les variables x et y
        x = leMouseEvent.getX();
        y = leMouseEvent.getY();
        // repaint() provoque un nouvel appel à paint()
        repaint();
    }

    // Il faut absolument définir les autres méthodes pour les autres
    // événements de souris même s'ils ne font rien
    public void mouseClicked(MouseEvent leMouseEvent) {}

    public void mouseEntered(MouseEvent leMouseEvent) {}

    public void mouseExited(MouseEvent leMouseEvent) {}

    public void mouseReleased(MouseEvent leMouseEvent) {}

    public void paintCrabe(Graphics g, int largeur, int hauteur) {
        // La méthode d'objet utilise directement les variables d'objet x et y
        g.setColor(Color.green);
        // Le corps
        g.fillOval(
            x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);
    }
}
```

```

g.setColor(Color.black);
// L'oeil gauche
g.fillRect(
    x + largeur * 3 / 8,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// L'oeil droit
g.fillRect(
    x + largeur * 5 / 8 - largeur / 12,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// La bouche
g.drawLine(
    x + largeur * 3 / 8,
    y + hauteur * 5 / 8,
    x + largeur * 5 / 8,
    y + hauteur * 5 / 8);

g.setColor(Color.red);
// Les pinces
g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
g.fillRect(
    x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public void paint(Graphics g) {
    super.paint(g);
    paintCrabe(g, 60, 40);
    g.drawString("x=" + x + " y=" + y, 10, 550);
}

public static void main(String[] args) {
    new ExempleMethodeObjetPaintCrabe();
    new ExempleMethodeObjetPaintCrabe();
}
}

```

Dans cet exemple, la méthode d'objet *paintCrabe()* ne peut être une méthode de classe car elle accède aux variables d'objet *x* et *y*. Si la méthode *paintCrabe()* était déclarée *static*, une erreur de compilation serait signalée.

5.4 Constantes (final)

Lorsqu'il précède une variable, l'identificateur réservé *final* indique que la variable ne peut être modifiée. Une telle variable désigne donc une *constante* dans le programme Java. Une pratique souvent employée consiste à définir les constantes utilisées dans un programme comme des variables *final*. Ceci

améliore la lisibilité du programme. D'autre part, si le programmeur doit changer la valeur de la constante, il évite d'avoir à parcourir le programme pour retrouver toutes les occurrences de la valeur constante pour la modifier.

Dans l'exemple suivant qui reprend l'exemple précédent, la hauteur et la largeur de la fenêtre sont définies comme des variables *final*. Ces constantes sont employées dans le constructeur de fenêtre *ExempleConstantesFinal()* et la méthode *paint()* pour ajuster la dimension du crabe et la position du message affichée.

Exemple. [JavaPasAPas/chapitre_4/ExempleConstantesFinal.java](#)

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExempleConstantesFinal extends JFrame
    implements MouseListener {
    // Constantes pour la taille de la fenêtre
    private static final int LARGEURFENETRE = 400;
    private static final int HAUTEURFENETRE = 600;

    // Variables d'objet qui contiennent les coordonnées de la souris
    // Le premier sera dessiné à la coordonnée (0,0)
    private int x = 0; // Coordonnée x du crabe à dessiner
    private int y = 0; // Coordonnée y du crabe à dessiner

    public ExempleConstantesFinal() {
        super("Exemple de traitement d'événements de la souris");

        // Le paramètre this de addMouseListener() indique que l'objet qui
        // doit réagir aux événements de souris est l'objet
        // qui est créé par ce constructeur
        addMouseListener(this);

        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(LARGEURFENETRE, HAUTEURFENETRE);
        this.setVisible(true);
    }

    // Méthode d'objet de la classe ExempleConstantesFinal
    // qui est appelée si le bouton de souris est enfoncé
    public void mousePressed(MouseEvent leMouseEvent) {
        // Place les coordonnées de la souris dans les variables x et y
        x = leMouseEvent.getX();
        y = leMouseEvent.getY();
        // repaint() provoque un nouvel appel à paint()
        repaint();
    }

    // Il faut absolument définir les autres méthodes pour les autres
    // événements de souris même s'ils ne font rien
    public void mouseClicked(MouseEvent leMouseEvent) {}
}
```

```
public void mouseEntered(MouseEvent leMouseEvent) {}

public void mouseExited(MouseEvent leMouseEvent) {}

public void mouseReleased(MouseEvent leMouseEvent) {}

public void paintCrabe(Graphics g, int largeur, int hauteur) {
    // La méthode d'objet utilise directement les variables d'objet x et y
    g.setColor(Color.green);
    // Le corps
    g.fillOval(
        x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

    g.setColor(Color.black);
    // L'oeil gauche
    g.fillRect(
        x + largeur * 3 / 8,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // L'oeil droit
    g.fillRect(
        x + largeur * 5 / 8 - largeur / 12,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // La bouche
    g.drawLine(
        x + largeur * 3 / 8,
        y + hauteur * 5 / 8,
        x + largeur * 5 / 8,
        y + hauteur * 5 / 8);

    g.setColor(Color.red);
    // Les pinces
    g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
    g.fillRect(
        x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public void paint(Graphics g) {
    super.paint(g);
    paintCrabe(g, LARGEURFENETRE / 6, HAUTEURFENETRE / 15);
    g.drawString("x=" + x + " y=" + y, 10, HAUTEURFENETRE - 50);
}

public static void main(String[] args) {
    new ExempleConstantesFinal();
    new ExempleConstantesFinal();
}
}
```

Exercice. Modifiez le programme précédent en définissant dans la classe *ExempleConstantesFinal* deux constantes `LARGEURCRABE` et `HAUTEURCRABE` qui seront utilisées dans l'appel à `paintCrabe()`. La valeur de ces constantes sera calculée à partir des constantes `LARGEURFENETRE` et `HAUTEURFENETRE`.

- **Ensemble de constantes (types de données énumérés)**

À noter que la constante `Color.green` qui représente une couleur de dessin est en fait un nom de variable *public static final* de la classe `Color`. Il en est de même pour les autres couleurs prédéfinies énumérées précédemment. Ainsi cet ensemble des couleurs est un ensemble de constantes dans la classe `Color`. Voici un extrait de la classe `Color` qui montre quelques déclarations de couleurs. Une couleur est un objet de la classe `Color` créé avec le constructeur `Color(int r, int g, int b)`. Chacun des paramètres est un entier entre 0 et 255 qui spécifie l'intensité d'une des trois composantes de la couleur : rouge, vert et bleu. Cette manière de représenter les couleurs correspond au système RGB.

```
public class Color
    implements java.awt.Paint, java.io.Serializable
{
...
    public static final java.awt.Color white = new Color(255,255,255);
    public static final java.awt.Color lightGray = new Color(192,192,192);
    public static final java.awt.Color gray = new Color(128,128,128);
    public static final java.awt.Color darkGray = new Color(64,64,64);
    public static final java.awt.Color black = new Color(0,0,0);
    public static final java.awt.Color red = new Color(255,0,0);
    public static final java.awt.Color pink = new Color(255,175,175);
    public static final java.awt.Color orange = new Color(255,200,0);
    public static final java.awt.Color yellow = new Color(255,255,0);
    public static final java.awt.Color green = new Color(0,255,0);
    public static final java.awt.Color magenta = new Color(255,0,255);
    public static final java.awt.Color cyan = new Color(0,255,255);
    public static final java.awt.Color blue = new Color(0,0,255);
...

    // Constructeur de couleur
    public Color(int r, int g, int b)
    {...
```



```

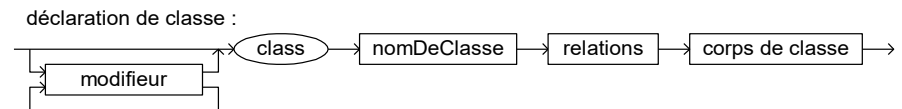
    }
...

```

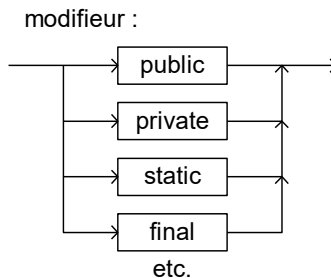
Pour accéder à une variable *public* d'une autre classe, il faut préfixer le nom de la variable avec le nom de sa classe, d'où l'utilisation de la syntaxe *Color.green* dans les exemples de programmes vus jusqu'à présent.

5.5 Sommaire d'une déclaration de classe

Résumons les concepts vus jusqu'à présent en portant un regard sommaire au sujet de la déclaration d'une classe dont le diagramme syntaxique est le suivant.

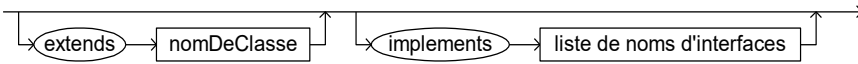


L'identificateur réservé *class* est optionnellement précédé d'une suite de modifieurs. Nous n'avons rencontré que *public* dans le cas d'une classe. Le sens des autres modifieurs sera expliqué ultérieurement.



Après le nom de classe, les relations avec d'autres classes sont spécifiées (super-classe et interfaces à implémenter). Il ne peut y avoir qu'une seule super-classe mais il peut y avoir plus d'une interface à implémenter. Lorsque la clause *extends* est absente, par défaut, la classe est une sous-classe de *java.lang.Object* qui est la racine de la hiérarchie des classes Java. Cette classe contient des méthodes supportées par tous les objets. Par exemple, la méthode *getClass()* de *Object* retourne la classe de l'objet.

relations :

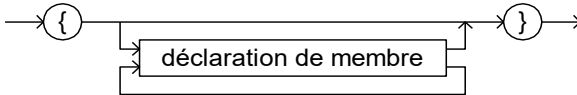


Ensuite, vient le corps de la classe qui contient les déclarations de ses membres.

Membre d'une classe

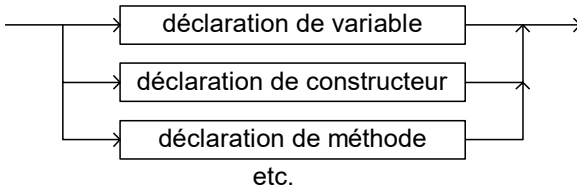
Les variables et méthodes définies au niveau d'une classe sont appelées des membres de cette classe.

corps de classe :



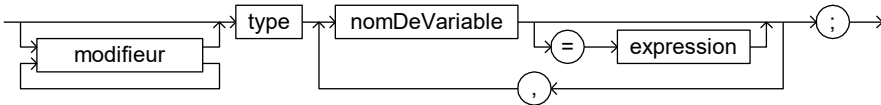
Le corps est une suite de déclarations de membres.

déclaration de membre :

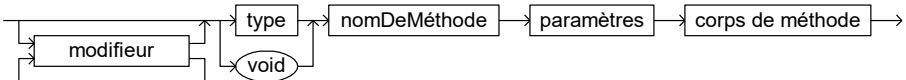


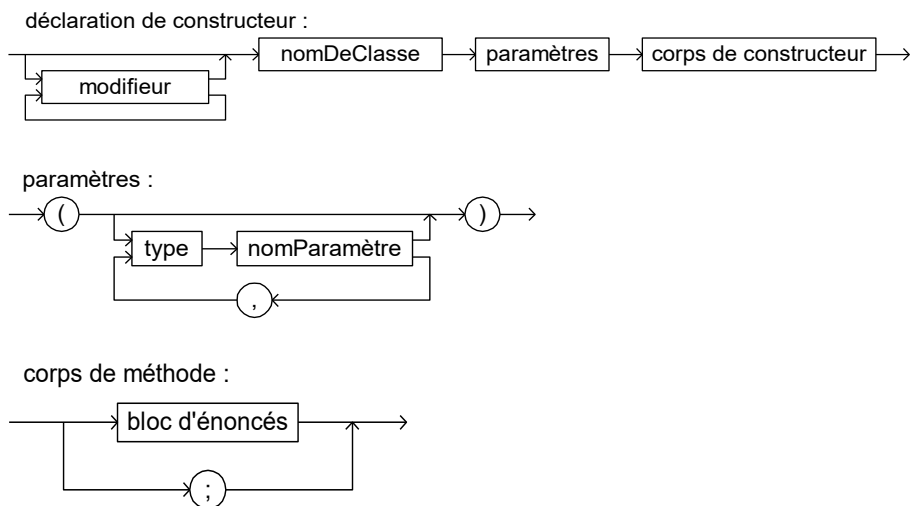
Voici la syntaxe pour chacun des types de membres.

déclarationDeVariable :



déclaration de méthode :





Le modifieur *public* pour un membre signifie qu'il peut être accédé de l'extérieur de la classe par opposition à *private*. Le modifieur *static* spécifie qu'il s'agit d'une variable ou méthode de classe. Une méthode de classe ne peut accéder qu'aux variables de classe alors qu'une méthode d'objet peut accéder aux variables de classes et d'objet tel qu'indiqué dans le tableau suivant :

	Variable d'objet	Variable de classe
Méthode de classe	Accès interdit	Accès permis
Méthode d'objet	Accès permis	Accès permis

Pour une variable, le modifieur *final* signifie qu'elle ne peut être modifiée (constante).

• Portée des variables

Lorsqu'un nom de variable est employé dans le corps d'une méthode, ce nom de variable peut correspondre à une des quatre possibilités suivantes :

- variable de classe
- variable d'objet
- variable locale
- paramètre formel

Lorsqu'une variable locale, disons *v*, est déclarée et qu'elle porte le même nom qu'une variable de classe ou d'objet, le nom de variable *v* désigne la variable locale. Ainsi, la variable locale masque en quelque sorte une variable d'objet ou de classe du même nom. Pour désigner la variable de classe qui est masquée, on peut employer la syntaxe *nomClasse.v*. Dans le cas d'une méthode d'objet, la syntaxe *this.v* est aussi permise.

Variable de classe masquée par une variable locale

Exemple. [JavaPasAPas/chapitre_4/ExempleVariablesLocales.java](#)

```
public class ExempleVariablesLocales {
    public static int x = 0;

    public ExempleVariablesLocales() {}

    public static void m1() {
        int x = 1;
        System.out.println(
            "Valeur de la variable locale x dans la méthode de classe m1() ="
            + x);
        System.out.println(
            "Valeur de la variable de classe x dans la méthode de classe m1()
="
            + ExempleVariablesLocales.x);
    }

    public void m2() {
        int x = 2;
        System.out.println(
            "Valeur de la variable locale x dans la méthode d'objet m2() ="
            + x);
        System.out.println(
            "Valeur de la variable de classe x dans la méthode d'objet m2() ="
            + this.x);
        System.out.println(
            "Valeur de la variable de classe x dans la méthode d'objet m2() ="
            + ExempleVariablesLocales.x);
    }
}
```

```
public static void main(String[] args) {  
  
    System.out.println("Appel de la méthode de classe m1() :");  
    ExempleVariablesLocales.m1();  
  
    System.out.println("Création d'un objet :");  
    ExempleVariablesLocales unObjet = new ExempleVariablesLocales();  
    System.out.println("Appel de la méthode de d'objet m2() :");  
    unObjet.m2();  
}  
}
```

Résultat :

```
Appel de la méthode de classe m1() :  
Valeur de la variable locale x dans la méthode de classe m1() =1  
Valeur de la variable de classe x dans la méthode de classe m1() =0  
Création d'un objet :  
Appel de la méthode de d'objet m2() :  
Valeur de la variable locale x dans la méthode d'objet m2() =2  
Valeur de la variable de classe x dans la méthode d'objet m2() =0  
Valeur de la variable de classe x dans la méthode d'objet m2() =0
```


6. Introduction à l'animation 2D

« *La seule façon de faire du grand travail est d'aimer ce que vous faites.* » — Steve Jobs

Ce chapitre présente les concepts de base de l'animation graphique 2D. L'idée de base de l'animation est de dessiner une série d'images de manière suffisamment rapide pour donner l'impression d'un mouvement continu. Chacune des images produites est appelée une *scène* de l'animation. La prochaine section illustre le principe de base d'affichage d'une séquence de scènes par une animation élémentaire. Le mécanisme de traitement d'exceptions Java sera en même temps introduit. La section deux ajoute un raffinement à l'animation avec le mécanisme de double tampon.

6.1 Une première tentative d'animation

Le programme suivant est une première tentative d'animation simple du crabe. L'objectif est de faire bouger le crabe de gauche à droite dans une fenêtre.

```
Exemple. JavaPasAPas/chapitre\_5/ExempleJFrameAvecAnimationRatee.java
// Tentative d'animation par itération d'affichage de gauche à droite
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExempleJFrameAvecAnimationRatee
    extends JFrame {

    // Constantes pour la taille de la fenêtre et du crabe
    private static final int LARGEURFENETRE = 400;
    private static final int HAUTEURFENETRE = 600;
    private static final int LARGEURCRABE = LARGEURFENETRE / 4;
    private static final int HAUTEURCRABE = LARGEURCRABE * 2 / 3;

    public ExempleJFrameAvecAnimationRatee() {
        super("Exemple d'animation ratée");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(LARGEURFENETRE, HAUTEURFENETRE);
        this.setVisible(true);
    }

    // Méthode qui dessine un crabe dans un objet Graphics g
    // à l'échelle dans un rectangle englobant de paramètres
    // x,y,largeur,hauteur
    public void paintCrabe(
        Graphics g, int x, int y, int largeur, int hauteur) {
```

```

g.setColor(Color.green);
// Le corps
g.fillOval(
    x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

g.setColor(Color.black);
// L'oeil gauche
g.fillRect(
    x + largeur * 3 / 8,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// L'oeil droit
g.fillRect(
    x + largeur * 5 / 8 - largeur / 12,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// La bouche
g.drawLine(
    x + largeur * 3 / 8,
    y + hauteur * 5 / 8,
    x + largeur * 5 / 8,
    y + hauteur * 5 / 8);

g.setColor(Color.red);
// Les pinces
g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
g.fillRect(
    x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public void paint(Graphics g) {
    super.paint(g);
    for (int x = 0; x <= LARGEURFENETRE - LARGEURCRABE; x = x + 5) {
        paintCrabe(g, x, HAUTEURFENETRE - 2 * HAUTEURCRABE, LARGEURCRABE,
            HAUTEURCRABE);
        try {
            Thread.sleep(50);
        } catch (InterruptedException uneException) {
            System.out.println(uneException.toString());
        }
    }
}

public static void main(String[] args) {
    new ExempleJFrameAvecAnimationRatee();
}
}

```

La méthode *paint()* dessine le crabe à répétition dans une boucle en variant graduellement la coordonnée *x* du crabe de 0 à *LARGEURFENETRE*-*LARGEURCRABE*. La ligne suivante de *paint()* est le *for* de la boucle d'animation.


```
for(int x=0; x<=LARGEURFENETRE-LARGEURCRABE; x = x + 5){
```

Dans la boucle, chacune des scènes est générée en dessinant le crabe à la coordonnée (x, HAUTEURFENETRE-2*HAUTEURCRABE) où x varie de 0 à LARGEURFENETRE-LARGEURCRABE.

```
    paintCrabe(g,x,HAUTEURFENETRE-
2*HAUTEURCRABE,LARGEURCRABE,HAUTEURCRABE);
```

L'appel à *Thread.sleep*(50) introduit un délai de 50 ms afin de ralentir l'animation.

```
Thread.sleep(50);
```

Cet appel est encadré dans un énoncé *try* Java.

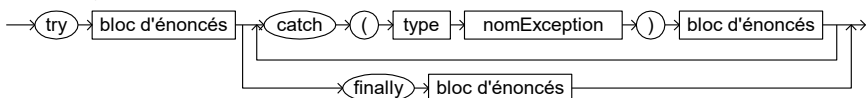
Énoncé *try* et exceptions

Une méthode Java peut interrompre la séquence normale d'exécution avec un énoncé *throw NomException*. Ceci est utile pour signaler que la méthode a rencontré quelque chose d'anormal pendant son exécution. En Java, on distingue deux catégories d'exceptions : *vérifiées* ou non *vérifiées*. Une exception vérifiée exige que l'appel de la méthode se fasse de manière à prévoir la possibilité d'une exception alors que cette gestion de la condition d'erreur est optionnelle dans le cas d'une exception non vérifiée. Par exemple, plusieurs fonctions en Java peuvent générer une exception non vérifiée liée à un manque de mémoire (exception de type *OutOfMemoryError*) et ce type d'exception n'est généralement pas géré explicitement : dans ce cas, le programme se termine avec une erreur.

On gère les exceptions en encadrant l'appel dans un énoncé *try*. Par exemple, la méthode *Thread.sleep()* peut soulever une exception dans la catégorie vérifiée. La méthode qui contient l'appel *Thread.sleep()* attrape l'exception avec un énoncé *try*. Si la fonction ne peut gérer l'exception, elle peut s'en remettre aux fonctions l'appelant en déclarant qu'elle génère des exceptions en ajoutant une clause *throws* à la définition de la fonction.

Pour attraper l'exception avec un énoncé *try*, la syntaxe est :

énoncé *try* :



Il peut y avoir plusieurs *catch* si la méthode soulève plusieurs types d'exceptions. Le type d'une exception est une classe Java. Un certain nombre de classes d'exception sont prédéfinies. Le bloc d'énoncé du *catch* est exécuté si l'exception du type spécifié est levée par la méthode appelée.

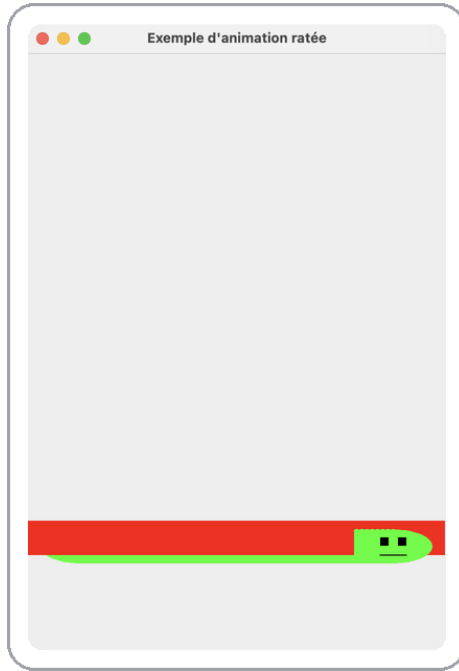
Dans notre exemple, si une exception de type *InterruptedException* est levée par l'appel à *Thread.sleep()*, un message est affiché par :

```
System.err.println(uneException.toString()); }
```

Il est à noter que l'objet *System.err* est employé pour l'affichage plutôt que *System.out*. Cet objet représente une zone de texte réservée aux messages d'erreurs. Dans le cas d'une exécution avec la ligne de commande Windows, *System.err* et *System.out* sont tous les deux assignés à la fenêtre de l'invite de commande. Dans d'autres environnements, ces deux zones sont souvent affichées dans des fenêtres différentes.

Le message est produit par *uneException.toString()* qui retourne un message correspondant à l'exception levée. La partie *finally* est optionnelle et n'apparaît pas dans notre exemple. Le bloc d'énoncé du *finally* est exécuté que l'exception soit levée ou pas.

L'exécution du programme produit le résultat final suivant qui n'est pas ce qui est visé ... Un problème vient du fait qu'avant de dessiner le crabe à une nouvelle position, il faut l'effacer de la position précédente.



Exemple. L'effacement du crabe peut être effectué en appelant *clearRect()* en fin de boucle dans la méthode *paint()*. Cette méthode rétablit la couleur de fond d'écran pour le rectangle spécifié par les paramètres.

```
public void paint (Graphics g) {  
    super.paint(g);  
    for(int x=0; x<=LARGEURFENETRE-LARGEURCRABE; x = x + 5){  
        paintCrabe(g,x,HAUTEURFENETRE-  
2*HAUTEURCRABE, LARGEURCRABE, HAUTEURCRABE);  
        try {  
            Thread.sleep(50);  
        }  
        catch(InterruptedException uneException){  
            System.out.println(uneException.toString());  
        }  
        g.clearRect(x,HAUTEURFENETRE-2*HAUTEURCRABE, LARGEURCRABE,  
            HAUTEURCRABE);  
    }  
}
```

Malheureusement, ceci produit un effet désagréable de scintillement parce que les opérations de dessin ne sont pas instantanées. L'œil perçoit le processus de dessin, ce qui produit cet effet.

6.2 Animation par double tampon

Pour éviter le scintillement, il faut employer la technique de *double tampon*. Plutôt que de dessiner le crabe directement sur le contexte graphique de la fenêtre, on utilise un autre contexte graphique pour le dessin de la nouvelle scène. La figure suivante montre les deux contextes graphiques au début d'une itération d'animation pour notre exemple de déplacement du crabe de gauche à droite.



Figure 21. Double tampon.

Les opérations de dessin sont effectuées dans le deuxième contexte graphique. La position du crabe est légèrement décalée vers la droite dans le tampon.

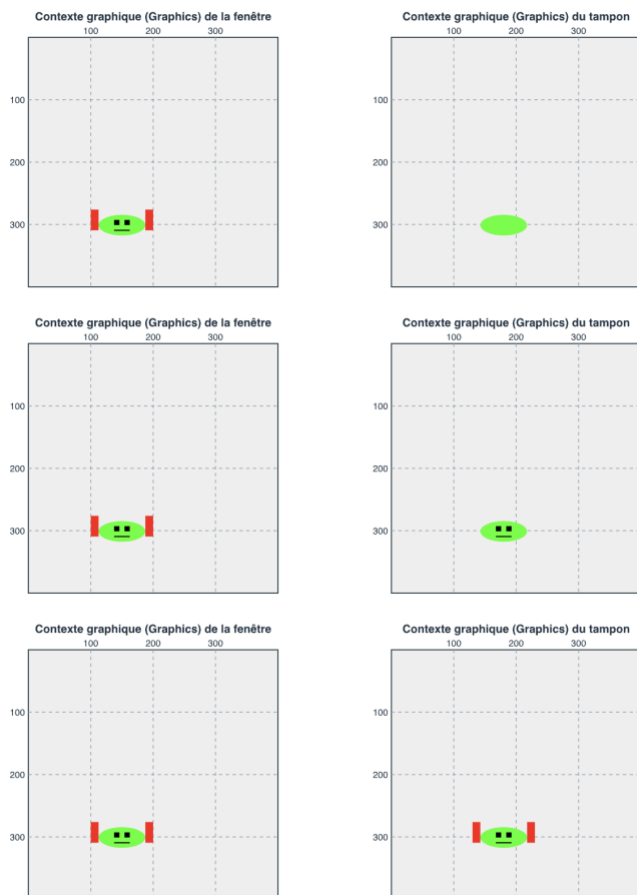


Figure 22. Dessin effectué sur le tampon.

Lorsque le dessin est terminé, l'image du deuxième contexte graphique est copiée directement d'un coup sur le contexte graphique de la fenêtre puis le crabe est effacé dans le tampon pour l'itération suivante. Ceci produit un effet d'animation beaucoup plus convaincant.

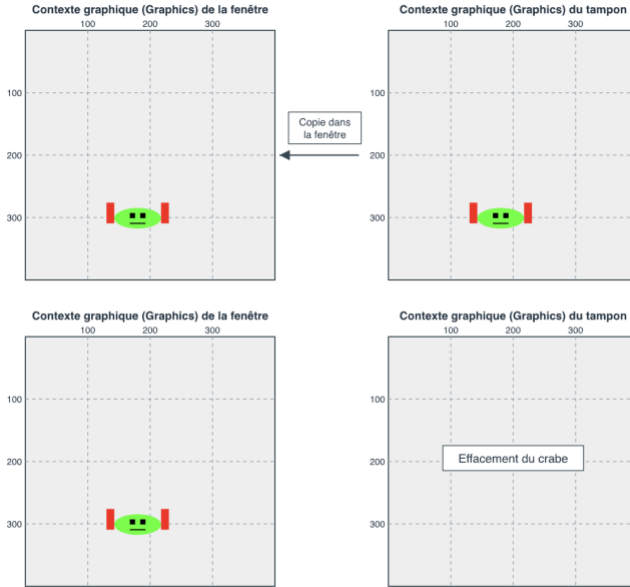


Figure 23. Copie de l'image du tampon dans le contexte graphique de la fenêtre.

Alternance de tampon (page flipping)

Dans les applications graphiques à haute performance, en particulier pour les jeux d'ordinateur, il est avantageux d'éviter la copie de tampon en alternant la zone de mémoire associée à l'écran. Le mécanisme d'alternance peut être effectué par la carte graphique afin de le rendre rapide. Les deux contextes graphiques ont ainsi un rôle symétrique. Il est possible de réaliser cette stratégie d'animation en Java si la carte graphique le permet. Nous reviendrons sur ces détails de mise en œuvre par la suite.

Le programme suivant produit l'animation du crabe par double tampon.

```
Exemple. JavaPasAPas/chapitre_5/ExempleJFrameAnimationDoubleTampon.java
// Animation par double tampon
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExempleJFrameAnimationDoubleTampon
    extends JFrame {
```

```
// Constantes pour la taille de la fenêtre et du crabe
private static final int LARGEURFENETRE = 400;
private static final int HAUTEURFENETRE = 400;
private static final int LARGEURCRABE = LARGEURFENETRE / 4;
private static final int HAUTEURCRABE = LARGEURCRABE * 2 / 3;

// Tampon pour construire l'image avant d'afficher
Graphics tamponGraphics;
Image tamponImage;

public ExempleJFrameAnimationDoubleTampon() {
    super("Exemple d'animation par double tampon");
    this.setDefaultCloseOperation(EXIT_ON_CLOSE);
    this.setSize(LARGEURFENETRE, HAUTEURFENETRE);
    this.setVisible(true);
}

// Méthode qui dessine un crabe dans un objet Graphics g
// à l'échelle dans un rectangle englobant de paramètres
// x,y,largeur,hauteur
public void paintCrabe(
    Graphics g, int x, int y, int largeur, int hauteur) {
    g.setColor(Color.green);
    // Le corps
    g.fillOval(
        x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

    g.setColor(Color.black);
    // L'oeil gauche
    g.fillRect(
        x + largeur * 3 / 8,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // L'oeil droit
    g.fillRect(
        x + largeur * 5 / 8 - largeur / 12,
        y + hauteur * 3 / 8,
        largeur / 12,
        hauteur / 8);
    // La bouche
    g.drawLine(
        x + largeur * 3 / 8,
        y + hauteur * 5 / 8,
        x + largeur * 5 / 8,
        y + hauteur * 5 / 8);

    g.setColor(Color.red);
    // Les pinces
    g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
    g.fillRect(
        x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public void paint(Graphics g) {
    super.paint(g);
```

```

tamponImage = createImage(LARGEURFENETRE, HAUTEURFENETRE);
tamponGraphics = tamponImage.getGraphics();
for (int x = 0; x <= LARGEURFENETRE - LARGEURCRABE; x = x + 5) {
    // Dessine le crabe dans le tampon
    paintCrabe(tamponGraphics, x, HAUTEURFENETRE - 2 * HAUTEURCRABE,
        LARGEURCRABE, HAUTEURCRABE);
    // Copie le tampon dans le contexte graphique de la fenêtre
    g.drawImage(tamponImage, 0, 0, this);
    try {
        Thread.sleep(50);
    } catch (InterruptedException uneException) {
        System.out.println(uneException.toString());
    }
    // Efface le crabe
    tamponGraphics.clearRect(x, HAUTEURFENETRE - 2 * HAUTEURCRABE,
        LARGEURCRABE, HAUTEURCRABE);
}

public static void main(String[] args) {
    new ExempleJFrameAnimationDoubleTampon();
}
}

```

Dans la méthode *paint()* de programme, la ligne suivante crée un objet de la classe *Image* qui sert de tampon :

```
tamponImage = createImage(LARGEURFENETRE, HAUTEURFENETRE);
```

La ligne suivante extrait le contexte graphique *tamponGraphics* qui correspond à l'objet *tamponImage* :

```
tamponGraphics = tamponImage.getGraphics();
```

Les dessins sont par la suite effectués dans ce contexte graphique, par exemple :

```

paintCrabe(tamponGraphics, x, HAUTEURFENETRE -
2 * HAUTEURCRABE, LARGEURCRABE, HAUTEURCRABE);

```

L'image *tamponImage* est ensuite copiée dans le contexte graphique *g* de la fenêtre à l'écran par :


```
g.drawImage(tamponImage,0,0,this);
```

Exercice. Modifiez le programme précédent de manière à ce que le crabe inverse sa direction de déplacement lorsqu'il atteint le bord de la fenêtre.

Solution. [JavaPasAPas/chapitre_5/ExerciceJFrameAvecCrabeRebondissant.java](#)

```
// Animation par double tampon
// Le crabe rebondit lorsqu'il atteint le bord de la fenêtre
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExerciceJFrameAvecCrabeRebondissant
    extends JFrame {

    // Constantes pour la taille de la fenêtre et du crabe
    private static final int LARGEURFENETRE = 400;
    private static final int HAUTEURFENETRE = 400;
    private static final int LARGEURCRABE = LARGEURFENETRE / 4;
    private static final int HAUTEURCRABE = LARGEURCRABE * 2 / 3;

    // Tampon pour construire l'image avant d'afficher
    Graphics tamponGraphics;
    Image tamponImage;

    public ExerciceJFrameAvecCrabeRebondissant() {
        super("Crabe rebondissant");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(LARGEURFENETRE, HAUTEURFENETRE);
        this.setVisible(true);
    }

    // Méthode qui dessine un crabe dans un objet Graphics g
    // à l'échelle dans un rectangle englobant de paramètres
    // x,y,largeur,hauteur
    public void paintCrabe(
        Graphics g, int x, int y, int largeur, int hauteur) {
        g.setColor(Color.green);
        // Le corps
        g.fillOval(
            x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

        g.setColor(Color.black);
        // L'oeil gauche
        g.fillRect(
            x + largeur * 3 / 8,
            y + hauteur * 3 / 8,
            largeur / 12,
            hauteur / 8);
        // L'oeil droit
        g.fillRect(
            x + largeur * 5 / 8 - largeur / 12,
            y + hauteur * 3 / 8,
```

```

        largeur / 12,
        hauteur / 8);
// La bouche
g.drawLine(
    x + largeur * 3 / 8,
    y + hauteur * 5 / 8,
    x + largeur * 5 / 8,
    y + hauteur * 5 / 8);

g.setColor(Color.red);
// Les pinces
g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
g.fillRect(
    x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public void paint(Graphics g) {
    super.paint(g);
    tamponImage = createImage(LARGEURFENETRE, HAUTEURFENETRE);
    tamponGraphics = tamponImage.getGraphics();
    int x = 0; // Coordonnée x du crabe
    int directionCrabe = 1; // +1 vers la droite et -1 vers la gauche
    // Nombre d'unités de déplacement à chaque itération de la boucle
    int vitesseCrabe = 5;
    while (true) {
        // Dessine le crabe dans le tampon
        paintCrabe(tamponGraphics, x, HAUTEURFENETRE - 2 * HAUTEURCRABE,
            LARGEURCRABE, HAUTEURCRABE);
        // Copie le tampon dans le contexte graphique de la fenêtre
        g.drawImage(tamponImage, 0, 0, this);
        try {
            Thread.sleep(50);
        } catch (InterruptedException uneException) {
            System.out.println(uneException.toString());
        }
        // Efface le crabe
        tamponGraphics.clearRect(x, HAUTEURFENETRE - 2 * HAUTEURCRABE,
            LARGEURCRABE, HAUTEURCRABE);
        // Déplace le crabe
        if (x + LARGEURCRABE >= LARGEURFENETRE | x < 0) // Si atteint le bord
            directionCrabe = -directionCrabe; // Inverser la direction
        x = x + vitesseCrabe * directionCrabe; // Déplacement du crabe
    }
}

public static void main(String[] args) {
    new ExerciceJFrameAvecCrabeRebondissant();
}
}

```

Note : la fenêtre ne peut être fermée !

Un problème avec l'animation précédente est le fait que la boucle d'animation est infinie ! Lorsque l'on tente de fermer la fenêtre, rien ne se produit car la méthode *paint()* ne peut être interrompue. La méthode *paint()* ne devrait pas être employée de cette manière dans une application. Nous employons cette approche dans un premier temps afin de simplifier la présentation des concepts d'animation.

Pour arrêter le programme dans une fenêtre de commande Windows, vous pouvez taper <ctrl>-C²⁴. Nous verrons par la suite une solution à ce problème qui consiste à effectuer la boucle d'animation à l'extérieur de la méthode *paint()*.

Exercice. Animez votre personnage préféré (par exemple, la soucoupe) en le déplaçant de haut en bas et lorsqu'il touche au bord de la fenêtre, inverser la direction.

```
Solution. JavaPasAPas/chapitre_5/
ExerciceJFrameAvecSoucoupeRebondissante.java
// Animation par double tampon
// La soucoupe se déplace à la verticale
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExerciceJFrameAvecSoucoupeRebondissante
    extends JFrame {

    // Constantes pour la taille de la fenêtre et de la soucoupe
    private static final int LARGEURFENETRE = 400;
    private static final int HAUTEURFENETRE = 400;
    private static final int LARGEURSOUCOPE = LARGEURFENETRE / 4;
    private static final int HAUTEURSOUCOPE = LARGEURSOUCOPE / 2;

    // Tampon pour construire l'image avant d'afficher
    Graphics tamponGraphics;
    Image tamponImage;

    public ExerciceJFrameAvecSoucoupeRebondissante() {
        super("Soucoupe rebondissante");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(LARGEURFENETRE, HAUTEURFENETRE);
        this.setVisible(true);
    }

    // Méthode qui dessine une soucoupe dans un objet Graphics g
```

²⁴ Tapez la letter c alors que la touche «ctrl» est enfoncée.

```

// à l'échelle dans un rectangle englobant de paramètres
// x,y,largeur,hauteur
public void paintSoucoupe(
    Graphics g, int x, int y, int largeur, int hauteur) {
    g.setColor(Color.cyan);
    // Le dôme
    g.fillOval(x + largeur / 4, y, largeur / 2, hauteur * 2 / 3);

    g.setColor(Color.red);
    // Le disque
    g.fillOval(x, y + hauteur / 3, largeur, hauteur * 2 / 3);

    g.setColor(Color.yellow);
    // Les hublots
    g.fillOval(x + largeur / 6, y + hauteur / 2, largeur / 8, hauteur / 4);
    g.fillOval(
        x + largeur / 2 - largeur / 16, y + hauteur / 2, largeur / 8,
        hauteur / 4);
    g.fillOval(
        x + largeur * 5 / 6 - largeur / 8, y + hauteur / 2, largeur / 8,
        hauteur / 4);
}

public void paint(Graphics g) {
    super.paint(g);
    tamponImage = createImage(LARGEURFENETRE, HAUTEURFENETRE);
    tamponGraphics = tamponImage.getGraphics();
    int x = LARGEURFENETRE - 2 * LARGEURSOUCOUPE; // Coordonnée x fixe
    int y = 0; // Coordonnée y de la soucoupe
    int directionSoucoupe = 1; // +1 vers le bas et -1 vers le haut
    // Nombre d'unités de déplacement à chaque itération de la boucle
    int vitesseSoucoupe = 5;
    while (true) {
        // Dessine la soucoupe dans le tampon
        paintSoucoupe(tamponGraphics, x, y, LARGEURSOUCOUPE,
            HAUTEURSOUCOUPE);
        // Copie le tampon dans le contexte graphique de la fenêtre
        g.drawImage(tamponImage, 0, 0, this);
        try {
            Thread.sleep(50);
        } catch (InterruptedException uneException) {
            System.out.println(uneException.toString());
        }
        // Efface la soucoupe
        tamponGraphics.clearRect(x, y, LARGEURSOUCOUPE, HAUTEURSOUCOUPE);
        // Déplace la soucoupe
        // Si atteint le bord
        if (y + HAUTEURSOUCOUPE >= HAUTEURFENETRE | y < 0)
            directionSoucoupe = -directionSoucoupe; // Inverser la direction
        y = y + vitesseSoucoupe * directionSoucoupe; // Déplacement
    }
}

public static void main(String[] args) {
    new ExerciceJFrameAvecSoucoupeRebondissante();
}

```

```
}  
}
```

Note : L'espace 2D du contexte graphique de la fenêtre correspond à toute la surface de la fenêtre incluant les bordures. Ainsi, la partie supérieure du contexte graphique est occupée par la barre de titre de la fenêtre. Les dessins effectués dans cette portion du contexte graphique sont cachés par la barre de titre. Ceci est apparent lorsque la soucoupe atteint le haut de la fenêtre. Pour éviter ce problème, il faut arrêter la soucoupe à la valeur de y qui correspond à la hauteur de la barre de titre.

Le crabe peut se déplacer en diagonale avec une vitesse de déplacement selon l'axe x et une autre selon l'axe y . Lorsqu'un bord de fenêtre gauche ou droit est atteint, ceci inverse la direction dans l'axe x . Lorsqu'un bord supérieur ou inférieur est atteint, ceci inverse la direction dans l'axe y .

Exercice. `JavaPasAPas/chapitre_5/ ExerciceJFrameAvecPingPongCrabe.java`

```
// Le crabe se déplace en diagonale  
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;  
  
public class ExerciceJFrameAvecPingPongCrabe  
    extends JFrame {  
  
    // Constantes pour la taille de la fenêtre et du crabe  
    private static final int LARGEURFENETRE = 400;  
    private static final int HAUTEURFENETRE = 400;  
    private static final int LARGEURCRABE = LARGEURFENETRE / 4;  
    private static final int HAUTEURCRABE = LARGEURCRABE * 2 / 3;  
  
    // Tampon pour construire l'image avant d'afficher  
    Graphics tamponGraphics;  
    Image tamponImage;  
  
    public ExerciceJFrameAvecPingPongCrabe() {  
        super("Ping pong crabe");  
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);  
        this.setSize(LARGEURFENETRE, HAUTEURFENETRE);  
        this.setVisible(true);  
    }  
  
    // Méthode qui dessine un crabe dans un objet Graphics g  
    // à l'échelle dans un rectangle englobant de paramètres  
    // x,y,largeur,hauteur  
    public void paintCrabe(  
        Graphics g, int x, int y, int largeur, int hauteur) {  
        g.setColor(Color.green);
```

```

// Le corps
g.fillOval(
    x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

g.setColor(Color.black);
// L'oeil gauche
g.fillRect(
    x + largeur * 3 / 8,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// L'oeil droit
g.fillRect(
    x + largeur * 5 / 8 - largeur / 12,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// La bouche
g.drawLine(
    x + largeur * 3 / 8,
    y + hauteur * 5 / 8,
    x + largeur * 5 / 8,
    y + hauteur * 5 / 8);

g.setColor(Color.red);
// Les pinces
g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
g.fillRect(
    x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

public void paint(Graphics g) {
    super.paint(g);
    tamponImage = createImage(LARGEURFENETRE, HAUTEURFENETRE);
    tamponGraphics = tamponImage.getGraphics();

    int xCrabe = 0; // Coordonnées du crabe
    int yCrabe = 0;
    int vitesseXCrabe = 5; // Vitesse du crabe
    int vitesseYCrabe = 10;

    while (true) {
        // Dessine le crabe dans le tampon
        paintCrabe(tamponGraphics, xCrabe, yCrabe, LARGEURCRABE,
            HAUTEURCRABE);
        // Copie le tampon dans le contexte graphique de la fenêtre
        g.drawImage(tamponImage, 0, 0, this);
        try {
            Thread.sleep(50);
        } catch (InterruptedException uneException) {
            System.out.println(uneException.toString());
        }
        // Efface le crabe
        tamponGraphics.clearRect(xCrabe, yCrabe, LARGEURCRABE,
            HAUTEURCRABE);
        // Déplace le crabe
    }
}

```

```

    // Si atteint le bord, inverser la direction selon x
    if (xCrabe + LARGEURCRABE >= LARGEURFENETRE | xCrabe < 0)
        vitesseXCrabe = -vitesseXCrabe;
    xCrabe = xCrabe + vitesseXCrabe; // Déplacement du crabe selon x
    // Si atteint le bord, inverser la direction selon y
    if (yCrabe + HAUTEURCRABE >= HAUTEURFENETRE | yCrabe < 0)
        vitesseYCrabe = -vitesseYCrabe;
    yCrabe = yCrabe + vitesseYCrabe; // Déplacement du crabe selon y
}
}

public static void main(String[] args) {
    new ExerciceJFrameAvecPingPongCrabe();
}
}

```

Exercice. Maintenant combinez quelques crabes et soucoupes dans la même animation avec des positions initiales, vitesses et tailles différentes.

Solution. [JavaPasAPas/chapitre_5/](#)

ExerciceJFrameAvecPingPongCrabesEtSoucoupes.java

```

// Plusieurs crabes et soucoupes qui bougent
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExerciceJFrameAvecPingPongCrabesEtSoucoupes
    extends JFrame {

    // Constantes pour la taille de la fenêtre
    private static final int LARGEURFENETRE = 400;
    private static final int HAUTEURFENETRE = 400;

    // Tampon pour construire l'image avant d'afficher
    Graphics tamponGraphics;
    Image tamponImage;

    public ExerciceJFrameAvecPingPongCrabesEtSoucoupes() {
        super("Ping pong crabes et soucoupes");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(LARGEURFENETRE, HAUTEURFENETRE);
        this.setVisible(true);
    }

    // Méthode qui dessine un crabe dans un objet Graphics g
    // à l'échelle dans un rectangle englobant de paramètres
    // x,y,largeur,hauteur
    public void paintCrabe(
        Graphics g, int x, int y, int largeur, int hauteur) {
        g.setColor(Color.green);
        // Le corps
        g.fillOval(
            x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);
    }
}

```

```

g.setColor(Color.black);
// L'oeil gauche
g.fillRect(
    x + largeur * 3 / 8,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// L'oeil droit
g.fillRect(
    x + largeur * 5 / 8 - largeur / 12,
    y + hauteur * 3 / 8,
    largeur / 12,
    hauteur / 8);
// La bouche
g.drawLine(
    x + largeur * 3 / 8,
    y + hauteur * 5 / 8,
    x + largeur * 5 / 8,
    y + hauteur * 5 / 8);

g.setColor(Color.red);
// Les pinces
g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
g.fillRect(
    x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
}

// Méthode qui dessine une soucoupe dans un objet Graphics g
// à l'échelle dans un rectangle englobant de paramètres
// x,y,largeur,hauteur
public void paintSoucoupe(
    Graphics g, int x, int y, int largeur, int hauteur) {
    g.setColor(Color.cyan);
    // Le dôme
    g.fillOval(x + largeur / 4, y, largeur / 2, hauteur * 2 / 3);

    g.setColor(Color.red);
    // Le disque
    g.fillOval(x, y + hauteur / 3, largeur, hauteur * 2 / 3);

    g.setColor(Color.yellow);
    // Les hublots
    g.fillOval(x + largeur / 6, y + hauteur / 2, largeur / 8, hauteur / 4);
    g.fillOval(
        x + largeur / 2 - largeur / 16, y + hauteur / 2, largeur / 8,
        hauteur / 4);
    g.fillOval(
        x + largeur * 5 / 6 - largeur / 8, y + hauteur / 2, largeur / 8,
        hauteur / 4);
}

public void paint(Graphics g) {
    super.paint(g);
    tamponImage = createImage(LARGEURFENETRE, HAUTEURFENETRE);
    tamponGraphics = tamponImage.getGraphics();
}

```



```
int xCrabe1 = 0; // Coordonnées du crabe1
int yCrabe1 = 100;
int vitesseXCrabe1 = 5; // Vitesse du crabe1
int vitesseYCrabe1 = 0;
int largeurCrabe1 = 120; // Taille du crabe1
int hauteurCrabe1 = 80;

int xCrabe2 = 100; // Coordonnées du crabe2
int yCrabe2 = 100;
int vitesseXCrabe2 = -10; // Vitesse du crabe2
int vitesseYCrabe2 = 5;
int largeurCrabe2 = 75; // Taille du crabe2
int hauteurCrabe2 = 50;

int xSoucoupe1 = 200; // Coordonnées de la soucoupe1
int ySoucoupe1 = 300;
int vitesseXSoucoupe1 = 6; // Vitesse de la soucoupe1
int vitesseYSoucoupe1 = 6;
int largeurSoucoupe1 = 100; // Taille de la soucoupe1
int hauteurSoucoupe1 = 44;

int xSoucoupe2 = 200; // Coordonnées de la soucoupe2
int ySoucoupe2 = 0;
int vitesseXSoucoupe2 = 0; // Vitesse de la soucoupe2
int vitesseYSoucoupe2 = 10;
int largeurSoucoupe2 = 64; // Taille de la soucoupe2
int hauteurSoucoupe2 = 28;

while (true) {
    // Dessine les crabes et les soucoupes
    paintCrabe(tamponGraphics, xCrabe1, yCrabe1,
        largeurCrabe1, hauteurCrabe1);
    paintCrabe(tamponGraphics, xCrabe2, yCrabe2,
        largeurCrabe2, hauteurCrabe2);
    paintSoucoupe(tamponGraphics, xSoucoupe1, ySoucoupe1,
        largeurSoucoupe1, hauteurSoucoupe1);
    paintSoucoupe(tamponGraphics, xSoucoupe2, ySoucoupe2,
        largeurSoucoupe2, hauteurSoucoupe2);
    // Copie le tampon dans le contexte graphique de la fenêtre
    g.drawImage(tamponImage, 0, 0, this);
    try {
        Thread.sleep(50);
    } catch (InterruptedException uneException) {
        System.out.println(uneException.toString());
    }
    // Efface les crabes et les soucoupes
    tamponGraphics.clearRect(xCrabe1, yCrabe1,
        largeurCrabe1, hauteurCrabe1);
    tamponGraphics.clearRect(xCrabe2, yCrabe2,
        largeurCrabe2, hauteurCrabe2);
    tamponGraphics.clearRect(xSoucoupe1, ySoucoupe1,
        largeurSoucoupe1, hauteurSoucoupe1);
    tamponGraphics.clearRect(xSoucoupe2, ySoucoupe2,
        largeurSoucoupe2, hauteurSoucoupe2);
}
```

```

// Déplace le crabe1 (au bord, inverser la direction)
if (xCrabe1 + largeurCrabe1 >= LARGEURFENETRE | xCrabe1 < 0)
    vitesseXCrabe1 = -vitesseXCrabe1;
xCrabe1 = xCrabe1 + vitesseXCrabe1; // Déplacement selon x
if (yCrabe1 + hauteurCrabe1 >= HAUTEURFENETRE | yCrabe1 < 0)
    vitesseYCrabe1 = -vitesseYCrabe1;
yCrabe1 = yCrabe1 + vitesseYCrabe1; // Déplacement selon y

// Déplace le crabe2 (au bord, inverser la direction)
if (xCrabe2 + largeurCrabe2 >= LARGEURFENETRE | xCrabe2 < 0)
    vitesseXCrabe2 = -vitesseXCrabe2;
xCrabe2 = xCrabe2 + vitesseXCrabe2; // Déplacement selon x
if (yCrabe2 + hauteurCrabe2 >= HAUTEURFENETRE | yCrabe2 < 0)
    vitesseYCrabe2 = -vitesseYCrabe2;
yCrabe2 = yCrabe2 + vitesseYCrabe2; // Déplacement selon y

// Déplace la soucoupe1 (au bord, inverser la direction)
if (xSoucoupe1 + largeurSoucoupe1 >= LARGEURFENETRE | xSoucoupe1 < 0)
    vitesseXSoucoupe1 = -vitesseXSoucoupe1;
xSoucoupe1 = xSoucoupe1 + vitesseXSoucoupe1; // Déplacement selon x
if (ySoucoupe1 + hauteurSoucoupe1 >= HAUTEURFENETRE | ySoucoupe1 < 0)
    vitesseYSoucoupe1 = -vitesseYSoucoupe1;
ySoucoupe1 = ySoucoupe1 + vitesseYSoucoupe1; // Déplacement selon y

// Déplace la soucoupe2 (au bord, inverser la direction)
if (xSoucoupe2 + largeurSoucoupe2 >= LARGEURFENETRE | xSoucoupe2 < 0)
    vitesseXSoucoupe2 = -vitesseXSoucoupe2;
xSoucoupe2 = xSoucoupe2 + vitesseXSoucoupe2; // Déplacement selon x
if (ySoucoupe2 + hauteurSoucoupe2 >= HAUTEURFENETRE | ySoucoupe2 < 0)
    vitesseYSoucoupe2 = -vitesseYSoucoupe2;
ySoucoupe2 = ySoucoupe2 + vitesseYSoucoupe2; // Déplacement selon y
    }
}

public static void main(String[] args) {
    new ExerciceJFrameAvecPingPongCrabesEtSoucoupes();
}
}

```

Le programme de l'exercice précédent est assez compliqué et les possibilités d'erreurs de codage se multiplient ! Le prochain chapitre montre comment mieux organiser le programme en exploitant de manière judicieuse la notion d'objet et de classe Java.

7. Développement de classes : conception objet

« Ne vous répétez pas : chaque élément de connaissance doit avoir une seule représentation non ambiguë et faisant autorité dans un système. » — Andy Hunt et Dave Thomas

La *conception objet* désigne le problème de conception d'un programme en utilisant des objets. Un aspect particulièrement important à considérer est le découpage du programme en classes. Ce chapitre passe en revue quelques notions de base de la conception objet : diviser pour régner, encapsulation, interface, cohésion, couplage, et relation d'héritage. Les principes de Java touchant à l'organisation, la compilation et l'exécution d'un programme composé de plusieurs classes sont aussi abordés.

7.1 Découpage d'un programme en classes

Une manière typique de faciliter le développement d'un programme Java complexe est de le découper en plusieurs classes. C'est le principe de la tarte.

Génie logiciel : principe de la tarte (diviser pour régner)

Si la tarte est trop grosse pour être gobée d'un coup, il est préférable de la couper en morceaux²⁵.

La notion de classe en programmation objet permet de tirer profit de ce principe d'une manière très efficace en regroupant ensemble les variables et méthodes qui sont fortement liées.

Exemple. L'exemple suivant opère une réorganisation du programme du dernier exercice du chapitre précédent avec plusieurs crabes et soucoupes. Il est recommandé d'étudier la solution *ExerciceJFrameAvecPingPongCrabesEtSoucoupes* avant de poursuivre ! Plutôt que de tout mettre dans une classe *ExerciceJFrameAvecPingPongCrabesEtSoucoupes*, deux nouvelles classes sont créées, une pour les crabes, appelée *CrabeRebondissant*, et une autre pour les soucoupes, *SoucoupeRebondissante*.

Ce principe est aussi appelé « diviser pour régner ». D'autre part, si le programmeur est une tarte, il est préférable de le changer plutôt que de le découper en morceaux ...

Chacun des crabes à animer sera représenté dans le programme par un objet de la classe *CrabeRebondissant*, et de même pour les soucoupes.

Voici le code de la classe *CrabeRebondissant*. Rappelons que l'absence de *extends* est équivalente à *extends java.lang.Object*.

```
JavaPasAPas/chapitre_6/Envahisseurs/CrabeRebondissant.java
import java.awt.*;

public class CrabeRebondissant {
    // Variables d'objet qui décrivent l'état de l'objet
    private int x, y; // Coordonnées du crabe
    private int largeur, hauteur; // Taille du crabe
    private int vitesseX; // Vitesse de déplacement dans l'axe x
    private int vitesseY; // Vitesse de déplacement dans l'axe y

    // Constructeur pour initialiser l'état du CrabeRebondissant
    public CrabeRebondissant(
        int x, int y, int largeur, int hauteur, int vitesseX, int vitesseY) {
        this.x = x;
        this.y = y;
        this.hauteur = hauteur;
        this.largeur = largeur;
        this.vitesseX = vitesseX;
        this.vitesseY = vitesseY;
    }

    // Déplacement pour la prochaine itération
    public void deplacer(int largeurFenetre, int hauteurFenetre) {
        if (x + largeur >= largeurFenetre | x < 0) // Au bord selon x
            vitesseX = -vitesseX; // Inverser la direction selon x
        x = x + vitesseX; // Déplacement selon x
        if (y + hauteur >= hauteurFenetre | y < 0) // Au bord selon y
            vitesseY = -vitesseY; // Inverser la direction selon y
        y = y + vitesseY; // Déplacement selon y
    }

    // Dessin du crabe
    public void paint(Graphics g) {
        g.setColor(Color.green);
        // Le corps
        g.fillOval(
            x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

        g.setColor(Color.black);
        // L'oeil gauche
        g.fillRect(
            x + largeur * 3 / 8,
            y + hauteur * 3 / 8,
            largeur / 12,
            hauteur / 8);
        // L'oeil droit
        g.fillRect(
            x + largeur * 5 / 8 - largeur / 12,
```

```
        y + hauteur * 3 / 8,  
        largeur / 12,  
        hauteur / 8);  
    // La bouche  
    g.drawLine(  
        x + largeur * 3 / 8,  
        y + hauteur * 5 / 8,  
        x + largeur * 5 / 8,  
        y + hauteur * 5 / 8);  
  
    g.setColor(Color.red);  
    // Les pinces  
    g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);  
    g.fillRect(  
        x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);  
}  
  
// Effacer le rectangle du crabe dans tamponGraphics  
public void effacer(Graphics tamponGraphics) {  
    tamponGraphics.clearRect(x, y, largeur, hauteur);  
}  
}
```

La classe *CrabeRebondissant* contient les variables et méthodes qui concernent un objet crabe. Les variables qui décrivent un objet crabe (coordonnées, taille et vitesse) sont :

```
private int x,y;           // Coordonnées x du crabe  
private int largeur,hauteur; // Taille du crabe  
private int vitesseX;      // Vitesse de déplacement dans l'axe x  
private int vitesseY;      // Vitesse de déplacement dans l'axe y
```

État d'un objet

Les variables d'objet représentent l'*état* d'un objet. Un objet évolue en passant d'un état à un autre suite aux modifications apportées aux variables de l'objet.

Le constructeur *CrabeRebondissant()* initialise les variables d'objet. Ici, le constructeur ne fait qu'assigner les valeurs des paramètres aux variables d'objet correspondantes.

```
// Constructeur pour initialiser l'état du CrabeRebondissant
public CrabeRebondissant(
    int x, int y, int largeur, int hauteur,
    int vitesseX, int vitesseY) {
    this.x = x; this.y = y;
    this.hauteur = hauteur; this.largeur = largeur;
    this.vitesseX = vitesseX; this.vitesseY = vitesseY;
}
```

Identificateur réservé *this* suivi d'un nom de variable

Le *this* suivi d'un nom de variable désigne la variable d'objet correspondante.

Le *this* est nécessaire ici pour distinguer le paramètre *x* et la variable d'objet *x* car ils portent le même nom ! Il en est de même pour les autres variables.

En plus du constructeur, les méthodes particulières à un objet *CrabeRebondissant* sont *deplacer()*, *paint()* et *effacer()*. Ce sont des méthodes d'objet car elles accèdent aux variables d'objet de la classe.

Comportement d'un objet

Les méthodes représentent le comportement d'un objet. Elles décrivent les opérations applicables à un objet de la classe.

La méthode *deplacer()* décrit le comportement de déplacement d'un *CrabeRebondissant* pour passer à la scène suivante.

```
// Déplacement pour la prochaine itération
public void deplacer(int largeurFenetre, int hauteurFenetre){
    if (x+largeur>=largeurFenetre | x < 0) // Si atteint le bord selon x
        vitesseX = -vitesseX; // Inverser la direction selon x
    x = x + vitesseX; // Déplacement selon x
    if (y+hauteur>=hauteurFenetre | y < 0) // Si atteint le bord selon y
        vitesseY = -vitesseY; // Inverser la direction selon y
    y = y + vitesseY; // Déplacement selon y
}
```

La méthode utilise toutes les variables d'objet de la classe (*x*, *y*, *largeur*, *hauteur*, *vitesseX* et *vitesseY*). Il est à noter que *largeurFenetre* et *hauteurFenetre* sont passés en paramètre à *deplacer()*.

La méthode *paint()* utilise les variables d'objet *x*, *y*, *largeur* et *hauteur* qui ne sont plus passées en paramètre par opposé à la version sans classes. La méthode *effacer()* utilise les mêmes variables.

La classe *SoucoupeRebondissante* est semblable. La différence avec *CrabeRebondissant* est la méthode *paint()*.

```
JavaPasAPas/chapitre_6/Envahisseurs/SoucoupeRebondissante.java
import java.awt.*;

public class SoucoupeRebondissante {
    // Variables d'objet qui décrivent l'état de l'objet
    private int x, y; // Coordonnées de la soucoupe
    private int largeur, hauteur; // Taille de la soucoupe
    private int vitesseX; // Vitesse de déplacement dans l'axe x
    private int vitesseY; // Vitesse de déplacement dans l'axe y

    // Constructeur pour initialiser l'état de la SoucoupeRebondissante
    public SoucoupeRebondissante(
        int x, int y, int largeur, int hauteur, int vitesseX, int vitesseY) {
        this.x = x;
        this.y = y;
        this.hauteur = hauteur;
        this.largeur = largeur;
        this.vitesseX = vitesseX;
        this.vitesseY = vitesseY;
    }

    // Déplacement pour la prochaine itération
    public void deplacer(int largeurFenetre, int hauteurFenetre) {
        if (x + largeur >= largeurFenetre | x < 0) // Au bord selon x
            vitesseX = -vitesseX; // Inverser la direction selon x
        x = x + vitesseX; // Déplacement selon x
        if (y + hauteur >= hauteurFenetre | y < 0) // Au bord selon y
            vitesseY = -vitesseY; // Inverser la direction selon y
        y = y + vitesseY; // Déplacement selon y
    }

    // Dessin de la soucoupe
    public void paint(Graphics g) {
        g.setColor(Color.cyan);
        // Le dôme
        g.fillOval(x + largeur / 4, y, largeur / 2, hauteur * 2 / 3);

        g.setColor(Color.red);
        // Le disque
        g.fillOval(x, y + hauteur / 3, largeur, hauteur * 2 / 3);

        g.setColor(Color.yellow);
        // Les hublots
        g.fillOval(x + largeur / 6, y + hauteur / 2, largeur / 8, hauteur / 4);
        g.fillOval(
            x + largeur / 2 - largeur / 16, y + hauteur / 2, largeur / 8,
            hauteur / 4);
        g.fillOval(
            x + largeur * 5 / 6 - largeur / 8, y + hauteur / 2, largeur / 8,
            hauteur / 4);
    }
}
```

```
// Effacer le rectangle de la soucoupe dans tamponGraphics
public void effacer(Graphics tamponGraphics) {
    tamponGraphics.clearRect(x, y, largeur, hauteur);
}
}
```

Finalement, voici le code de la classe de la fenêtre *ExempleJFrameAvecClassesPourCrabeEtSoucoupe* qui utilise les deux classes précédentes.

JavaPasAPas/chapitre_6/ ExempleJFrameAvecClassesPourCrabeEtSoucoupe.java

```
// Plusieurs crabes et soucoupes qui bougent
// Utilise les classes CrabeRebondissant et SoucoupeRebondissante
// du package Envahisseurs
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExempleJFrameAvecClassesPourCrabeEtSoucoupe
    extends JFrame {

    // Constantes pour la taille de la fenêtre
    private static final int LARGEURFENETRE = 400;
    private static final int HAUTEURFENETRE = 400;

    // Tampon pour construire l'image avant d'afficher
    Graphics tamponGraphics;
    Image tamponImage;

    public ExempleJFrameAvecClassesPourCrabeEtSoucoupe() {
        super("Ping pong avec classes pour crabes et soucoupes");
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(LARGEURFENETRE, HAUTEURFENETRE);
        this.setVisible(true);
    }

    public void paint(Graphics g) {
        tamponImage = createImage(LARGEURFENETRE, HAUTEURFENETRE);
        tamponGraphics = tamponImage.getGraphics();

        Envahisseurs.CrabeRebondissant crabe1 =
            new Envahisseurs.CrabeRebondissant(0, 100, 120, 80, 5, 0);
        Envahisseurs.CrabeRebondissant crabe2 =
            new Envahisseurs.CrabeRebondissant(100, 100, 75, 50, -10, 5);
        Envahisseurs.SoucoupeRebondissante soucoupe1 =
            new Envahisseurs.SoucoupeRebondissante(200, 300, 100, 44, 6, 6);
        Envahisseurs.SoucoupeRebondissante soucoupe2 =
            new Envahisseurs.SoucoupeRebondissante(200, 0, 64, 28, 0, 10);

        while (true) {
            // Dessine les crabes et les soucoupes
            crabe1.paint(tamponGraphics);
            crabe2.paint(tamponGraphics);
            soucoupe1.paint(tamponGraphics);
            soucoupe2.paint(tamponGraphics);
        }
    }
}
```



```

// Copie le tampon dans le contexte graphique de la fenêtre
g.drawImage(tamponImage, 0, 0, this);
try {
    Thread.sleep(50);
} catch (InterruptedException uneException) {
    System.out.println(uneException.toString());
}
// Efface les crabes et les soucoupes du tampon
crabe1 effacer(tamponGraphics);
crabe2 effacer(tamponGraphics);
soucoupe1 effacer(tamponGraphics);
soucoupe2 effacer(tamponGraphics);

// Déplace les crabes et les soucoupes
crabe1.deplacer(LARGEURFENETRE, HAUTEURFENETRE);
crabe2.deplacer(LARGEURFENETRE, HAUTEURFENETRE);
soucoupe1.deplacer(LARGEURFENETRE, HAUTEURFENETRE);
soucoupe2.deplacer(LARGEURFENETRE, HAUTEURFENETRE);
}
}

public static void main(String[] args) {
    new ExempleJFrameAvecClassesPourCrabeEtSoucoupe();
}
}

```

La classe *ExempleJFrameAvecClassesPourCrabeEtSoucoupe* utilise les classes *CrabeRebondissant* et *SoucoupeRebondissante*. On dit que la classe *fait appel aux services* fournis par les classes *CrabeRebondissant* et *SoucoupeRebondissante*. Le diagramme de Figure 24 montre comment représenter le fait que *ExempleJFrameAvecClassesPourCrabeEtSoucoupe* utilise les classes *CrabeRebondissant* et *SoucoupeRebondissante* par une flèche pointillée appelée relation de dépendance en UML. Le diagramme montre aussi comment représenter les attributs et les méthodes dans un diagramme de classe UML. Les attributs apparaissent dans le deuxième sous-rectangle à l'intérieur du rectangle de la classe et les méthodes dans le troisième. Le nom d'un attribut peut être suivi de : et du type de l'attribut. Le nom d'une méthode est suivi optionnellement de ses paramètres. Le souligné désigne une propriété (attribut ou méthode) de classe (*static* en Java). Le symbole - avant une propriété correspond à la visibilité *private* de Java et le +, à *public*. Lorsque le nombre de classes devient important, un tel diagramme permet d'en faciliter la compréhension.

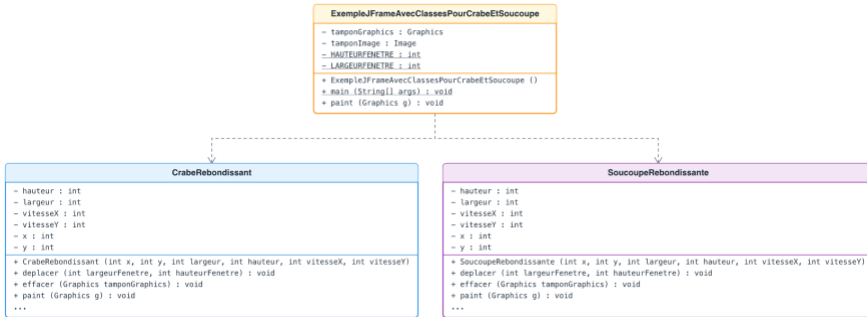


Figure 24. Diagramme UML des classes.

Tous les détails du fonctionnement des crabes et des soucoupes sont cachés dans les classes *CrabeRebondissant* et *SoucoupeRebondissante* du point de vue de la classe *ExempleJFrameAvecClassesPourCrabeEtSoucoupe*.

Encapsulation, interface programmatique (Application Programming Interface - API), service, client

Cette manière d'isoler une classe de détails d'une autre classe est une caractéristique de la programmation objet appelée l'*encapsulation*. Tout ce que la classe *ExempleJFrameAvecClassesPourCrabeEtSoucoupe* doit savoir, c'est comment appeler les méthodes appropriées de la classe *CrabeRebondissant* (*CrabeRebondissant()*, *deplacer()*, *paint()*). Elle n'a pas besoin de comprendre comment cela se passe à l'intérieur des méthodes appelées. Ainsi la classe *CrabeRebondissant* fournit une abstraction de mécanismes complexes sous forme d'un ensemble de méthodes simples à appeler. Dans le langage objet, cet ensemble de méthode est appelé une *interface programmatique*. On dit aussi que la classe *CrabeRebondissant* fournit un *service* à la classe *ExempleJFrameAvecClassesPourCrabeEtSoucoupe* qui est le *client* de ce service. Le client voit l'interface programmatique mais pas la mise en œuvre.

Principes de génie logiciel : cohésion forte et couplage faible entre classes

Une question fondamentale au cœur de la conception d'un programme objet est la manière de répartir les variables et méthodes entre les classes. Deux principes fondamentaux sont de chercher à maximiser la cohésion des classes et de minimiser le couplage entre les classes. La *cohésion* à l'intérieur

d'une classe est forte lorsque les variables et méthodes de la classe sont fortement liées. Le couplage entre deux classes est *faible* lorsqu'il y a peu de dépendances entre les classes. Concrètement la dépendance entre classes est déterminée par l'utilisation d'une autre classe en passant par des déclarations, utilisation des variables ou appels de méthodes et passage de paramètres.

Dans notre exemple, les méthodes *deplacer()*, *paint()* et *effacer()* utilisent toutes une grande proportion des variables de classe *CrabeRebondissant*, ce qui est un signe de grande cohésion de cette classe.

D'autre part, le fait de passer les valeurs *largeurFenetre* et *hauteurFenetre* en paramètres à *deplacer()* est un indice de couplage entre *ExempleJFrameAvecClassesPourCrabeEtSoucoupe* et *CrabeRebondissant/SoucoupeRebondissante* car cette méthode est appelée dans *paint()* de *ExempleJFrameAvecClassesPourCrabeEtSoucoupe*.

Minimiser le couplage signifie, entre autres, de chercher à réduire le nombre de paramètres passés lorsque cela est approprié. Par exemple, une possibilité serait de faire des variables *largeurFenetre* et *hauteurFenetre*, des variables d'objet des classes *CrabeRebondissant/SoucoupeRebondissante*. Comme avantage, on pourrait éviter de passer ces paramètres à chacun des appels à *deplacer()*. Comme inconvénient, il faudrait répéter la même information dans chacun des objets de *CrabeRebondissant/SoucoupeRebondissante*. Généralement, le fait de répéter la même information à plusieurs endroits est à éviter. Pour limiter la répétition des données entre les objets, il serait approprié d'en faire des variables de classe. Ceci éviterait la répétition dans chacun des objets concernés mais il y aurait tout de même une répétition car les données seraient répétées dans les classes *CrabeRebondissant* et *SoucoupeRebondissante*, et *ExempleJFrameAvecClassesPourCrabeEtSoucoupe*. Cet exemple illustre des enjeux subtils de la conception d'un programme objet.

Pour trouver la solution appropriée, d'une manière abstraite, il faut se demander si la *largeurFenetre* et la *hauteurFenetre* sont des données plus particulièrement caractéristiques de la fenêtre ou d'un crabe ? Dans notre exemple, la réponse est que ce sont des caractéristiques de la fenêtre, et de ce point de vue, ces variables devraient plutôt être maintenues dans la classe fenêtre. Cependant, comme le déplacement d'un crabe est contraint par la taille de la fenêtre, la méthode *deplacer()* a besoin de ces informations, d'où la

nécessité de passer les paramètres à l'appel. Le choix d'en faire des paramètres de la méthode *deplacer()* est donc approprié dans notre exemple.

Note au sujet de l'initialisation des variables d'objet

Les variables d'objet ou de classe sont automatiquement initialisées avec des valeurs de défaut énumérées dans le tableau suivant.

Type	Valeur de défaut
boolean	false
byte	0
char	\u0000
short	0
int	0
long	0
float	0.0
double	0.0
String or object	null

Il est cependant préférable de ne pas se fier sur cette initialisation implicite afin de faciliter la compréhension du programme.

7.2 Compilation et exécution d'un programme composé de plusieurs classes et de packages

Un programme Java est habituellement composé de plusieurs classes. Un aspect important du développement d'un programme Java est de pouvoir localiser toutes les classes nécessaires à un programme lors de la compilation et de l'exécution. Il y a plusieurs manières de procéder.

- **Cas simple : répertoire courant sans packages**

Regardons d'abord un cas simple sans packages où toutes les classes sont dans le répertoire courant.

Exemple. Prenons notre exemple précédent qui inclut trois classes *CrabeRebondissant* et *SoucoupeRebondissante*, et *ExempleJFrameAvecClassesPourCrabeEtSoucoupe*. Une manière simple de procéder consiste à placer tous les fichiers sources (.java) dans un dossier, par exemple, le dossier *C:\MesProgrammes* et compiler le fichier qui contient la méthode *main()*, soit *ExempleJFrameAvecClassesPourCrabeEtSoucoupe.java*, dans notre exemple, comme pour le cas d'une seule classe :

```
C:\MesProgrammes>javac ExempleJFrameAvecClassesPourCrabeEtSoucoupe.java
```

En compilant *ExempleJFrameAvecClassesPourCrabeEtSoucoupe.java*, le compilateur va automatiquement rechercher les fichiers sources des classes utilisées par la classe *ExempleJFrameAvecClassesPourCrabeEtSoucoupe.java* soit, *CrabeRebondissant.java* *SoucoupeRebondissante.java*, dans notre exemple. La recherche est effectuée de la même manière que pour une seule classe. Si la variable d'environnement *classpath* contient le dossier courant²⁶, le compilateur regarde dans *C:\MesProgrammes*.

Si ces fichiers source ne sont pas déjà compilés ou si le code compilé (.class) n'est pas à jour, le compilateur compile les fichiers source automatiquement et les place dans le même dossier *C:\MesProgrammes*. Dans notre exemple, les fichiers *CrabeRebondissant.java* et *SoucoupeRebondissante.java* seraient compilés.

Ensuite, il est possible d'exécuter le fichier .class de la classe qui contient le *main()* comme si ce n'était qu'une seule classe.

```
C:\MesProgrammes>java ExempleJFrameAvecClassesPourCrabeEtSoucoupe
```

Le même principe pour la recherche des classes est applicable à l'exécution. En exécutant *ExempleJFrameAvecClassesPourCrabeEtSoucoupe.class*, la JVM va automatiquement rechercher les fichiers .class des classes utilisées par *ExempleJFrameAvecClassesPourCrabeEtSoucoupe.class* soit, *CrabeRebondissant.class* *SoucoupeRebondissante.class*, dans notre exemple. Si la variable *classpath* contient le dossier courant, le compilateur regarde dans *C:\MesProgrammes*.

²⁶ Rappelons que le «.» dans le classpath représente le dossier courant

- **Cas général incluant des packages**

Les outils Java incorporent des moyens très flexibles pour la recherche des fichiers. Lorsque le compilateur recherche un fichier, et que le chemin complet n'est pas donné, la recherche se fait de manière relative à un ensemble de répertoires appelé *répertoires racines*. Les répertoires racines sont recherchés dans l'ordre suivant :

1. *Bootclasspath* et *extension directories*. Ces répertoires sont fixés lors de l'installation du JDK. Ils peuvent être modifiés par des options lors de l'appel du compilateur. Consultez la documentation du JDK pour le détail des options du compilateur.

On retrouve dans ces répertoires les packages prédéfinis de Java (java, javax, ...). Normalement, ces répertoires ne sont pas utilisés pour les classes du programme d'application en développement.

2. Le *classpath* de l'utilisateur (*user classpath*). C'est ici que le programmeur place les fichiers de ses classes. Ce *classpath* peut être fixé de trois manières.
 - a. Par l'option *-classpath* du compilateur
 - b. Sinon, par la variable d'environnement *classpath*
 - c. Sinon, c'est le répertoire courant

Si une classe est dans un package, la recherche est plus compliquée. Supposons que le compilateur recherche une classe en utilisant le répertoire racine *C:\MesProgrammes* qui est en l'occurrence le répertoire courant. Si le fichier recherché, appelée *Classe1.java* est dans le package *p1.p2.p3* (c'est-à-dire que le nom complet de la classe est *p1.p2.p3.Classe1*), le compilateur recherche *Classe1.java* dans le répertoire suivant :

```
C:\MesProgrammes\p1\p2\p3
```

En d'autres mots, le répertoire racine est utilisé comme point de départ de la recherche et chacun des packages Java correspond effectivement à une suite de répertoires imbriqués sous le répertoire racine. Ainsi le chemin complet du fichier est formé par la concaténation d'un répertoire racine avec la séquence des répertoires correspondant à la séquence des noms de

packages, suivi du nom du fichier lui-même. Ceci implique que le programmeur doit créer les répertoires dont le nom correspond au nom du package et y placer les fichiers appropriés.

Cet aspect de la programmation Java devient rapidement très fastidieux et sujet à erreur. C'est pourquoi, lorsque la complexité des programmes augmente et conduit à l'utilisation de packages pour organiser les classes, il est intéressant d'employer un environnement de développement intégré qui s'occupe automatiquement de la gestion des répertoires correspondant aux packages.

Enfin, une autre option peut être employée. Plutôt que de rechercher dans le système de gestion de fichier, la recherche peut se faire directement à l'intérieur d'un fichier d'archive *jar* ou *zip* (extension *.jar* ou *.zip*) qui contient la structure de répertoire des packages.

D'autre part, plutôt que de placer les fichiers compilés (.class) dans le même répertoire que les fichiers sources (.java), il est possible de spécifier un autre répertoire racine pour les fichiers compilés par l'option -d du compilateur. Ainsi, on obtient deux structures de répertoire identiques l'une pour les fichiers sources (.java), l'autre pour les fichiers compilés (.class). Le compilateur crée de nouveaux répertoires au besoin pour y placer le code compilé.

Le même principe est applicable dans le cas de la recherche de fichiers *.class* lors de l'exécution avec *java*.

Plaçons les deux classes *CrabeRebondissant* et *SoucoupeRebondissante* dans le package *Envahisseurs*. À cet effet, il faut d'une part ajouter un énoncé *package* dans chacun des fichiers *CrabeRebondissant.java* et *SoucoupeRebondissante.java*. Cet énoncé est placé au début du code Java :

```
Exemple. JavaPasAPas/chapitre_6/Envahisseurs/  
package Envahisseurs;  
import java.awt.*;  
public class CrabeRebondissant {  
...  
}
```

```
package Envahisseurs;
import java.awt.*;
public class SoucoupeRebondissante {

...
}
```

Dans la classe *ExempleJFrameAvecClassesPourCrabeEtSoucoupe*, on peut ajouter un énoncé *import* :

```
// Plusieurs crabes et soucoupes qui bougent
// Utilise les classes CrabeRebondissant et SoucoupeRebondissante
// du package Envahisseurs
import Envahisseurs.*;
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

public class ExempleJFrameAvecClassesPourCrabeEtSoucoupe
    extends JFrame {

...
}
```

Il faut placer les deux fichiers *CrabeRebondissant.java* et *SoucoupeRebondissante.java* dans un répertoire appelé aussi *Envahisseurs*.

```
C:\MesProgrammes>dir /b Envahisseurs
CrabeRebondissant.java
SoucoupeRebondissante.java
```

Si le répertoire racine est *C:\MesProgrammes*, il faut placer le répertoire *Envahisseurs* sous le répertoire *C:\MesProgrammes* :

```
C:\MesProgrammes>dir /b
Envahisseurs
ExempleJFrameAvecClassesPourCrabeEtSoucoupe.java
```

Voici un scénario de compilation. On peut voir que le compilateur a produit le fichier *ExempleJFrameAvecClassesPourCrabeEtSoucoupe.class* dans le répertoire *C:\MesProgrammes* comme dans le cas simple. Maintenant, les fichiers compilés *CrabeRebondissant.class* et *SoucoupeRebondissante.class* ont été placés dans *C:\MesProgrammes\Envahisseurs*.


```
C:\MesProgrammes>javac ExempleJFrameAvecClassesPourCrabeEtSoucoupe.java

C:\MesProgrammes>dir /b
Envahisseurs
ExempleJFrameAvecClassesPourCrabeEtSoucoupe.class
ExempleJFrameAvecClassesPourCrabeEtSoucoupe.java

C:\MesProgrammes>dir /b Envahisseurs
CrabeRebondissant.class
CrabeRebondissant.java
SoucoupeRebondissante.class
SoucoupeRebondissante.java
```

On exécute le programme comme dans le cas simple.

```
C:\MesProgrammes>java ExempleJFrameAvecClassesPourCrabeEtSoucoupe
```

Il est à noter que plutôt que d'utiliser un énoncé *import Envahisseurs.** dans *ExempleJFrameAvecClassesPourCrabeEtSoucoupe*, on peut spécifier le nom complet *Envahisseurs.CrabeRebondissant* et *Envahisseurs.SoucoupeRebondissante*, incluant le nom du package en préfixe :

```
// Plusieurs crabes et soucoupes qui bougent
// Utilise les classes CrabeRebondissant et SoucoupeRebondissante
// du package Envahisseurs
import java.awt.*;
import javax.swing.*;
import java.awt.event.*;

...
    Envahisseurs.CrabeRebondissant crabe1 =
        new Envahisseurs.CrabeRebondissant(0, 100, 120, 80, 5, 0);
    Envahisseurs.CrabeRebondissant crabe2 =
        new Envahisseurs.CrabeRebondissant(100, 100, 75, 50, -10, 5);
    Envahisseurs.SoucoupeRebondissante soucoupe1 =
        new Envahisseurs.SoucoupeRebondissante(200, 300, 100, 44, 6, 6);
    Envahisseurs.SoucoupeRebondissante soucoupe2 =
        new Envahisseurs.SoucoupeRebondissante(200, 0, 64, 28, 0, 10);
...
}
```

Le nom du package peut être spécifié avant le nom de classe dans le diagramme UML tel qu'illustré à la Figure 25.

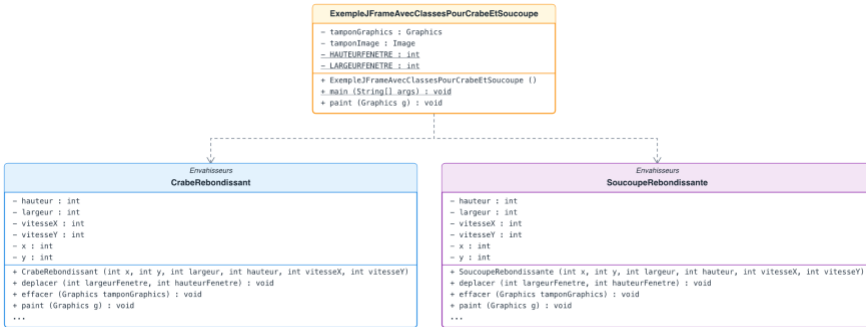


Figure 25. Package *Envahisseurs*

Exercice. Ajoutez à l'exemple précédent une nouvelle classe pour votre personnage préféré que vous placerez aussi dans le package *Envahisseurs*. Créez quelques objets de cette classe dans l'animation.

Exercice. Supposons que la soucoupe se déplace dans l'axe z . Coder une méthode *deplacer()* spécifique à la soucoupe afin de donner cette impression. Ceci peut être effectué en modifiant la taille de la soucoupe plutôt que sa position !

7.3 Limiter la répétition de code par la création d'une super-classe

L'utilisation de méthodes pour limiter la répétition des mêmes groupes d'énoncés dans le code du programme a déjà été expliquée. Un autre moyen qui permet de limiter la répétition de code est l'utilisation de super-classes.

Exemple.

Dans l'exemple précédent, il y a beaucoup de répétition de code entre les deux classes *CrabeRebondissant* et *SoucoupeRebondissante*. Les variables sont les mêmes ainsi que les méthodes *deplacer()* et *effacer()*. Une manière d'éviter cette redondance de code est de créer une super-classe des deux classes qui isole les aspects communs des deux classes dans la super-classe commune. Appelons cette super-classe *EntiteRebondissante*. La classe *EntiteRebondissante* déclare les variables et méthodes communes aux deux classes *CrabeRebondissant* et *SoucoupeRebondissante* de l'exemple précédent (voir Figure 26).

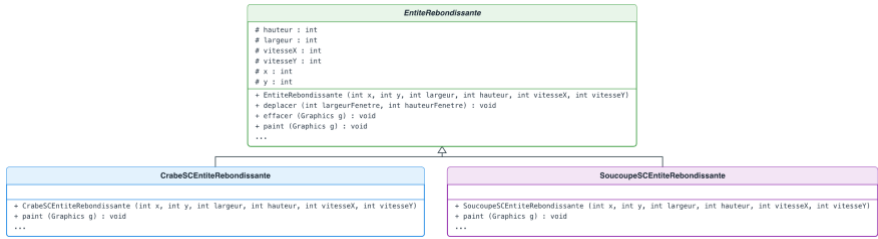


Figure 26. Abstraction des aspects communs par création d'une super-classe.

JavaPasAPas/chapitre_6/EntiteRebondissante.java

```

import java.awt.*;

public abstract class EntiteRebondissante {
    // Variables d'objet qui décrivent l'état d'un objet EntiteRebondissante
    // Protected permet aux sous-classes d'utiliser les variables
    protected int x, y; // Coordonnées de l'entité
    protected int largeur, hauteur; // Taille de l'entité
    protected int vitesseX; // Vitesse de déplacement dans l'axe x
    protected int vitesseY; // Vitesse de déplacement dans l'axe y

    // Constructeur
    public EntiteRebondissante(
        int x, int y, int largeur, int hauteur, int vitesseX, int vitesseY) {
        this.x = x;
        this.y = y;
        this.hauteur = hauteur;
        this.largeur = largeur;
        this.vitesseX = vitesseX;
        this.vitesseY = vitesseY;
    }

    // Déplacement pour la prochaine itération
    public void deplacer(int largeurFenetre, int hauteurFenetre) {
        if (x + largeur >= largeurFenetre | x < 0) // Au bord selon x
            vitesseX = -vitesseX; // Inverser la direction selon x
        x = x + vitesseX; // Déplacement selon x
        if (y + hauteur >= hauteurFenetre | y < 0) // Au bord selon y
            vitesseY = -vitesseY; // Inverser la direction selon y
        y = y + vitesseY; // Déplacement selon y
    }

    // Méthode abstraite : corps doit être précisé dans la sous-classe
    // concrète
    public abstract void paint(Graphics g);

    // Effacer le rectangle dans tamponGraphics
    public void effacer(Graphics g) {
        g.clearRect(x, y, largeur, hauteur);
    }
}

```

```
}  
}
```

Modifieur *abstract* pour une classe, classe abstraite, classe concrète

Le modifieur *abstract* dans la déclaration d'une classe signifie qu'il n'est pas possible de créer directement un objet de cette classe (avec *new*). La classe dite *abstraite* n'est utile que comme super-classe d'autres classes non abstraites dites *concrètes* qui, elles, sont utilisées pour créer des objets.

La classe *EntiteRebondissante* est abstraite car ce sont les sous-classes, *CrabeSCEntiteRebondissante* et *SoucoupeSCEntiteRebondissante*, de la classe *EntiteRebondissante* qui seront utilisées pour créer les objets.

```
public abstract class EntiteRebondissante {
```

En UML, le modifieur *abstract* est représenté en mettant le nom en italique (voir Figure 26).

Modifieur *abstract* pour une méthode

Le modifieur *abstract* pour une méthode signifie que la méthode n'a pas de corps. Le corps d'une méthode abstraite doit être précisé une sous-classe concrète.

Le modifieur *abstract* de la méthode *paint()* signifie que la méthode n'a pas de corps dans la classe *EntiteRebondissante*.

```
public abstract void paint (Graphics g) ;
```

Donc, cette déclaration précise qu'une sous-classe de *EntiteRebondissante* doit pouvoir répondre à un appel de la méthode *paint()*. Cette déclaration indique que les sous-classes concrètes doivent spécifier un corps de la méthode abstraite *paint()*.

Modifieur *protected* pour une variable

Le modifieur *protected* pour une variable signifie que la variable n'est pas accessible aux autres classes (comme *private*) mais en revanche, elle peut être accédée par une méthode d'une sous-classe (par opposé à *private*).

Maintenant, les classes pour le crabe et la soucoupe sont les sous-classes *CrabeSCEntiteRebondissante* et *SoucoupeSCEntiteRebondissante* de la classe *EntiteRebondissante*.

```
import java.awt.*;

public class CrabeSCEntiteRebondissante extends EntiteRebondissante {

    // Constructeur
    public CrabeSCEntiteRebondissante(
        int x, int y, int largeur, int hauteur, int vitesseX, int vitesseY) {
        super(x, y, largeur, hauteur, vitesseX, vitesseY);
    }

    // Corps de la méthode abstraite héritée de la super-classe
    public void paint(Graphics g) {
        g.setColor(Color.green);
        // Le corps
        g.fillOval(
            x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

        g.setColor(Color.black);
        // L'oeil gauche
        g.fillRect(
            x + largeur * 3 / 8,
            y + hauteur * 3 / 8,
            largeur / 12,
            hauteur / 8);
        // L'oeil droit
        g.fillRect(
            x + largeur * 5 / 8 - largeur / 12,
            y + hauteur * 3 / 8,
            largeur / 12,
            hauteur / 8);
        // La bouche
        g.drawLine(
            x + largeur * 3 / 8,
            y + hauteur * 5 / 8,
            x + largeur * 5 / 8,
            y + hauteur * 5 / 8);

        g.setColor(Color.red);
        // Les pinces
        g.fillRect(x, y + hauteur / 8, largeur / 8, hauteur / 2);
        g.fillRect(
            x + largeur * 7 / 8, y + hauteur / 8, largeur / 8, hauteur / 2);
    }
}
```

```
}  
}
```

Dans la déclaration de la classe, la clause *extends* indique la super-classe.

```
public class CrabeSCEntiteRebondissante extends EntiteRebondissante {
```

La classe *ExempleJFrameAvecSuperClassePourCrabeEtSoucoupe* est identique à la version sans super-classe sauf pour les noms des sous-classes *CrabeSCEntiteRebondissante* et *SoucoupeSCEntiteRebondissante* qui remplacent *CrabeRebondissant* et *SoucoupeRebondissante*. NB On aurait pu conserver exactement les mêmes noms pour les sous-classes. Le programme qui utilise une classe n'a pas besoin de connaître ses super-classes.

JavaPasAPas/chapitre_6/

ExempleJFrameAvecSuperClassePourCrabeEtSoucoupe.java

```
// Plusieurs crabes et soucoupes qui bougent  
// Version avec super-classe EntiteRebondissante  
import java.awt.*;  
import java.awt.event.*;  
import javax.swing.*;  
  
public class ExempleJFrameAvecSuperClassePourCrabeEtSoucoupe  
    extends JFrame {  
  
    // Constantes pour la taille de la fenêtre  
    private static final int LARGEURFENETRE = 400;  
    private static final int HAUTEURFENETRE = 400;  
  
    // Tampon pour construire l'image avant d'afficher  
    Graphics tamponGraphics;  
    Image tamponImage;  
  
    public ExempleJFrameAvecSuperClassePourCrabeEtSoucoupe() {  
        super("Ping pong avec classes pour crabes et soucoupes");  
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);  
        this.setSize(LARGEURFENETRE, HAUTEURFENETRE);  
        this.setVisible(true);  
    }  
  
    public void paint(Graphics g) {  
        tamponImage = createImage(LARGEURFENETRE, HAUTEURFENETRE);  
        tamponGraphics = tamponImage.getGraphics();  
  
        CrabeSCEntiteRebondissante crabe1 =  
            new CrabeSCEntiteRebondissante(0, 100, 120, 80, 5, 0);  
        CrabeSCEntiteRebondissante crabe2 =  
            new CrabeSCEntiteRebondissante(100, 100, 75, 50, -10, 5);
```

```

SoucoupeSCEntiteRebondissante soucoupe1 =
    new SoucoupeSCEntiteRebondissante(200, 300, 100, 44, 6, 6);
SoucoupeSCEntiteRebondissante soucoupe2 =
    new SoucoupeSCEntiteRebondissante(200, 0, 64, 28, 0, 10);

while (true) {
    // Dessine les crabes et les soucoupes
    crabe1.paint(tamponGraphics);
    crabe2.paint(tamponGraphics);
    soucoupe1.paint(tamponGraphics);
    soucoupe2.paint(tamponGraphics);

    // Copie le tampon dans le contexte graphique de la fenêtre
    g.drawImage(tamponImage, 0, 0, this);
    try {
        Thread.sleep(50);
    } catch (InterruptedException uneException) {
        System.out.println(uneException.toString());
    }
    // Efface les crabes et les soucoupes du tampon
    crabe1 effacer(tamponGraphics);
    crabe2 effacer(tamponGraphics);
    soucoupe1 effacer(tamponGraphics);
    soucoupe2 effacer(tamponGraphics);

    // Déplace les crabes et les soucoupes
    crabe1.deplacer(LARGEURFENETRE, HAUTEURFENETRE);
    crabe2.deplacer(LARGEURFENETRE, HAUTEURFENETRE);
    soucoupe1.deplacer(LARGEURFENETRE, HAUTEURFENETRE);
    soucoupe2.deplacer(LARGEURFENETRE, HAUTEURFENETRE);
}

public static void main(String[] args) {
    new ExempleJFrameAvecSuperClassePourCrabeEtSoucoupe();
}

```

Refactorisation de code (refactoring)

Dans l'exemple *ExempleJFrameAvecSuperClassePourCrabeEtSoucoupe*, le code de *ExempleJFrameAvecClassesPourCrabeEtSoucoupe* a été transformé en déplaçant des méthodes et variables dans une super-classe mais sans en changer le comportement. Ce processus de transformation qui préserve le même comportement est appelé une *refactorisation* du code (*refactoring*).

Exercice. Reprenez les deux exercices de la section précédente en employant la super-classe *EntiteRebondissante*.

8. Animation 2D et développement d'un jeu simple

« *Un jeu est une série de choix intéressants.* » — *Sid Meier*

Ce chapitre étudie le développement d'un jeu simple qui combine l'animation 2D et une interactivité de base au moyen de la souris. Inspiré des jeux d'arcade classiques, ce jeu oppose le canon du joueur à une formation d'envahisseurs venus de l'espace. Ce faisant, l'approche d'animation introduite au chapitre 6 sera raffinée. D'autre part, les acteurs seront raffinés en ajoutant des sons et des gestes. Le code des acteurs de notre animation sera réorganisé d'une manière plus cohérente en regroupant les éléments qui touche au monde du jeu. Ceci permet de réutiliser les classes du jeu dans une variété de contextes.

8.1 Animation avec un *Timer* dans une sous-classe de *JPanel*

Dans les chapitres précédents, une boucle explicite contrôle l'animation et le principe du double tampon est exploité pour la production de la séquence de scènes d'une animation. Ce mécanisme d'animation est raffiné ici en utilisant de manière plus judicieuse les classes de Java.

Plutôt que d'employer une boucle pour produire la séquence des scènes de l'animation, un objet de la classe `javax.swing.Timer` va générer un événement à intervalles réguliers afin de déclencher la production de la prochaine scène. Cet événement provoque l'appel d'une méthode prédéfinie, la méthode `actionPerformed()` qui servira à dessiner la prochaine scène et à préparer la scène suivante. L'objet `Timer` joue ainsi le rôle d'un chronomètre d'animation. Un avantage d'un chronomètre d'animation est de mieux contrôler l'intervalle de temps entre l'affichage des scènes en le rendant indépendant du temps de calcul nécessaire à la production de la prochaine scène.

Note : paramétrage de la prochaine scène par le délai écoulé

Une autre approche de contrôle de la séquence d'animation serait de paramétrer la prochaine scène (e.g. le déplacement des objets) en fonction

du délai écoulé entre les scènes évitant ainsi de dépendre d'un délai fixé à l'avance.

D'autre part, plutôt que d'exploiter directement une sous-classe de `JFrame`, une sous-classe de `javax.swing.JPanel` contiendra le contexte graphique de l'animation. Ceci a comme avantage de favoriser la réutilisabilité de l'animation car l'objet `JPanel` peut être inclus dans diverses formes d'interfaces graphiques tel qu'une fenêtre `JFrame` ou une boîte de dialogue.

Exemple. L'exemple suivant reprend l'animation du chapitre précédent avec plusieurs crabes et soucoupes. La classe `ExempleJPanelAvecAnimationParTimer` est une sous-classe de `javax.swing.JPanel` qui contient le contexte graphique de l'animation. Un `JPanel` est une zone d'affichage simple qui doit être incluse dans un autre objet qui fournit le support à l'affichage sur l'écran de l'ordinateur. Ici, le `JPanel` est inclus dans la fenêtre `ExempleJFrameIncluantJPanelAvecAnimationParTimer` sous-classe de `JFrame`.

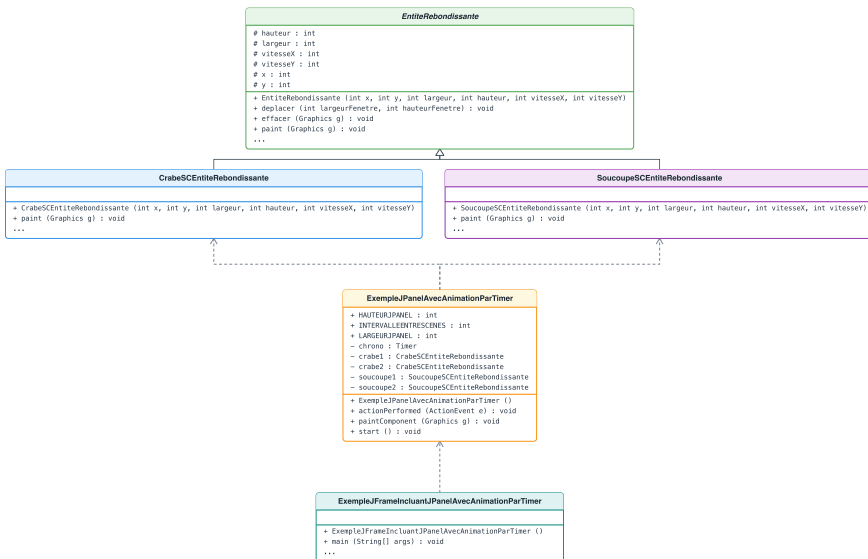


Figure 27. Nouvelle organisation des classes qui passe par un JPanel.

Le délai entre deux scènes de la séquence d'animation est contrôlé par un objet de la classe `Timer` qui génère un événement à toutes les 50 ms. Voici

le code de la classe *ExempleJPanelAvecAnimationParTimer*, sous-classe de *javax.swing.JPanel*.

```
JavaPasAPas/chapitre_7/ExempleJPanelAvecAnimationParTimer.java
// Plusieurs crabes et soucoupes qui bougent dans un JPanel avec Timer
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class ExempleJPanelAvecAnimationParTimer extends JPanel
    implements ActionListener {

    public static final int INTERVALLEENTRESCENES = 50; // En ms

    // Le chrono génère un événement à chaque intervalle
    private Timer chrono;
    // Entités du monde à animer
    private CrabeSCEntiteRebondissante crabe1;
    private CrabeSCEntiteRebondissante crabe2;
    private SoucoupeSCEntiteRebondissante soucoupe1;
    private SoucoupeSCEntiteRebondissante soucoupe2;

    // Taille du JPanel
    public static final int LARGEURJPANEL = 400;
    public static final int HAUTEURJPANEL = 400;

    // Constructeur initialise les entités à animer
    public ExempleJPanelAvecAnimationParTimer() {
        crabe1 = new CrabeSCEntiteRebondissante(0, 100, 120, 80, 5, 0);
        crabe2 = new CrabeSCEntiteRebondissante(100, 100, 75, 50, -10, 5);
        soucoupe1 =
            new SoucoupeSCEntiteRebondissante(200, 300, 100, 44, 6, 6);
        soucoupe2 = new SoucoupeSCEntiteRebondissante(200, 0, 64, 28, 0, 10);
    }

    public void start() {
        if (chrono == null) {
            chrono = new Timer(INTERVALLEENTRESCENES, this);
            chrono.start();
        }
    }

    // Le chrono appelle actionPerformed périodiquement (boucle d'animation)
    public void actionPerformed(ActionEvent e) {
        // Affiche la scène
        repaint();

        // Déplace les entités à animer pour la prochaine scène
        crabe1.deplacer(LARGEURJPANEL, HAUTEURJPANEL);
        crabe2.deplacer(LARGEURJPANEL, HAUTEURJPANEL);
        soucoupe1.deplacer(LARGEURJPANEL, HAUTEURJPANEL);
        soucoupe2.deplacer(LARGEURJPANEL, HAUTEURJPANEL);
    }

    // paintComponent() est appelée indirectement par repaint()
    // N.B. Swing utilise le double tampon : pas besoin d'effacer !
}
```

```

public void paintComponent(Graphics g) {
    super.paintComponent(g);

    // Dessine les entités de l'animation
    crabe1.paint(g);
    crabe2.paint(g);
    soucoupe1.paint(g);
    soucoupe2.paint(g);
}
}

```

Regardons maintenant le détail de la classe. Le constructeur initialise les entités à animer :

```

public ExempleJPanelAvecAnimationParTimer() {
    crabe1 = new CrabeSCEntiteRebondissante(0, 100, 120, 80, 5, 0);
    crabe2 = new CrabeSCEntiteRebondissante(100, 100, 75, 50, -10, 5);
    soucoupe1 =
        new SoucoupeSCEntiteRebondissante(200, 300, 100, 44, 6, 6);
    soucoupe2 = new SoucoupeSCEntiteRebondissante(200, 0, 64, 28, 0, 10);
}

```

La méthode *start()* doit être appelée pour démarrer l'animation. Dans notre exemple, elle sera appelée par la méthode *main()* de la classe *ExempleJFrameIncluantJPanelAvecAnimationParTimer* que nous verrons plus loin. La méthode *start()* crée le chronomètre (objet de la classe *Timer*) et le démarre.

```

public void start(){
    if (chrono == null){
        chrono = new Timer(intervalleEntreScenes, this);
        chrono.start();
    }
}

```

Si la variable *chrono* est *null*, un nouvel objet *chrono* de la classe *Timer* est créé. Le premier paramètre du constructeur est l'intervalle de temps et le second est l'objet qui doit répondre à l'événement généré par le *Timer*. Dans notre exemple, le second paramètre est *this*, ce qui signifie que la méthode de réponse à l'événement du *Timer*, la méthode *actionPerformed()*, doit se trouver dans la classe *ExempleJPanelAvecAnimationParTimer*.

Lorsqu'un événement est généré par l'objet *Timer*, la méthode *actionPerformed()* est appelée automatiquement sur l'objet de la classe *ExempleJPanelAvecAnimationParTimer* qui doit répondre à l'événement.

```
public void actionPerformed( ActionEvent e){
    // Affiche la scène
    repaint();

    // Déplace les entités à animer pour la prochaine scène
    crabe1.deplacer(LARGEURJPANEL, HAUTEURJPANEL);
    crabe2.deplacer(LARGEURJPANEL, HAUTEURJPANEL);
    soucoupe1.deplacer(LARGEURJPANEL, HAUTEURJPANEL);
    soucoupe2.deplacer(LARGEURJPANEL, HAUTEURJPANEL);
}
```

Nous avons décrit au chapitre 5, le concept d'écouteur d'événement de souris qui doit implémenter l'interface *java.awt.event.MouseListener*. Par analogie pour répondre à l'événement du *Timer*, la classe *ExempleJPanelAvecAnimationParTimer* doit implémenter l'interface *java.awt.event.ActionListener*:

```
public class ExempleJPanelAvecAnimationParTimer extends JPanel implements
ActionListener{
```

La méthode *actionPerformed()* appelle d'abord la méthode *repaint()* qui provoque l'appel de la méthode *paintComponent()* pour afficher la prochaine scène. La méthode *paintComponent()* de *JPanel* joue un rôle analogue à la méthode *paint()* de *JFrame* que nous avons employé jusqu'à présent. Ensuite, la méthode *actionPerformed()* mets les entités à jour pour la prochaine scène. En effectuant l'appel à *repaint()* avant de produire la prochaine scène, l'intervalle entre l'affichage de deux scènes est indépendant du temps de calcul nécessaire à la production de la prochaine scène.

Comme vu précédemment pour *paint()*, la méthode *paintComponent()* appelle d'abord la méthode correspondante de la super-classe. Ensuite, la scène est dessinée. Il est à noter qu'il n'est pas nécessaire de dessiner dans un tampon car l'approche du double tampon est automatiquement employée lorsque l'on passe par *repaint()* pour dessiner la prochaine scène ! D'autre part, il n'est pas nécessaire d'effacer les entités de la scène précédente car le fond de l'écran est rétabli par *repaint()* avant l'appel à *paintComponent()*.

```
public void paintComponent (Graphics g) {
    super.paintComponent(g);

    // Dessine les entités de l'animation
    crabe1.paint(g); crabe2.paint(g);
    soucoupe1.paint(g); soucoupe2.paint(g);
}
```

```
}
```

La classe *ExempleJFrameIncluantJPanelAvecAnimationParTimer* est une sous-classe de `JFrame` qui représente la fenêtre contenant l'objet *ExempleJPanelAvecAnimationParTimer*.

JavaPasAPas/chapitre_7/

ExempleJFrameIncluantJPanelAvecAnimationParTimer.java

```
import javax.swing.JFrame;
```

```
public class ExempleJFrameIncluantJPanelAvecAnimationParTimer
    extends JFrame {
```

```
    public ExempleJFrameIncluantJPanelAvecAnimationParTimer() {
        super("Animation dans JPanel avec Timer");
        ExempleJPanelAvecAnimationParTimer leJPanelAnimation =
            new ExempleJPanelAvecAnimationParTimer();
        this.getContentPane().add(leJPanelAnimation);
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(
            ExempleJPanelAvecAnimationParTimer.LARGEURJPANEL,
            ExempleJPanelAvecAnimationParTimer.HAUTEURJPANEL + 30);
        this.setVisible(true);
        leJPanelAnimation.start();
    }

    public static void main(String[] args) {
        new ExempleJFrameIncluantJPanelAvecAnimationParTimer();
    }
}
```

La ligne suivante crée l'objet d'animation *leJPanelAnimation*:

```
ExempleJPanelAvecAnimationParTimer leJPanelAnimation =
    new ExempleJPanelAvecAnimationParTimer();
```

La ligne suivante ajoute l'objet d'animation *leJPanelAnimation* comme une composante de la fenêtre `JPanel`. Par conséquent, l'objet *leJPanelAnimation* sera affiché dans la fenêtre.

```
this.getContentPane().add(leJPanelAnimation);
```

L'appel à la méthode *leJPanelAnimation.start()* démarre l'animation :

```
leJPanelAnimation.start();
```

La figure suivante montre la fenêtre produite par l'animation. Les crabes et les soucoupes se déplacent et rebondissent sur les bords du *JPanel*.

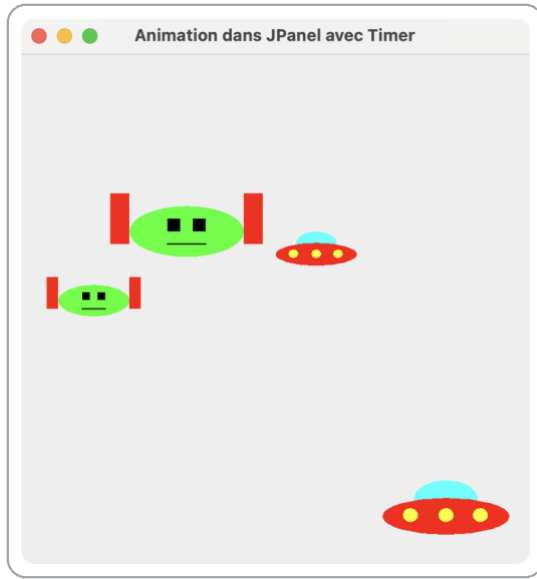


Figure 28. Animation de crabes et de soucoupes dans un *JPanel* avec *Timer*.

Exercice. Reprenez l'application précédente en incluant votre entité préférée.

8.2 Isoler le monde à animer du mécanisme d'animation

Cette section présente une refactorisation de l'exemple précédent (animation de plusieurs crabes et soucoupes) en isolant dans une classe *MondeAnime* les aspects du programme qui concerne le monde que nous voulons animer, indépendamment du mécanisme d'animation lui-même. Cette classe peut ainsi être utilisée soit dans une animation par boucle ou encore dans une animation avec *Timer*. Ceci illustre un des aspects les plus puissants de la programmation objet. En regroupant de manière judicieuse les variables et méthodes sous forme de classe, on obtient une composante logicielle facile à comprendre et à réutiliser à travers différents contextes. Java inclut d'ailleurs un grand nombre de classes réutilisables. En développant ses propres classes, il est important de chercher à favoriser la réutilisabilité par un découpage judicieux des éléments du programme en classes.

JavaPasAPas/chapitre_7/MondeAnime.java

```
// Monde à animer
import java.awt.*;

public class MondeAnime {
    // Taille du monde
    public static int LARGEURMONDE = 400;
    public static int HAUTEURMONDE = 400;
    // Entités du monde à animer
    private CrabeSCEntiteRebondissante crabe1 =
        new CrabeSCEntiteRebondissante(0, 100, 120, 80, 5, 0);
    private CrabeSCEntiteRebondissante crabe2 =
        new CrabeSCEntiteRebondissante(100, 100, 75, 50, -10, 5);
    private SoucoupeSCEntiteRebondissante soucoupe1 =
        new SoucoupeSCEntiteRebondissante(200, 300, 100, 44, 6, 6);
    private SoucoupeSCEntiteRebondissante soucoupe2 =
        new SoucoupeSCEntiteRebondissante(200, 0, 64, 28, 0, 10);

    public MondeAnime() {
        // Initialise les entités à animer pour la première scène
        crabe1 = new CrabeSCEntiteRebondissante(0, 100, 120, 80, 5, 0);
        crabe2 = new CrabeSCEntiteRebondissante(100, 100, 75, 50, -10, 5);
        soucoupe1 = new SoucoupeSCEntiteRebondissante(200, 300, 100, 44, 6, 6);
        soucoupe2 = new SoucoupeSCEntiteRebondissante(200, 0, 64, 28, 0, 10);
    }

    public void prochaineScene() {
        // Modifie les entités à animer pour la prochaine scène
        crabe1.deplacer(LARGEURMONDE, HAUTEURMONDE);
        crabe2.deplacer(LARGEURMONDE, HAUTEURMONDE);
        soucoupe1.deplacer(LARGEURMONDE, HAUTEURMONDE);
        soucoupe2.deplacer(LARGEURMONDE, HAUTEURMONDE);
    }

    public void paint(Graphics g) {
        // Dessine la scène
        crabe1.paint(g);
        crabe2.paint(g);
        soucoupe1.paint(g);
        soucoupe2.paint(g);
    }
}
```

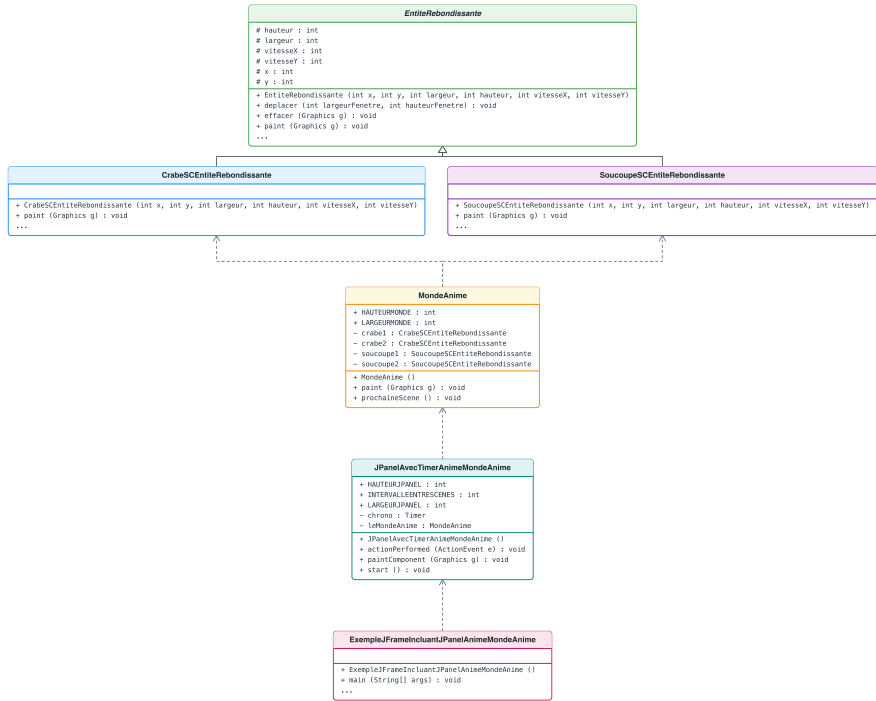



Figure 29. Isolation du monde à animer dans la classe *MondeAnime*.

La classe *JPanelAvecTimerAnimeMondeAnime* contient le mécanisme d'animation par *Timer*. Elle fait appel aux méthodes de la classe *MondeAnime* qui contient les détails du monde à animer. L'interaction entre la classe du monde à animer et le mécanisme d'animation est limité à l'appel de trois méthodes : le constructeur *MondeAnime()*, *prochaineScene()* et *paint()*. Les variables de classe constantes *MondeAnime.LARGEURMONDE* et *MondeAnime.HAUTEURMONDE* définissent la taille du monde 2D dans lequel les entités sont animées.

```

JavaPasAPas/chapitre_7/ JPanelAvecTimerAnimeMondeAnime.java
// JPanel qui anime un objet de MondeAnime
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class JPanelAvecTimerAnimeMondeAnime extends JPanel
    implements ActionListener {

    public static final int INTERVALLEENTRESCENES = 50; // En ms
    
```

```

// Le chrono génère un événement à chaque intervalle
private Timer chrono;
// Le monde à animer
private MondeAnime leMondeAnime;

// Taille du JPanel
public static final int LARGEURJPANEL = MondeAnime.LARGEURMONDE;
public static final int HAUTEURJPANEL = MondeAnime.HAUTEURMONDE;

// Constructeur initialise le monde à animer
public JPanelAvecTimerAnimeMondeAnime() {
    leMondeAnime = new MondeAnime();
}

public void start() {
    if (chrono == null) {
        chrono = new Timer(INTERVALLEENTRESCENES, this);
        chrono.start();
    }
}
// Le chrono appelle actionPerformed périodiquement (boucle d'animation)
public void actionPerformed(ActionEvent e) {
    repaint();
    // Produire la prochaine scène du monde à animer
    leMondeAnime.prochaineScene();
}

// paintComponent() est appelée indirectement par repaint()
// N.B. Swing utilise le double tampon : pas besoin d'effacer !
public void paintComponent(Graphics g) {
    super.paintComponent(g);

    // Dessine les entités de l'animation
    leMondeAnime.paint(g);
}
}

```

Enfin, l'objet de la classe *JPanelAvecTimerAnimeMondeAnime* est inclus dans une fenêtre de la classe *ExempleJFrameIncluantJPanelAnimeMondeAnime*.

```

JavaPasAPas/chapitre_7/ ExempleJFrameIncluantJPanelAnimeMondeAnime.java
// JFrame qui inclut le JPanelAnimeMondeAnime
import javax.swing.JFrame;

public class ExempleJFrameIncluantJPanelAnimeMondeAnime extends JFrame {

    public ExempleJFrameIncluantJPanelAnimeMondeAnime() {
        super("Animation dans JPanel avec Timer");
        JPanelAvecTimerAnimeMondeAnime leJPanelAnimation = new
            JPanelAvecTimerAnimeMondeAnime();
        this.getContentPane().add(leJPanelAnimation);
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(
            JPanelAvecTimerAnimeMondeAnime.LARGEURJPANEL,
            JPanelAvecTimerAnimeMondeAnime.HAUTEURJPANEL + 30);
        this.setVisible(true);
        leJPanelAnimation.start();
    }

    public static void main(String[] args) {
        new ExempleJFrameIncluantJPanelAnimeMondeAnime();
    }
}

```

Il est important d'examiner attentivement la différence entre cette version du programme et la précédente (section 8.1). Le regroupement des aspects spécifiques au monde à animer, de manière indépendante du mécanisme d'animation, facilite la compréhension du programme. Les différents aspects du monde à animer ont été identifiés et isolés de manière précise. Ce partage des responsabilités permet de changer le mécanisme d'animation en réutilisant la même classe *MondeAnime*.

- **Animation par boucle explicite**

Dans l'exemple suivant, un `JPanel` utilise une boucle explicite pour animer le monde de la classe *MondeAnime* plutôt qu'un *Timer*. La classe *MondeAnime* est réutilisée telle quelle sans changement !

Exemple. Voici le code de la sous-classe de `JPanel` qui fournit le mécanisme d'animation sous forme d'une boucle dans la méthode *start()*.

```

JavaPasAPas/chapitre_7/ JPanelAvecBoucleAnimeMondeAnime.java
// JPanel avec boucle qui anime un objet de MondeAnime
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class JPanelAvecBoucleAnimeMondeAnime extends JPanel {

    public static final int INTERVALLEENTRESCENES = 50; // En ms

    // Le chrono génère un évènement à chaque intervalle
    // private Timer chrono;
    // Le monde à animer
    private MondeAnime leMondeAnime;

    // Taille du JPanel
    public static final int LARGEURJPANEL = MondeAnime.LARGEURMONDE;
    public static final int HAUTEURJPANEL = MondeAnime.HAUTEURMONDE;

    // Constructeur initialise le monde à animer
    public JPanelAvecBoucleAnimeMondeAnime() {
        leMondeAnime = new MondeAnime();
    }

    public void start() {
        while (true) {
            repaint();
            try {
                Thread.sleep(INTERVALLEENTRESCENES);
            } catch (InterruptedException exception) {
                System.err.println(exception.toString());
            }
            leMondeAnime.prochaineScene();
        }
    }

    // paintComponent() est appelée indirectement par repaint()
    // N.B. Swing utilise le double tampon : pas besoin d'effacer !
    public void paintComponent(Graphics g) {
        super.paintComponent(g);

        // Dessine les entités de l'animation
        leMondeAnime.paint(g);
    }
}

```

Comme pour la version avec *Timer*, l'animation est démarrée avec la méthode *start()* du *JPanel*. Contrairement aux exemples de boucle d'animation vus précédemment où la boucle était placée dans la méthode *paint()*, la fermeture de fenêtre fonctionne correctement car la boucle infinie dans *start()* peut être interrompue par un événement de souris. Dans la

boucle, *repaint()* est appelée pour afficher la scène courante, ensuite *leMondeAnime.prochaineScene()* produit les changements du monde pour la prochaine scène.

```
public void start(){
    while (true){
        repaint();
        try {
            Thread.sleep(intervalleEntreScenes);
        }
        catch (InterruptedException exception){
            System.err.println(exception.toString());
        }
        leMondeAnime.prochaineScene();
    }
}
```

La sous-classe *JFrameIncluantJPanelAvecBoucleAnimeMondeAnime* de *JFrame* inclut l'objet de la classe *JPanelAvecBoucleAnimeMondeAnime* pour afficher l'animation dans une fenêtre :

JavaPasAPas/chapitre_7/JFrameIncluantJPanelAvecBoucleAnimeMondeAnime.java
import javax.swing.JFrame;
public class JFrameIncluantJPanelAvecBoucleAnimeMondeAnime extends JFrame{

```
public JFrameIncluantJPanelAvecBoucleAnimeMondeAnime() {
    super("Animation dans JPanel avec boucle");
    JPanelAvecBoucleAnimeMondeAnime leJPanelAnimation =
        new JPanelAvecBoucleAnimeMondeAnime();
    this.getContentPane().add(leJPanelAnimation);
    this.setDefaultCloseOperation(EXIT_ON_CLOSE);
    this.setSize(
        JPanelAvecBoucleAnimeMondeAnime.LARGEURJPANEL,
        JPanelAvecBoucleAnimeMondeAnime.HAUTEURJPANEL+30);
    this.setVisible(true);
    leJPanelAnimation.start();
}
```

```
public static void main (String[] args) {
    new JFrameIncluantJPanelAvecBoucleAnimeMondeAnime();
}
}
```

8.3 Développement du jeu

Cette section développe un jeu simple 2D, *les envahisseurs*, en raffinant les classes développées précédemment. Le jeu démarre en animant une formation d'envahisseurs qui se déplace latéralement et descend d'un cran chaque fois qu'elle atteint un bord de la fenêtre. Une soucoupe volante traverse le haut de l'écran. Le joueur contrôle un canon avec la souris : le

canon suit les déplacements de la souris et chaque clic lance un tir. Le but de l'utilisateur est de détruire les envahisseurs avec ces tirs. Lorsqu'une entité est touchée, elle disparaît en poussant un cri de désarroi. Les envahisseurs lâchent des bombes : la partie est perdue si une bombe touche le canon ou si la formation atteint le canon. La figure suivante montre une partie en cours.



Figure 30. Une partie du jeu des envahisseurs.

Nous allons d'abord raffiner la hiérarchie des entités du monde à animer en ajoutant des comportements plus sophistiqués (gestes, cris). La figure suivante montre les classes à développer.

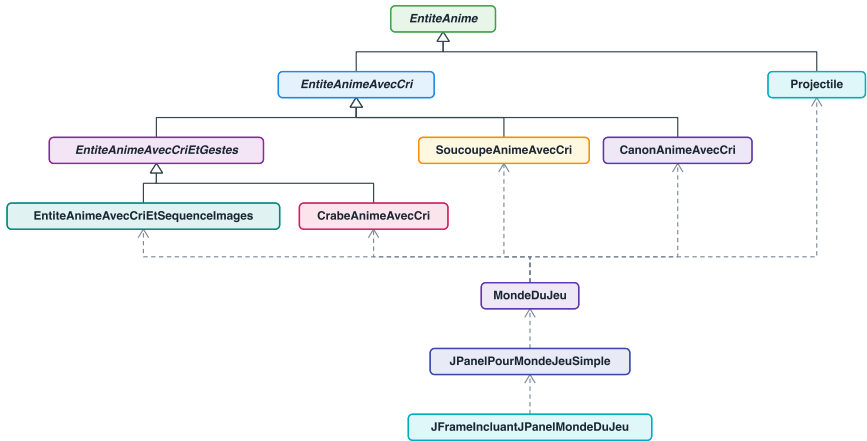


Figure 31. Classes du jeu des envahisseurs.

La classe *EntiteAnime* du package *JeuSimple* ajoute les aspects suivants à la classe *EntiteAnime* développée au chapitre 7 :

- Visibilité : une entité peut être visible ou pas (variable d'objet *visible*). Si elle est invisible, un appel à *paintSiVisible()* ne la dessine pas.
- La méthode *touche()* : cette méthode retourne vrai si la coordonnée (*unX, unY*) est à l'intérieur du rectangle englobant de l'entité. Elle sert à vérifier si un projectile a touché l'entité dans le contexte du jeu.
- Méthodes de lecture (*lecteurs/getters*) et d'écriture (*modifieurs/setters*) de variables d'objet. Il est souvent nécessaire de pouvoir extraire ou modifier une variable membre d'une classe à partir de l'extérieur de cette classe. Il est possible de spécifier que la variable est *public* à cet effet. Cependant, cette approche est généralement à éviter pour minimiser le couplage entre classes. Il est préférable d'utiliser des méthodes pour extraire (lecteur) ou modifier (modifieur) le contenu des variables. Une convention souvent employée en Java est de nommer par *getNomVariable()* et *setNomVariable()* les méthodes qui vont extraire le contenu et modifier le contenu de la variable appelée *nomVariable*. Des méthodes de lecture et d'écriture ont été ajoutées pour la plupart des variables d'objet de la classe *EntiteAnime* car ceci est nécessaire dans notre programme.

JavaPasAPas/JeuSimple/EntiteAnime.java

```
package JeuSimple;

import java.awt.*;

public abstract class EntiteAnime {
    // Variables d'objet qui décrivent l'état d'un objet EntiteAnime
    protected int x, y; // Coordonnées de l'entité
    protected int largeur, hauteur; // Taille du rectangle englobant
    protected int vitesseX; // Vitesse de déplacement dans l'axe x
    protected int vitesseY; // Vitesse de déplacement dans l'axe y
    protected boolean visible; // Indique si l'entité doit être affichée

    // Constructeur
    public EntiteAnime(
        int x, int y, int largeur, int hauteur, int vitesseX, int vitesseY,
        boolean visible) {
        this.x = x;
        this.y = y;
        this.hauteur = hauteur;
        this.largeur = largeur;
        this.vitesseX = vitesseX;
        this.vitesseY = vitesseY;
        this.visible = visible;
    }
    // Méthode de lecture et d'écriture des variables d'objet
    public void setX(int x) {
        this.x = x;
    }

    public void setY(int y) {
        this.y = y;
    }

    public void setLargeur(int largeur) {
        this.largeur = largeur;
    }

    public void setHauteur(int hauteur) {
        this.hauteur = hauteur;
    }

    public void setVisible(boolean visible) {
        this.visible = visible;
    }

    public void setVitesseX(int vitesseX) {
        this.vitesseX = vitesseX;
    }
    public void setVitesseY(int vitesseY) {
        this.vitesseY = vitesseY;
    }

    public int getX() {
        return x;
    }
}
```



```
public int getY() {
    return y;
}

public int getLargeur() {
    return largeur;
}

public int getHauteur() {
    return hauteur;
}

public boolean getVisible() {
    return visible;
}
public int getVitesseX() {
    return vitesseX;
}
public int getVitesseY() {
    return vitesseY;
}
// Modifier les Coordonnées pour la prochaine scène
public void prochaineScene(int largeurMonde, int hauteurMonde) {
    if (x + largeur >= largeurMonde | x < 0) // Si atteint le bord selon x
        vitesseX = -vitesseX; // Inverser la direction selon x
    x = x + vitesseX; // déplacement selon x
    if (y + hauteur >= hauteurMonde | y < 0) // Si atteint le bord selon y
        vitesseY = -vitesseY; // Inverser la direction selon y
    y = y + vitesseY; // déplacement selon y
}

// Détermine si la coordonnée unX,unY touche au rectangle englobant de
// l'entité
public boolean touche(int unX, int unY) {
    return ((unX >= x) && (unX <= x + largeur) && (unY >= y) && (unY <= y
        + hauteur));
}

// Méthode abstraite de dessin de l'entité
public abstract void paint(Graphics g);
// Dessine seulement si visible
public void paintSiVisible(Graphics g) {
    if (visible) {
        paint(g);
    }
}

// Effacer l'entité
public void effacer(Graphics g) {
    g.clearRect(x, y, largeur, hauteur);
}
}
```

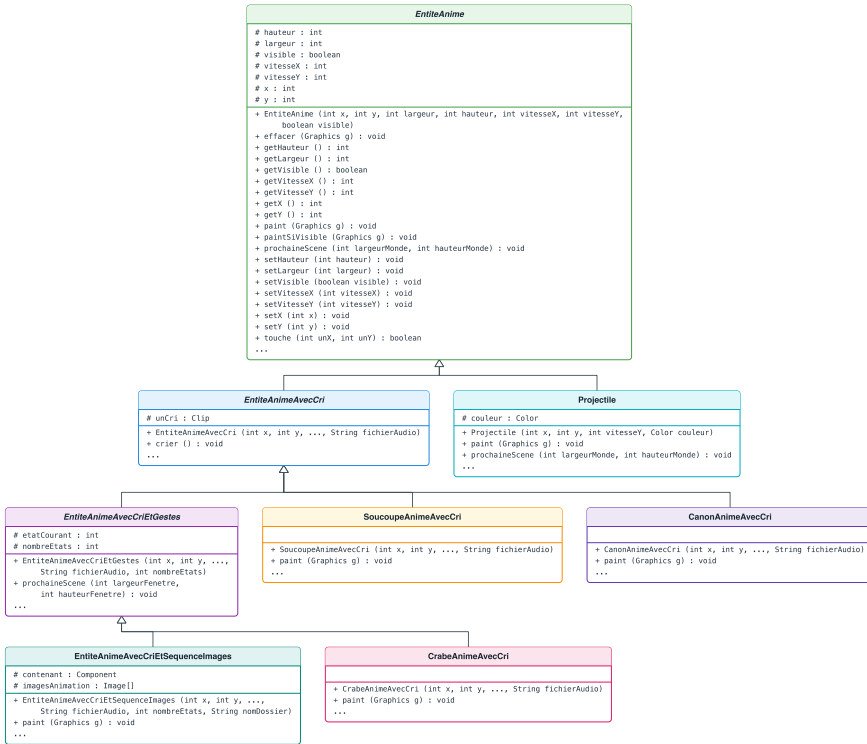


Figure 32. Hiérarchie des entités animées.

- **Traitement du son avec *Clip***

La classe *EntiteAnimeAvecCrie*, sous-classe de *EntiteAnime*, vise à associer un son à l'entité qui servira dans le contexte du jeu. Lorsqu'une entité est touchée par un projectile, elle pousse un cri de désarroi.

```

JavaPasAPas/JeuSimple/EntiteAnimeAvecCrie.java
package JeuSimple;

import java.net.URL;
import javax.sound.sampled.*;

public abstract class EntiteAnimeAvecCrie extends EntiteAnime {
    protected Clip unCrie; // Cri de l'entité

    public EntiteAnimeAvecCrie(
        int x,

```

```

    int y,
    int largeur,
    int hauteur,
    int vitesseX,
    int vitesseY,
    boolean visible,
    String nomFichierAudio) {
    super(x, y, largeur, hauteur, vitesseX, vitesseY, visible);
    // Charge le son dans le fichier nomFichierAudio
    // Le fichier est dans le dossier de EntiteAnimeAvecCri.class
    // Cherche l'URL du fichier
    URL url = EntiteAnimeAvecCri.class.getResource(nomFichierAudio);
    try {
        // Charge le clip audio à partir de l'URL
        unCri = AudioSystem.getClip();
        unCri.open(AudioSystem.getAudioInputStream(url));
    } catch (Exception e) {
        // Sans dispositif audio, le jeu fonctionne en silence
        System.err.println("Son non disponible : " + e);
        unCri = null;
    }
}

public void crier() {
    if (unCri != null) {
        unCri.setFramePosition(0); // Revient au début du clip
        unCri.start();
    }
}
}

```

La variable d'objet *unCri* de la classe *javax.sound.sampled.Clip* contient un clip audio :

```
protected Clip unCri; // Cri de l'entité
```

Le clip est lu d'un fichier dont le nom, *nomFichierAudio*, est passé en paramètre au constructeur. L'appel suivant construit une adresse sous forme d'URL qui fait référence au fichier qui contient le clip audio. Sans entrer dans les détails de la notion d'URL, mentionnons que dans notre contexte, l'URL représente le chemin du fichier. On suppose que le fichier se trouve dans le même dossier que le fichier *EntiteAnimeAvecCri.class*.

```
URL url = EntiteAnimeAvecCri.class.getResource(nomFichierAudio);
```

Les appels suivants obtiennent un objet *Clip* auprès du système audio (classe *AudioSystem*), puis l'ouvrent avec le flux audio lu à partir de l'URL. Le chargement du son peut échouer, par exemple sur un ordinateur sans dispositif audio. Les appels sont donc placés dans un bloc *try...catch* : en cas d'erreur, un message est affiché, la variable *unCri* reste à *null* et le jeu fonctionne en silence.

```
unCri = AudioSystem.getClip();
unCri.open(AudioSystem.getAudioInputStream(ur1));
```

La méthode *crier()* ramène le clip à son début avec *setFramePosition(0)*, puis appelle la méthode *start()* de l'objet *unCri* pour jouer le clip audio. Rien n'est joué si le son n'a pas pu être chargé.

```
public void crier() {
    if (unCri != null) {
        unCri.setFramePosition(0); // Revient au début du clip
        unCri.start();
    }
}
```

La méthode *paint()* de la classe *SoucoupeAnimeAvecCri*, sous-classe de *EntiteAnimeAvecCri*, précise la manière de dessiner la soucoupe volante. Le dessin est le même que celui de la classe *SoucoupeSCEntiteRebondissante* vue au chapitre précédent.

```
JavaPasAPas/JeuSimple/SoucoupeAnimeAvecCri.java
package JeuSimple;

import java.awt.*;

public class SoucoupeAnimeAvecCri extends EntiteAnimeAvecCri {

    public SoucoupeAnimeAvecCri(
        int x,
        int y,
        int largeur,
        int hauteur,
        int vitesseX,
        int vitesseY,
        boolean visible,
        String fichierAudio) {
        super(x, y, largeur, hauteur, vitesseX, vitesseY, visible,
            fichierAudio);
    }

    public void paint(Graphics g) {
        g.setColor(Color.cyan);
        // Le dôme
        g.fillOval(x + largeur / 4, y, largeur / 2, hauteur * 2 / 3);

        g.setColor(Color.red);
        // Le disque
        g.fillOval(x, y + hauteur / 3, largeur, hauteur * 2 / 3);

        g.setColor(Color.yellow); // Les hublots
        g.fillOval(x + largeur / 6, y + hauteur / 2, largeur / 8, hauteur / 4);
        g.fillOval(
```

```
        x + largeur / 2 - largeur / 16, y + hauteur / 2, largeur / 8,
        hauteur / 4);
g.fillOval(
    x + largeur * 5 / 6 - largeur / 8, y + hauteur / 2, largeur / 8,
    hauteur / 4);
    }
}
```

La classe *CanonAnimeAvecCri* est une autre entité animée, sous-classe de *EntiteAnimeAvecCri*, dont la méthode *paint()* dessine le canon du joueur avec trois rectangles. Le canon ne se déplace pas par lui-même : comme nous le verrons, c'est la souris qui fixe sa position. Son cri est l'explosion que l'on entend lorsqu'une bombe le touche.

```
JavaPasAPas/JeuSimple/CanonAnimeAvecCri.java
package JeuSimple;

import java.awt.*;

public class CanonAnimeAvecCri extends EntiteAnimeAvecCri {

    public CanonAnimeAvecCri(
        int x,
        int y,
        int largeur,
        int hauteur,
        int vitesseX,
        int vitesseY,
        boolean visible,
        String fichierAudio) {
        super(x, y, largeur, hauteur, vitesseX, vitesseY, visible,
            fichierAudio);
    }

    public void paint(Graphics g) {
        g.setColor(Color.white);
        // La base
        g.fillRect(x, y + hauteur / 2, largeur, hauteur / 2);
        // Le dessus de la base
        g.fillRect(
            x + largeur / 10, y + hauteur / 3, largeur * 8 / 10, hauteur / 6);
        // Le tube du canon
        g.fillRect(
            x + largeur / 2 - largeur / 16, y, largeur / 8, hauteur / 3);
    }
}
```

- **Animation vectorielle de gestes (par opérations de dessin)**

Pour rendre l'animation plus intéressante, une entité peut non seulement se déplacer mais aussi effectuer des gestes. Pour simplifier, supposons que l'entité effectue toujours la même série de gestes en répétant la même séquence d'images. Généralement, on distingue deux techniques de production des images simulant les gestes :

- *Animation vectorielle*. Chaque image est produite en invoquant des opérations de dessins, par exemple, avec Java 2D.
- *Animation bitmap*. Chaque image est préalablement créée. Cette approche évite le calcul répété des images à dessiner mais nécessite le stockage des images. Dans le cas d'images très complexes, il peut être très difficile de produire le résultat recherché avec des opérations de dessin.

Pour produire la séquence d'animation des gestes, il faut un moyen de déterminer la forme graphique appropriée pour la prochaine scène. La classe *EntiteAnimeAvecCriEtGestes* sous-classe de *EntiteAnimeAvecCri* ajoute deux variables d'objets à cet effet. La variable d'objet *nombreEtats* représente le nombre de formes graphiques de l'entité et *etatCourant* détermine la prochaine forme graphique de l'entité. La méthode *prochaineScene()* fait passer à la forme suivante en ajoutant 1 à *etatCourant* (modulo le *nombreEtats* afin de produire un effet de répétition du mouvement). L'opération modulo (reste après division entière) est représentée par le symbole %.

```
JavaPasAPas/JeuSimple/EntiteAnimeAvecCriEtGestes.java
```

```
package JeuSimple;

public abstract class EntiteAnimeAvecCriEtGestes
    extends EntiteAnimeAvecCri {
    protected int etatCourant = 0;
    protected int nombreEtats = 1;

    public EntiteAnimeAvecCriEtGestes(
        int x,
        int y,
        int largeur,
        int hauteur,
        int vitesseX,
        int vitesseY,
        boolean visible,
        String nomFichierAudio,
        int nombreEtats) {
```

```

        super(x, y, largeur, hauteur, vitesseX, vitesseY, visible,
              nomFichierAudio);
        this.nombreEtats = nombreEtats;
    }

    public void prochaineScene(int largeurFenetre, int hauteurFenetre) {
        super.prochaineScene(largeurFenetre, hauteurFenetre);
        etatCourant = (etatCourant + 1) % nombreEtats;
    }
}

```

La méthode *paint()* de la sous-classe *CrabeAnimeAvecCri* tient compte de la variable *etatCourant* dans le dessin du crabe en faisant monter et descendre ses pinces. Le crabe a deux états (0 et 1) : la position des pinces dépend de la valeur de *etatCourant*.

```

JavaPasAPas/JeuSimple/CrabeAnimeAvecCri.java
package JeuSimple;

import java.awt.*;

public class CrabeAnimeAvecCri extends EntiteAnimeAvecCriEtGestes {
    public CrabeAnimeAvecCri(
        int x,
        int y,
        int largeur,
        int hauteur,
        int vitesseX,
        int vitesseY,
        boolean visible,
        String fichierAudio) {
        super(x, y, largeur, hauteur, vitesseX, vitesseY, visible,
              fichierAudio, 2);
    }

    public void paint(Graphics g) {
        g.setColor(Color.green);
        // Le corps
        g.fillOval(
            x + largeur / 8, y + hauteur / 4, largeur * 3 / 4, hauteur / 2);

        g.setColor(Color.black);
        // Les yeux
        g.fillRect(
            x + largeur * 3 / 8, y + hauteur * 3 / 8, largeur / 12,
            hauteur / 8);
        g.fillRect(
            x + largeur * 5 / 8 - largeur / 12, y + hauteur * 3 / 8,
            largeur / 12, hauteur / 8);
        // La bouche
        g.drawLine(

```

```

        x + largeur * 3 / 8, y + hauteur * 5 / 8, x + largeur * 5 / 8,
        y + hauteur * 5 / 8);

g.setColor(Color.red);
// Les pinces : leur hauteur dépend de etatCourant
int hautPinces = y + hauteur / 8 + (hauteur / 4) * etatCourant;
g.fillRect(x, hautPinces, largeur / 8, hauteur / 2);
g.fillRect(x + largeur * 7 / 8, hautPinces, largeur / 8, hauteur / 2);
    }
}

```

- Animation bitmap de gestes

La sous-classe *EntiteAnimeAvecCriEtSequenceImages* illustre une autre manière de produire l'animation :

```

JavaPasAPas/JeuSimple/EntiteAnimeAvecCriEtSequenceImages.java
package JeuSimple;

import java.awt.*;
import java.net.URL;
import javax.swing.*;

public class EntiteAnimeAvecCriEtSequenceImages
    extends EntiteAnimeAvecCriEtGestes {

    protected Image[] imagesAnimation;

    public EntiteAnimeAvecCriEtSequenceImages(
        int x,
        int y,
        int largeur,
        int hauteur,
        int vitesseX,
        int vitesseY,
        boolean visible,
        String fichierAudio,
        int nombreEtats,
        String nomDossier) {
        super(x, y, largeur, hauteur, vitesseX, vitesseY, visible,
            fichierAudio, nombreEtats);

        // Charge les images de l'animation
        // On suppose que les fichiers .gif se trouvent dans un dossier nommé
        // nomDossier dans le répertoire du code compilé et que les noms de
        // fichiers gif sont de la forme nomDossiern.gif, n = 1 .. nombreEtats
        this.imagesAnimation = new Image[nombreEtats];
        for (int i = 0; i < nombreEtats; i++) {
            URL url =
                getClass().getResource(nomDossier + "/" + nomDossier + (i + 1)
                    + ".gif");
            this.imagesAnimation[i] = new ImageIcon(url).getImage();
        }
    }
}

```



```

    }
}

public void paint(Graphics g) {
    g.drawImage(imagesAnimation[etatCourant], x, y, largeur, hauteur,
        null);
}
}

```

Plutôt que d'employer les méthodes de dessin de *Graphics*, une séquence d'images pré-enregistrées est utilisée. Java inclut des classes pour la manipulation des images selon différentes normes telles GIF (*Graphics Interchange Format*), JPEG (*Joint Photographic Experts Group*) et PNG (*Portable Network Graphics*). Notre classe suppose que les images sont des fichiers de type GIF.

Exemple. La figure suivante montre le contenu de 4 fichiers GIF qui représentent une séquence d'images utilisées pour produire une animation d'une pieuvre qui agite ses tentacules. Le nom du fichier est montré sous l'image. L'extension *.gif* est une convention pour les fichiers d'images encodés selon la norme GIF. Le jeu utilise de la même façon une séquence de 2 images pour animer le calmar.

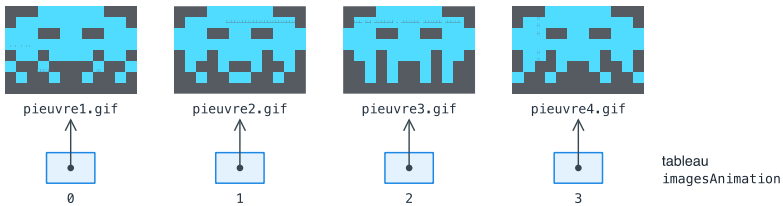


Figure 33. Série d'images pour l'animation de la pieuvre qui agite ses tentacules.

- **Notion de tableau**

Dans notre programme, une image est représentée par un objet de la classe *Image*. Il faut un objet pour chacune des images (4 pour la pieuvre). Lorsqu'il est nécessaire de manipuler une série d'objets du même type, il est possible de déclarer une variable pour chacun des objets mais ceci devient rapidement lourd lorsque le nombre croît. Une solution consiste à employer une structure de donnée telle qu'un *tableau*.

La ligne suivante déclare un tableau nommé *imagesAnimation* illustré à la figure précédente. Les crochets `[]` qui suivent le nom de la classe *Image* indiquent que c'est un tableau.

```
protected Image[] imagesAnimation;
```

Le nom de classe *Image* qui précède les crochets signifie que chacune des cases du tableau représente en quelque sorte une variable de type *Image* et donc fait référence à un objet de la classe *Image*.

En Java, un tableau est considéré comme un objet. Comme pour un objet, le *new* crée l'objet tableau. Le nombre d'éléments du tableau est spécifié entre crochets (ici *nombreEtats*):

```
this.imagesAnimation = new Image[nombreEtats];
```

La boucle suivante crée les objets *Image* à partir des fichiers GIF et les affectent aux cases du tableau.

```
for (int i = 0; i < nombreEtats; i++){
```

L'URL est créé comme précédemment pour le fichier audio avec *getResource()* :

```
URL url =
    getClass().getResource(nomDossier + "/" + nomDossier + (i + 1)
        + ".gif");
```

Le paramètre

```
nomDossier+"/"+nomDossier+(i+1)+".gif"
```

est le chemin du fichier relativement au dossier de la classe *EntiteAnime.AvecCriEt.SequenceImages.class*. Pour notre exemple de pieuvre, les fichiers sont nommés *pieuvre*n*.gif* où *n* est un entier de 1 à 4. L'entier désigne l'ordre de l'image dans la séquence d'animation. De plus, les images sont dans un dossier nommé *pieuvre* dans le répertoire du fichier compilé *EntiteAnime.AvecCriEt.SequenceImages.class*.

La ligne suivante crée l'objet de la classe *Image* à partir de l'URL du fichier et l'affecte à la case *i* du tableau *imagesAnimation* :

```
this.imagesAnimation[i] = new ImageIcon(url).getImage();
```

La notation avec crochets *imagesAnimation[i]* désigne la case d'indice *i* du tableau. Il est à noter que, en Java, les cases d'un tableau sont numérotées de 0 à *n*-1 où *n* est le nombre de cases. Le numéro de case est appelé un *indice*. Par exemple, pour la pieuvre, le tableau a 4 cases dont les indices vont de 0 à 3 comme illustré à la figure précédente. Dans la figure, il faut comprendre qu'une case fait référence à un objet de la classe *Image* qui contient l'image produite à partir du fichier. Par exemple, *imagesAnimation[3]* fait référence à un objet de la classe *Image* qui contient l'image du fichier *pieuvre4.gif*.

Dans la classe *Entite.Anime.AvecCriEtSequenceImages*, la méthode *paint()* affiche successivement les images du tableau en se servant de *etatCourant* comme indice :

```
g.drawImage(imagesAnimation[etatCourant],x,y,largeur,hauteur,null);
```

Modes de transparence

Lorsqu'une image est affichée, chacun des pixels peut être affiché selon trois modes de transparence :

Opaque. Le pixel apparaît sans altération en remplaçant la couleur existante.

Transparent. Le pixel n'est pas affiché. On voit donc ce qui est déjà affiché. Dans notre exemple de pieuvre, le gris foncé qui entoure la pieuvre à la figure précédente correspond à des pixels transparents. Dans le jeu, le fond noir de l'espace apparaît donc autour des envahisseurs.

Translucide. Le pixel est affiché en laissant transparaître partiellement ce qui est déjà affiché. Ceci est réalisé en combinant la couleur existante avec la couleur du pixel à afficher.

Le mode de transparence du pixel est spécifié dans l'encodage de l'image. Le format GIF permet à chacun des pixels d'être transparent ou opaque. Le format PNG permet les trois modes alors que le format JPEG ne permet

que le mode opaque. Les éditeurs d'images permettent de préciser le mode de transparence de chacun des pixels.

Caractéristiques des tableaux Java

Un tableau Java est considéré comme un objet et chacun des éléments du tableau doit être du même type. Les éléments peuvent être d'un type primitif ou objet. Le nombre d'éléments d'un tableau ne peut changer après sa création. Les Collections Java, introduites plus loin, permettent de traiter des groupes de taille variable au besoin. A la création, les éléments d'un tableau sont initialisés avec une valeur de défaut. Il est aussi possible de préciser des valeurs initiales à la création du tableau en affectant au tableau une suite d'éléments entre accolades.

Exemple. L'exemple suivant illustre l'utilisation d'un tableau.

```
public class ExempleTableau {
    public static void main(String[] args) {
        int[] tableauDe5Int = {12, 3, 154, -5, 17};
        for (int i = 0; i < tableauDe5Int.length; i++)
            System.out.println(i + " " + tableauDe5Int[i]);
        for (int unInt : tableauDe5Int)
            System.out.println(unInt);
    }
}
```

Le tableau *tableauDe5Int* est initialisé avec les cinq entiers entre accolades :

```
int[] tableauDe5Int = {12, 3, 154, -5, 17};
```

Le premier *for* permet d'itérer sur chacun des éléments du tableau à l'aide du compteur *i* qui joue le rôle d'indice qui prend les valeurs de 0 à la taille du tableau contenue dans *tableauDe5Int.length*. Le corps du *for* imprime l'indice *i* avec l'élément correspondant du tableau désigné par *tableauDe5Int[i]*.

```
for (int i = 0; i < tableauDe5Int.length; i++)
    System.out.println(i + " " + tableauDe5Int[i]);
```

Le deuxième *for* montre une forme spéciale du *for* adaptée à l'itération des éléments du tableau sans avoir à spécifier un compteur explicite.

```
for (int unInt : tableauDe5Int)
    System.out.println(unInt);
```

Le *for* permet de spécifier une variable, ici *unInt*, qui représente un élément du tableau à l'intérieur du *for*. Le type de la variable est déclaré en entête devant le nom de la variable. La variable est suivie de «:» et du nom du tableau à parcourir. Le *for* parcourt ainsi les éléments du tableau du premier au dernier et affecte l'élément courant de la boucle à la variable spécifiée, soit *unInt*, dans l'exemple.

Exercice. Ajoutez à l'exemple précédent le calcul de la moyenne des éléments du tableau et la différence de chacun des éléments par rapport à la moyenne.

Tableaux multi-dimensionnels

En Java il est possible que chacun des éléments d'un tableau soit lui-même un tableau (par nécessairement de même dimension).

Exemple. L'exemple suivant illustre l'utilisation d'un tableau à deux dimensions (matrice).

```
public class ExempleTableau2D {
    public static void main(String[] args) {
        int[][] matrice2par3 = {{11, 3, 2}, {-5, 7, 2}};
        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 3; j++) {
                System.out.print(matrice2par3[i][j] + " ");
                System.out.println();
            }
            for (int[] ligne : matrice2par3) {
                for (int unInt : ligne)
                    System.out.print(unInt + " ");
                System.out.println();
            }
        }
    }
}
```

La déclaration du type multi-dimensionnel est faite par une séquence de [] pour désigner chacune des dimensions. Le code suivant déclare un tableau *matrice2par3* à deux dimensions, 2 lignes par 3 colonnes.

```
int[][] matrice2par3 = {{11, 3, 2}, {-5, 7, 2}};
```

Les éléments sont énumérés ligne par ligne. La notation *matrice2par3*[1][2] représente l'élément de la deuxième ligne et de la troisième colonne, puisque les indices commencent à 0.

Deux boucles imbriquées avec compteur parcourent les éléments ligne par ligne, une boucle pour chacune des deux dimensions. Le compteur *i* correspond à la ligne et *j* à la colonne.

```
for (int i = 0; i < 2; i++){
    for (int j = 0; j < 3; j++){
        System.out.print(matrice2par3 [i][j] + " ");
        System.out.println();
    }
}
```

La forme étendue du *for* peut aussi être employée :

```
for (int[] ligne : matrice2par3) {
    for (int unInt : ligne)
        System.out.print(unInt + " ");
    System.out.println();
}
```

Exercice. Afficher le produit de deux matrices.

- **Classe *MondeDuJeu***

La classe *MondeDuJeu* reproduit la même organisation que la classe *MondeAnime* développée précédemment. En plus des méthodes *prochaineScene()* et *paint()*, elle ajoute deux nouvelles méthodes : *mousePressed()*, qui doit être appelée pour répondre au click de la souris, et *mouseMoved()*, qui doit être appelée lorsque la souris se déplace. La classe utilise des entités animées créées à partir des classes précédentes : une formation d'envahisseurs (calmars, crabes et pieuvres), une soucoupe, le canon du joueur ainsi que des projectiles (les tirs du canon et les bombes des envahisseurs).

JavaPasAPas/JeuSimple/MondeDuJeu.java

```

package JeuSimple;
/*
 * MondeDuJeu.java
 * Les envahisseurs : une formation d'envahisseurs dans un Vector
 * Le crabe bouge ses pinces (changement du dessin à chaque état)
 * Le calmar et la pieuvre sont animés par séquence d'images
 * Le canon suit la souris et tire à chaque clic
 */
import java.awt.*;
import java.awt.event.*;
import java.util.*;

public class MondeDuJeu {

    // Taille du monde
    public static final int LARGEURMONDE = 800;
    public static final int HAUTEURMONDE = 600;
    // Descente de la formation lorsqu'elle atteint un bord
    public static final int DESCENTE = 20;

    protected Vector vecteurEntites; // Les envahisseurs
    protected Vector vecteurProjectiles; // Les tirs et les bombes
    protected SoucoupeAnimeAvecCri soucoupe;
    protected CanonAnimeAvecCri canon;
    protected Random hasard = new Random();
    protected int score = 0;
    protected boolean partieTerminee = false;

    public MondeDuJeu() {
        vecteurEntites = new Vector();
        for (int colonne = 0; colonne < 8; colonne++) {
            int x = 40 + colonne * 64;
            vecteurEntites.addElement(
                new EntiteAnimeAvecCriEtSequenceImages(
                    x, 80, 48, 32, 3, 0, true, "explosion1.wav", 2, "calmar"));
            vecteurEntites.addElement(
                new CrabeAnimeAvecCri(x, 130, 48, 32, 3, 0, true,
                    "explosion2.wav"));
            vecteurEntites.addElement(
                new EntiteAnimeAvecCriEtSequenceImages(
                    x, 180, 48, 32, 3, 0, true, "explosion3.wav", 4, "pieuvre"));
        }
        vecteurProjectiles = new Vector();
        soucoupe = new SoucoupeAnimeAvecCri(0, 30, 64, 28, 5, 0, true,
            "soucoupe.wav");
        canon =
            new CanonAnimeAvecCri(370, 530, 60, 30, 0, 0, true, "canon.wav");
    }

    public void prochaineScene() {
        if (partieTerminee) {
            return; // Le monde ne bouge plus
        }
        // Si la formation atteint un bord, elle descend et change de direction
        if (formationAuBord()) {

```

```

    for (Iterator unIterator = vecteurEntites.iterator();
         unIterator.hasNext(); ) {
        EntiteAnime uneEntite = (EntiteAnime) unIterator.next();
        uneEntite.setVitesseX(-uneEntite.getVitesseX());
        uneEntite.setY(uneEntite.getY() + DESCENTE);
    }
}
for (Iterator unIterator = vecteurEntites.iterator();
     unIterator.hasNext(); ) {
    ((EntiteAnime) unIterator.next()).prochaineScene(LARGEURMONDE,
        HAUTEURMONDE);
}
soucoupe.prochaineScene(LARGEURMONDE, HAUTEURMONDE);
lancerBombe();
// Déplace les projectiles et retire ceux qui sont devenus invisibles
for (Iterator unIterator = vecteurProjectiles.iterator();
     unIterator.hasNext(); ) {
    Projectile unProjectile = (Projectile) unIterator.next();
    unProjectile.prochaineScene(LARGEURMONDE, HAUTEURMONDE);
    traiterCollisions(unProjectile);
    if (!unProjectile.getVisible()) {
        unIterator.remove();
    }
}
}
verifierFinDePartie();
}

// Vrai si un envahisseur visible va dépasser un bord du monde
private boolean formationAuBord() {
    for (Iterator unIterator = vecteurEntites.iterator();
         unIterator.hasNext(); ) {
        EntiteAnime uneEntite = (EntiteAnime) unIterator.next();
        int prochainX = uneEntite.getX() + uneEntite.getVitesseX();
        if (uneEntite.getVisible()
            && (prochainX < 0
                || prochainX + uneEntite.getLargeur() >= LARGEURMONDE)) {
            return true;
        }
    }
    return false;
}

// De temps en temps, un envahisseur choisi au hasard lâche une bombe
private void lancerBombe() {
    if (hasard.nextInt(20) == 0) {
        int indice = hasard.nextInt(vecteurEntites.size());
        EntiteAnime uneEntite =
            (EntiteAnime) vecteurEntites.elementAt(indice);
        if (uneEntite.getVisible()) {
            vecteurProjectiles.addElement(
                new Projectile(
                    uneEntite.getX() + uneEntite.getLargeur() / 2,
                    uneEntite.getY() + uneEntite.getHauteur(),
                    6,
                    Color.yellow));
        }
    }
}

```



```
    }  
}  
  
// Si le projectile touche l'entité, les deux disparaissent  
// et l'entité pousse un cri  
private boolean collision(  
    EntiteAnimeAvecCrie uneEntite, Projectile unProjectile) {  
    if (uneEntite.getVisible()  
        && unProjectile.getVisible()  
        && uneEntite.touche(unProjectile.getX(), unProjectile.getY())) {  
        uneEntite.setVisible(false);  
        uneEntite.crier();  
        unProjectile.setVisible(false);  
        return true;  
    }  
    return false;  
}  
  
// Une bombe peut toucher le canon. Un tir du canon peut toucher la  
// soucoupe ou un envahisseur.  
private void traiterCollisions(Projectile unProjectile) {  
    if (unProjectile.getVitesseY() > 0) { // Une bombe descend  
        if (collision(canon, unProjectile)) {  
            partieTerminee = true;  
        }  
    } else { // Un tir monte  
        if (collision(soucoupe, unProjectile)) {  
            score = score + 100;  
        }  
        for (Iterator unIterator = vecteurEntites.iterator();  
            unIterator.hasNext(); ) {  
            EntiteAnimeAvecCrie uneEntite =  
                (EntiteAnimeAvecCrie) unIterator.next();  
            if (collision(uneEntite, unProjectile)) {  
                score = score + 10;  
            }  
        }  
    }  
}  
  
// La partie est gagnée lorsqu'il ne reste plus d'envahisseurs et  
// perdue lorsqu'un envahisseur atteint le canon  
private void verifierFinDePartie() {  
    boolean resteDesEnvahisseurs = false;  
    for (Iterator unIterator = vecteurEntites.iterator();  
        unIterator.hasNext(); ) {  
        EntiteAnime uneEntite = (EntiteAnime) unIterator.next();  
        if (uneEntite.getVisible()) {  
            resteDesEnvahisseurs = true;  
            if (uneEntite.getY() + uneEntite.getHauteur() >= canon.getY()) {  
                canon.setVisible(false);  
                canon.crier();  
                partieTerminee = true;  
            }  
        }  
    }  
}
```

```

    if (!resteDesEnvahisseurs) {
        partieTerminee = true;
    }
}

public void paint(Graphics g) {
    for (Iterator unIterator = vecteurEntites.iterator();
        unIterator.hasNext(); ) {
        ((EntiteAnime) unIterator.next()).paintSiVisible(g);
    }
    for (Iterator unIterator = vecteurProjectiles.iterator();
        unIterator.hasNext(); ) {
        ((EntiteAnime) unIterator.next()).paintSiVisible(g);
    }
    soucoupe.paintSiVisible(g);
    canon.paintSiVisible(g);
    // Le score et le message de fin de partie
    g.setColor(Color.white);
    g.setFont(new Font("SansSerif", Font.BOLD, 18));
    g.drawString("Score : " + score, 10, 20);
    if (partieTerminee) {
        g.setFont(new Font("SansSerif", Font.BOLD, 48));
        if (canon.getVisible()) {
            g.drawString("Victoire !", 280, 320);
        } else {
            g.drawString("Partie perdue", 240, 320);
        }
    }
}

// Le canon suit la souris
public void mouseMoved(MouseEvent e) {
    canon.setX(e.getX() - canon.getLargeur() / 2);
}

// Un clic lance un tir à partir du canon
public void mousePressed(MouseEvent e) {
    if (!partieTerminee) {
        vecteurProjectiles.addElement(
            new Projectile(
                canon.getX() + canon.getLargeur() / 2 - 2,
                canon.getY(),
                -12,
                Color.white));
    }
}
}

```

- Collections en Java

L'ensemble des envahisseurs du jeu est représenté par un objet *vecteurEntites* de la classe *java.util.Vector* :

```
vecteurEntites = new Vector();
```

Un tel objet correspond au concept mathématique de vecteur et il est semblable à un tableau au sens où il permet de remplacer plusieurs variables par une seule. Comme pour un tableau, il est possible d'accéder à chacun des objets dans un **Vector**. Cependant, un **Vector** se distingue d'un tableau par deux aspects. La manière d'accéder aux objets est différente et le nombre d'objets d'un **Vector** n'est pas fixe. Pour les envahisseurs, dont le nombre est connu à l'avance, on aurait aussi bien pu employer un tableau Java. Par contre, le nombre de projectiles varie sans cesse au cours de la partie : un tir est ajouté à chaque clic et un projectile est retiré lorsqu'il sort du monde ou touche sa cible. Le second **Vector**, **vecteurProjectiles**, profite donc pleinement de la taille variable d'un **Vector**.

La classe **Vector** fait partie d'un ensemble de classes Java qui implémentent l'interface `java.util.Collection`. Ces classes permettent toutes de manipuler des collections d'objets mais elles se distinguent surtout par différentes possibilités pour accéder aux objets.

La méthode **addElement(E obj)** sert à ajouter un objet à un **Vector**. Le constructeur de *MondeDuJeu* fait une série d'appels à **addElement()**, dans une boucle **for** qui parcourt les 8 colonnes de la formation, afin d'ajouter les objets correspondant aux envahisseurs dans le **Vector** **vecteurEntites**. Par exemple, l'appel suivant ajoute un nouvel objet de la classe *CrabeAnimeAvecCri* :

```
vecteurEntites.addElement(  
    new CrabeAnimeAvecCri(x, 130, 48, 32, 3, 0, true,  
        "explosion2.wav"));
```

L'objet est toujours ajouté à la suite des autres objets déjà contenus dans le **Vector**. Les objets sont ainsi ordonnés en fonction de l'ordre des appels à **addElement()**. Contrairement au tableau, on ne précise pas à quelle position exacte l'objet est ajouté. Comme pour un tableau, l'utilisation d'un **Vector** simplifie énormément le codage lorsqu'il y a une collection d'entités à traiter.

On peut parcourir les objets du **Vector**, un par un, en passant par un objet **Iterator** produit à partir du **Vector**. Par exemple, dans la méthode *prochaineScene()* de *MondeDuJeu*, on appelle la méthode

prochaineScene(LARGEURMONDE,HAUTEURMONDE) sur chacun des objets (entités) de vecteurEntites de la manière suivante :

```
for (Iterator unIterator = vecteurEntites.iterator();
    unIterator.hasNext(); ) {
    ((EntiteAnime) unIterator.next()).prochaineScene(LARGEURMONDE,
        HAUTEURMONDE);
}
```

Pour accéder aux objets, un objet **Iterator** est créé à partir du **Vector** dans la partie initialisation du *for*. L'objet **Iterator** maintient une position courante dans le **Vector**. La méthode *hasNext()* vérifie s'il reste encore un objet à parcourir. La méthode *next()* retourne le prochain objet en modifiant la position courante. Au départ, la position courante est avant le premier objet. Par opposition à un tableau, il n'est pas nécessaire de préciser la position de l'objet par un indice. La méthode *next()* retourne l'objet suivant en fonction de la position courante dans le **Vector** qui est maintenue dans l'objet **Iterator**.

A noter l'utilisation de la conversion de type:

```
((EntiteAnime)unIterator.next())
```

La méthode *next()* retourne un objet de la classe *java.lang.Object* super-classe de toutes les classes Java. Pour appeler la méthode *prochaineScene()* sur cet objet, il faut d'abord convertir l'objet en un *EntiteAnime*. Rappelons que c'est le principe de surcharge dynamique qui détermine la méthode *prochaineScene()* qui est effectivement exécutée. Dans le cas d'un objet de la classe *EntiteAnimeAvecCriEtSequenceImages* ou *CrabeAnimeAvecCri*, c'est la méthode définie dans la super-classe *EntiteAnimeAvecCriEtGestes* qui est exécutée. De même, l'appel *soucoupe.prochaineScene()* exécute la méthode de la super-classe *EntiteAnime*, alors que pour un objet *Projectile*, c'est la méthode redéfinie dans la classe *Projectile* qui est exécutée : le projectile file en ligne droite et devient invisible à la sortie du monde. Cet exemple illustre un aspect très puissant de la programmation objet. En regroupant un ensemble d'objets possiblement de classes différentes dans une collection, on peut par la suite appeler une méthode sur tous les objets de la collection par une itération même si le traitement effectué varie en fonction de la classe de l'objet.

Le même principe est employé dans la méthode *paint()* qui appelle *paintSiVisible()* sur chacun des objets du *Vector*.

```
for (Iterator unIterator = vecteurEntites.iterator();
     unIterator.hasNext(); ) {
    ((EntiteAnime) unIterator.next()).paintSiVisible(g);
}
```

La méthode *mousePressed()* est invoquée lors d'un click de la souris. Elle ajoute au *Vector vecteurProjectiles* un nouveau tir qui part du milieu du canon vers le haut (vitesse négative selon l'axe y) :

```
public void mousePressed(MouseEvent e) {
    if (!partieTerminee) {
        vecteurProjectiles.addElement(
            new Projectile(
                canon.getX() + canon.getLargeur() / 2 - 2,
                canon.getY(),
                -12,
                Color.white));
    }
}
```

La méthode *mouseMoved()* est invoquée lorsque la souris se déplace. Elle place le canon sous le curseur de la souris :

```
public void mouseMoved(MouseEvent e) {
    canon.setX(e.getX() - canon.getLargeur() / 2);
}
```

- **Déroulement d'une scène du jeu**

La méthode *prochaineScene()* de *MondeDuJeu* produit chaque scène du jeu en cinq étapes. D'abord, si la formation atteint un bord, elle change de direction et descend. Ensuite, chaque entité produit sa prochaine scène. Puis, un envahisseur lâche parfois une bombe. Les projectiles se déplacent alors et les collisions sont traitées. Enfin, on vérifie si la partie est terminée. Pour garder *prochaineScene()* courte et lisible, chaque étape importante est confiée à une méthode *private*, qui ne sert qu'à l'intérieur de la classe.

La méthode *formationAuBord()* vérifie si un envahisseur visible dépasserait un bord du monde à la prochaine scène. Dans ce cas, toutes les entités de la formation changent de direction et descendent de *DESCENTE* pixels. C'est ici que les méthodes de lecture et d'écriture de *EntiteAnime* sont nécessaires :

getVitesseX(), *setVitesseX()*, *getY()* et *setY()* permettent à *MondeDuJeu* de modifier les entités de l'extérieur.

```
EntiteAnime uneEntite = (EntiteAnime) unIterator.next();
uneEntite.setVitesseX(-uneEntite.getVitesseX());
uneEntite.setY(uneEntite.getY() + DESCENTE);
```

La méthode *lancerBombe()* emploie un objet de la classe *java.util.Random*. L'appel *hasard.nextInt(20)* retourne un entier choisi au hasard entre 0 et 19 : en moyenne une fois sur vingt, un envahisseur est choisi au hasard et lâche une bombe. L'envahisseur est obtenu par son indice dans le Vector avec la méthode *element.At()*, comme on le ferait avec un tableau.

Les collisions sont détectées avec la méthode *touche()* de *EntiteAnime* : un projectile touche une entité si son coin supérieur gauche est à l'intérieur du rectangle englobant de l'entité. La méthode *collision()* rend alors invisibles l'entité et le projectile et fait crier l'entité. La méthode *traiterCollisions()* distingue les bombes, qui descendent (vitesse positive selon l'axe y) et peuvent toucher le canon, des tirs du canon, qui montent et peuvent toucher la soucoupe (100 points) ou un envahisseur (10 points).

Les projectiles devenus invisibles sont retirés du Vector *vecteurProjectiles*. Il ne faut pas retirer un objet d'une collection avec la méthode *remove()* de la collection pendant qu'on la parcourt : la collection serait modifiée à l'insu de l'itérateur, ce qui provoque une exception *ConcurrentModificationException*. La méthode *remove()* de l'objet *Iterator* retire sans danger le dernier objet retourné par *next()* :

```
for (Iterator unIterator = vecteurProjectiles.iterator();
     unIterator.hasNext(); ) {
    Projectile unProjectile = (Projectile) unIterator.next();
    unProjectile.prochaineScene(LARGEURMONDE, HAUTEURMONDE);
    traiterCollisions(unProjectile);
    if (!unProjectile.getVisible()) {
        unIterator.remove();
    }
}
```

Enfin, la méthode *verifierFinDePartie()* termine la partie lorsqu'il ne reste plus d'envahisseurs visibles (victoire) ou lorsqu'un envahisseur atteint la hauteur du canon (défaite). Une fois la partie terminée, le monde ne bouge plus et la méthode *paint()* affiche un message avec *drawString()*, sous le score.

Voici le code de la classe *JPanelPourMondeJeuSimple*.

```
JavaPasAPas/JeuSimple/JPanelPourMondeJeuSimple.java
package JeuSimple;

import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class JPanelPourMondeJeuSimple extends JPanel
    implements ActionListener, MouseListener, MouseMotionListener {

    public static final int INTERVALLEENTRESCENES = 50; // En ms

    // Le chrono génère un événement à chaque intervalle
    private Timer chrono;
    // Le monde à animer
    private MondeDuJeu leMondeDuJeu;

    // Taille du JPanel
    public static final int LARGEURJPANEL = MondeDuJeu.LARGEURMONDE;
    public static final int HAUTEURJPANEL = MondeDuJeu.HAUTEURMONDE;

    // Constructeur initialise le monde à animer
    public JPanelPourMondeJeuSimple() {
        leMondeDuJeu = new MondeDuJeu();
        setBackground(Color.black); // Le fond noir de l'espace
        addMouseListener(this);
        addMouseMotionListener(this);
    }

    public void start() {
        if (chrono == null) {
            chrono = new Timer(INTERVALLEENTRESCENES, this);
            chrono.start();
        }
    }

    // Le chrono appelle actionPerformed périodiquement (boucle d'animation)
    public void actionPerformed(ActionEvent e) {
        repaint();
        // Produire la prochaine scène du monde à animer
        leMondeDuJeu.prochaineScene();
    }

    // paintComponent() est appelée indirectement par repaint()
    // N.B. Swing utilise le double tampon : pas besoin d'effacer !
    public void paintComponent(Graphics g) {
        super.paintComponent(g);

        // Dessine les entités de l'animation
        leMondeDuJeu.paint(g);
    }

    public void mousePressed(MouseEvent leMouseEvent) {
```

```

    leMondeDuJeu.mousePressed(leMouseEvent);
}

public void mouseMoved(MouseEvent leMouseEvent) {
    leMondeDuJeu.mouseMoved(leMouseEvent);
}

// Le canon suit aussi la souris lorsqu'un bouton est enfoncé
public void mouseDragged(MouseEvent leMouseEvent) {
    leMondeDuJeu.mouseMoved(leMouseEvent);
}

// Il faut absolument définir les autres méthodes pour les autres
// événements de souris même s'ils ne font rien
public void mouseClicked(MouseEvent leMouseEvent) {}

public void mouseEntered(MouseEvent leMouseEvent) {}

public void mouseExited(MouseEvent leMouseEvent) {}

public void mouseReleased(MouseEvent leMouseEvent) {}
}

```

Tel qu'illustré à la section précédente, la classe *JPanelPourMondeJeuSimple* anime le *MondeDuJeu* par la stratégie du chronomètre. Elle implémente donc l'interface `java.awt.event.ActionListener` et la méthode `actionPerformed(ActionEvent e)` pour répondre aux événement du *Timer*:

```

public void actionPerformed( ActionEvent e){
    repaint();
    // Produire la prochaine scène du monde à animer
    leMondeDuJeu.prochaineScene();
}

```

La classe *JPanelPourMondeJeuSimple* est désignée comme écouteur de l'événement du click de la souris. Nous avons déjà illustré le mécanisme d'écouteur au chapitre 5. Nous avons vu qu'il faut désigner un objet qui implémente l'interface `java.awt.event.MouseListener` et les méthodes correspondantes. La ligne suivante désigne l'objet de la classe *JPanelPourMondeJeuSimple* comme écouteur des événements de la souris :

```

addMouseListener (this);

```

Les déplacements de la souris sont des événements distincts, traités par un écouteur qui implémente l'interface `java.awt.event.MouseMotionListener` avec ses

deux méthodes `mouseMoved()` et `mouseDragged()` (déplacement avec un bouton enfoncé). La classe `JPanelPourMondeJeuSimple` implémente aussi cette interface et s'enregistre comme écouteur avec `addMouseListener()`. Le constructeur fixe aussi la couleur de fond du `JPanel` au noir de l'espace avec `setBackground(Color.black)`:

```
setBackground(Color.black); // Le fond noir de l'espace
addMouseListener(this);
addMouseMotionListener(this);
```

La méthode `mousePressed(MouseEvent e)` spécifie la réponse à l'événement de click de la souris :

```
public void mousePressed(MouseEvent leMouseEvent){
    leMondeDuJeu.mousePressed(leMouseEvent);
}
```

Cette méthode délègue le travail à effectuer au `MondeDuJeu` en appelant la méthode de même nom. Les méthodes `mouseMoved()` et `mouseDragged()` délèguent de la même façon le déplacement de la souris au monde du jeu, si bien que le canon suit la souris même lorsqu'un bouton est enfoncé. La classe `JFrameIncluantJPanelMondeDuJeu` est la fenêtre qui contient le `JPanel` à animer.

```
JavaPasAPas/JeuSimple/ JFrameIncluantJPanelMondeDuJeu.java
package JeuSimple;

import javax.swing.JFrame;

public class JFrameIncluantJPanelMondeDuJeu extends JFrame {

    public JFrameIncluantJPanelMondeDuJeu() {
        super("Les envahisseurs");
        JPanelPourMondeJeuSimple leJPanelAnimation =
            new JPanelPourMondeJeuSimple();
        this.getContentPane().add(leJPanelAnimation);
        this.setDefaultCloseOperation(EXIT_ON_CLOSE);
        this.setSize(
            JPanelPourMondeJeuSimple.LARGEURJPANEL,
            JPanelPourMondeJeuSimple.HAUTEURJPANEL + 30);
        this.setVisible(true);
        leJPanelAnimation.start();
    }

    public static void main(String[] args) {
```

```
new JFrameIncluantJPanelMondeDuJeu();
}
}
```

8.4 Génériques

Introduite à la version 5 de Java, la généricité permet d'employer un type comme valeur de paramètre. Cette possibilité est exploitée en particulier avec les collections génériques. Il est ainsi possible de préciser le type des éléments d'une collection à sa déclaration. Dans l'exemple suivant, un vecteur est déclaré en spécifiant que les éléments sont des objets de la classe *EntiteAnimeAvecCri*.

```
Vector<EntiteAnimeAvecCri> vecteurEntites;
```

Le constructeur peut aussi spécifier le type des éléments :

```
vecteurEntites = new Vector<EntiteAnimeAvecCri>();
```

Le *for* permet l'itération sur les éléments de la collection sans avoir à passer explicitement par un *Iterator* et une conversion de type tel qu'illustré par l'exemple suivant :

```
for(EntiteAnimeAvecCri uneEntite : vecteurEntites){
    uneEntite.prochaineScene(LARGEURMONDE,HAUTEURMONDE);
}
```

L'exemple suivant est une version du monde du jeu qui exploite deux collections génériques : *Vector<EntiteAnimeAvecCri>* pour les envahisseurs et *Vector<Projectile>* pour les projectiles. Les conversions de type disparaissent. Notez toutefois que la boucle qui retire les projectiles invisibles passe toujours par un *Iterator* explicite, ici de type *Iterator<Projectile>* : la forme étendue du *for* ne donne pas accès à la méthode *remove()* de l'itérateur.

```
JavaPasAPas/JeuSimple/MondeDuJeuVectorGen.java
package JeuSimple;
/*
 * MondeDuJeuVectorGen.java
 * Les envahisseurs dans un Vector<EntiteAnimeAvecCri>
 * Les projectiles dans un Vector<Projectile>
 */
```

```
import java.awt.*;
import java.awt.event.*;
import java.util.*;

public class MondeDuJeuVectorGen {

    // Taille du monde
    public static final int LARGEURMONDE = 800;
    public static final int HAUTEURMONDE = 600;
    // Descente de la formation lorsqu'elle atteint un bord
    public static final int DESCENTE = 20;

    protected Vector<EntiteAnimeAvecCri> vecteurEntites; // Les envahisseurs
    protected Vector<Projectile> vecteurProjectiles; // Tirs et bombes
    protected SoucoupeAnimeAvecCri soucoupe;
    protected CanonAnimeAvecCri canon;
    protected Random hasard = new Random();
    protected int score = 0;
    protected boolean partieTerminee = false;

    public MondeDuJeuVectorGen() {
        vecteurEntites = new Vector<EntiteAnimeAvecCri>();
        for (int colonne = 0; colonne < 8; colonne++) {
            int x = 40 + colonne * 64;
            vecteurEntites.addElement(
                new EntiteAnimeAvecCriEtSequenceImages(
                    x, 80, 48, 32, 3, 0, true, "explosion1.wav", 2, "calmar"));
            vecteurEntites.addElement(
                new CrabeAnimeAvecCri(x, 130, 48, 32, 3, 0, true,
                    "explosion2.wav"));
            vecteurEntites.addElement(
                new EntiteAnimeAvecCriEtSequenceImages(
                    x, 180, 48, 32, 3, 0, true, "explosion3.wav", 4, "pieuvre"));
        }
        vecteurProjectiles = new Vector<Projectile>();
        soucoupe = new SoucoupeAnimeAvecCri(0, 30, 64, 28, 5, 0, true,
            "soucoupe.wav");
        canon =
            new CanonAnimeAvecCri(370, 530, 60, 30, 0, 0, true, "canon.wav");
    }

    public void prochaineScene() {
        if (partieTerminee) {
            return; // Le monde ne bouge plus
        }
        // Si la formation atteint un bord, elle descend et change de direction
        if (formationAuBord()) {
            for (EntiteAnimeAvecCri uneEntite : vecteurEntites) {
                uneEntite.setVitesseX(-uneEntite.getVitesseX());
                uneEntite.setY(uneEntite.getY() + DESCENTE);
            }
        }
        for (EntiteAnimeAvecCri uneEntite : vecteurEntites) {
            uneEntite.prochaineScene(LARGEURMONDE, HAUTEURMONDE);
        }
        soucoupe.prochaineScene(LARGEURMONDE, HAUTEURMONDE);
    }
}
```

```

    lancerBombe();
    // Un Iterator explicite est nécessaire pour retirer des éléments
    for (Iterator<Projectile> unIterator = vecteurProjectiles.iterator();
        unIterator.hasNext(); ) {
        Projectile unProjectile = unIterator.next();
        unProjectile.prochaineScene(LARGEURMONDE, HAUTEURMONDE);
        traiterCollisions(unProjectile);
        if (!unProjectile.getVisible()) {
            unIterator.remove();
        }
    }
    verifierFinDePartie();
}

// Vrai si un envahisseur visible va dépasser un bord du monde
private boolean formationAuBord() {
    for (EntiteAnimeAvecCrie uneEntite : vecteurEntites) {
        int prochainX = uneEntite.getX() + uneEntite.getVitesseX();
        if (uneEntite.getVisible()
            && (prochainX < 0
                || prochainX + uneEntite.getLargeur() >= LARGEURMONDE)) {
            return true;
        }
    }
    return false;
}

// De temps en temps, un envahisseur choisi au hasard lâche une bombe
private void lancerBombe() {
    if (hasard.nextInt(20) == 0) {
        int indice = hasard.nextInt(vecteurEntites.size());
        EntiteAnimeAvecCrie uneEntite = vecteurEntites.elementAt(indice);
        if (uneEntite.getVisible()) {
            vecteurProjectiles.addElement(
                new Projectile(
                    uneEntite.getX() + uneEntite.getLargeur() / 2,
                    uneEntite.getY() + uneEntite.getHauteur(),
                    6,
                    Color.yellow));
        }
    }
}

// Si le projectile touche l'entité, les deux disparaissent
// et l'entité pousse un cri
private boolean collision(
    EntiteAnimeAvecCrie uneEntite, Projectile unProjectile) {
    if (uneEntite.getVisible()
        && unProjectile.getVisible()
        && uneEntite.touche(unProjectile.getX(), unProjectile.getY())) {
        uneEntite.setVisible(false);
        uneEntite.crier();
        unProjectile.setVisible(false);
        return true;
    }
    return false;
}

```

```
}

// Une bombe peut toucher le canon. Un tir du canon peut toucher la
// soucoupe ou un envahisseur.
private void traiterCollisions(Projectile unProjectile) {
    if (unProjectile.getVitesseY() > 0) { // Une bombe descend
        if (collision(canon, unProjectile)) {
            partieTerminee = true;
        }
    } else { // Un tir monte
        if (collision(soucoupe, unProjectile)) {
            score = score + 100;
        }
        for (EntiteAnimeAvecCri uneEntite : vecteurEntites) {
            if (collision(uneEntite, unProjectile)) {
                score = score + 10;
            }
        }
    }
}

// La partie est gagnée lorsqu'il ne reste plus d'envahisseurs et
// perdue lorsqu'un envahisseur atteint le canon
private void verifierFinDePartie() {
    boolean resteDesEnvahisseurs = false;
    for (EntiteAnimeAvecCri uneEntite : vecteurEntites) {
        if (uneEntite.getVisible()) {
            resteDesEnvahisseurs = true;
            if (uneEntite.getY() + uneEntite.getHauteur() >= canon.getY()) {
                canon.setVisible(false);
                canon.crier();
                partieTerminee = true;
            }
        }
    }
    if (!resteDesEnvahisseurs) {
        partieTerminee = true;
    }
}

public void paint(Graphics g) {
    for (EntiteAnimeAvecCri uneEntite : vecteurEntites) {
        uneEntite.paintSiVisible(g);
    }
    for (Projectile unProjectile : vecteurProjectiles) {
        unProjectile.paintSiVisible(g);
    }
    soucoupe.paintSiVisible(g);
    canon.paintSiVisible(g);
    // Le score et le message de fin de partie
    g.setColor(Color.white);
    g.setFont(new Font("SansSerif", Font.BOLD, 18));
    g.drawString("Score : " + score, 10, 20);
    if (partieTerminee) {
        g.setFont(new Font("SansSerif", Font.BOLD, 48));
        if (canon.getVisible()) {
```

```

        g.drawString("Victoire !", 280, 320);
    } else {
        g.drawString("Partie perdue", 240, 320);
    }
}

// Le canon suit la souris
public void mouseMoved(MouseEvent e) {
    canon.setX(e.getX() - canon.getLargeur() / 2);
}

// Un clic lance un tir à partir du canon
public void mousePressed(MouseEvent e) {
    if (!partieTerminee) {
        vecteurProjectiles.addElement(
            new Projectile(
                canon.getX() + canon.getLargeur() / 2 - 2,
                canon.getY(),
                -12,
                Color.white));
    }
}
}

```

8.5 Autres collections

En plus de la classe **Vector**, Java offre plusieurs autres classes pour manipuler des collections d'objets : *Set*, *ArrayList*, *LinkedList*, *Map*, *Queue*, etc. Chacune des classes possèdent des avantages particuliers pour la manipulation de groupes d'objets. La classe **Vector** a la particularité d'être synchronisée (*Synchronized*), ce qui signifie qu'elle peut être partagée entre plusieurs fils (*Thread*) parallèles. Les mécanismes de contrôle de concurrence employés à cet effet entraînent une surcharge de calcul dans le cas où ce partage n'est pas nécessaire. Java offre plusieurs classes Collection au-delà de Vector qui ne sont pas synchronisées et qui évitent cette surcharge de travail. Dans notre exemple de jeu, il aurait été plus judicieux d'employer la classe **ArrayList**.

Exercice. Reprendre le jeu avec une collection **ArrayList** générique.

9. Traitement de fichiers

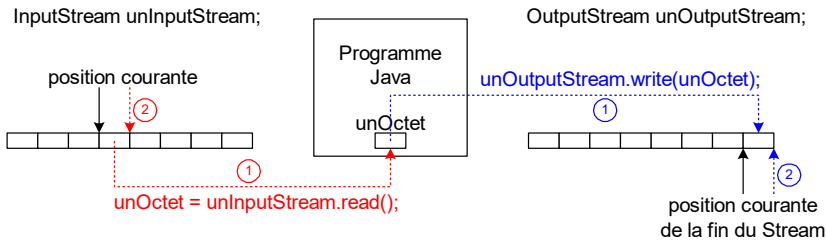
« N'importe quel code que vous avez écrit il y a six mois ou plus sans y regarder pourrait tout aussi bien avoir été écrit par quelqu'un d'autre. » — Alan Eagleson

Les données en mémoire Java ne sont pas conservées après la fin du programme. Pour conserver des données de manière persistante à long terme, il faut employer les mémoires secondaires. Les langages de programmation fournissent des interfaces programmatiques pour la manipulation des fichiers en mémoire secondaire. Ces interfaces réalisent des abstractions simples qui isolent le client de plusieurs des détails de bas niveau des mécanismes des mémoires secondaires. La simplicité de ces interfaces est pertinente pour le développement d'applications basiques nécessitant la persistance des données. Par contre, ces interfaces sont souvent trop limitées pour des applications complexes nécessitant des services plus sophistiqués. Les systèmes de gestion de bases de données sont prévus à cet effet.

Dans le cas de Java, le package [java.io](#) contient une hiérarchie élaborée de classes dédiées aux entrées-sorties sur fichier et sur d'autres types de flux de données. Dans ce chapitre, l'emphasis est mise sur l'utilisation des classes pour le traitement de fichiers. Les classes de [java.io](#) permettent d'utiliser les fichiers selon deux modes de base :

- par *accès sériel* comme des flux de données (*stream*). Les octets du fichier sont lus ou écrits en série les uns après les autres.
- par *accès direct* (*random access*). Les octets peuvent être lus ou écrits dans un ordre quelconque.

Dans le cas de l'accès par flux, des méthodes permettent d'itérer sur les données de manière sérielle les unes après les autres. Un flux d'entrée correspondant à la classe abstraite [InputStream](#) permet de lire une suite d'octets. Un flux de sortie correspondant à la classe abstraite [OutputStream](#) permet d'écrire une suite d'octets. Comme illustré à la figure suivante, la méthode [read\(\)](#) de [InputStream](#) lit le prochain octet du flux d'entrée et la méthode [write\(int b\)](#) de [OutputStream](#) ajoute un octet au flux de sortie.

Figure 34. Concept d'*InputStream* et d'*OutputStream*.

Cette abstraction cache plusieurs détails de réalisation du stockage des données sous forme de fichiers. Ainsi le programme Java n'a pas à se préoccuper de la structure physique des unités périphériques, de l'allocation d'espace au fichier, etc. Le fichier apparaît tout simplement comme une série d'octets. Le package *java.io* fournit ainsi une abstraction de base qui permet l'accès au niveau octet en traduisant les opérations sur les octets en termes des opérations du niveau de la structure physique de l'unité périphérique.

Les flux sont utilisés non seulement pour l'accès aux mémoires secondaires mais aussi pour d'autres unités périphériques tel que le clavier, l'écran, l'imprimante, le réseau, etc. L'accès à un flux est toujours sériel. Ceci est parfois insuffisant, par exemple, dans le cas d'une application qui doit pouvoir accéder directement à une donnée sans devoir itérer sur tout le fichier. Pour obtenir cette souplesse d'accès, il faut employer la classe [RandomAccessFile](#) qui permet un accès direct à n'importe quel octet d'un fichier.

La Figure 35 montre les classes pour les flux d'entrée d'octets. La Figure 36 montre les classes des flux de sortie d'octets. La Figure 37 montre la classe [RandomAccessFile](#) qui sert à la fois d'entrée et de sortie. Les flux de base [InputStream](#) et [OutputStream](#) agissent au niveau binaire en terme d'octets. Le programme voit l'unité d'entrée ou de sortie comme une série d'octets. Dans le cas d'un fichier, le programme ne voit donc pas de données structurées mais uniquement une suite de bits regroupés en octets.

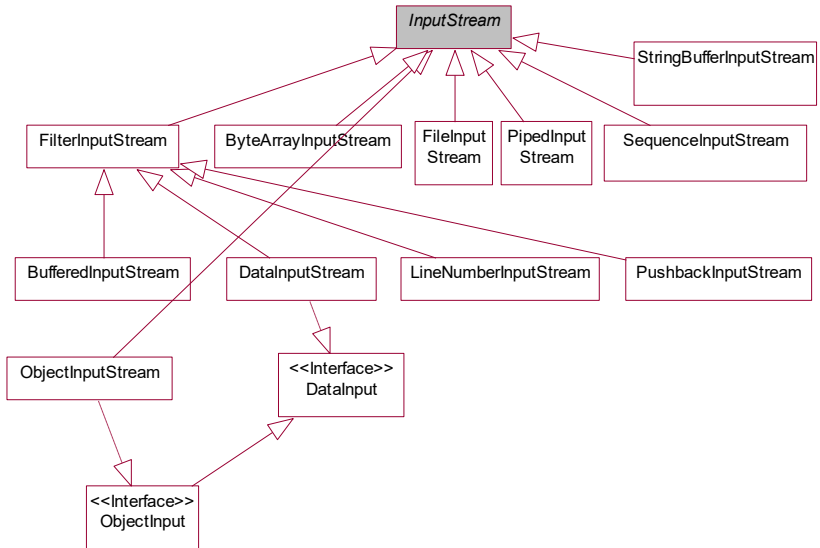


Figure 35. Classes de java.io pour les flux d'entrée d'octets (*InputStream*).

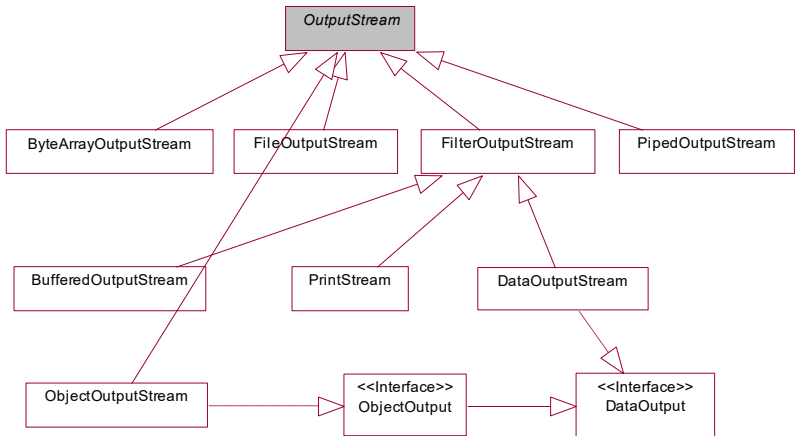
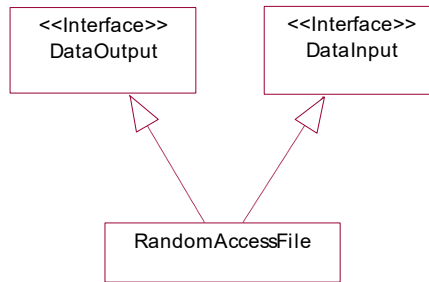


Figure 36. Classes de java.io pour les flux de sortie d'octets.

Figure 37. Classe *RandomAccessFile*.

- Lecture d'un flux d'octets provenant d'un fichier

L'exemple suivant introduit les concepts de base concernant la lecture d'un fichier vu comme une suite d'octets. Le programme utilise la classe `FileInputStream`, sous-classe de `InputStream`, dédiée au traitement des fichiers.

Exemple. Le programme Java suivant lit le contenu du fichier en consommant les octets, un octet à la fois, par l'utilisation de la classe `FileInputStream` et compte le nombre d'octets contenus dans le fichier.

```

JavaPasAPas/chapitre_8/CompterOctetsFichier.java
/* Lire un fichier et en compter le nombre d'octets */

import java.io.*;

public class CompterOctetsFichier {
    public static void main(String[] args) {
        int unOctet;
        int compteurOctet = 0;
        // Le fichier est fermé automatiquement à la fin du bloc try
        try (FileInputStream unFichier = new FileInputStream("Fichier1.txt")) {
            while ((unOctet = unFichier.read()) != -1) compteurOctet++;
            System.out.println(
                "Nombre d'octets du fichier Fichier1.txt : " + compteurOctet);
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
    }
}
  
```

L'énoncé suivant crée un objet de la classe `FileInputStream` en ouvrant le fichier dont le chemin est passé en paramètre au constructeur. Le fichier est

déclaré entre parenthèses, juste après le mot *try* : c'est un énoncé *try* avec ressources (*try-with-resources*) :

```
try (FileInputStream unFichier =
    new FileInputStream("Fichier1.txt")) {
```

Si le fichier est inexistant, une exception est levée. Le constructeur de la classe `FileInputStream` retourne une référence à un objet qui représente le fichier²⁷. Ainsi la variable *unFichier* est par la suite utilisée pour faire référence au fichier comme dans :

```
unOctet = unFichier.read()
```

La méthode `read()` de la classe `FileInputStream` retourne le prochain octet lu sous forme d'un entier. Pour lire tous les octets du fichier un par un, la lecture est incluse dans une boucle. Le fichier apparaît ainsi au programme comme un flux d'octets. Par convention, lorsque la fin du fichier est atteinte, la valeur `-1` est retournée²⁸ par `read()`. La boucle incrémente le compteur *compteurOctet* à chacune des itérations.

Il n'est pas nécessaire d'appeler la méthode `close()` pour fermer le fichier : à la fin du bloc *try*, Java appelle automatiquement *unFichier.close()*, même si une exception a été levée. C'est le principal avantage du *try* avec ressources : on ne peut pas oublier de fermer le fichier. Tous les exemples de ce chapitre emploient cette forme.

Après avoir fermé le fichier, les ressources associées sont libérées et le fichier n'est plus accessible par le programme.

Exercice. Comptez le nombre d'occurrences de la lettre « a » dans le fichier.

²⁷ . A noter que le«/» est utilisé plutôt que le «\» dans le chemin de fichier même si la plate-forme est Windows. Les deux formes de séparateur sont acceptées.

²⁸ Cependant, la fin de fichier est habituellement représentée par un code particulier du jeu de caractères qui n'est pas réellement la valeur entière `-1`. Ce code peut varier selon la plate-forme sous-jacente. Le `read()` retourne `-1` afin de faciliter l'indépendance du code vis-à-vis la plate-forme.

L'exemple suivant étend l'exemple de lecture d'un fichier en écrivant les octets lus dans un nouveau fichier.

Exemple.

```
JavaPasAPas/chapitre_8/CopierFichier.java
/* Copier un fichier octet par octet */

import java.io.*;

public class CopierFichier {
    public static void main(String[] args) {
        int unOctet;
        // Les deux fichiers sont fermés automatiquement à la fin du bloc try
        try (FileInputStream unFileInputStream =
            new FileInputStream("Fichier1.txt");
            FileOutputStream unFileOutputStream =
            new FileOutputStream("Fichier2.txt")) {
            while ((unOctet = unFileInputStream.read()) != -1)
                unFileOutputStream.write(unOctet);
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
    }
}
```

L'énoncé

```
new FileOutputStream("Fichier2.txt");
```

ouvre le fichier désigné par son chemin en vue de l'écriture. Si le fichier n'existe pas, il est créé. S'il existe déjà, le contenu précédent sera détruit et remplacé par les octets écrits par la suite.

La méthode [write\(int b\)](#) de [FileOutputStream](#) écrit l'octet lu dans le fichier :

```
unFileOutputStream.write(unOctet);
```

Seul l'octet le moins significatif de l'entier est employé.

9.1 Fichier binaire (*FileOutputStream*, *FileInputStream*)

Cette section montre comment conserver et récupérer une donnée dans un fichier en employant les opérations au niveau octet.

Exemple. Supposons que l'on veuille écrire un nombre entier (*int*) dans un fichier afin de le récupérer par la suite. Malheureusement, la classe `FileOutputStream` ne permet d'écrire que des octets qui ne sont pas interprétés comme des données d'un type de plus haut niveau comme un entier, un réel ou une chaîne de caractères²⁹. Comme l'interface est en termes d'octets, il faut d'abord convertir l'entier à écrire sous forme du tableau *tampon* de quatre octets. Le tableau *tampon* est ensuite écrit octet par octet par `write(tampon)`.

```
JavaPasAPas/chapitre_8/EcrireEntierEnOctets.java
/* Création d'un fichier et écriture d'un entier sous forme d'une suite
 * d'octets dans le fichier */
import java.io.*;

public class EcrireEntierEnOctets {
    public static void main(String[] args) {
        try (FileOutputStream unFichier = new FileOutputStream("Octets.dat")) {
            int unEntier = 1629696561; // "a#21" vu comme 4 octets
            // Convertir unEntier en un tableau de 4 octets
            byte[] tampon = new byte[4];
            for (int i = 3; i >= 0; i--) {
                // Extrait l'octet le moins significatif
                tampon[i] = (byte) (unEntier & 0xFF);
                unEntier >>= 8; // Décalage de 8 bits (remplissage à 0)
            }
            unFichier.write(tampon);
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
    }
}
```

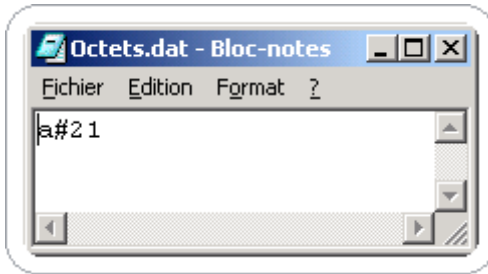
L'énoncé

```
try (FileOutputStream unFichier =
    new FileOutputStream("Octets.dat")) {
```

ouvre le fichier.

Par ailleurs, si ce fichier est ouvert avec l'éditeur de texte *Bloc-notes* de *Windows*, le contenu semble incompréhensible parce que le programme *Bloc-notes* interprète le contenu du fichier comme une suite caractères et non un entier de 4 octets :

²⁹ Il n'y a pas de méthode `writeInt(int unEntier)` supportée par `FileOutputStream`.



On dit souvent de ce genre de fichier qu'il est *binaire* par opposition à un fichier de type *texte* que nous étudierons plus loin. Pour lire le fichier avec `FileInputStream`, et l'interpréter correctement, il faut convertir les octets lus en entier.

Exemple.

```
JavaPasAPas/chapitre_8/LireEntierEnOctets.java
/* Lecture dans le fichier d'un entier sous forme d'une suite d'octets et
 * conversion en int */

import java.io.*;

public class LireEntierEnOctets {
    public static void main(String[] args) {
        try (FileInputStream unFichier = new FileInputStream("Octets.dat")) {
            byte[] tampon = new byte[4];
            unFichier.read(tampon); // Lecture des 4 octets

            // Convertir le tableau d'octets tampon en int unEntier
            int unEntier = 0;
            for (int i = 0; i <= 3; i++) {
                unEntier <= 8;
                unEntier += ((int) tampon[i]) & 0xFF;
            }
            System.out.println("Valeur décimale de l'entier : " + unEntier);
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
    }
}
```

Résultat :

Valeur décimale de l'entier : 1629696561
--

Exercice. Etendre l'exemple précédent en écrivant une suite de 3 entiers dans un fichier. Ensuite relire les trois entiers du fichier dans un autre programme.

Les classes [InputStream](#) et [OutputStream](#) sont rarement utilisées directement à cause des conversions fastidieuses à effectuer comme l'illustre les exemples précédents. D'autres sous-classes fournissent des abstractions de plus haut niveau. Les classes [DataInputStream](#) et [DataOutputStream](#) permettent de lire et d'écrire directement des types de base (*int*, *long*, *float*, ...) sans avoir à les convertir en suite d'octets. Les classes [ObjectInputStream](#) et [ObjectOutputStream](#) permettent de lire et d'écrire directement des objets. D'autre part, les classes [Reader](#) et [Writer](#) permettent de traiter les fichiers de type texte qui sont directement lisibles par les humains.

9.2 DataInputStream et DataOutputStream

Les classes [DataInputStream](#) et [DataOutputStream](#) permettent de lire et d'écrire les types de base. La figure suivante montre les méthodes supportées.

DataInputStream	DataOutputStream
DataInputStream(arg0 : InputStream) read(arg0 : byte[]) : int read(arg0 : byte[], arg1 : int, arg2 : int) : int readFully(arg0 : byte[]) : void readFully(arg0 : byte[], arg1 : int, arg2 : int) : void skipBytes(arg0 : int) : int readBoolean() : boolean readByte() : byte readUnsignedByte() : int readShort() : short readUnsignedShort() : int readChar() : char readInt() : int readLong() : long readFloat() : float readDouble() : double readLine() : String readUTF() : String readUTF(arg0 : DataInput) : String	DataOutputStream(arg0 : OutputStream) write(arg0 : int) : void write(arg0 : byte[], arg1 : int, arg2 : int) : void flush() : void writeBoolean(arg0 : boolean) : void writeByte(arg0 : int) : void writeShort(arg0 : int) : void writeChar(arg0 : int) : void writeInt(arg0 : int) : void writeLong(arg0 : long) : void writeFloat(arg0 : float) : void writeDouble(arg0 : double) : void writeBytes(arg0 : String) : void writeChars(arg0 : String) : void writeUTF(arg0 : String) : void size() : int

Figure 38. Méthodes des classes *DataInputStream* et *DataOutputStream*.

Pour illustrer l'utilisation de ces classes, reprenons l'exemple précédent d'écriture et de lecture d'un entier.

Exemple. Comme dans l'exemple précédent, le programme suivant stocke un entier dans un fichier, mais cette fois-ci, en utilisant la méthode [writeInt](#)(int v) de la classe [DataOutputStream](#) évitant ainsi la conversion en suite d'octets.

```

JavaPasAPas/chapitre_8/EcrireEntier.java
/* Création d'un DataOutputStream à partir d'un fichier et écriture d'un
 * entier dans le fichier */

import java.io.*;

public class EcrireEntier {
    public static void main(String[] args) {
        try (DataOutputStream unFichier =
            new DataOutputStream(new FileOutputStream("UnEntier.dat"))) {
            int unEntier = 1629696561; // "a#21" vu comme 4 octets
            unFichier.writeInt(unEntier);
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
    }
}

```

L'objet [DataOutputStream](#) est construit à partir d'un [FileOutputStream](#) avec l'instruction³⁰ :

```
new DataOutputStream(new FileOutputStream("UnEntier.dat"))
```

Il est ensuite possible d'écrire directement l'entier dans le fichier avec :

```
unFichier.writeInt(unEntier);
```

Exemple. Le programme suivant lit l'entier avec [readInt\(\)](#).

```

JavaPasAPas/chapitre_9/LireEntier.java
/* Lecture dans le fichier d'un entier à l'aide d'un DataInputStream */
import java.io.*;

public class LireEntier {
    public static void main(String[] args) {
        try (DataInputStream unFichier =
            new DataInputStream(new FileInputStream("UnEntier.dat"))) {
            int unEntier = unFichier.readInt();
            System.out.println("Valeur décimale de l'entier : " + unEntier);
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
    }
}

```

³⁰ Cette manière d'ajouter des fonctionnalités correspond au patron bien connu *décorateur*.

Exercice. Etendre l'exemple précédent en écrivant une suite de 3 entiers dans un fichier. Ensuite relire les trois entiers du fichier dans un autre programme.

9.3 Fichier texte

Les classes abstraites [Reader](#) et [Writer](#) sont analogues aux classes [InputStream](#) et [OutputStream](#) sauf qu'elles interprètent les flux d'octets comme des suites de caractères d'un jeu de caractère particulier. Elles gèrent plusieurs jeux de caractères standards (ASCII, ISO-Latin-1, Unicode, etc.). Par défaut, le jeu de caractère de la plate-forme sous-jacente est utilisé. La conversion des octets en caractères est effectuée par ces classes.

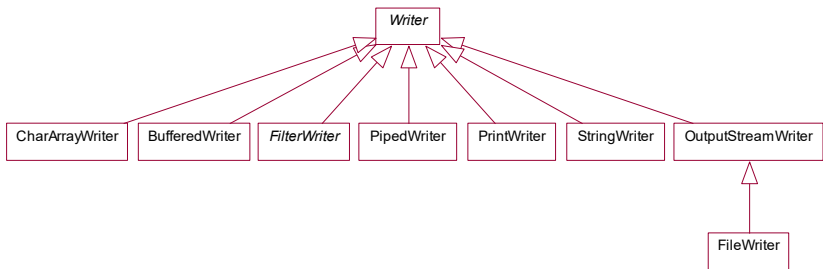


Figure 39. Sous-hiérarchie des classes *Writer*.

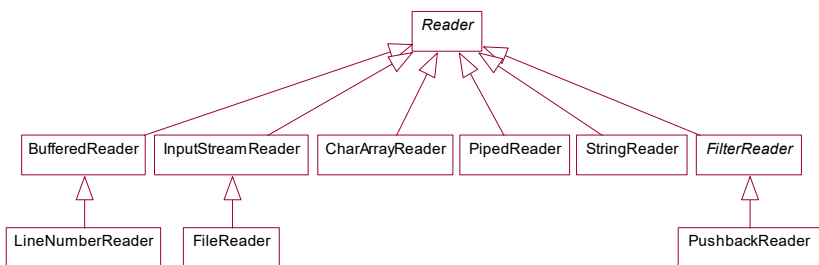


Figure 40. Sous-hiérarchie des classes *Reader*.

Exemple. L'exemple suivant écrit une chaîne de caractère qui représente un entier dans un fichier.

```

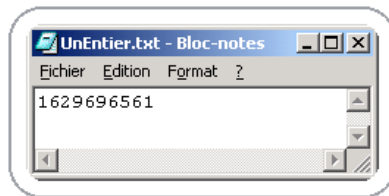
JavaPasAPas/chapitre_8/EcrireEntierTexte.java
/* Création d'un FileWriter à partir d'un fichier et écriture d'un entier
 * dans le fichier sous forme d'une chaîne de caractères */
import java.io.*;
import java.nio.charset.StandardCharsets;

public class EcrireEntierTexte {
    public static void main(String[] args) {
        // Le texte est encodé en UTF-8
        try (FileWriter unFichier =
            new FileWriter("UnEntier.txt", StandardCharsets.UTF_8)) {
            unFichier.write("1629696561");
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
    }
}

```

Le second paramètre du constructeur de *FileWriter*, *StandardCharsets.UTF_8*, précise le jeu de caractères employé pour encoder le texte, ici UTF-8. Il est préférable de toujours le préciser : autrement, le contenu du fichier dépend du jeu de caractères par défaut de l'ordinateur.

Comme le contenu du fichier *UnEntier.txt* est sous forme de texte, il peut être consulté avec un éditeur de texte. En ouvrant ce fichier avec l'éditeur Notepad, on voit donc l'entier sous une forme lisible :



Le même effet est obtenu par le programme suivant qui utilise plutôt un **PrintWriter**. Ceci permet d'écrire un *int* qui sera automatiquement converti sous forme d'une chaîne de caractère. Le *print()* accepte tous les types de base.

Exemple.

```
JavaPasAPas/chapitre_8/EcrireEntierTextePrintWriter.java
/* Création d'un PrintWriter à partir d'un fichier et écriture d'un entier
 * dans le fichier sous forme d'une chaîne de caractères */

import java.io.*;
import java.nio.charset.StandardCharsets;

public class EcrireEntierTextePrintWriter {
    public static void main(String[] args) {
        try (PrintWriter unPrintWriter = new PrintWriter(
            new FileWriter("UnEntierPW.txt", StandardCharsets.UTF_8))) {
            unPrintWriter.print(1629696561);
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
    }
}
```

PrintStream

La classe PrintStream permet aussi de faire un *print* des types de base. Pour des raisons historiques, les objets prédéfinis *System.out* (sortie standard, habituellement l'écran) et *System.err* sont des PrintStream plutôt que des PrintWriter.

Exemple. Le programme suivant lit l'entier sous forme de texte dans un tableau de caractères. Ensuite le tableau est converti en un *int* en passant par la méthode Integer.parseInt(String s).

```
JavaPasAPas/chapitre_9/LireEntierTexte.java
/* Lecture dans le fichier d'un entier sous forme de texte à l'aide d'un
 * FileReader */

import java.io.*;
import java.nio.charset.StandardCharsets;

public class LireEntierTexte {
    public static void main(String[] args) {
        try (FileReader unFichier =
            new FileReader("UnEntier.txt", StandardCharsets.UTF_8)) {
            char[] tableauChar = new char[10];
            unFichier.read(tableauChar, 0, 10);
            int unEntier = Integer.parseInt(new String(tableauChar, 0, 10));
            System.out.println("Valeur décimale de l'entier : " + unEntier);
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
    }
}
```

```
}  
}
```

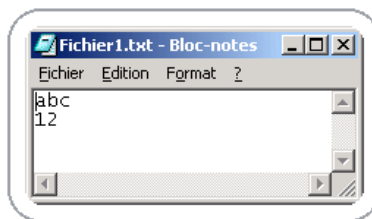
Une difficulté importante de la lecture de données sous forme de texte est que la taille de la donnée à lire n'est pas toujours connue à l'avance et peut être variable. Dans l'appel à `read(char[] cbuf, int off, int len)`, il faut spécifier l'indice du début de la chaîne et nombre de caractères à lire :

```
unFichier.read(tableauChar,0,10);
```

Si le nombre de caractères utilisé pour représenter l'entier est différent de 10, le programme ne donnera pas le bon résultat. Habituellement, lorsque des données sont placées dans un fichier texte, la fin d'une donnée est spécifiée par un délimiteur déterminé par une convention prédéfinie. Dans notre petit exemple, on pourrait spécifier que la fin de l'entier est déterminée par la fin du fichier. Le programme pourrait alors lire caractère par caractère jusqu'à ce que la fin du fichier soit atteinte. Le cas général du découpage d'un texte en éléments délimités est étudié plus loin.

9.3.1 Représentation interne des caractères et traitement des fins de ligne

Supposons que le fichier *Fichier1.txt* soit édité avec l'éditeur de texte *Notepad* et que le contenu soit celui affiché à la figure suivante :



L'éditeur de texte nous affiche le contenu du fichier sous forme de caractères. En réalité, le contenu du fichier est une séquence de bits. Avec *Notepad* sous *Windows*, par défaut, le jeu de caractère utilisé est le code ASCII. Chacun des caractères du texte est représenté par un code ASCII de huit bits (un octet). La première ligne de l'exemple suivant montre le contenu réel du fichier. La deuxième ligne indique les caractères correspondants.

Exemple. Contenu du fichier *Fichier1.txt* en binaire.

01100001	01100010	01100011	00001101	00001010	00110001	00110010	00001101	00001010
a	b	c	\r	\n	1	2	\r	\n

Sous Windows, la fin de ligne est représentée par la séquence des caractères spéciaux ASCII, *retour de chariot* (noté <CR> ou '\r' en Java) et *saut de ligne* (<LF> ou '\n' en Java)³¹. Un code supplémentaire, absent dans l'exemple, est ajouté à la fin de la séquence pour indiquer la fin du fichier (0X0A sur Windows).

Avec cet exemple, le programme *CompterOctetsFichier* afficherait le résultat suivant à l'écran :

```
Nombre d'octets du fichier Fichier1.txt : 9
```

Pour un [FileInputStream](#), les octets lus n'ont pas de signification particulière. Les octets représentant les caractères de contrôle, de retour de chariot ('\r') et de saut de ligne ('\n') sont des octets comme les autres et ils sont donc comptés par le programme. C'est pourquoi même si à l'écran de *Notepad*, on ne voit que 5 caractères, chacun étant représenté par un octet ASCII, le programme compte 9 octets. Un [InputStream](#) voit donc un fichier sous forme binaire, c'est-à-dire sans interpréter les octets lus.

Exemple. Le fichier *Fichier1.txt* pourrait être produit en écrivant directement les octets à l'aide d'un [FileOutputStream](#).

```
JavaPasAPas/chapitre_9/EcrireOctetsFichier.java
/* Création d'un fichier et écriture d'une suite d'octets */
import java.io.*;

public class EcrireOctetsFichier {
    public static void main(String[] args) {
        try (FileOutputStream unFichier =
            new FileOutputStream("Fichier1.txt")) {
            unFichier.write(0X61);
            unFichier.write(0X62);
            unFichier.write(0X63);
            unFichier.write(0X0D);
            unFichier.write(0X0A);
            unFichier.write(0X31);
            unFichier.write(0X32);
            unFichier.write(0X0D);
            unFichier.write(0X0A);
        } catch (IOException e) {
```

³¹ La manière de représenter les fins de ligne peut différer en fonction de la plate-forme. Unix, par exemple, emploie uniquement le saut de ligne (<LF>).

```

        System.err.println("Exception\n" + e.toString());
    }
}

```

On peut produire le même effet avec les classes **PrintWriter** et *PrintStream* qui possèdent des méthodes pour traiter les fins de ligne. En particulier, les méthodes *println()* sont analogues aux méthodes *print()* et ajoutent une fin de ligne après la donnée.

Exemple. Production de *Fichier1.txt* avec un **PrintWriter**.

```

JavaPasAPas/chapitre_8/EcrireTexteabc12.java
/* Création d'un PrintWriter à partir d'un fichier et écriture de texte
 * avec println */

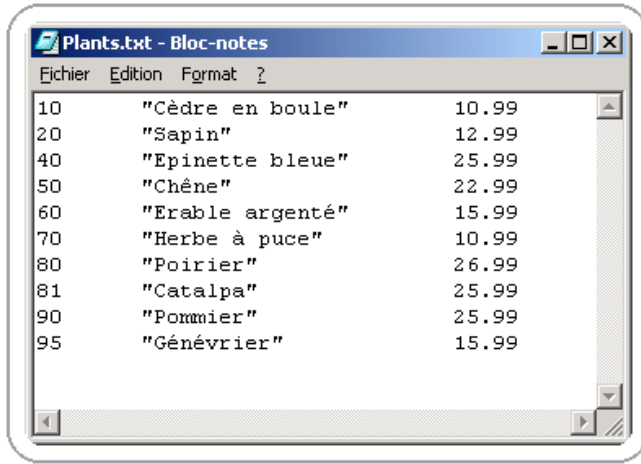
import java.io.*;
import java.nio.charset.StandardCharsets;

public class EcrireTexteabc12 {
    public static void main(String[] args) {
        try (PrintWriter unPrintWriter = new PrintWriter(
            new FileWriter("Fichier1.txt", StandardCharsets.UTF_8))) {
            unPrintWriter.println("abc");
            unPrintWriter.println(12);
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
    }
}

```

9.3.2 Analyse lexicale avec la classe **StreamTokenizer**

Un fichier texte est souvent utilisé pour saisir des données ou pour échanger des données entre applications. Les données sont alors placées en séquence selon un format prédéfini. Par exemple, supposons que le fichier *plants.txt* contienne des données sur les plants du catalogue de la pépinière *PleinDeFoin* sous la forme suivante :



10	"Cèdre en boule"	10.99
20	"Sapin"	12.99
40	"Épinette bleue"	25.99
50	"Chêne"	22.99
60	"Érable argenté"	15.99
70	"Herbe à puce"	10.99
80	"Poirier"	26.99
81	"Catalpa"	25.99
90	"Pommier"	25.99
95	"Génévrier"	15.99

On peut imaginer que les données ont été saisies à l'aide d'un éditeur de texte conventionnel. Ces données correspondent au numéro de catalogue, nom et prix d'un ensemble de plants. Supposons que les conventions suivantes ont été établies. Une donnée se termine par un ou plusieurs caractères parmi les suivants : espaces (' '), tabulation ('\t'), retour de chariot('\r'), fin de ligne ('\n'). Ces caractères jouent le rôle de *délimiteurs*. Un cas particulier est souvent nécessaire pour les chaînes de caractères qui peuvent contenir des délimiteurs. C'est le cas des descriptions de plants dans notre exemple. Par convention, elles sont encadrées par des guillemets (") qui jouent ainsi le rôle de *délimiteur de chaîne*. Le découpage du texte en ses éléments constitutifs, appelés *jetons (token)* est un problème bien connu, appelé analyse lexicale, qui fait appel à des techniques éprouvées. La classe **StreamTokenizer** incorpore les algorithmes nécessaires.

Exemple. Le programme suivant illustre l'utilisation d'un **StreamTokenizer** qui permet de découper un texte en jetons. Les données lues sont stockées dans un vecteur (un objet `ArrayList`, voir le chapitre 8) d'objets de la classe `Plant` et sont affichées à l'écran.

```
JavaPasAPas/chapitre_8/ExempleStreamTokenizer.java
/* Illustration du StreamTokenizer
 * Lit le fichier Plants.txt, affiche à l'écran chacun des jetons
 * (noPlant, description, prixUnitaire) et stocke le contenu dans un
 * vecteur (ArrayList) d'objets Plant */

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.*;
```

```

public class ExempleStreamTokenizer {
    public static void main(String[] args) {
        ArrayList<Plant> vecteurDePlants = new ArrayList<Plant>();
        try (FileReader unFichier =
            new FileReader("Plants.txt", StandardCharsets.UTF_8)) {
            StreamTokenizer unStreamTokenizer = new StreamTokenizer(unFichier);
            // Les 5 lignes suivantes ne sont pas nécessaires car les paramètres
            // donnés sont les valeurs de défaut
            unStreamTokenizer.quoteChar((int) '\'');
            unStreamTokenizer.whitespaceChars((int) '\r', (int) '\r');
            unStreamTokenizer.whitespaceChars((int) '\n', (int) '\n');
            unStreamTokenizer.whitespaceChars((int) '\t', (int) '\t');
            unStreamTokenizer.whitespaceChars((int) ' ', (int) ' ');

            int noPlant = 0;
            String description = "";
            double prixUnitaire = 0.0;

            // Jusqu'à la fin du fichier
            while (unStreamTokenizer.nextToken() != StreamTokenizer.TT_EOF) {
                // Lecture du noPlant : est-ce bien un nombre ?
                if (unStreamTokenizer.ttype == StreamTokenizer.TT_NUMBER) {
                    noPlant = (int) unStreamTokenizer.nval; // nval est un double !
                } else {
                    System.out.println("Format incorrect : noPlant attendu");
                    System.exit(1);
                }
                // Lecture de la description : est-ce bien une chaîne entre " ?
                unStreamTokenizer.nextToken();
                if (unStreamTokenizer.ttype == (int) '\'') {
                    description = unStreamTokenizer.sval;
                } else {
                    System.out.println("Format incorrect : description attendue");
                    System.exit(1);
                }
                // Lecture du prixUnitaire : est-ce bien un nombre ?
                unStreamTokenizer.nextToken();
                if (unStreamTokenizer.ttype == StreamTokenizer.TT_NUMBER) {
                    prixUnitaire = unStreamTokenizer.nval;
                } else {
                    System.out.println("Format incorrect : prix attendu");
                    System.exit(1);
                }

                // Création de l'objet Plant
                Plant unPlant = new Plant(noPlant, description, prixUnitaire);
                System.out.println(
                    noPlant + " " + description + " " + prixUnitaire);
                vecteurDePlants.add(unPlant);
            }
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
    }
}

```



```
}  
}
```

JavaPasAPas/chapitre_8/Plant.java

```
import java.io.*;  
import java.nio.charset.StandardCharsets;  
  
public class Plant implements Serializable {  
    private int noPlant; // numéro de catalogue du plant  
    private String description; // description du plant  
    private double prixUnitaire; // prix unitaire du plant  
  
    public Plant(int noPlant, String description, double prixUnitaire) {  
        this.noPlant = noPlant;  
        this.description = description;  
        this.prixUnitaire = prixUnitaire;  
    }  
  
    public void setNoPlant(int noPlant) {  
        this.noPlant = noPlant;  
    }  
  
    public int getNoPlant() {  
        return noPlant;  
    }  
  
    public void setDescription(String description) {  
        this.description = description;  
    }  
  
    public String getDescription() {  
        return description;  
    }  
  
    public void setPrixUnitaire(double prixUnitaire) {  
        this.prixUnitaire = prixUnitaire;  
    }  
  
    public double getPrixUnitaire() {  
        return prixUnitaire;  
    }  
  
    public void ecrireEnregistrementTailleMax(RandomAccessFile unFichier)  
        throws Exception {  
        // 4 + 4 + 8 octets pour noPlant, la taille et prixUnitaire  
        int tailleZoneDescription = tailleMaxEnregistrement() - 16;  
        // Un octet par caractère (ISO-8859-1, qui comprend les accents)  
        byte[] octets = description.getBytes(StandardCharsets.ISO_8859_1);  
        if (octets.length > tailleZoneDescription) {  
            System.exit(1);  
        }  
        unFichier.writeInt(noPlant); // 4 octets  
        unFichier.writeInt(octets.length); // 4 octets  
        unFichier.write(octets);  
    }  
}
```

```

// Compléter la zone description à 34 octets
unFichier.write(new byte[tailleZoneDescription - octets.length]);
unFichier.writeDouble(prixUnitaire); // 8 octets
}

public void lireEnregistrementTailleMax(RandomAccessFile unFichier)
    throws Exception {
    int tailleZoneDescription = tailleMaxEnregistrement() - 16;
    noPlant = unFichier.readInt();
    int tailleDescription = unFichier.readInt();
    byte[] tampon = new byte[tailleDescription];
    unFichier.readFully(tampon);
    description = new String(tampon, StandardCharsets.ISO_8859_1);
    unFichier.skipBytes(tailleZoneDescription - tailleDescription);
    prixUnitaire = unFichier.readDouble();
}

public static int tailleMaxEnregistrement() {
    return 50;
}
}

```

La méthode `nextToken()` retourne le type du prochain jeton rencontré et stocke cette valeur dans la variable publique *type*. Les types possibles sont les suivants :

- *StreamTokenizer.TT_WORD* : une chaîne a été rencontré. La valeur est stockée dans la variable String *sval*.
- *StreamTokenizer.TT_NUMBER* : un nombre a été rencontré. La valeur est stockée dans la variable double *mval*.
- *StreamTokenizer.TT_EOF* : la fin du fichier a été rencontrée.
- *StreamTokenizer.TT_EOL* : une fin de ligne a été rencontrée. Par défaut les fins de ligne ne sont pas traitées. La méthode *collsSignificant*(boolean flag) permet d'activer la détection des fins de ligne.
- Lorsqu'une chaîne délimitée par un caractère délimiteur de chaîne (*quoteChar*) est rencontrée, le type retourné est la valeur entière du caractère délimiteur de chaîne.
- Dans le cas de caractères spéciaux, c'est la valeur entière du caractère lui-même qui est retourné.

Plusieurs paramètres qui contrôlent le *StreamTokenizer* peuvent être modifiés au besoin par les méthodes suivantes entre autres :

- **whitespaceChars**(int low, int hi). Spécifie que les caractères de l'intervalle $[low, hi]$ sont des délimiteurs.
- **quoteChar**(int ch). Spécifie que le caractère *ch* est un délimiteur de chaîne.

Un problème important lors du décodage d'un fichier de texte est la validation de son format. Dans notre petit exemple, le format est assez simple. La validation est rudimentaire et ne tient pas compte de toutes les possibilités. Par exemple, si un nombre réel est rencontré à la place d'un entier, la conversion tronque le réel en entier sans aucun avertissement.

Exercice. Reprendre l'exemple précédent pour un fichier qui contient les noms et numéros de téléphone de contacts fictifs.

Avec la popularité du Web qui est un medium de transmission basé sur le texte, l'échange de données sous forme de fichier de texte prend une importance croissante. Ceci a conduit à l'établissement de la norme XML (*eXtensible Markup Language*). La section suivante introduit XML et des outils Java permettant de manipuler les fichiers XML.

9.3.3 Traitement d'un document XML avec SAX et DOM

XML permet de spécifier la grammaire d'un document textuel par un *Data Type Definition* – DTD ou par un schéma XML.

Exemple. Le contenu du fichier *Plants.xml* suivant est un exemple de document XML avec DTD incluse représentant le catalogue des plants de la pépinière *PleinDeFoin*.

```
JavaPasAPas/chapitre_8/Plants.xml
<?xml version="1.0" encoding="ISO-8859-1" standalone=
    "yes"?> <!DOCTYPE Catalogue [
    <!ELEMENT Catalogue (Plant+)>
    <!ELEMENT Plant (noPlant,description,prixUnitaire)>
    <!ELEMENT noPlant (#PCDATA)>
    <!ELEMENT description (#PCDATA)>
    <!ELEMENT prixUnitaire (#PCDATA)>
]>
<Catalogue>
```

```
<Plant>
<noPlant>10</noPlant>
<description>Cèdre en boule</description>
<prixUnitaire>10.99</prixUnitaire>
</Plant>
<Plant>
<noPlant>20</noPlant>
<description>Sapin</description>
<prixUnitaire>12.99</prixUnitaire>
</Plant>
<Plant>
<noPlant>40</noPlant>
<description>Epinette bleue</description>
<prixUnitaire>25.99</prixUnitaire>
</Plant>
<Plant>
<noPlant>50</noPlant>
<description>Chêne</description>
<prixUnitaire>22.99</prixUnitaire>
</Plant>

<Plant>
<noPlant>60</noPlant>
<description>Erable argenté</description>
<prixUnitaire>15.99</prixUnitaire>
</Plant>

<Plant>
<noPlant>70</noPlant>
<description>Erable argenté</description>
<prixUnitaire>15.99</prixUnitaire>
</Plant>
<Plant>
<noPlant>80</noPlant>
<description>Herbe à puce</description>
<prixUnitaire>10.99</prixUnitaire>
</Plant>

<Plant>
<noPlant>81</noPlant>
<description>Catalpa</description>
<prixUnitaire>25.99</prixUnitaire>
</Plant>
<Plant>
<noPlant>90</noPlant>
<description>Pommier</description>
<prixUnitaire>25.99</prixUnitaire>
</Plant>
<Plant>
<noPlant>95</noPlant>
<description>Génévrier</description>
<prixUnitaire>15.99</prixUnitaire>
</Plant>
</Catalogue>
```

L'entête

```
<?xml version="1.0" encoding="ISO-8859-1" standalone="yes"?>
```

indique la version utilisée de la norme XML, le jeu de caractère choisi et si le document fait référence à des documents externes. Le DOCTYPE définit un nom de type pour la structure du document :

```
<!DOCTYPE Catalogue [
```

Les clauses ELEMENT décrivent la grammaire du document. Chaque clause

```
!ELEMENT nomElement syntaxeElement
```

définit un élément qui est encodé dans le texte à l'aide d'une paire de *balises* de la forme

```
<nomElement> contenu </nomElement>
```

dont le contenu suit la syntaxe définie par *syntaxeElement*. Par opposition à HTML, les balises XML n'ont pas pour objectif de préciser le style d'affichage des données mais plutôt d'en indiquer la signification. La clause

```
<!ELEMENT Catalogue (Plant+)>
```

de la DTD définit l'élément nommé *Catalogue* dont le contenu est un ensemble (au moins un à cause du +) d'éléments *Plant* (au moins un à cause du +).

On peut ainsi observer que le contenu du document lui-même débute par la balise ouvrante

```
<Catalogue>
```

et se termine par la balise fermante

```
</Catalogue>
```

Entre les deux se trouvent une suite d'éléments *Plant*. La clause

```
<!ELEMENT Plant (noPlant,description,prixUnitaire)>
```

définit qu'un élément *Plant* est composé de trois éléments nommés : *noPlant*, *description* et *prixUnitaire*. On peut observer cette structure dans l'exemple suivant du contenu du document :

```
<Plant>
  <noPlant>10</noPlant>
  <description>Cèdre en boule</description>
  <prixUnitaire>10.99</prixUnitaire>
</Plant>
```

La présence des balises avec les données permet d'interpréter le texte du document soit pour un programme ou un humain. La clause

```
<!ELEMENT noPlant (#PCDATA)>
```

définit qu'un *noPlant* est une chaîne de caractère. Le symbole *#PCDATA* indique une feuille de la grammaire dont la syntaxe est une suite quelconque de caractères. XML ne permet pas de définir de types pour les données elle-même. C'est à l'application de vérifier que la chaîne est du bon type (un entier, un réel, etc.). Cette limitation de XML fait d'ailleurs l'objet d'une autre norme, *XML Schema* qui vise en particulier à combler cette lacune. D'autre part, la grammaire en *XML Schema* est elle-même codée en suivant la norme XML. Contrairement à notre petit exemple, la DTD d'un document n'est habituellement pas incluse directement dans le document lui-même mais est plutôt placée dans un fichier à part auquel le document XML fait référence. Ceci permet de partager la même DTD entre plusieurs documents XML sans avoir à la répéter.

Des outils, appelés parseurs XML, permettent de décoder un fichier XML, de vérifier la validité du contenu par rapport à la grammaire et de stocker le contenu en mémoire du programme sous une forme objet standard, le *Document Object Model* (DOM). Des parseurs Java sont disponibles et suivent eux-mêmes une norme pour l'interface programmatique appelée JAXP (*Java API for XML Processing Specification*). En fournissant la grammaire du document avec son contenu, il devient possible de vérifier la validité du contenu du document, ce qui garantit une meilleure intégrité des documents échangés.

- **Interface programmatique JAXP pour le parsing d'un document XML**

L'interface programmatique JAXP permet de traiter un document XML. L'exemple suivant lit le fichier *Plants.xml*, le parse avec un parseur de JAXP. Deux interfaces sont disponibles pour le traitement d'un document XML. L'interface DOM (*Document Object Model*) construit une représentation objet du document XML sous forme d'un arbre. Des méthodes permettent de naviguer dans la structure d'arbre DOM afin

d'en extraire les éléments. L'interface SAX (*Simple API for XML*) permet d'itérer sur les éléments de manière sérielle selon une approche de programmation par événement analogue à l'approche utilisée dans les interfaces graphiques.

Exemple. Le programme *ExempleJAXPPlants* analyse le fichier *Plants.xml* et en extrait les données avec l'interface DOM. Les données du fichier XML sont extraites par navigation dans l'arbre DOM, puis stockées dans un vecteur d'objets de la classe *Plant* et affichées à l'écran. Le résultat est donc le même que pour *ExempleStreamTokenizer* de la section précédente.

```
JavaPasAPas/chapitre_8/ExempleJAXPPlants.java
/*
 * Création d'un arbre DOM avec JAXP. Parcours de l'arbre pour extraire les
 * données et les insérer dans un vecteur (ArrayList) d'objets Plant
 */

import java.io.*;
import java.util.*;
// Packages de JAXP
import javax.xml.parsers.*;
import org.w3c.dom.*;

public class ExempleJAXPPlants {

    public static void main(String[] args) throws Exception {
        // Création d'un DocumentBuilderFactory et configuration des paramètres
        DocumentBuilderFactory unDocBuildFact =
            DocumentBuilderFactory.newInstance();
        unDocBuildFact.setValidating(true);
        unDocBuildFact.setIgnoringElementContentWhitespace(true);

        // Création d'un DocumentBuilder
        DocumentBuilder unDocumentBuilder =
            unDocBuildFact.newDocumentBuilder();

        // Parsage du document
        File leFile = new File("Plants.xml");
        Document unDocument = unDocumentBuilder.parse(leFile);
        ArrayList<Plant> vecteurDePlants = new ArrayList<Plant>();
        // Recherche l'élément racine <Catalogue>
        Node unElementCatalogue = unDocument.getDocumentElement();

        // Itérer sur les noeuds <Plant> qui sont les enfants de <Catalogue>
        NodeList listeNodePlants = unElementCatalogue.getChildNodes();
        int tailleListe = listeNodePlants.getLength();
        for (int i = 0; i < tailleListe; i++) {
            Node unNodePlant = listeNodePlants.item(i); // ELEMENT <Plant>
```

```

// ELEMENT <noPlant> : la valeur est dans le premier enfant
Node unNodeNoPlant = unNodePlant.getFirstChild();
int noPlant =
    Integer.parseInt(unNodeNoPlant.getFirstChild().getNodeValue());

// ELEMENT <description>
Node unNodeDescription = unNodeNoPlant.getNextSibling();
String description =
    unNodeDescription.getFirstChild().getNodeValue();

// ELEMENT <prixUnitaire>
Node unNodePrixUnitaire = unNodeDescription.getNextSibling();
double prixUnitaire = Double.parseDouble(
    unNodePrixUnitaire.getFirstChild().getNodeValue());

Plant unPlant = new Plant(noPlant, description, prixUnitaire);
System.out.println(noPlant + " " + description + " " + prixUnitaire);
vecteurDePlants.add(unPlant);
    }
}
}

```

Si le fichier n'est pas valide par rapport à la DTD, une exception est levée par le parseur et retournée par le programme. Bien que notre exemple n'en fasse pas la démonstration, il est possible de fournir au besoin une classe chargée de répondre aux exceptions. La structure exacte de l'arbre [DOM](#) est assez compliquée et nous ne présentons pas une description exhaustive du sujet.

Une bonne manière de se familiariser avec DOM est d'afficher la structure d'un document en utilisant le programme [DOMEcho](#). Le programme montre la structure d'arbre du DOM produit à partir d'un document XML. Le résultat partiel pour *Plants.xml* est le suivant :


```
DOC: nodeName="#document"
  DOC_TYPE: nodeName="Catalogue"
    ELEM: nodeName="Catalogue" local="Catalogue"
      ELEM: nodeName="Plant" local="Plant"
        ELEM: nodeName="noPlant" local="noPlant"
          TEXT: nodeName="#text" nodeValue="10"
        ELEM: nodeName="description" local="description"
          TEXT: nodeName="#text" nodeValue="Cèdre en boule"
        ELEM: nodeName="prixUnitaire" local="prixUnitaire"
          TEXT: nodeName="#text" nodeValue="10.99"
      ELEM: nodeName="Plant" local="Plant"
        ELEM: nodeName="noPlant" local="noPlant"
          TEXT: nodeName="#text" nodeValue="20"
        ELEM: nodeName="description" local="description"
          TEXT: nodeName="#text" nodeValue="Sapin"
        ELEM: nodeName="prixUnitaire" local="prixUnitaire"
          TEXT: nodeName="#text" nodeValue="12.99"
    ...
```

Chacun des nœuds de l'arbre selon la norme DOM est un objet de la classe **Node**. Chacun des **Node** possède des attributs tel que le type du **Node**, son nom, sa valeur, etc. La racine de l'arbre est un **Node** de type DOC nommé "#document". Dans notre exemple, ce **Node** a deux enfants de type DOC_TYPE et ELEM. Le ELEM nommé "Catalogue" correspond à l'élément `<Catalogue>` du document XML. Sous lui, on retrouve la liste des éléments `<Plant>`, etc. On peut se servir de cette information afin de déterminer la manière exacte de cheminer dans l'arbre.

Exercice. Reprendre l'exemple précédent pour un fichier xml qui contient les noms et numéros de téléphone de contacts fictifs.

9.4 Gestion de fichiers et répertoires avec java.io.File

La classe java.io.**File** permet de manipuler la hiérarchie des répertoires de fichiers. Elle permet de vérifier si un fichier ou répertoire existe ou non, de créer des fichiers ou des répertoires, de parcourir la hiérarchie, de renommer un fichier ou un répertoire, etc.

▪ Vérifier l'existence d'un fichier

Exemple. L'exemple suivant reprend l'exemple *EcrireEntierEnOctets* d'écriture d'octets dans le fichier *Octets.dat* en vérifiant d'abord si le fichier existe. Si c'est bien le cas, une confirmation est exigée de l'utilisateur avant d'écraser le contenu actuel du fichier.

```
JavaPasAPas/chapitre_8/VerifierExistenceFichier.java
import java.io.*;
import javax.swing.JOptionPane;

public class VerifierExistenceFichier {
    public static void main(String[] args) {
        File leFile = new File("Octets.dat");
        if (leFile.exists()) {
            String reponse = JOptionPane.showInputDialog(
                "Voulez-vous détruire le contenu existant (oui ou non)?");
            if (reponse.equals("non")) {
                System.out.println("Le fichier demeure tel quel");
                System.exit(0);
            }
        }
        try (FileOutputStream unFichier = new FileOutputStream(leFile)) {
            int unEntier = 1629696561; // "a#21" vu comme 4 octets
            // Convertir unEntier en un tableau de 4 octets
            byte[] tampon = new byte[4];
            for (int i = 3; i >= 0; i--) {
                // Extrait l'octet le moins significatif
                tampon[i] = (byte) (unEntier & 0xFF);
                unEntier >>= 8; // Décalage de 8 bits (remplissage à 0)
            }
            unFichier.write(tampon);
        } catch (IOException e) {
            System.err.println("Exception\n" + e.toString());
        }
        System.exit(0);
    }
}
```

Les différences à noter par rapport à l'exemple *EcrireEntierEnOctets* sont les suivantes. L'énoncé

```
File leFile =
new File("Octets.dat");
```

crée un nouvel objet *unFile* de la classe *File* qui correspond à un chemin spécifié en paramètre. La création d'un objet **File** n'ouvre pas le fichier en vue de manipuler son contenu. La classe **File** ne permet que de manipuler les informations du répertoire de fichier, et non pas le contenu des fichiers.

L'énoncé

```
if (leFile.exists()){
```

vérifie si le fichier existe.

L'énoncé

```
try (FileOutputStream unFichier = new FileOutputStream(leFile)) {
```

ouvre le fichier en écriture à partir de l'objet *leFile* de la classe *File*.

■ Créer de nouveaux dossiers

La classe **File** peut aussi être utilisée pour créer de nouvelles branches dans la hiérarchie de répertoires.

Exemple. L'exemple suivant crée de nouveaux dossiers pour obtenir un chemin spécifié (ici DossierA/DossierB, relativement au dossier courant).

```
import java.io.*;

public class CreerRepertoire {
    public static void main(String[] args) throws Exception {
        File unFile = new File("DossierA/DossierB");
        unFile.mkdirs();
        if (unFile.exists()) {
            System.out.println("Il a été créé");
        } else {
            System.out.println("Il n'a pas été créé");
        }
    }
}
```

La ligne

```
File unFile = new File("DossierA/DossierB");
```

spécifie le chemin désiré.

La méthode **mkdirs()** de la classe *File* est utilisé pour créer les dossiers nécessaires :

```
unFile.mkdirs();
```

Tous les dossiers manquants du chemin spécifié sont créés.

9.4.1 Dialogue de sélection de fichier avec la classe **JFileChooser**

Souvent, l'utilisateur doit choisir un fichier de manière interactive en naviguant dans un répertoire. L'outil **JFileChooser** est une classe GUI dédiée à ce traitement.

Exemple. L'exemple suivant reprend l'exemple *EcrireEntierEnOctets* d'écriture d'octets dans un fichier. Cependant, ici, un **JFileChooser** permet à l'utilisateur de spécifier le fichier à créer.

```
JavaPasApas/chapitre_8/CreerFichierFileChooser.java
import java.io.*;
import javax.swing.*;

public class CreerFichierFileChooser extends JFrame {
    public CreerFichierFileChooser() throws Exception {
        JFileChooser unFileChooser = new JFileChooser();
        unFileChooser.setSelectionMode(JFileChooser.FILES_ONLY);
        int resultat = unFileChooser.showSaveDialog(this);
        if (resultat != JFileChooser.CANCEL_OPTION) {
            File leFile = unFileChooser.getSelectedFile();
            if (leFile != null && !(leFile.getName().equals("")) {
                try (FileOutputStream unFichier = new FileOutputStream(leFile)) {
                    int unEntier = 1629696561; // "a#21" vu comme 4 octets
                    // Convertir unEntier en un tableau de 4 octets
                    byte[] tampon = new byte[4];
                    for (int i = 3; i >= 0; i--) {
                        // Extrait l'octet le moins significatif
                        tampon[i] = (byte) (unEntier & 0xFF);
                        unEntier >>= 8; // Décalage de 8 bits (remplissage à 0)
                    }
                    unFichier.write(tampon);
                }
            } else {
                System.out.println("Nom de fichier invalide");
            }
        } else {
            System.out.println("Fichier non choisi");
        }
        System.exit(0);
    }

    public static void main(String[] args) throws Exception {
        new CreerFichierFileChooser();
    }
}
```

La ligne

```
JFileChooser unFileChooser = new JFileChooser();
```

crée un **JFileChooser**.

La ligne,

```
unFileChooser.setFileSelectionMode(JFileChooser.FILES_ONLY);
```

restreint le choix des éléments qu'il est possible de sélectionner. Il sera par la suite possible de sélectionner uniquement un fichier et pas un répertoire (dossier).

La ligne

```
int resultat = unFileChooser.showSaveDialog(this);
```

affiche le dialogue et permet à l'utilisateur de sélectionner un fichier. La sélection est confirmée en cliquant le bouton *Save*. Le paramètre *this* est la composante parente. Ici, la classe parente est *CreerFichierFileChooser* qui est un *Jframe*. Le parent doit être un *JComponent*.

L'entier retourné permet de vérifier si le choix a bien été effectué. La ligne suivante

```
if (resultat != JFileChooser.CANCEL_OPTION){
```

vérifie que le bouton *Cancel* n'a pas été cliqué.

La ligne suivante extrait l'objet *leFile* de la classe *File* qui représente le fichier choisi :

```
File leFile = unFileChooser.getSelectedFile();
```

Ensuite, la ligne suivante vérifie que la sélection n'est pas nulle ou la chaîne vide :

```
if (leFile != null && !(leFile.getName().equals(""))){
```

Enfin la ligne suivante crée un *FileOutputStream* à partir du *File*.

```
try (FileOutputStream unFichier = new FileOutputStream(leFile)) {
```

Exemple. L'exemple suivant reprend l'exemple *LireEntierEnOctets* de lecture d'octets d'un fichier mais en permettant à l'utilisateur de sélectionner le fichier à lire à l'aide d'un dialogue **JFileChooser**.

```
JavaPasAPas/chapitre_8/LireFichierFileChooser.java
```

```
import java.io.*;
import javax.swing.*;

public class LireFichierFileChooser extends JFrame {
    public LireFichierFileChooser() throws Exception {
        JFileChooser unFileChooser = new JFileChooser();
        unFileChooser.setFileSelectionMode(JFileChooser.FILES_ONLY);
        int resultat = unFileChooser.showOpenDialog(this);
        if (resultat != JFileChooser.CANCEL_OPTION) {
            File leFile = unFileChooser.getSelectedFile();
            if (leFile != null && !(leFile.getName().equals("")) {
                try (FileInputStream unFichier = new FileInputStream(leFile)) {
                    byte[] tampon = new byte[4];
                    unFichier.read(tampon); // Lecture des 4 octets
```

```

        // Convertir le tableau d'octets tampon en int unEntier
        int unEntier = 0;
        for (int i = 0; i <= 3; i++) {
            unEntier <<= 8;
            unEntier += ((int) tampon[i]) & 0xFF;
        }
        System.out.println("Valeur décimale de l'entier : " + unEntier);
    }
} else {
    System.out.println("Nom de fichier invalide");
}
} else {
    System.out.println("Fichier non choisi");
}
}
System.exit(0);
}

public static void main(String[] args) throws Exception {
    new LireFichierFileChooser();
}
}

```

La différence à noter en comparaison avec l'exemple d'écriture est l'emploi de la méthode `showOpenDialog()` plutôt que `showSaveDialog()` :

```
int resultat = unFileChooser.showOpenDialog(this);
```

9.5 Fichier d'objets en Java

Dans l'exemple de lecture des données du catalogue de plants dans un vecteur, les données ne sont pas conservées après la fin du programme. Souvent, on voudra stocker ce genre de données en mémoire secondaire pour qu'elles soient conservées à long terme au-delà de l'exécution des programmes qui les manipulent. Cette section montre comment stocker des objets dans un fichier de manière à pouvoir les récupérer sans avoir à les convertir explicitement sous forme d'une série d'octets en employant des classes prévues à cet effet.

9.5.1 Fichier sériel d'objets en Java

Voyons d'abord comment créer un fichier qui contient une suite d'objets en Java avec la classe `ObjectOutputStream`. Ce genre de fichier est souvent appelé fichier sériel ou séquentiel. La méthode `writeObject(Object obj)` de la classe `ObjectOutputStream` permet d'écrire directement un objet et symétriquement la méthode `readObject()` de la classe `ObjectInputStream` permet de lire cet objet par la suite. Java n'impose pas de structure comme tel au fichier au sens où c'est le programme qui

détermine sa structure par les écritures qu'il effectue. Il est ainsi possible d'écrire n'importe quelle suite d'objets provenant possiblement de classes différentes dans le même fichier. Pour retrouver ces objets correctement, ils doivent être lus dans le même ordre. Le programme d'application doit donc s'assurer que les données sont écrites et lues selon une discipline cohérente.

Pour illustrer le concept de fichier sériel, nous allons dans un premier temps créer un fichier qui contient les objets du catalogue de plants sous forme d'une suite d'objets. Comme point de départ, nous supposons que les données se trouvent dans un vecteur d'objets de la classe *Plant* tel que produit par l'exemple de programme vu précédemment *ExempleStreamTokenizer*. La classe **ObjectOutputStream** est utilisée pour écrire directement un objet dans le fichier sans avoir à se préoccuper de la conversion de l'objet en une suite d'octets. La classe **ObjectOutputStream** se sert du mécanisme de *sérialisation* de Java qui permet de convertir un objet sous forme d'une suite d'octets.

Exemple. Le programme *EcrireFluxPlants* lit les données de *Plants.txt* les convertit en un vecteur d'objets et écrits les objets du vecteur les uns à la suite des autres dans le fichier *FluxPlants.dat*.

Dans le code suivant la méthode *lirePlantsFichierTexte()* reprend essentiellement le code de *ExempleStreamTokenizer*. Cette méthode lit le fichier texte *Plants.txt* et retourne un vecteur d'objets de la classe *Plant*, *vecteurDePlants*. La méthode *écrireFichierFluxPlants* écrit les objets de *vecteurDePlants* les uns à la suite des autres dans le fichier *FluxPlants.dat*.

```
JavaPasAPas/chapitre_8/EcrireFluxPlants.java
/* Illustration de la création d'un fichier d'objets sériel
 * Lit le fichier Plants.txt, stocke le contenu dans un vecteur (ArrayList)
 * d'objets Plant et crée ensuite le fichier d'objets FluxPlants.dat par
 * accès sériel */

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.*;

public class EcrireFluxPlants {

    // La méthode lit les données de Plants.txt et les retourne dans un
    // vecteur (ArrayList) d'objets de la classe Plant
    // Reprend essentiellement le code de ExempleStreamTokenizer
    public static ArrayList<Plant> lirePlantsFichierTexte()
        throws Exception {
        ArrayList<Plant> vecteurDePlants = new ArrayList<Plant>();
```

```

try (FileReader unFichier =
    new FileReader("Plants.txt", StandardCharsets.UTF_8)) {
    StreamTokenizer unStreamTokenizer = new StreamTokenizer(unFichier);
    // Les 5 lignes suivantes ne sont pas nécessaires car les paramètres
    // donnés sont les valeurs de défaut
    unStreamTokenizer.quoteChar((int) '"');
    unStreamTokenizer.whitespaceChars((int) '\r', (int) '\r');
    unStreamTokenizer.whitespaceChars((int) '\n', (int) '\n');
    unStreamTokenizer.whitespaceChars((int) '\t', (int) '\t');
    unStreamTokenizer.whitespaceChars((int) ' ', (int) ' ');

    int noPlant = 0;
    String description = "";
    double prixUnitaire = 0.0;

    // Jusqu'à la fin du fichier
    while (unStreamTokenizer.nextToken() != StreamTokenizer.TT_EOF) {
        // Lecture du noPlant : est-ce bien un nombre ?
        if (unStreamTokenizer.ttype == StreamTokenizer.TT_NUMBER) {
            noPlant = (int) unStreamTokenizer.nval; // nval est un double !
        } else {
            System.out.println("Format incorrect : noPlant attendu");
            System.exit(1);
        }
        // Lecture de la description : est-ce bien une chaîne entre " ?
        unStreamTokenizer.nextToken();
        if (unStreamTokenizer.ttype == (int) '"') {
            description = unStreamTokenizer.sval;
        } else {
            System.out.println("Format incorrect : description attendue");
            System.exit(1);
        }
        // Lecture du prixUnitaire : est-ce bien un nombre ?
        unStreamTokenizer.nextToken();
        if (unStreamTokenizer.ttype == StreamTokenizer.TT_NUMBER) {
            prixUnitaire = unStreamTokenizer.nval;
        } else {
            System.out.println("Format incorrect : prix attendu");
            System.exit(1);
        }

        // Création de l'objet Plant
        Plant unPlant = new Plant(noPlant, description, prixUnitaire);
        System.out.println(
            noPlant + " " + description + " " + prixUnitaire);
        vecteurDePlants.add(unPlant);
    }
}
return vecteurDePlants;
}

// La méthode suivante écrit les objets de vecteurDePlants les uns à la
// suite des autres dans le fichier FluxPlants.dat par accès sériel
public static void ecrireFichierFluxPlants(
    ArrayList<Plant> vecteurDePlants) throws Exception {
    try (ObjectOutputStream fichierFluxPlants =

```



```

        new ObjectOutputStream(new FileOutputStream("FluxPlants.dat")) {
        for (Plant unPlant : vecteurDePlants) {
            // writeObject ajoute le nouvel objet à la fin du fichier
            fichierFluxPlants.writeObject(unPlant);
        }
    }

    public static void main(String[] args) throws Exception {
        ArrayList<Plant> vecteurDePlants = lirePlantsFichierTexte();
        ecrireFichierFluxPlants(vecteurDePlants);
    }
}

```

Pour ajouter un objet de la classe *Plant* dans le fichier, il suffit d'appeler [writeObject\(Object obj\)](#) qui fait tout le travail de conversion de l'objet sous forme d'une suite d'octets et qui ajoute les octets à la fin du fichier. Pour pouvoir écrire un objet de la classe *Plant* avec la méthode [writeObject\(Object obj\)](#), il faut que *Plant* implémente l'interface *Serializable*. Dans notre exemple, il n'y a qu'à s'assurer de la présence de la clause *implements Serializable* tel qu'illustré par le code suivant :

```

import java.io.Serializable;

public class Plant implements Serializable {
    private int noPlant; // numéro de catalogue du plant
    private String description; // description du plant
    private double prixUnitaire; // prix unitaire du plant

    public Plant(int noPlant, String description, double prixUnitaire) {
        this.noPlant = noPlant;
        this.description = description;
        this.prixUnitaire = prixUnitaire;
    }

    public void setNoPlant(int noPlant) {
        this.noPlant = noPlant;
    }

    public int getNoPlant() {
        return noPlant;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public String getDescription() {
        return description;
    }
}

```

```
public void setPrixUnitaire(double prixUnitaire) {  
    this.prixUnitaire = prixUnitaire;  
}  
  
public double getPrixUnitaire() {  
    return prixUnitaire;  
}  
}
```

Sérialisation (*serialization*) Java

Le mécanisme de *sérialisation* Java effectue la conversion d'un objet en une suite d'octets. Le mécanisme inverse est appelé *désérialisation*. Le mécanisme de sérialisation Java est très puissant et il permet même de sérialiser un objet complexe qui fait référence à d'autres objets en incluant les objets référencés dans la sérialisation. On pourrait d'ailleurs écrire tout le vecteur d'objets *Plant* d'un coup en une seule écriture.

Le mécanisme de sérialisation n'est pas le plus économique d'un point de vue de l'occupation d'espace. La suite d'octets produite contient non seulement les données, c'est-à-dire les valeurs des attributs mais aussi des *métadonnées* qui décrivent l'objet, c'est-à-dire le nom de la classe incluant son package, le nom et le type des attributs, etc. Par exemple, un objet sérialisé de la classe *Plant* occupe environ 120 octets. En écrivant uniquement les données, comme nous le verrons à la section suivante, une cinquantaine d'octets par objet serait suffisant. Les métadonnées ne sont pas nécessaires lorsque le programme connaît le contenu du fichier à priori. Cependant, l'ajout de ces informations assure un plus grand niveau de fiabilité car une exception est levée dans le cas où l'objet lu n'est pas de la classe attendue.

Allocation automatique d'espace au fichier

Lors de l'écriture d'un objet, il est ajouté à la fin du fichier et l'espace nécessaire est automatiquement alloué au besoin. Le programme n'a donc pas à se préoccuper de l'allocation d'espace. Évidemment, ce manque de contrôle au niveau de la stratégie d'allocation d'espace peut conduire à une fragmentation importante du fichier.

Exemple. Le programme *LireFluxPlants* lit les objets un par un en partant du premier jusqu'à ce que la fin du fichier soit atteinte.

```
JavaPasAPas/chapitre_8/LireFluxPlants.java
/* Illustration de la lecture d'un fichier d'objets par itération sérielle
 * Lit le fichier FluxPlants.dat et en affiche le contenu */

import java.io.*;

public class LireFluxPlants {

    public static void main(String[] args) throws Exception {
        try (ObjectInputStream fichierFluxPlants =
            new ObjectInputStream(new FileInputStream("FluxPlants.dat"))) {
            while (true) {
                Plant unPlant;
                try { // Lecture de l'objet suivant
                    unPlant = (Plant) fichierFluxPlants.readObject();
                } catch (EOFException e) {
                    break; // Fin du fichier
                }
                System.out.println(unPlant.getNoPlant() + " "
                    + unPlant.getDescription() + " " + unPlant.getPrixUnitaire());
            }
        }
    }
}
```

L'appel à *readObject()* retourne le prochain objet lu relativement à la position courante dans le fichier par le mécanisme de désérialisation. Lors de l'ouverture du fichier, la position courante est le début du fichier. Une exception est levée lorsque la fin du fichier est atteinte.

Cette manière d'organiser un fichier est très restrictive. En particulier, supposons que l'on veuille simplement modifier une donnée, par exemple, le prix d'un plant. La seule façon de procéder consiste à lire tous les objets et à les réécrire dans un nouveau fichier en modifiant le prix au passage. Ceci serait très inefficace en particulier dans le cas d'un gros volume de données.

S'il est important de pouvoir accéder sélectivement à un ou quelques enregistrements d'un fichier, une organisation à accès direct peut être utilisée. La classe *RandomAccessFile* offre cette possibilité.

- Écriture d'un objet complexe

Tel que mentionné précédemment, il est possible de sérialiser un objet complexe qui fait référence à d'autres objets. Dans l'exemple précédent

l'objet *vecteurDePlants* est un exemple d'objet complexe. C'est un vecteur qui contient des références aux objets de la classe *Plant*. Plutôt que d'écrire les objets du vecteur un à un dans le fichier, il est possible d'écrire tout le vecteur d'un coup.

Exemple. Le programme suivant écrit l'objet complexe *vecteurDePlants* dans le fichier *VecteurPlants.dat*.

```
JavaPasAPas/chapitre_8/EcrireVecteurPlants.java
/* Illustration de l'écriture d'un objet complexe dans un fichier par
 * sérialisation
 * Lit le fichier Plants.txt, stocke le contenu dans un vecteur (ArrayList)
 * d'objets Plant et écrit ensuite le vecteur dans le fichier
 * VecteurPlants.dat */

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.*;

public class EcrireVecteurPlants {

    // La méthode lit les données de Plants.txt et les retourne dans un
    // vecteur (ArrayList) d'objets de la classe Plant
    // Reprend essentiellement le code de ExempleStreamTokenizer
    public static ArrayList<Plant> lirePlantsFichierTexte()
        throws Exception {
        ArrayList<Plant> vecteurDePlants = new ArrayList<Plant>();
        try (FileReader unFichier =
            new FileReader("Plants.txt", StandardCharsets.UTF_8)) {
            StreamTokenizer unStreamTokenizer = new StreamTokenizer(unFichier);
            // Les 5 lignes suivantes ne sont pas nécessaires car les paramètres
            // donnés sont les valeurs de défaut
            unStreamTokenizer.quoteChar((int) '"');
            unStreamTokenizer.whitespaceChars((int) '\r', (int) '\r');
            unStreamTokenizer.whitespaceChars((int) '\n', (int) '\n');
            unStreamTokenizer.whitespaceChars((int) '\t', (int) '\t');
            unStreamTokenizer.whitespaceChars((int) ' ', (int) ' ');

            int noPlant = 0;
            String description = "";
            double prixUnitaire = 0.0;

            // Jusqu'à la fin du fichier
            while (unStreamTokenizer.nextToken() != StreamTokenizer.TT_EOF) {
                // Lecture du noPlant : est-ce bien un nombre ?
                if (unStreamTokenizer.ttype == StreamTokenizer.TT_NUMBER) {
                    noPlant = (int) unStreamTokenizer.nval; // nval est un double !
                } else {
                    System.out.println("Format incorrect : noPlant attendu");
                    System.exit(1);
                }
                // Lecture de la description : est-ce bien une chaîne entre " ?
                unStreamTokenizer.nextToken();
```

```

        if (unStreamTokenizer.ttype == (int) '"') {
            description = unStreamTokenizer.sval;
        } else {
            System.out.println("Format incorrect : description attendue");
            System.exit(1);
        }
        // Lecture du prixUnitaire : est-ce bien un nombre ?
        unStreamTokenizer.nextToken();
        if (unStreamTokenizer.ttype == StreamTokenizer.TT_NUMBER) {
            prixUnitaire = unStreamTokenizer.nval;
        } else {
            System.out.println("Format incorrect : prix attendu");
            System.exit(1);
        }

        // Création de l'objet Plant
        Plant unPlant = new Plant(noPlant, description, prixUnitaire);
        System.out.println(
            noPlant + " " + description + " " + prixUnitaire);
        vecteurDePlants.add(unPlant);
    }
}
return vecteurDePlants;
}

// La méthode suivante écrit tout le vecteur d'un coup dans le fichier
// VecteurPlants.dat
public static void ecrireFichierVecteurPlants(
    ArrayList<Plant> vecteurDePlants) throws Exception {
    try (ObjectOutputStream fichierFluxPlants = new ObjectOutputStream(
        new FileOutputStream("VecteurPlants.dat"))) {
        fichierFluxPlants.writeObject(vecteurDePlants);
    }
}

public static void main(String[] args) throws Exception {
    ArrayList<Plant> vecteurDePlants = lirePlantsFichierTexte();
    ecrireFichierVecteurPlants(vecteurDePlants);
}
}

```

L'énoncé

```
fichierFluxPlants.writeObject(vecteurDePlants);
```

écrit tout le vecteur d'un coup. Comme le vecteur contient des objets de la classe *Plant*, ces objets sont sérialisés automatiquement.

Inversement, on peut lire tout le vecteur d'un coup.

Exemple. Le programme suivant lit l'objet complexe `ArrayList` contenant des objets de la classe *Plant* par désérialisation.

```

JavaPasAPas/chapitre_8/LireVecteurPlants.java
/* Illustration de la lecture d'un objet complexe par désérialisation
 * Lit un vecteur (ArrayList) de plants de VecteurPlants.dat et en affiche
 * le contenu */

import java.io.*;
import java.util.*;

public class LireVecteurPlants {

    // La conversion de type de readObject() ne peut être vérifiée
    @SuppressWarnings("unchecked")
    public static void main(String[] args) throws Exception {
        ArrayList<Plant> vecteurDePlants;
        try (ObjectInputStream fichierFluxPlants = new ObjectInputStream(
            new FileInputStream("VecteurPlants.dat"))) {
            vecteurDePlants = (ArrayList<Plant>) fichierFluxPlants.readObject();
        }
        for (Plant unPlant : vecteurDePlants) {
            System.out.println(unPlant.getNoPlant() + " "
                + unPlant.getDescription() + " " + unPlant.getPrixUnitaire());
        }
    }
}

```

L'instruction suivante lit le vecteur dans une seule invocation *readObject()* :

```
vecteurDePlants = (ArrayList<Plant>) fichierFluxPlants.readObject();
```

La sérialisation du vecteur de plants va occuper plus d'espace que la séquence des objets *Plant* sérialisés. En effet, la structure de données utilisées pour le vecteur lui-même doit aussi être sérialisée.

9.5.2 Fichier à adressage relatif en Java avec `RandomAccessFile`

Cette section présente un exemple d'implémentation d'un fichier à adressage relatif en Java en passant par la classe `RandomAccessFile`. Ceci permet de manipuler les données d'une manière plus performante en accédant directement aux données sans devoir passer par le traitement de tous les objets du fichier comme c'est le cas du fichier sérial. Comme pour le cas précédent, les données sont disposées dans le fichier les unes à la suite des autres. Les données d'un objet sont représentées par un ensemble d'octets appelé un enregistrement (*record*). Les données sont écrites et lues sans passer par la sérialisation. Il est possible d'accéder directement aux données d'un objet particulier soit pour le lire ou soit pour l'écrire. A cet effet, un numéro d'enregistrement relatif (NER) est assigné à chacun des enregistrements du fichier en fonction de sa position.

Un nouvel enregistrement est toujours ajouté à la fin du fichier. Pour simplifier, la suppression d'un enregistrement n'est pas permise. Tel qu'illustré à la figure suivante, le nombre courant d'enregistrements est stocké sous forme d'un *int* dans les quatre premiers octets du fichier. Lors de la création d'un nouvel enregistrement, cette information sert à déterminer le NER du nouvel enregistrement.

Dans notre figure, nous supposons que la taille d'un enregistrement est fixée à 50. L'emploi d'une taille fixe permet de retrouver facilement la position d'un enregistrement dans le fichier. Comme les octets sont numérotés à partir de 0, la position du premier octet d'un enregistrement d'adresse NER est :

$$\text{Position du premier octet} = \text{NER} * 50 + 4$$

Dans notre implémentation, une méthode statique *tailleMaxEnregistrement()* de la classe *Plant* retourne la taille maximale d'un enregistrement. De fait, tout l'espace est réservé pour l'enregistrement même si en réalité, sa taille peut être inférieure. Dans ce cas, les derniers octets sont inutilisés mais tout de même alloués.

La méthode *ecrireEnregistrementTailleMax* est ajoutée à *Plant* afin d'écrire la suite d'octets de l'objet *Plant* dans le fichier. Contrairement à l'implémentation de fichier à accès sériel de la section précédente, la sérialisation n'est pas utilisée. Ceci permet d'avoir un contrôle précis sur les octets écrits et de réduire l'espace occupé par l'enregistrement correspondant à un objet parce qu'il n'y a pas de métadonnées stockées avec les données.

Exemple. Classe *Plant* modifiée.

```
JavaPasAPas/chapitre_8/Plant.java
import java.io.*;
import java.nio.charset.StandardCharsets;

public class Plant implements Serializable {
    private int noPlant; // numéro de catalogue du plant
    private String description; // description du plant
    private double prixUnitaire; // prix unitaire du plant

    public Plant(int noPlant, String description, double prixUnitaire) {
        this.noPlant = noPlant;
        this.description = description;
        this.prixUnitaire = prixUnitaire;
    }
}
```

```

public void setNoPlant(int noPlant) {
    this.noPlant = noPlant;
}

public int getNoPlant() {
    return noPlant;
}

public void setDescription(String description) {
    this.description = description;
}

public String getDescription() {
    return description;
}

public void setPrixUnitaire(double prixUnitaire) {
    this.prixUnitaire = prixUnitaire;
}

public double getPrixUnitaire() {
    return prixUnitaire;
}

public void ecrireEnregistrementTailleMax(RandomAccessFile unFichier)
    throws Exception {
    // 4 + 4 + 8 octets pour noPlant, la taille et prixUnitaire
    int tailleZoneDescription = tailleMaxEnregistrement() - 16;
    // Un octet par caractère (ISO-8859-1, qui comprend les accents)
    byte[] octets = description.getBytes(StandardCharsets.ISO_8859_1);
    if (octets.length > tailleZoneDescription) {
        System.exit(1);
    }
    unFichier.writeInt(noPlant); // 4 octets
    unFichier.writeInt(octets.length); // 4 octets
    unFichier.write(octets);
    // Compléter la zone description à 34 octets
    unFichier.write(new byte[tailleZoneDescription - octets.length]);
    unFichier.writeDouble(prixUnitaire); // 8 octets
}

public void lireEnregistrementTailleMax(RandomAccessFile unFichier)
    throws Exception {
    int tailleZoneDescription = tailleMaxEnregistrement() - 16;
    noPlant = unFichier.readInt();
    int tailleDescription = unFichier.readInt();
    byte[] tampon = new byte[tailleDescription];
    unFichier.readFully(tampon);
    description = new String(tampon, StandardCharsets.ISO_8859_1);
    unFichier.skipBytes(tailleZoneDescription - tailleDescription);
    prixUnitaire = unFichier.readDouble();
}

public static int tailleMaxEnregistrement() {
    return 50;
}

```



```
}  
}
```

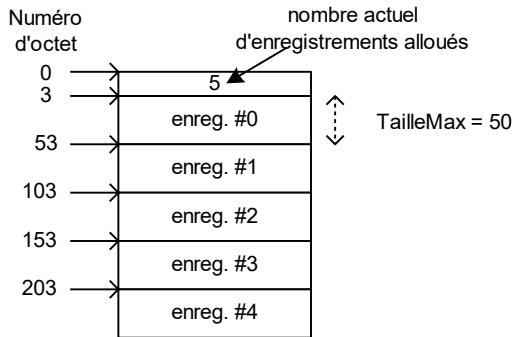


Figure 41. Organisation du fichier à adressage relatif.

Pour simplifier l'exemple, les seules opérations permises sont :

- Créer un nouvel enregistrement.
- Sélectionner un enregistrement à partir du NER.
- Modifier le prix d'un enregistrement sélectionné à partir du NER.

Exemple. Le programme suivant illustre l'accès direct et l'adressage relatif.

```
JavaPasAPas/chapitre_8/AccesDirect.java  
/* Illustration de l'accès direct avec un fichier à adressage relatif  
 * Opérations permises :  
 *   sélectionner un enregistrement par son NER  
 *   modifier le prix d'un enregistrement sélectionné par son NER  
 *   créer un nouvel enregistrement (toujours à la fin)  
 *   (ne permet pas la suppression)  
 */  
  
import java.io.*;  
import javax.swing.JOptionPane;  
  
public class AccesDirect {  
  
    // Affiche le contenu d'un enregistrement  
    public static void afficher(int numeroER, Plant unPlant) {  
        JOptionPane.showMessageDialog(null,  
            "NER :" + numeroER  
            + "\nnoPlant :" + unPlant.getNoPlant()  
        );  
    }  
}
```

```

        + "\ndescription :" + unPlant.getDescription()
        + "\nprixUnitaire :" + unPlant.getPrixUnitaire());
    }

    public static void main(String[] args) throws Exception {
        File leFichier = new File("DirectPlants.dat");
        boolean existe = leFichier.exists();
        // Ouverture du fichier ou création s'il n'existe pas
        // Le fichier est fermé automatiquement à la fin du bloc try
        try (RandomAccessFile fichierDirectPlants =
            new RandomAccessFile(leFichier, "rw")) {
            int nombreAlloue; // nombre d'enregistrements actuellement alloués
            if (existe) {
                // Cherche le nombre d'enregistrements actuellement alloués
                nombreAlloue = fichierDirectPlants.readInt();
            } else { // Le fichier n'existe pas, il faut initialiser nombreAlloue
                nombreAlloue = 0;
                fichierDirectPlants.writeInt(nombreAlloue);
            }

            String chaineNER;
            int numeroER;
            Plant unPlant = new Plant(0, "", 0.0);
            boolean continuer = true;

            while (continuer) {
                String chaineChoix = JOptionPane.showInputDialog(
                    "Menu: 1(lire); 2(modifier prix); 3(ajouter) ; 0 (terminer)");
                int choix = Integer.parseInt(chaineChoix);

                switch (choix) {
                    case 1: // Lire et afficher l'enregistrement
                        chaineNER = JOptionPane.showInputDialog(
                            "Entrez le numéro d'enregistrement relatif :");
                        numeroER = Integer.parseInt(chaineNER);
                        if (numeroER >= 0 && numeroER < nombreAlloue) {
                            // Sélectionner un enregistrement par son NER
                            fichierDirectPlants.seek(
                                numeroER * Plant.tailleMaxEnregistrement() + 4);
                            unPlant.lireEnregistrementTailleMax(fichierDirectPlants);
                            afficher(numeroER, unPlant);
                        } else {
                            JOptionPane.showMessageDialog(
                                null, "Numéro incorrect :" + numeroER);
                        }
                        break;

                    case 2: // Modifier un enregistrement
                        chaineNER = JOptionPane.showInputDialog(
                            "Entrez le numéro d'enregistrement relatif :");
                        numeroER = Integer.parseInt(chaineNER);
                        if (numeroER >= 0 && numeroER < nombreAlloue) {
                            // D'abord sélectionner l'enregistrement par son NER
                            fichierDirectPlants.seek(
                                numeroER * Plant.tailleMaxEnregistrement() + 4);
                            unPlant.lireEnregistrementTailleMax(fichierDirectPlants);

```

```

        // Modifier son prix en mémoire centrale
        String chainePrix =
            JOptionPane.showInputDialog("Entrez le nouveau prix :");
        unPlant.setPrixUnitaire(Double.parseDouble(chainePrix));

        // Écrire l'enregistrement modifié
        fichierDirectPlants.seek(
            numeroER * Plant.tailleMaxEnregistrement() + 4);
        unPlant.ecrireEnregistrementTailleMax(fichierDirectPlants);
        afficher(numeroER, unPlant);
    } else {
        JOptionPane.showMessageDialog(
            null, "Numéro incorrect :" + numeroER);
    }
    break;

case 3: // Créer un enregistrement
    String chaineNoPlant =
        JOptionPane.showInputDialog("Entrez le noPlant :");
    unPlant.setNoPlant(Integer.parseInt(chaineNoPlant));
    unPlant.setDescription(
        JOptionPane.showInputDialog("Entrez la description :"));
    String chainePrix =
        JOptionPane.showInputDialog("Entrez le prixUnitaire :");
    unPlant.setPrixUnitaire(Double.parseDouble(chainePrix));

    // Allocation sérielle : NER du nouvel enregistrement =
    // nombreAlloue
    fichierDirectPlants.seek(
        nombreAlloue * Plant.tailleMaxEnregistrement() + 4);
    unPlant.ecrireEnregistrementTailleMax(fichierDirectPlants);
    afficher(nombreAlloue, unPlant);

    // Incrémenter le nombre d'enregistrements alloués
    nombreAlloue++;
    fichierDirectPlants.seek(0);
    fichierDirectPlants.writeInt(nombreAlloue);
    break;

case 0: // Terminer
    continuer = false;
    break;

default:
    JOptionPane.showMessageDialog(
        null, "Choix incorrect :" + choix);
    }
    }
    }
    }
    System.exit(0);
}
}

```

La méthode `seek()` de [RandomAccessFile](#) fixe la position courante dans le fichier à un numéro d'octet particulier. Ainsi, l'instruction

```
fichierDirectPlants.seek(numeroER*Plant.tailleMaxEnregistrement()+4);
```

établit la position courante au début de l'enregistrement *numeroER*. Ensuite l'instruction

```
unPlant.lireEnregistrementTailleMax(fichierDirectPlants);
```

lit l'enregistrement à cette position.

L'instruction

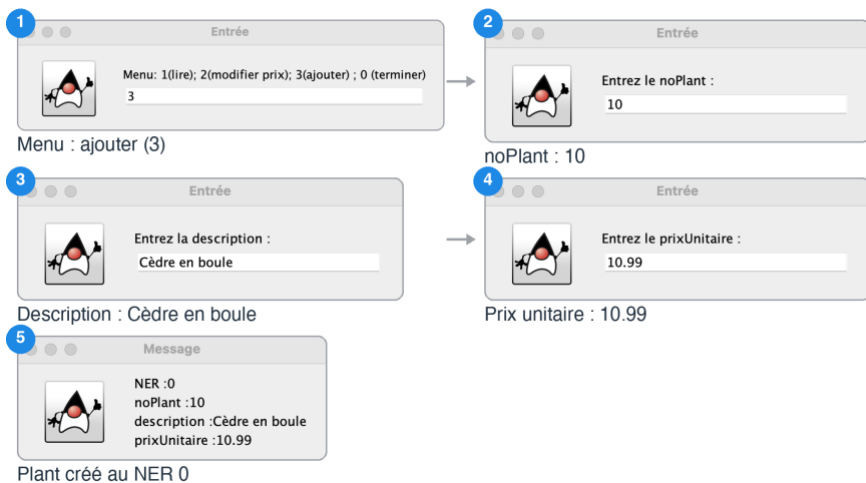
```
unPlant.ecrireEnregistrementTailleMax(fichierDirectPlants);
```

écrit un enregistrement à cette position. Ceci remplace effectivement le contenu précédant par le contenu de l'objet *Plant*.

L'espace est alloué automatiquement lors de l'écriture si nécessaire. Par contre, une tentative de lecture au-delà du dernier enregistrement écrit provoque une exception.

Exemple. Les deux figures suivantes illustrent l'exécution du programme : on ajoute deux plants, on consulte le premier, puis on modifie son prix. Les dialogues sont numérotés dans l'ordre où ils apparaissent.

Ajouter un premier plant



Ajouter un second plant

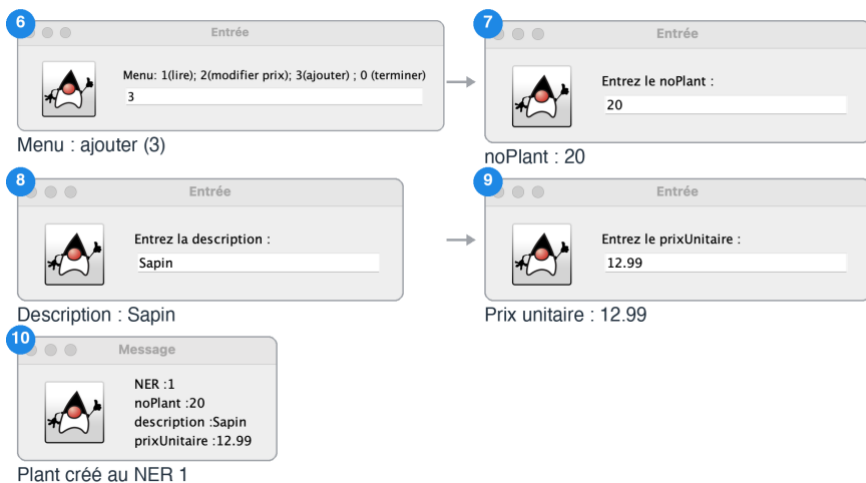
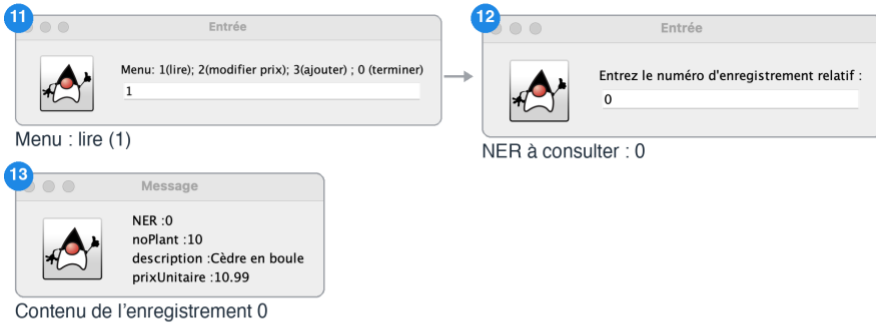


Figure 42. Scénario d'exécution d'AccesDirect : ajout de deux plants.

Consulter l'enregistrement 0



Modifier le prix de l'enregistrement 0

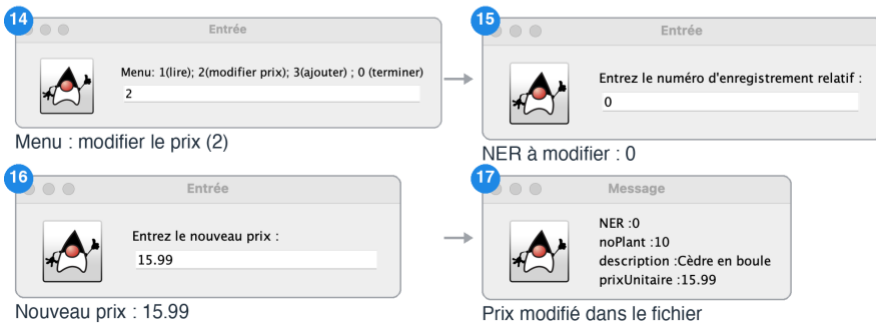


Figure 43. Scénario d'exécution d'AccesDirect : consultation et modification du prix.

Exercice. Charger le fichier à adressage relatif *DirectPlants.dat* avec les données du catalogue de plants saisies du fichier *Plants.txt*.

```
JavaPasAPas/chapitre_8/CreerFichierDirect.java
/* Création d'un fichier à adressage relatif
 * Lit le fichier Plants.txt, stocke le contenu dans un vecteur (ArrayList)
 * d'objets Plant et crée ensuite le fichier à accès direct
 * DirectPlants.dat */

import java.io.*;
import java.nio.charset.StandardCharsets;
import java.util.*;

public class CreerFichierDirect {

    // La méthode lit les données de Plants.txt et les retourne dans un
    // vecteur (ArrayList) d'objets de la classe Plant
    // Reprend essentiellement le code de ExempleStreamTokenizer
    public static ArrayList<Plant> lirePlantsFichierTexte()
        throws Exception {
        ArrayList<Plant> vecteurDePlants = new ArrayList<Plant>();
```

```

try (FileReader unFichier =
    new FileReader("Plants.txt", StandardCharsets.UTF_8)) {
    StreamTokenizer unStreamTokenizer = new StreamTokenizer(unFichier);
    // Les 5 lignes suivantes ne sont pas nécessaires car les paramètres
    // donnés sont les valeurs de défaut
    unStreamTokenizer.quoteChar((int) '"');
    unStreamTokenizer.whitespaceChars((int) '\r', (int) '\r');
    unStreamTokenizer.whitespaceChars((int) '\n', (int) '\n');
    unStreamTokenizer.whitespaceChars((int) '\t', (int) '\t');
    unStreamTokenizer.whitespaceChars((int) ' ', (int) ' ');

    int noPlant = 0;
    String description = "";
    double prixUnitaire = 0.0;

    // Jusqu'à la fin du fichier
    while (unStreamTokenizer.nextToken() != StreamTokenizer.TT_EOF) {
        // Lecture du noPlant : est-ce bien un nombre ?
        if (unStreamTokenizer.ttype == StreamTokenizer.TT_NUMBER) {
            noPlant = (int) unStreamTokenizer.nval; // nval est un double !
        } else {
            System.out.println("Format incorrect : noPlant attendu");
            System.exit(1);
        }
        // Lecture de la description : est-ce bien une chaîne entre " ?
        unStreamTokenizer.nextToken();
        if (unStreamTokenizer.ttype == (int) '"') {
            description = unStreamTokenizer.sval;
        } else {
            System.out.println("Format incorrect : description attendue");
            System.exit(1);
        }
        // Lecture du prixUnitaire : est-ce bien un nombre ?
        unStreamTokenizer.nextToken();
        if (unStreamTokenizer.ttype == StreamTokenizer.TT_NUMBER) {
            prixUnitaire = unStreamTokenizer.nval;
        } else {
            System.out.println("Format incorrect : prix attendu");
            System.exit(1);
        }

        // Création de l'objet Plant
        Plant unPlant = new Plant(noPlant, description, prixUnitaire);
        System.out.println(
            noPlant + " " + description + " " + prixUnitaire);
        vecteurDePlants.add(unPlant);
    }
}
return vecteurDePlants;
}

// La méthode suivante écrit les objets de vecteurDePlants dans le
// fichier à adressage relatif DirectPlants.dat
public static void ecrireFichierDirectPlants(
    ArrayList<Plant> vecteurDePlants) throws Exception {
    try (RandomAccessFile fichierDirectPlants =

```

```

    new RandomAccessFile("DirectPlants.dat", "rw")) {
int numeroEnregistrementRelatif = 0;
for (Plant unPlant : vecteurDePlants) {
    // Créer un nouvel enregistrement
    // Le seek établit la position courante
    // NB Les quatre premiers octets du fichier contiennent le nombre
    // d'enregistrements créés
    fichierDirectPlants.seek(
        numeroEnregistrementRelatif * Plant.tailleMaxEnregistrement()
        + 4);
    unPlant.ecrireEnregistrementTailleMax(fichierDirectPlants);
    numeroEnregistrementRelatif++;
}
// Stocke le nombre d'enregistrements dans les octets 0 à 3
fichierDirectPlants.seek(0);
fichierDirectPlants.writeInt(numeroEnregistrementRelatif);
}
}

public static void main(String[] args) throws Exception {
    ArrayList<Plant> vecteurDePlants = lirePlantsFichierTexte();
    ecrireFichierDirectPlants(vecteurDePlants);
}
}

```

Exercice. Etendre le programme précédent en permettant de modifier le numéro de plant et sa description en plus du prix. Ajouter la possibilité de sélectionner un *Plant* par son numéro.

Exercice. Dans l'exercice précédent, cherchez une solution pour sélectionner un enregistrement par son numéro qui permet d'éviter de lire tout le fichier dans le pire cas.

Exercice. Ajoutez la possibilité de supprimer un *Plant*.

À propos des auteurs

Robert Godin est professeur retraité du Département d'informatique de l'Université du Québec à Montréal (UQAM). Au cours de sa carrière, ses enseignements et ses travaux de recherche ont porté notamment sur la programmation, le génie logiciel et les bases de données. Il est l'auteur de plusieurs ouvrages pédagogiques en informatique.

Daniel Lemire est professeur d'informatique à l'Université TÉLUQ (Université du Québec). Ses recherches portent sur la performance des logiciels et les techniques d'indexation en science des données. Plusieurs de ses bibliothèques logicielles libres — comme Roaring Bitmaps et des algorithmes de compression vectorisés (SIMD) — sont utilisées dans des navigateurs web, dans Git et dans les bibliothèques standard de plusieurs langages de programmation. Il figure parmi les chercheurs les plus cités au monde (classement de l'Université Stanford) et tient un blogue largement suivi à l'adresse <https://lemire.me>.
